PYTHON DATA SCIENCE · COMPLETE ANSWER KEY

Python Data Science

Every self-check item answered and explained, plus reference notes for each lab, for all 21 sessions. Use for grading and reteaching. Confidential instructor material.

21 sessions | 252 self-check items + lab solutions | Confidential · do not distribute

CONTENTS

S01 **Toolkit and First Arrays**

- 0.1 Your Data Science Toolkit
- 1.1 Arrays vs Lists

S02 **Vectorized Operations and Broadcasting**

- 1.2 Vectorized Operations
- 1.3 Broadcasting

S03 **Randomness and Linear Algebra**

- 1.4 Random Number Generation
- 1.5 Linear Algebra Essentials

S04 **DataFrames and Loading Data**

- 2.1 DataFrames and Series
- 2.2 Loading Data

S05 **Cleaning and Grouping**

- 2.3 Cleaning Data
- 2.4 GroupBy Operations

S06 **Merging and Reshaping**

- 2.5 Merging and Joining
- 2.6 Pivot Tables and Reshaping

S07 **Time Series and Windows**

- 2.7 Time Series
- 2.8 Window Functions

S08 **Features and First Charts**

- 2.9 Feature Engineering Basics
- 3.1 Line, Scatter and Bar Charts

S09 **Composing and Customizing Figures**

- 3.2 Subplots and Composition
- 3.3 Customizing Plots

S10 **Saving Figures; Center and Spread**

- 3.4 Saving Figures
- 4.1 Descriptive Statistics I: Center and Spread

S11 **Shape, Outliers and Distributions**

- 4.2 Descriptive Statistics II: Shape, Position and Outliers
- 4.3 Probability Distributions

S12 **Correlation and Hypothesis Tests**

- 4.4 Correlation Analysis
- 4.5 Hypothesis Testing

S13 **A/B Testing and the EDA Workflow**

- 4.6 A/B Testing
- 4.7 An EDA Workflow

S14 **SQL from Python: Queries and Joins**

- 5.1 Database Connections
- 5.2 Complex Queries

S15 **SQL Windows and Query Speed**

- 5.3 SQL Window Functions
- 5.4 Query Optimization

S16 **Documents and HTTP**

- 5.5 NoSQL with MongoDB
- 6.1 HTTP with Requests

S17 **Scraping Static and Dynamic Pages**

- 6.2 Parsing HTML with Beautiful Soup
- 6.3 Dynamic Pages with Selenium

S18 **APIs: Keys, Tokens and Polite Clients**

6.4 API Authentication

6.5 Rate Limiting and Retries

S19 **Seaborn and Plotly**

7.1 Statistical Graphics with Seaborn

7.2 Interactive Charts with Plotly

S20 **Bokeh and Streamlit**

7.3 Interactive Charts with Bokeh

7.4 Dashboards with Streamlit

S21 **Capstone Studio and Showcase**

8.1 The Pinecrest Report

INSTRUCTOR KIT · ANSWER KEY

Session 1: Toolkit and First Arrays

Answers with reasoning for everything in the Session 1 workbook: the opening diagnostic, the twelve self-check items (with why each wrong option is wrong), the predict-then-run items, reference notes for every lab and handout, the lesson pages' own Quick Quiz answers, the exit ticket, and what the common wrong answers tell you about a learner.

Session 1 | 12 self-check items + 6 diagnostic | Confidential

> **NUMBERING MATCHES THE WORKBOOK**
> The diagnostic (D1 to D6), the self-check items (Q1 to Q12), the predict items (P1 to P3 per lesson) and the exit ticket (E1 to E3) use the same numbers, wording and option letters as the participant workbook: all three documents are generated from one source. Every output printed here came from running the code on the course data with Python 3.12, NumPy 2.5 and pandas 3.0. Timings are the only figures that will differ from machine to machine.

START HERE · DIAGNOSTIC

Not graded. The reasoning for each answer also says where the skill comes back in this course.

D1

What does this print?

```
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
print(days[-2:], days[1:3])
```

A) `['Sat', 'Sun'] ['Tue', 'Wed']`

B) `['Sun'] ['Tue', 'Wed', 'Thu']`

C) `['Sat', 'Sun'] ['Mon', 'Tue', 'Wed']`

D) `['Fri', 'Sat'] ['Tue', 'Wed']`

> **Why:** A negative start counts from the end, and a slice leaves out its stop position. Array slicing in Lesson 1.1 uses exactly these rules, then adds a second dimension. **Skill:** slicing. **Review if missed:** Intro Python: lists and slicing.

D2

What does this print?

```
a = [1, 2, 3]
b = a
c = a[:]
b.append(4)
print(a, c)
```

A) `[1, 2, 3] [1, 2, 3]`

B) `[1, 2, 3, 4] [1, 2, 3, 4]`

C) `[1, 2, 3, 4] [1, 2, 3]`

D) `[1, 2, 3] [1, 2, 3, 4]`

> **Why:** `b = a` is a second name for the same list; `a[:]` is a copy. This is the idea behind views and copies today, with one twist: a slice of an ARRAY is not a copy. **Skill:** names and copies. **Review if missed:** Intro Python: lists are mutable; two names, one list.

D3

What does this print?

```
stock = {"tent": 4, "stove": 0}
stock["lamp"] = stock.get("lamp", 0) + 2
print(len(stock), stock["lamp"])
```

A) `2 2`

B) `3 2`

C) A `KeyError`

D) `3 None`

> **Why:** `.get("lamp", 0)` returns 0 for a missing key instead of raising an error, so the new key is created with 0 + 2. Counting with a dictionary is what `value_counts()` does for you in pandas. **Skill:** dictionaries. **Review if missed:** Intro Python: dictionaries and .get().

D4

What does this print?

```
def with_tax(price, rate=0.08):
    return round(price * (1 + rate), 2)

print(with_tax(50), with_tax(50, rate=0.10))
```

A) `54.0 54.0`

B) `4.0 5.0`

C) `None None`

D) `54.0 55.0`

> **Why:** The first call uses the default rate, the second overrides it with a keyword argument, and the function returns its result. Every library call in this course works this way: `read_csv(path, parse_dates=[...])`. **Skill:** functions. **Review if missed:** Intro Python: functions, return values, default arguments.

D5

What does this print?

```
totals = [434.90, 75.90, 399.80, 26.71]
big = [t for t in totals if t >= 100]
print(len(big), max(big))
```

A) `2 434.9`

B) `4 434.9`

C) `2 399.8`

D) `3 434.9`

> **Why:** The condition keeps 434.90 and 399.80. Filtering rows with a condition is the most common move in data analysis; Lesson 1.2 does it without the loop. **Skill:** comprehensions. **Review if missed:** Intro Python: list comprehensions with a condition.

D6

A row read from a CSV file with the `csv` module looks like this. What does the code print?

```
row = {"store": "Boise", "revenue": "5962.0"}
print(row["revenue"] * 2)
```

A) `11924.0`

B) `5962.05962.0`

C) A `TypeError`

D) `5962.0 2`

> **Why:** Everything in a text file arrives as text, and text times 2 repeats. `float(row["revenue"]) * 2` gives 11924.0. NumPy and pandas do this conversion for you, which is why checking the dtype matters. **Skill:** text from files. **Review if missed:** Intermediate Python: reading files; values in a text file are strings.

0.1 · YOUR DATA SCIENCE TOOLKIT: SELF-CHECK

Q1

Your terminal prompt starts with `(.venv)`. What does that tell you?

A) Python is running inside a notebook

B) This terminal will use the project's own Python and the packages installed for it

C) The course packages are being downloaded right now

D) The `data` folder was found

> **Why:** The prefix appears when the virtual environment is active in this terminal window. From then on `python` and `python -m pip` mean the ones inside `.venv`.
> **A** is wrong: A notebook runs in a browser tab or an editor; the prompt prefix is about the terminal.
> **C** is wrong: Activating installs nothing. Installing is a separate command, run once.
> **D** is wrong: Activation knows nothing about your data. `Path("data/orders.csv").exists()` answers that question.

Q2

Which command installs every package the course needs, using the `pip` that belongs to the active environment?

A) `pip install requirements.txt`

B) `python requirements.txt`

C) `python -m venv requirements.txt`

D) `python -m pip install -r requirements.txt`

> **Why:** `python -m pip` runs the pip of the Python that is active, and `-r` tells it to read package names from the file.
> **A** is wrong: Without `-r` pip looks for a package that is literally named requirements.txt. A bare `pip` may also belong to a different Python.
> **B** is wrong: That asks Python to run the text file as a program, which fails on the first line that is not Python.
> **C** is wrong: That would create a new environment in a folder called requirements.txt.

Q3

The status column of `orders` holds completed 18,762 times, returned 1,143 times and cancelled 397 times. What does this print?

```
print(len(orders[orders["status"] == "returned"]))
```

`1143`

> **Why:** The expression in the brackets keeps only the rows where the status is exactly "returned", and `len` counts the rows that are left.

Q4

A script stops with this traceback. What is the most useful first step?

```
Traceback (most recent call last):
  File "/home/you/pinecrest-analysis/report.py", line 4, in <module>
    print(orders["Order Total"].sum())
  ...
KeyError: 'Order Total'
```

A) Print `orders.columns` and copy the exact column name from there

B) Reinstall pandas, because the error happened inside it

C) Restart the computer and run the script again

D) Change the path to use backslashes

Why: Read the last line first: a `KeyError` from a table means there is no column with exactly that name. The real name is `order_total`.

B is wrong: The last line names your key, not a problem with pandas. Library lines in a traceback are rarely where the mistake is.

C is wrong: The same code gives the same error every time.

D is wrong: The error is about a column name, not about a file path.

Q5

In a notebook, cell 1 loads `orders` and cell 3 prints `orders.shape`. You restart the kernel and run only cell 3. What happens?

A) Cell 3 prints the shape, because the notebook file remembers the table

B) A `FileNotFoundError`, because the data folder is closed

C) A `NameError`: the name `orders` is not defined

D) Nothing is printed and there is no error

Why: Restarting the kernel clears every variable. Until cell 1 runs again there is nothing called `orders`. Run the cells from the top.

A is wrong: The file stores your code and the old outputs, not the variables in memory.

B is wrong: No file is being opened in cell 3.

D is wrong: Using a name that does not exist always raises an error.

Q6

Python is working in your project folder, which contains `data/orders.csv` (and no file called order.csv). What does this print?

```
from pathlib import Path

print(Path("data/orders.csv").exists(), Path("data/order.csv").exists())
```

```
True False
```

> **Why:** A relative path is looked up from the folder Python is working in. The first file is there; the second name is missing an s, so it is not.

0.1 · YOUR DATA SCIENCE TOOLKIT: PREDICT, THEN RUN

P1

Two numbers. Which one is bigger, and roughly by how much?

```
print(len(orders), len(orders[orders["channel"] == "store"]))
```

```
20302 7448
```

> **Why:** All 20,302 orders, then only the 7,448 placed in a store. A filter can only keep rows, never add them, so the second number is never larger than the first.

P2

The file covers two calendar years. What does this print?

```
print(orders["order_date"].min().year, orders["order_date"].max().year)
```

```
2024 2025
```

> **Why:** `.min()` and `.max()` work on dates as they do on numbers: the earliest order is in 2024 and the latest in 2025. This only works because `parse_dates` turned the text into real dates when the file was loaded.

P3

This line has a mistake in it. Predict the kind of error and its message.

```
orders["Status"]
```

```
KeyError: 'Status'
```

Why: Column names must match exactly, capitals included. The column is `status`. `print(orders.columns)` shows the real names.

1.1 · ARRAYS VS LISTS: SELF-CHECK

Q7

What does this print?

```
a = [1, 2, 3]
b = np.array([1, 2, 3])
print(a * 2, b * 2)
```

A) `[1, 2, 3, 1, 2, 3] [2 4 6]`

B) `[2, 4, 6] [2 4 6]`

C) `[1, 2, 3, 1, 2, 3] [1 2 3 1 2 3]`

D) A `TypeError`

Why: For a list, `*` means repeat. For an array, `*` is arithmetic on every element.
B is wrong: A list never does arithmetic on its items; you would need a loop or a comprehension.
C is wrong: An array never repeats itself with `*`; it multiplies.
D is wrong: Both operations are allowed. They just mean different things.

Q8

What does this print?

```
t = np.arange(12).reshape(3, 4)
print(t[1, 2], t[:, 0])
```

`6 [0 4 8]`

> **Why:** The table is 0 to 11 in three rows of four. Row 1, column 2 is 6. A lone colon means every row, so `t[:, 0]` is the first column: 0, 4, 8.

Q9

What does this print?

```
a = np.array([5, 6, 7, 8])
b = a[1:3].copy()
b[0] = 0
print(a)
```

A) `[5 0 7 8]`

B) `[5 6 7 8]`

C) `[0 6 7 8]`

D) `[5 0 0 8]`

> **Why:** `.copy()` makes an independent array, so writing to `b` cannot reach `a`.
> **A** is wrong: That is what happens WITHOUT `.copy()`: a plain slice is a view, and `b[0]` is `a[1]`.
> **C** is wrong: `b[0]` is the first element of the slice, which started at position 1 of `a`, not position 0.
> **D** is wrong: Only one element was assigned.

Q10

What is the dtype of `np.array([1, 2.5, 3])`?

```
print(np.array([1, 2.5, 3]).dtype)
```

A) `int64`

B) `object`

C) `<U32`

D) `float64`

> **Why:** An array has one type. A float can hold 1 and 3, but an integer cannot hold 2.5, so NumPy promotes everything to `float64`.
> **A** is wrong: An integer array would have to drop the .5.
> **B** is wrong: NumPy only falls back to `object` when you ask for it or when the values have no common type.
> **C** is wrong: That is a text dtype; it appears when a string is mixed in with the numbers.

Q11

What does this print?

```
r = np.array([10.0, np.nan, 30.0])
print(r.mean(), np.nanmean(r))
```

`nan 20.0`

> **Why:** Any arithmetic that touches a `nan` gives `nan`, so the plain mean is `nan`. `np.nanmean` skips the missing value and averages the two that are present.

Q12

Which job suits a Python list better than a NumPy array?

A) Raising two million prices by 8%

B) Totaling each column of a days-by-stores table

C) Collecting sensor readings one at a time when you do not know how many there will be

D) Storing five million revenue figures in as little memory as possible

> **Why:** An array's size is fixed, so every `np.append` copies everything. Collect in a list, convert once with `np.array(the_list)`, then compute.
> **A** is wrong: Whole-array arithmetic is the array's best job: one expression, compiled speed.
> **B** is wrong: Columns of a 2-D array are one expression, `table[:, k].sum()`; a list of lists needs a loop.
> **D** is wrong: An array stores 8 bytes per number; a list needs about 32 to 36.

1.1 · ARRAYS VS LISTS: PREDICT, THEN RUN

P1

What does this print?

```
x = np.array([1, 2, 3])
print(x + x, [1, 2, 3] + [1, 2, 3])
```

```
[2 4 6] [1, 2, 3, 1, 2, 3]
```

> **Why:** `+` on arrays adds element by element; `+` on lists joins them end to end. Same symbol, two meanings, just like `*`.

P2

What does this print?

```
g = np.arange(6).reshape(2, 3)
row = g[0]
row[:] = 9
print(g)
```

```
[[9 9 9]
 [3 4 5]]
```

Why: `g[0]` is a view of the first row, so filling `row` with 9 writes into `g`. `row = g[0].copy()` would have left `g` as it was.

P3

What does this print?

```
z = np.zeros(3, dtype=np.int64)
z[0] = 2.99
z[1] = True
print(z)
```

```
[2 1 0]
```

Why: The array is `int64` and stays `int64`. 2.99 is chopped to 2 (not rounded to 3), `True` becomes 1, and the third value was never touched.

LAB REFERENCE SOLUTIONS

Every lab has a runnable `_solution.py` next to its starter in `labs/module-01/`. The notes below give what a correct run prints, the lines that matter and why, and the wrong results you are most likely to see while circulating. Check all eight files at once with `python3 labs/check_labs.py module-01`.

ENVIRONMENT CHECK: LABS/MODULE-01/L0_1-ORIENTATION/CHECK_ENVIRONMENT.PY

A tool, not an exercise, so it has no starter and solution. It checks Python 3.12 or newer, an active virtual environment, every package in `requirements.txt` (printing each version), and the data folder (all 12 data files, and 20,302 rows in `orders.csv`). A ready machine ends with `READY for Python Data Science`. Anything else ends with `NOT READY. Fix before Session 2:` and a numbered list that says exactly what to type.

JupyterLab and kaleido are reported as notes, not failures: a learner who works only in VS Code does not need JupyterLab, and kaleido is needed only for static Plotly images in Lesson 7.2. Every other package missing is a FIX

line.

The check looks for the data in the folder it is run from and every folder above it, so run it from `pinecrest-analysis` (or copy it there). The "running from" line shows which Python actually ran: if it does not contain `.venv`, the environment was not active.

DIAGNOSTIC PRACTICE: LABS/MODULE-01/L0_1-ORIENTATION/DIAGNOSTIC_STARTER.PY → DIAGNOSTIC_SOLUTION.PY

Six small functions, one per diagnostic skill, with a checker that prints PASS or "not yet" for each and never stops at the first failure. The starter prints six "not yet" lines and `0 of 6 checks pass.` The solution prints six PASS lines, `6 of 6 checks pass.` and `Two years of store revenue: $21,539,319.88` (the same total Lesson 1.1 reports).

What each one tests: `weekend()` slicing (`days[-2:]`); `with_extra()` returning a new list without changing the caller's (`items + [extra]`; `items.append(extra)` fails the check because the original grows); `count_values()` the `.get(value, 0) + 1` counting pattern; `average()` a default argument and an empty-list guard; `big_orders()` a comprehension over dictionaries; `column_total()` reading a CSV with `csv.DictReader`, converting text with `float()` and skipping the six blank revenue cells.

Use it as homework for anyone who missed two or more diagnostic questions. A learner who cannot get `column_total()` to pass after a real try needs the Intermediate Python file lessons before Module 2.

LAB 0.1-A: LABS/MODULE-01/L0_1-ORIENTATION/PAYMENT_METHODS_STARTER.PY → PAYMENT_METHODS_SOLUTION.PY

Reference output: `(20302, 11)`; card 12,392, digital_wallet 4,794, cash 1,614, gift_card 1,502; digital wallet share 23.6 (percent); 1,502 gift-card orders averaging $193.22; the cash orders' channel count shows `store 1614` and nothing else; `Cash orders placed online: 0`.

The solution's key lines:

```
methods = orders["payment_method"].value_counts()
wallet_pct = round((orders["payment_method"] == "digital_wallet").mean() * 100, 1)
gift = orders[orders["payment_method"] == "gift_card"]
cash = orders[orders["payment_method"] == "cash"]
```

Why it is written this way: a comparison on a column gives a True/False column; its mean is the fraction that is True (Exercise 2 of the lesson uses the same idea with `.notna()`). The filter is the lesson's `completed` pattern with a different column and value.

Common wrong results: 4,794 printed instead of 23.6 (they used `.sum()`, a count, not `.mean()`); 0.2 or 0.236 (forgot to multiply by 100 or rounded too early); an empty table for cash (`"Cash"` with a capital: values are case-sensitive too); `SyntaxError` from a single `=` inside the filter.

HANDOUT: LABS/MODULE-01/L0_1-ORIENTATION/DEBUG_TRACEBACK.PY (DELIBERATELY BROKEN)

Three bugs, met one at a time. **1** `FileNotFoundError: ... order.csv`: the file is `orders.csv`. **2** `KeyError: 'Status'`: the column is `status`. **3** `NameError: name 'retuned' is not defined`: the variable is `returned`. Fixed, it prints `Orders: 20302`, `Returned orders: 1143`, `Largest returned order: $1,168.75`.

What to listen for: a learner who fixes all three at once by reading the code has skipped the exercise. The point is the routine: run, read the last line, find your line number, fix one thing, run again.

LAB 1.1-A: LABS/MODULE-01/L1_1-ARRAYS-VS-LISTS/TRANSACTIONS_WEEK_STARTER.PY → TRANSACTIONS_WEEK_SOLUTION.PY

Reference output: first eight values `[51. 52. 68. 33. 39. 45. 73. 39.]`; shape `(2903,)`, dtype `float64`, ndim 1, size 2903; missing values 0; the week table (7 rows, 4 columns, first row `[61. 49. 63. 28.]`, last row `[57. 73. 130. 57.]`); Saturday `[93. 59. 151. 71.]`; Bend `[49. 42. 39. 46. 66. 59. 73.]`; Boise on Saturday 151.0; busiest store-day 151.0; `float64: 224 bytes | int16: 56 bytes`; same values True.

The solution's key lines:

```
week2 = transactions[28:56].reshape(7, 4)     # 7 days x 4 stores, a view
print(week2[5])         # Saturday, every store
print(week2[:, 1])      # Bend, every day
print(week2[5, 2])      # Boise on Saturday
small = week2.astype(np.int16)
```

Why: the file is sorted by date and then store, and early January has no gaps, so 28 consecutive values are exactly 7 days of 4 stores, and `reshape(7, 4)` fills row by row. The solution also prints two extras worth showing in the debrief: `week2` shares memory with the full column (True: slicing and reshaping copy nothing) but `small` does not (False: `astype` always builds a new array).

Common wrong results: `ValueError: cannot reshape array of size 27 into shape (7,4)` (stop of 55 instead of 56); a table of 4 rows whose first row is `[61. 49. 63. 28. 42. 42. 69.]`, with no error at all (that is `reshape(4, 7)`: each row now runs across two days and mixes the stores); Tuesday's row printed for Bend (`week2[1]` instead of `week2[:, 1]`); 112 bytes for the small table (`int32`, not `int16`).

LAB 1.1-B: LABS/MODULE-01/L1_1-ARRAYS-VS-LISTS/WHAT_IF_COPY_STARTER.PY → WHAT_IF_COPY_SOLUTION.PY

Reference output: the week-1 units table (first row `[110 105 148 69]`); `Bend, real total: 868`; `bend shares memory with week1: True`; `Bend, what-if total: 938`; `Bend, real total: 868`; `what_if shares memory with week1: False`; `stored 97.6, got: 97 | dtype: int64`; `float version: 97.6 | dtype: float64`; `Bend, real total (still): 868`.

The solution's key lines:

```
what_if = week1.copy()                    # independent table
what_if[:, 1] = what_if[:, 1] + 10        # Bend's column, in the copy only
trap = week1[:, 1].copy()
trap[0] = 97.6                            # int64 array: stored as 97
fixed = week1[:, 1].astype(np.float64)    # a float array can hold 97.6
```

Why: seven days times 10 units is 70, so 868 becomes 938 in the copy and the original must still say 868. The integer trap is Common Mistake 4 from the lesson on real numbers: an array never changes its dtype to fit a value.

Common wrong results: the real total prints 938 after the what-if (`what_if = week1` or `what_if = week1[:, :]`: both share memory with the original); `shares_memory` prints True for the what-if for the same reason; 98 expected for the trap (rounding was assumed; storing into an integer array chops).

HANDOUT: LABS/MODULE-01/L1_1-ARRAYS-VS-LISTS/DEBUG_SILENT_TRAPS.PY (WRONG ANSWERS, NO ERROR)

As shipped it prints `dtype: <U32`, `average price: ['24.5' '39.0']`, `total stock: 4`, `Monday is still: 0`. **Block 1:** the quotes around `"82.00"` turn the whole array into text; remove them and `prices.mean()` is 48.5. **Block 2:** 100 + 60 does not fit in `int8` (limit 127) and wraps to -96, so the total is 4; drop `dtype=np.int8` (or use `int64`) and the total is 260. **Block 3:** `what_if = boise_week[:3]` is a view; add `.copy()` and Monday is still 148.

THE LESSON PAGES' OWN QUICK QUIZZES

Each lesson page ends with a five-question Quick Quiz (three options each). Learners do them at home; these are the answers with the page's own explanation.

LESSON 0.1 · YOUR DATA SCIENCE TOOLKIT

1. **Question 1.** Yesterday everything worked. Today you open a new terminal, go to your project folder, run `python first_look.py`, and get `ModuleNotFoundError: No module named 'pandas'`. What is the most likely cause?
   Answer: C) The virtual environment is not activated in this terminal. Activation applies to one terminal window. In a new one, python means the system Python again, which does not have the course packages. Activate the environment, look for (.venv) in the prompt, and run the script again.

2. **Question 2.** A traceback fills your screen with thirty lines. Where should you look first?
   Answer: A) The last line, which gives the error type and message. The last line names the type of error and gives its message, which often tells you the cause outright. After that, scan upward for the line that belongs to your own file.

3. **Question 3.** In a requirements file, what does `pandas~=3.0.0` allow pip to install?
   Answer: B) Version 3.0.0 or any later 3.0.x release, but not 3.1. The compatible-release operator lets the last number written go up and nothing else. So 3.0.0, 3.0.6 and any other 3.0.x are allowed, while 3.1 and 4.0 are not.

4 **Question 4.** The file `orders.csv` is definitely inside your `data` folder, yet `pd.read_csv("data/orders.csv")` raises `FileNotFoundError` in a notebook. What should you check?
Answer: A) Which folder Python is working in, with `Path.cwd()`. A relative path is resolved from the folder Python is working in. If JupyterLab was started somewhere else, data/orders.csv points to a place that does not exist. Path.cwd() shows the working folder.

5 **Question 5.** You want to try several ways of charting a dataset, looking at each result before deciding on the next step. Which fits best, and why?
Answer: B) A notebook, because variables stay in memory and each result appears under its cell. A notebook keeps the loaded data in memory and shows each result under its cell, so you can change one step and rerun only that step. Once the analysis is settled, move it into a script so it runs the same way every time.

LESSON 1.1 · ARRAYS VS LISTS

1 **Question 1.** What does this print?

```
x = np.array([1, 2, 3])
y = x[:2]
y[0] = 99
print(x)
```

Answer: B) `[99 2 3]`. Slicing an array gives a view of the same memory, so writing to y[0] changes x[0] as well. Use x[:2].copy() when you need an independent array.

2 **Question 2.** One million whole numbers take about 36 MB as a Python list and 8 MB as an `int64` array. What is the main reason?
Answer: A) The list stores an address plus a full Python object for every number; the array stores only the raw values. A list stores an 8-byte address for each item, and each item is a full Python object of about 28 bytes carrying its own type information. The array stores the type once and then only the 8-byte values.

3 **Question 3.** What does this print?

```
z = np.array([7, 8, 9])
z[1] = 4.9
print(z)
```

Answer: C) `[7 4 9]`. z has dtype int64 and keeps it. Assigning 4.9 chops off the decimal part, leaving 4, with no warning. Create the array as float64 if decimals are possible.

4 **Question 4.** `sales` has shape `(3, 4)`: three days as rows, four stores as columns. Which expression gives every day's value for the store in column 1?
Answer: B) `sales[:, 1]`. Rows come before the comma and columns after. A lone colon means every row, and 1 picks column 1, so the result has three values, one per day.

5 **Question 5.** You are reading sensor values one at a time from a device and do not know how many there will be. Afterward you want their mean and maximum. What is the best approach?

Answer: C) Append each value to a list, then convert the list to an array once and compute there. Lists are built for cheap appends and arrays are built for math. Collect in a list while the size is unknown, convert once with np.array(), and compute on the array.

OPENING: WHAT A READY MACHINE PRINTS

`check_environment.py` on a ready machine: four numbered sections (Python, virtual environment, course packages with one version per line, course data) where every line starts with `[ok]`, then `READY for Python Data Science`. A `[note]` line (JupyterLab or kaleido missing) does not stop READY. A `[FIX]` line always comes with a numbered instruction at the end under `NOT READY. Fix before Session 2:`.

The three NOT READY results you will see most: **no environment active** (the fix list gives both activate commands); **packages missing** (it lists them and gives the install command); **data folder not found** (it gives the unzip command and says to run the check from the project folder).

EXIT TICKET

E1

Your prompt does not start with `(.venv)`. What will probably go wrong, and what do you type to fix it?

Python will not find the course packages (`ModuleNotFoundError`). Activate the environment: `.venv\Scripts\Activate.ps1` on Windows, `source .venv/bin/activate` on macOS and Linux, from the project folder.

E2

`a = np.array([1, 2, 3])`, then `b = a[:2]` and `b[0] = 9`. What is `a` now, and what one change would have protected it?

`[9 2 3]`: a slice is a view of the same memory. `b = a[:2].copy()` makes an independent array.

E3

The mean of a revenue array prints `nan`. What does that tell you, and what do you use instead?

At least one value is missing, and arithmetic with a missing value gives a missing value. `np.nanmean(revenue)` skips them (`np.isnan(revenue).sum()` counts them).

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

WHAT YOU SEE OR HEAR	WHAT IT REVEALS	WHAT TO DO
Q1 or E1 missed; "I installed pandas yesterday and now it is gone"	The learner thinks of activation as installation, or does not know it is per terminal window	Have them close the terminal, open a new one and get back to `(.venv)` without notes. Repeat until it is boring.
Q4 answered B ("reinstall pandas"); long stares at the top of a traceback	Reading tracebacks top-down and stopping at the first library line	Cover everything but the last line with a sheet of paper. Ask what it says, then uncover upward to their own file.
Q5 answered A; `NameError` panic after a restart	The notebook is seen as a document that remembers, not a conversation with a running Python	Restart and Run All together, twice. Point at the execution numbers in the brackets.
Q7 answered B; D6 missed	Arithmetic is assumed to "just work" on any container	Run `[1, 2, 3] * 2` and `"12" * 2` side by side with the array version. Ask what `*` means for each type.
Q9 answered A; P2 predicted an unchanged table; exit ticket 2 wrong	The list model of slicing (a slice is a copy) carried over to arrays	The five-number window example again, with `np.shares_memory` after each line. Then Lab B's Step 1.
Q8 written as `9 [0 1 2 3]` or similar	Rows and columns swapped, or the row index read as a count	The paper table: label rows and columns, circle the selection, then type it. "Rows, comma, columns."
Q10 answered A; P3 predicted `[2.99 1 0]` or `[3 1 0]`	Belief that an array adapts its type to each value, or that conversion rounds	`.dtype` before and after an assignment. An array's dtype never changes; converting chops.
Q11 written as `20.0 20.0`	Missing values are assumed to be skipped automatically	Run it. Then `np.isnan(r).sum()`. In pandas the default will be different, which is worth saying now.
Lab A: `reshape(4, 7)` with no complaint	Shape is treated as decoration; the fill order (row by row) is not in the learner's model yet	Ask what the first four values of the file are (one day, four stores) and where they went in each version of the table.

INSTRUCTOR KIT · ANSWER KEY

Session 2: Vectorized Operations and Broadcasting

Answers with reasoning for everything in the Session 2 workbook: the twelve self-check items (with why each wrong option is wrong), the predict-then-run items, reference notes for every lab and handout, the lesson pages' own Quick Quiz answers, the warm-up and exit ticket, and what the common wrong answers tell you about a learner.

Session 2 | 12 self-check items | Confidential

NUMBERING MATCHES THE WORKBOOK

The self-check items (Q1 to Q12), the predict items (P1 to P3 per lesson), the warm-up (W1 to W5) and the exit ticket (E1 to E3) use the same numbers, wording and option letters as the participant workbook and the facilitator guide: all three documents are generated from one source. Every output printed here came from running the code on the course data with Python 3.12 and NumPy 2.5.

1.2 · VECTORIZED OPERATIONS: SELF-CHECK

Q1

What does this print?

```
v = np.array([4, 9, 2, 7])
print((v > 3).sum(), (v > 3).mean())
```

A) `3 0.75`

B) `20 0.75`

C) `[4 9 7] 3`

D) `3 6.67`

Why: The comparison gives a mask, `[ True True False True]`. True counts as 1, so the sum is how many are true (3) and the mean is the fraction that are true (3 of 4).

B is wrong: 20 is the sum of the VALUES that pass. The sum of a mask counts; `v[v > 3].sum()` would add the values.

C is wrong: That is the filter `v[v > 3]`, not the sum of the mask.

D is wrong: 6.67 is the mean of the values that pass, `v[v > 3].mean()`. The mean of the mask is a fraction between 0 and 1.

Q2

`stores` (text) and `visitors` (numbers) are parallel arrays. Which line counts the Bend store-days with more than 500 visitors?

A) `(stores == "Bend" and visitors > 500).sum()`

B) `(stores == "Bend" & visitors > 500).sum()`

C) `((stores == "Bend") & (visitors > 500)).sum()`

D) `(stores == "Bend").sum() & (visitors > 500).sum()`

Why: Combine masks with `&` and put parentheses around each comparison. The sum of the combined mask is the count.

A is wrong: `and` wants one true-or-false value on each side and gets an array: `ValueError: The truth value of an array ... is ambiguous`.

B is wrong: Without parentheses Python evaluates `"Bend" & visitors` first and raises a `TypeError` that mentions `bitwise_and`.

D is wrong: That counts each group separately and then does a bitwise "and" of two whole numbers. It answers no question at all.

Q3

What does this print?

```
t = np.array([[1, 2, 3],
              [4, 5, 6]])
print(t.max(axis=1), t.sum(axis=0).shape)
```

`[3 6] (3,)`

Why: The axis you name is the axis that disappears. `axis=1` collapses the 3 columns and leaves one value per row: the row maximums 3 and 6. `axis=0` collapses the 2 rows and leaves one value per column, so the shape is `(3,)`.

Q4

What does this print?

```
x = np.array([5.0, np.nan, 12.0])
print((x > 8).sum() + (x <= 8).sum())
```

A) `3`

B) `2`

C) `nan`

D) `1`

Why: Every comparison with `nan` is False. The missing value is counted in neither group, so one value is above 8, one is not, and one has silently dropped out.

A is wrong: That assumes every value is in one group or the other. A missing value is in neither.

C is wrong: The masks hold only True and False, so their sums are ordinary whole numbers.

D is wrong: 1 is the count of one group alone.

Q5

What does this print?

```
p = np.array([40, 95, 60])
print(np.where(p >= 60, "pass", "retry"))
```

`['retry' 'pass' 'pass']`

> **Why:** `np.where(condition, value_if_true, value_if_false)` is an if/else for every element, and the result keeps the array's shape. 60 passes because the test is `>=`.

Q6

What does this print?

```
names = np.array(["Asheville", "Bend", "Boise", "Flagstaff"])
totals = np.array([2935, 3312, 4347, 2661])
print(names[totals.argmax()], totals.argmin())
```

A) `4347 2661`

B) `Boise Flagstaff`

C) `2 3`

D) `Boise 3`

> **Why:** `argmax` and `argmin` return positions, not values. Position 2 looked up in the parallel array `names` is Boise; the smallest total is at position 3.
>
> **A** is wrong: Those are `totals.max()` and `totals.min()`: the values, not their positions.
>
> **B** is wrong: The second part is printed as a bare position; nothing looked it up in `names`.
>
> **C** is wrong: The first part IS looked up in `names`, so it prints a store name.

1.2 · VECTORIZED OPERATIONS: PREDICT, THEN RUN

P1

What does this print?

```
s = np.array([3, 8, 5, 10])
m = s > 4
print(m, s[m], s[~m])
```

```
[False  True  True  True] [ 8  5 10] [3]
```

Why: The mask marks the values above 4. In brackets it keeps them; `~m` flips every True and False, so it keeps the rest.

P2

What does this print?

```
g = np.array([[1, 2],
              [3, 4],
              [5, 6]])
print(g.sum(axis=0), g.sum(axis=1), g.sum())
```

```
[ 9 12] [ 3  7 11] 21
```

Why: Shape `(3, 2)`. `axis=0` collapses the three rows and leaves one total per column (two numbers). `axis=1` collapses the two columns and leaves one total per row (three numbers). With no axis, everything is added.

P3

What does this print?

```
r = np.array([200.0, np.nan, 50.0])
print((r > 100).sum(), (r == np.nan).sum(), np.isnan(r).sum())
```

```
1 0 1
```

Why: One value is above 100. Nothing is ever equal to `nan`, not even `nan` itself, so the middle count is 0. `np.isnan` is the test that finds the missing value.

1.3 · BROADCASTING: SELF-CHECK

Q7

An array of shape `(6, 3)` is added to an array of shape `(3,)`. What is the result?

A) A `ValueError`, because the shapes are different

B) An array of shape `(6, 3)`

C) An array of shape `(6,)`

D) An array of shape `(3, 3)`

> **Why:** Line the shapes up from the right: 3 meets 3 (equal), and 6 meets a missing axis, which counts as 1. The row of three is reused on each of the six rows.
> **A** is wrong: Different shapes are allowed when every pair of lengths is equal or one of them is 1.
> **C** is wrong: The result takes the larger length from each pair; nothing is collapsed.
> **D** is wrong: The 6 does not shrink to match anything; the smaller array is the one that is reused.

Q8

`t` has shape `(30, 4)`: 30 days as rows, 4 stores as columns. Which expression subtracts each day's mean from that day's row?

A) `t - t.mean(axis=1)`

B) `t - t.mean(axis=0)`

C) `t - t.mean(axis=1, keepdims=True)`

D) `t - t.mean()`

> **Why:** Per-day means are one value per row, and to line up with the rows they must stand as a column of shape `(30, 1)`. `keepdims=True` keeps the collapsed axis with length 1.
> **A** is wrong: The 30 day-means come back flat, shape `(30,)`. From the right, 4 meets 30: `ValueError`.
> **B** is wrong: That runs, but it subtracts each STORE's mean from its column: a different question.
> **D** is wrong: That subtracts one number, the mean of everything, from every cell.

Q9

What does this print?

```
a = np.array([1, 2, 3])
b = np.array([[10], [20]])
print((a * b).shape, (a * b)[1, 2])
```

`(2, 3) 60`

Why: A row of three meets a column of two, so both are stretched into a 2 by 3 grid of every combination. Row 1, column 2 is 20 times 3.

Q10

Which pair of shapes can NOT be broadcast together?

A) `(8, 1)` and `(1, 5)`

B) `(8, 5)` and `(5,)`

C) `(8, 5)` and `(1,)`

D) `(8, 5)` and `(8,)`

Why: Shapes are matched from the right, so the 8 of the flat array is compared with the 5 columns. They are not equal and neither is 1.

A is wrong: Each pair has a 1 in it: a column meets a row and the result is `(8, 5)`.

B is wrong: 5 meets 5, and 8 meets a missing axis: a table and a row.

C is wrong: A length of 1 fits anything; this is the same as combining with a single number.

Q11

What does this print?

```
print(np.broadcast_shapes((4, 1), (3,)))
```

`(4, 3)`

Why: From the right: 1 meets 3 (compatible, result 3), then 4 meets a missing axis (compatible, result 4). A column of four against a row of three is a 4 by 3 grid.

Q12

`sales` has shape `(12,)` and `plan` has shape `(12, 1)`. What does `sales - plan` give?

A) An array of shape `(12, 12)`: every sales value minus every plan value, with no error

B) An array of shape `(12,)` holding the twelve differences

C) A `ValueError`, because the shapes are different

D) An array of shape `(12, 1)` holding the twelve differences

> **Why:** A flat array is treated as a row. A row meeting a column builds the grid of every combination: the accidental grid. Twelve values in should mean twelve out; `plan.ravel()` fixes it.
> **B** is wrong: That is what you wanted, and what you get only after flattening `plan`.
> **C** is wrong: The shapes are compatible by the rules (12 meets 1, then 12 meets a missing axis), which is exactly why this mistake is silent.
> **D** is wrong: The result takes the larger length from each pair, so both axes come out as 12.

1.3 · BROADCASTING: PREDICT, THEN RUN

P1

What does this print?

```
t = np.ones((3, 4))
print((t + np.array([1, 2, 3, 4])).shape, (t + np.array([[1], [2], [3]]))[2])
```

```
(3, 4) [4. 4. 4. 4.]
```

> **Why:** A table and a row of four, then a table and a column of three: both give a `(3, 4)` result. In the second, 3 is added to everything in row 2, so that row is four 4s.

P2

What does this print?

```
col = np.array([1, 2, 3])[:, np.newaxis]
row = np.array([10, 20])
print(col + row)
```

```
[[11 21]
 [12 22]
 [13 23]]
```

Why: `(3, 1)` with `(2,)`: a column meets a row and both are stretched into a 3 by 2 grid holding every sum.

P3

This line fails. Predict the two shapes in the error message, then say which pair of lengths is the problem.

```
np.ones((3, 4)) + np.array([1, 2, 3])
```

```
ValueError: operands could not be broadcast together with shapes (3,4) (3,)
```

Why: From the right, 4 meets 3: not equal, and neither is 1. The three values would need to stand as a column, shape `(3, 1)`, to be added to the three rows.

LAB REFERENCE SOLUTIONS

Every lab has a runnable `_solution.py` next to its starter in `labs/module-02/`. The notes give what a correct run prints, the lines that matter and why, and the wrong results you are most likely to see. Check all eight files with `python3 labs/check_labs.py module-02`.

- LAB 1.2-A: LABS/MODULE-02/L1_2-VECTORIZED-OPERATIONS/STAFFING_LOAD_STARTER.PY → STAFFING_LOAD_SOLUTION.PY

Reference output: `visitors per staff member: mean 55.3 | min 28.2 | max 116.5`; `814 stretched store-days, 28.0% of all store-days`; `stretched AND sale day: 26`; `stretched OR sale day: 884`; quiet 361, steady 1728, stretched 814; `most stretched: 2025-02-26 Bend | 233 visitors, 2 staff`; then a per-store block: staff 5.35 / 5.87 / 8.62 / 6.11 and visitors per staff 52.2 / 57.8 / 56.5 / 54.5 for Asheville, Bend, Boise, Flagstaff.

The solution's key lines:

```
per_staff = visitors / staff
stretched = per_staff >= 60
is_promo = promo == 1
print((stretched & is_promo).sum(), (stretched | is_promo).sum())
labels = np.select([per_staff >= 60, per_staff >= 45], ["stretched", "steady"], default
i = per_staff.argmax()
print(dates[i], stores[i], visitors[i], staff[i])
```

Why: every step is one expression on parallel arrays. The "or" count can be checked by hand: 814 stretched + 96 sale store-days - 26 that are both = 884. The three label counts add up to 2,903 because neither `visitors` nor `staff` has missing values. `argmax` gives one position, and the same position reads the date, the store and both counts.

Common wrong results: stretched count 0 and steady 2,542 (the `>= 45` condition listed first: first match wins); 910 for the "or" (added the two counts instead of combining the masks); `116.5` reported as "the most stretched day" (the value, not the row); the `bitwise_and` TypeError (no parentheses).

- LAB 1.2-B: LABS/MODULE-02/L1_2-VECTORIZED-OPERATIONS/EIGHT_WEEKS_AXIS_STARTER.PY → EIGHT_WEEKS_AXIS_SOLUTION.PY

Reference output: `(56, 4) 2024-01-01 to 2024-02-25`; total 13255.0; per store `[2935. 3312. 4347. 2661.]`; per day, first five `[204. 196. 196. 234. 287.]`; shapes `(4,) (56,)`; each store's busiest day `['2024-02-24' '2024-02-24' '2024-01-13' '2024-01-27']` with `[ 95. 105. 151. 85.]` transactions; three busiest days 2024-02-24, 2024-01-13, 2024-01-20 with totals `[392. 374. 360.]`; Asheville won 1, Bend 8, Boise 45, Flagstaff 2 of 56 days.

The solution's key lines:

```
store_total = table.sum(axis=0)        # the 56 days disappear: 4 values
day_total = table.sum(axis=1)          # the 4 stores disappear: 56 values
best_row = table.argmax(axis=0)        # for each store, the ROW of its busiest day
print(day[best_row])
top3 = np.argsort(day_total)[::-1][:3]
winner = table.argmax(axis=1)          # for each day, the COLUMN of the busiest store
```

Why: `argmax(axis=0)` answers "for each column, which row?", so its four results are row numbers and index the `day` array. `argmax(axis=1)` answers "for each row, which column?", so its 56 results are 0 to 3 and index `store_names`. All three top days are Saturdays.

Common wrong results: dates in early January for every store (`day[table.argmax(axis=1)]`: column numbers 0 to 3 used as row numbers); the three smallest totals (no `[::-1]`); a per-store total of 56 numbers (axis swapped). In each case the shape of the result gives it away.

- HANDOUT: LABS/MODULE-02/L1_2-VECTORIZED-OPERATIONS/DEBUG_MASKS.PY (DELIBERATELY BROKEN)

Block 1: `and` raises `ValueError: The truth value of an array with more than one element is ambiguous`; use `&`. Answer 15. **Block 2:** no parentheses raises `TypeError: ufunc 'bitwise_and' not supported...`; write `(revenue > 20000) & (stores == "Boise")`. Answer 15. **Block 3:** runs and prints 6, and the learner's job is to explain it: the six missing revenues fail both comparisons. `np.isnan(revenue).sum()` counts them directly. **Block 4:** `capped = revenue` is a second name for the same array, so the cap is written into the real data and the last line prints 15000.0 as the largest real revenue. With `capped = revenue.copy()` the largest real revenue is 27677.1 and 101 values in `capped` equal 15000.

LAB 1.3-A: LABS/MODULE-02/L1_3-BROADCASTING/FAIR_VISITORS_STARTER.PY → FAIR_VISITORS_SOLUTION.PY

Reference output: `(224, 4) 2024-01-01 to 2024-08-11`; store averages `[281.4 323.4 440.9 281.6]`; ratio shape `(224, 4)` with first rows `[0.71 0.62 0.76 0.51]` and `[0.74 0.68 0.7 0.65]`; busiest day, times normal `[2.51 2.45 2.3 2.4]`, all on 2024-05-25; `without keepdims: operands could not be broadcast together with shapes (224,4) (224,)`; rows add up to `[1. 1. 1.]`; average share per store `[0.213 0.244 0.332 0.211]`; Boise's largest share 0.406 on 2024-04-20, smallest 0.243 on 2024-02-10.

The solution's key lines:

```
store_mean = table.mean(axis=0)                    # shape (4,)
ratio = table / store_mean                         # (224, 4) with (4,) -> (224, 4)
print(ratio.max(axis=0), day[ratio.argmax(axis=0)])
day_total = table.sum(axis=1, keepdims=True)       # shape (224, 1): a column
share = table / day_total                          # (224, 4) with (224, 1) -> (224, 4)
```

Why: per-store values are a row and line up with the last axis as they are; per-day values must be a column, which `keepdims=True` provides. `ratio.argmax(axis=0)` gives one row position per store, and `day[...]` turns positions into dates (Session 2's first lesson, reused). Visitors have no missing values in this stretch, so plain `mean` and `sum` are safe; the lesson's project uses revenue and needs the `nan`-aware versions.

Common wrong results: 224 "store averages" (`axis=1`); the ValueError in Step 2 (they divided by the day totals there); a share table whose rows do not add up to 1 (divided by `table.sum(axis=0)`: each column then adds up to 1 instead, which is "share of the store's 224 days", a different question).

LAB 1.3-B: LABS/MODULE-02/L1_3-BROADCASTING/TARGET_GRID_STARTER.PY → TARGET_GRID_SOLUTION.PY

Reference output: current means `[ 5703.83 6997.87 10144.21 6889.75]`, shapes `(4,) (3,)`; `(4,)` with `(3,)` fails ("shape mismatch: objects cannot be broadcast to a single shape"); `(4, 1)` with `(3,)` gives `(4, 3)`; `(224, 4)` with `(4,)` gives `(224, 4)`; `(4,)` with `(4, 1)` gives `(4, 4)`; targets (rounded) `[[5875 5989 6274] [7208 7348 7698] [10449 10651 11159] [7096 7234 7579]]`; `targets above $7,500: 5 of 12`; per store `[0 1 3 1]`; wrong shape `(4, 4)` with average size 1753.02; fixed errors `[-196.17 197.87 -255.79 189.75]` with average size 209.9.

The solution's key lines:

```
targets = current[:, np.newaxis] * (1 + growth)      # (4, 1) with (3,) -> (4, 3)
above = targets > 7500
print(above.sum(), above.sum(axis=1))                # all twelve; then one count per s
errors = actual - forecast.ravel()                   # (4,) with (4,) -> (4,)
```

Why: stores must run down the rows, so the four store means become a column; the three growth rates stay a row. In Step 4 the forecast arrives as a column, and a row minus a column is every actual minus every forecast: sixteen numbers whose average size is meaningless. Flattening the forecast gives four errors.

Common wrong results: per store `[1 1 3]` (that is `axis=0`: one count per scenario); a `(3, 4)` grid (the new axis was put on `growth`: legal, but rows are now scenarios, so `axis=1` no longer means "per store"); 1753.02 left standing because the shape line was not read.

HANDOUT: LABS/MODULE-02/L1_3-BROADCASTING/DEBUG_SHAPES.PY (DELIBERATELY BROKEN)

Block 1: `week1 - week1.mean(axis=1)` raises `ValueError: operands could not be broadcast together with shapes (7,4) (7,)`; add `keepdims=True` and the row sums print as seven zeros. **Block 2:** `best` is `(7,)` and `worst` is `(7, 1)`, so `best - worst` is a 7 by 7 grid: it prints 49 gaps with a largest of 9326.91. Make the shapes match (drop `keepdims=True` from `worst`, or add it to `best`) and it prints 7 gaps with a largest of 6131.55. **Block 3:** the square-table trap from the lesson: four day-means fit as a row, so no error; as shipped it prints `[ 829.24 1563.14 620.76 3013.13]`; with `keepdims=True` it prints four zeros.

THE LESSON PAGES' OWN QUICK QUIZZES

Each lesson page ends with a five-question Quick Quiz (three options each). Learners do them at home; these are the answers with the page's own explanation.

LESSON 1.2 · VECTORIZED OPERATIONS

1. **Question 1.** What does this print?

```
q1 = np.array([3, 8, 5, 10])
print((q1 > 4).sum())
```

Answer: A) `3`. The comparison gives [False True True True]. In arithmetic True counts as 1, so the sum of a mask is the number of elements that passed the test.

2. **Question 2.** `revenue` and `promo` are float arrays of the same length. Which line correctly counts sale days with revenue above 10,000?
Answer: C) `((revenue > 10000) & (promo == 1)).sum()`. Each comparison is wrapped in parentheses so it is evaluated first, the ampersand combines the two masks element by element, and the sum counts the True values.

3. **Question 3.** What does this print?

```
q3 = np.array([[1, 2, 3],
               [4, 5, 6]])
print(q3.sum(axis=0))
```

Answer: B) `[5 7 9]`. axis=0 collapses the rows, leaving one total per column: 1+4, 2+5, 3+6. The shape goes from (2, 3) to (3,).

4 **Question 4.** What does this print?

```
q4 = np.array([-2, 5, -1, 3])
print(np.where(q4 > 0, q4, 0))
```

Answer: C) `[0 5 0 3]`. np.where keeps the shape. Where the condition is true it takes the value from q4, and everywhere else it takes 0.

5 **Question 5.** You run `high = revenue[revenue > 15000]` and then `high[0] = 0`. What happens to `revenue`?
Answer: B) Nothing: selecting with a mask builds a new array, so `high` is a copy. Mask indexing gathers the chosen elements into a new array, unlike a slice. To change the original through a mask, assign in one step, with the mask in brackets on the left of the equals sign.

LESSON 1.3 · BROADCASTING

1 **Question 1.** An array of shape `(5, 3)` is added to an array of shape `(3,)`. What is the result?
Answer: B) An array of shape `(5, 3)`. Lined up from the right, 3 meets 3, and the missing axis is treated as 1 and reused along the 5 rows. The three values are added to every row.

2 **Question 2.** What does this print?

```
q2a = np.array([1, 2, 3])
q2b = np.array([[10], [20]])
print((q2a + q2b).shape)
```

Answer: C) `(2, 3)`. Shapes (3,) and (2, 1): from the right, 3 meets 1 and gives 3, then the missing axis meets 2 and gives 2. A row meeting a column makes a grid.

3 **Question 3.** `t` has shape `(7, 4)`. Which expression divides every row by that row's total?
Answer: A) `t / t.sum(axis=1, keepdims=True)`. axis=1 gives one total per row, and keepdims=True keeps them as a column of shape (7, 1), which is reused across the 4 columns.

4 **Question 4.** `actual` has shape `(100,)` and `predicted` has shape `(100, 1)`. What does `actual - predicted` give?
Answer: A) An array of shape `(100, 100)`, with no error. From the right, 100 meets 1, then a missing axis

meets 100. Both are legal, so NumPy builds every actual minus every prediction: 10,000 numbers. Flatten one side first.

5 **Question 5.** Which pair of shapes can not be broadcast together?
Answer: C) `(3, 4)` **and** `(3,)`. Lined up from the right, 4 meets 3. They are not equal and neither is 1, so it fails. The 3 would have to be a column of shape (3, 1).

WARM-UP ANSWERS

W1

Name the three rules that are true of every NumPy array and not of a Python list.

One type for the whole array (the dtype), a fixed size, and operations that act on every element at once.

W2

What does `[1, 2, 3] * 2` give? What does `np.array([1, 2, 3]) * 2` give?

`[1, 2, 3, 1, 2, 3]` (a list repeats) and `[2 4 6]` (an array does arithmetic).

W3

`b = a[1:4]`, then `b[0] = 0`. What happened to `a`, and how do you prevent it?

`a[1]` is now 0, because a slice is a view. `b = a[1:4].copy()` makes an independent array.

W4

The mean of the revenue array printed `nan`. Why, and what did you use instead?

Six values are missing and arithmetic with a missing value gives `nan`. `np.nanmean(revenue)` skips them.

W5

`t` has shape `(7, 4)`: days by stores. What does `t[:, 2]` select, and what is its shape?

Every row of column 2: one store's value on each of the seven days. Shape `(7,)`.

EXIT TICKET

E1

`revenue` and `promo` are parallel arrays. Write one line that counts the sale days (`promo` equal to 1) with revenue above 10,000.

`((revenue > 10000) & (promo == 1)).sum()` : `&`, parentheses around each comparison, and the sum of the mask counts.

E2

A table has shape `(224, 4)` : days by stores. What is the shape of `table.mean(axis=0)`, and what does each value mean?

`(4,)` : one mean per store. The axis you name (the 224 days) is the one that disappears.

E3

You want every row of that table divided by its own row total. Write the expression, and say what goes wrong if you leave out the keyword.

`table / table.sum(axis=1, keepdims=True)`. Without `keepdims=True` the totals have shape `(224,)` ; from the right, 4 meets 224 and NumPy raises "could not be broadcast together".

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

WHAT YOU SEE OR HEAR	WHAT IT REVEALS	WHAT TO DO
Q1 answered B or D; "the sum of the mask" and "the sum of the values" used interchangeably	The mask and the filtered values are not yet separate ideas	Print `m`, `m.sum()`, `v[m]`, `v[m].sum()` on four lines and have the learner label each in words.
Q2 answered A or B; the two errors of 1.2 demo Step 10 in the lab	Python's `and` habit; operator precedence is invisible until it bites	"One pair of parentheses per question, `&` between them." Have them say the two questions aloud first.
Q3 written as `[5 7 9] (2,)`; 56 values where 4 were wanted in Lab B	`axis=0` read as "along the rows", meaning "for each row"	The squash gesture on a drawn table. Then always the same two questions: how many answers do I expect; which axis must disappear?

WHAT YOU SEE OR HEAR	WHAT IT REVEALS	WHAT TO DO
Q4 answered A; P3 predicted `1 1 1`; counts that do not add up to 2,903	Missing values are assumed to behave like small numbers or zeros	`np.nan > 0`, `np.nan <= 0`, `np.nan == np.nan` typed one at a time. Then `np.isnan`.
Q6 answered A	`argmax` read as `max`	"arg" means "which position". Have them use the position on a second array straight away.
Q8 answered A; exit ticket 3 without `keepdims`	Rule 1 (line up from the RIGHT) is not yet automatic; rows are assumed to match rows	Paper: write `(30, 4)` over `(30,)` right-aligned. Which numbers are under each other?
Q12 answered B or C; Lab B Step 4 left at 1753.02	A belief that NumPy would complain if shapes "did not match"	`np.broadcast_shapes((12,), (12, 1))`. Then the habit: n in, n out, print the shape.
Q10 answered B	A table and a row is thought to need matching FIRST numbers	Right-align on paper again; the lesson's "why from the right" answer in Common Questions helps.
Lab 1.3-A: shares divided by `table.sum(axis=0)` and accepted	No check was made on the result	"Shares of what? What should they add up to, and along which axis?" Make the check line part of the answer.

INSTRUCTOR KIT · ANSWER KEY

Session 3: Randomness and Linear Algebra

Answers with reasoning for everything in the Session 3 workbook: the twelve self-check items (with why each wrong option is wrong), the predict-then-run items, reference notes for every lab and handout, the lesson pages' own Quick Quiz answers, the warm-up and exit ticket, and what the common wrong answers tell you about a learner.

Session 3 | 12 self-check items | Confidential

NUMBERING MATCHES THE WORKBOOK

The self-check items (Q1 to Q12), the predict items (P1 to P3 per lesson), the warm-up (W1 to W5) and the exit ticket (E1 to E3) use the same numbers, wording and option letters as the participant workbook and the facilitator guide: all three documents are generated from one source. Every output printed here came from running the code on the course data with Python 3.12 and NumPy 2.5. Random results are from the seeds shown, drawn in the order shown.

1.4 · RANDOM NUMBER GENERATION: SELF-CHECK

Q1

What does this print?

```
a = np.random.default_rng(5)
b = np.random.default_rng(5)
x = a.integers(0, 100, size=3)
y = a.integers(0, 100, size=3)
z = b.integers(0, 100, size=3)
print(np.array_equal(x, y), np.array_equal(x, z))
```

A) `True True`

B) `False True`

C) `True False`

D) `False False`

Why: A generator moves forward with every draw, so the second draw from `a` is the next three numbers, not the same three. `b` has the same seed and has not been used yet, so its first draw repeats `a`'s first draw.

A is wrong: The seed fixes where the sequence starts, not what every later draw returns.

C is wrong: It is the other way round: the same generator gives new numbers, and a fresh generator with the same seed repeats them.

D is wrong: Two generators with the same seed always produce the same sequence.

Q2

Which line rolls ten ordinary six-sided dice (values 1 to 6)?

A) `rng.integers(1, 6, size=10)`

B) `rng.random(6, size=10)`

C) `rng.choice(6, size=10)`

D) `rng.integers(1, 7, size=10)`

Why: `rng.integers(low, high)` leaves out the upper end, like `range` and slicing, so 1 to 6 needs a `high` of 7 (or `endpoint=True`).

A is wrong: That draws 1 to 5: a die with five faces, and no error to warn you.

B is wrong: `rng.random` draws floats from 0 up to 1 and takes only a size; this call raises a `TypeError`.

C is wrong: `rng.choice(6, ...)` draws from 0 to 5.

Q3

What does this print?

```
d = np.arange(5)
r = np.random.default_rng(0).shuffle(d)
print(r, np.sort(d))
```

`None [0 1 2 3 4]`

> **Why:** `shuffle` rearranges the array in place and returns `None`, so `r` is `None`. The array `d` still holds the same five values in a new order; sorted, they are 0 to 4 again.

Q4

`orders` has `n` rows. Which line chooses 50 different rows to audit?

A) `rng.choice(n, size=50, replace=False)`

B) `rng.choice(n, size=50)`

C) `rng.integers(0, 50, size=n)`

D) `rng.permutation(50)`

> **Why:** With a whole number `n`, `choice` draws row positions from 0 to n - 1, and `replace=False` guarantees no position comes up twice.
>
> **B** is wrong: The default is WITH replacement, so the same row can be drawn twice and you may audit fewer than 50 different orders.
>
> **C** is wrong: That draws n numbers between 0 and 49: the wrong count and the wrong range.
>
> **D** is wrong: That is the numbers 0 to 49 in random order: always the first fifty rows of the file.

Q5

A simulation of 1,000,000 days says a store runs out of gift bags on 4.5% of days, and the estimate barely moves from run to run. The model assumed exactly 485 visitors every day. What is the best reading?

A) The answer is correct, because a million simulations cannot be wrong

B) The answer is useless, because the numbers were random

C) The estimate is precise, but it is only as true as the model; a better model of visitors could change it a lot

D) Run ten million simulations to remove the effect of the assumption

Why: More simulations make an estimate more precise. Only a better model makes it more true. In the lesson, drawing each day's visitors from the real counts moved the answer from about 4.5% to about 28%.
A is wrong: Stability tells you about the luck of the draw, not about whether the model describes the store.
B is wrong: Random draws from a stated model give a perfectly usable estimate of that model's answer.
D is wrong: The assumption is in the model, not in the sample size. Ten million runs would give the same wrong 4.5%, to more decimal places.

Q6

What does this print?

```
s = np.random.default_rng(3).integers(1, 4, size=(1000, 2))
print(s.shape, s.min(), s.max())
```

`(1000, 2) 1 3`

Why: `size` can be a shape, so this is a table of 1,000 rows and 2 columns. The upper end is left out, so the values are 1, 2 and 3; with two thousand draws both ends are certain to appear.

1.4 · RANDOM NUMBER GENERATION: PREDICT, THEN RUN

P1

What does this print?

```
r = np.random.default_rng(9)
first = r.random(3)
r = np.random.default_rng(9)
second = r.random(3)
third = r.random(3)
print(np.array_equal(first, second), np.array_equal(second, third))
```

> `True False`

> **Why:** Re-creating the generator with the same seed goes back to the first page, so `second` repeats `first`. Drawing again without re-creating it moves the bookmark on, so `third` is new.

P2

What does this print?

```
r = np.random.default_rng(1)
picks = r.choice(4, size=4, replace=False)
print(np.sort(picks), picks.sum())
```

> `[0 1 2 3] 6`

> **Why:** Four different values drawn from 0, 1, 2, 3 must be all four of them, in some order. Sorted they are `[0 1 2 3]` and they always add up to 6, whatever the seed.

P3

This line fails. Predict the kind of error and what the message says.

```
np.random.default_rng(2).choice(np.array(["A", "B", "C"]), size=5, replace=False)
```

> `ValueError: Cannot take a larger sample than population when replace is False`

> **Why:** There are only three values, and you cannot draw five different ones. Ask for at most three, or allow repeats by leaving `replace` at its default.

1.5 · LINEAR ALGEBRA ESSENTIALS: SELF-CHECK

Q7

What does this print?

```
u = np.array([2, 0, 1])
v = np.array([10, 5, 3])
print(u @ v, u * v)
```

A) `23 [20 0 3]`

B) `[20 0 3] 23`

C) `23 23`

D) A `ValueError`, because `@` needs 2-D arrays

Why: `@` multiplies matching positions and adds them up: 20 + 0 + 3. `*` multiplies position by position and keeps the shape.
B is wrong: The two results are the right ones in the wrong order: `@` gives the single number.
C is wrong: `*` never adds anything up; you would need `.sum()`.
D is wrong: Two 1-D arrays of the same length are exactly what a dot product takes.

Q8

`X` has shape `(100, 3)` (100 days, 3 features) and `w` has shape `(3,)`. What is the shape of `X @ w`?

A) `(100, 3)`

B) `(100,)`

C) `(3,)`

D) It fails, because the shapes are different

Why: A matrix times a vector takes the dot product of every row with the vector: one number per row, so 100 predictions.
A is wrong: That is the shape of `X * w`, which broadcasts and keeps the table.
C is wrong: The 3 is the inner number that has to match; it is the one that disappears.
D is wrong: The row length (3) matches the vector's length (3), which is all `@` needs.

Q9

What does this print?

```
A = np.array([[1, 2],
              [3, 4]])
print(A @ np.array([1, 1]), A.T[0])
```

`[3 7] [1 3]`

Why: Each row dotted with `[1, 1]` is that row's sum: 3 and 7. `.T` swaps rows and columns, so the first row of the transpose is the first column of `A`: 1, 3.

Q10

A least-squares fit with the columns (ones, visitors, sale day) returns the coefficients `[189.11, 20.34, 2502.17]`. What does the model predict for a sale day with 400 visitors?

```
print((np.array([1, 400, 1]) @ np.array([189.11, 20.34, 2502.17])).round(2))
```

A) `8325.11`

B) `8136.0`

C) `10827.28`

D) `2711.62`

Why: A prediction is the row of feature values dotted with the coefficients: 1 × 189.11 + 400 × 20.34 + 1 × 2502.17.

A is wrong: That leaves out the sale-day coefficient: it is the prediction for an ordinary day with 400 visitors.

B is wrong: That is 400 × 20.34 alone: no intercept and no sale-day term.

D is wrong: That adds the three coefficients without multiplying by the feature values.

Q11

What does this print?

```
print(np.linalg.norm(np.array([6.0, 8.0])),
      np.linalg.norm(np.array([1.0, 1.0]) - np.array([4.0, 5.0])))
```

`10.0 5.0`

Why: A norm is a length: the square root of 36 + 64 is 10. The distance between two vectors is the norm of their difference, `[-3, -4]`, which is 5.

Q12

You look for the product most like a given one by distance over two columns: price (in the hundreds) and weight in kilograms (single digits). The "nearest" product has almost the same price and twice the weight. What is the fix?

A) Use `*` in place of `@` when computing the distance

B) Add a column of ones to the table

C) Sort the table by price first

D) Standardize each column (subtract its mean, divide by its standard deviation) before measuring distance

> **Why:** A distance adds up differences across columns, so the column with the biggest numbers dominates. Putting every column on one scale lets each feature pull equal weight.
> **A** is wrong: The operator is not the problem; the scales of the columns are.
> **B** is wrong: A column of ones gives a least-squares model its intercept. It does nothing for a distance.
> **C** is wrong: Sorting changes the order of the rows, not the distances between them.

1.5 · LINEAR ALGEBRA ESSENTIALS: PREDICT, THEN RUN

P1

What does this print?

```
M = np.array([[1, 0, 2],
              [0, 3, 1]])
w = np.array([10, 1, 5])
print(M @ w, (M * w).shape)
```

```
[20  8] (2, 3)
```

> **Why:** `@` takes the dot product of each row with `w`: 10 + 0 + 10 and 0 + 3 + 5. `*` broadcasts the row of three across both rows and keeps the `(2, 3)` shape.

P2

What does this print?

```
a = np.array([[2.0, 0.0],
              [0.0, 4.0]])
print(np.linalg.solve(a, np.array([6.0, 8.0])))
```

```
[3. 2.]
```

Why: The two equations are 2 × first = 6 and 4 × second = 8, so the unknowns are 3 and 2. Check: `a @ [3, 2]` gives `[6, 8]`.

P3

This line fails. Say why, using the shape rule, before you run it.

```
np.ones((2, 3)) @ np.ones((2, 3))
```

```
ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0, with gu
```

Why: `(2, 3) @ (2, 3)`: the inner numbers are 3 and 2, and they must match. The end of the message says so. Transposing the second matrix, `(2, 3) @ (3, 2)`, would work and give `(2, 2)`.

LAB REFERENCE SOLUTIONS

Every lab has a runnable `_solution.py` next to its starter in `labs/module-03/`. The notes give what a correct run prints, the lines that matter and why, and the wrong results you are most likely to see. Check all eight files with `python3 labs/check_labs.py module-03`.

LAB 1.4-A: LABS/MODULE-03/L1_4-RANDOM-GENERATION/MYSTERY_SHOPPER_STARTER.PY → MYSTERY_SHOPPER_SOLUTION.PY

Reference output (seed 7): `361 Flagstaff rows in 2025`; `pilot stores: ['Boise' 'Flagstaff']`; shopper dates 2025-01-02, 01-20, 03-23, 04-14, 04-19, 07-02, 07-24, 10-03, 10-24, 11-11, 11-13, 11-26; `weekend days among them: 2 of 12`; `same schedule from the same seed: True`; `group sizes: 2032 871`; `mean visitors: 356.8 and 353.8 | all rows: 355.9`.

The solution's key lines:

```
rng = np.random.default_rng(7)
pilot = rng.choice(store_names, size=2, replace=False)
picks = rng.choice(flagstaff_2025, size=12, replace=False)     # row positions, no repe
print(np.sort(dates[picks]), (weekday[picks] >= 5).sum())
order = rng.permutation(visitors.size)
group_a, group_b = order[:cut], order[cut:]
```

Why: `flagstaff_2025` holds row positions (the starter builds it with `np.flatnonzero`, which returns the positions where a mask is True), so one draw of positions can index `dates`, `weekday` and anything else. Reproducing the schedule needs a new generator with the same seed AND the same draws in the same order: the pilot draw has to be repeated before the dates, or the bookmark is in a different place.

Common wrong results: different dates from a neighbor's (an extra or repeated draw; a different seed); `False` in Step 4 (only the second draw was repeated); 3 or more weekend days "expected" (2 of 12 is well within what chance gives when 2 days in 7 are weekend days); group means reported as proof that the split "worked" or "failed" (similar and not identical is exactly what a random split looks like).

LAB 1.4-B: LABS/MODULE-03/L1_4-RANDOM-GENERATION/RETURNS_DESK_STARTER.PY → RETURNS_DESK_SOLUTION.PY

Reference output (seed 14): `731 days | 1143 returned orders | first ten days: [1 1 1 2 3 1 0 1 0 4]`; `average returns per day: 1.564`; `4 or more returns: simulated 7.4% | real 7.3%`; `no returns at all: simulated 21.1% | real 20.7%`; `mean and variance of the real counts: 1.56 1.52`; `40 or more orders: simulated 1.7% | real 7.3%`; `mean and variance of real orders per day: 27.77 64.35`.

The solution's key lines:

```
rate = returns_per_day.mean()
sim = rng.poisson(lam=rate, size=100_000)              # model + repeat, in one call
print((sim >= 4).mean(), (returns_per_day >= 4).mean())    # count, and check against r
sim_orders = rng.poisson(lam=orders_per_day.mean(), size=100_000)
print((sim_orders >= 40).mean(), (orders_per_day >= 40).mean())
```

Why: a Poisson describes counts of events that happen at a steady average rate. Returned orders per day behave that way (the real mean and variance are nearly equal, as a Poisson's are), so the simulation agrees with the real days to within a few tenths of a point. Orders per day do not: weekends, seasons and sale days make busy days far more common than a single rate allows, and the real variance is more than twice the mean. The same code gives a trustworthy answer to one question and a wrong one to the other, and only the comparison with real data tells you which.

Notes for the debrief: "returns per day" here means how many of the orders PLACED on a day were later returned, because the data records an order's date and its final status. Small differences in the last digit are possible if a learner draws in a different order; the conclusions do not change. Common wrong results: a loop of 100,000 calls to

`rng.poisson(rate)` (correct but slow: ask for `size=`); `(sim > 4)` instead of `>= 4` (2.2% instead of 7.4%); concluding the model is "bad" from Step 6 alone, when it was a good model for the question it was built for.

HANDOUT: LABS/MODULE-03/L1_4-RANDOM-GENERATION/DEBUG_RANDOM.PY (DELIBERATELY BROKEN)

Block 1: `integers(1, 6)` draws 1 to 5; the check prints a largest roll of 5. Use `integers(1, 7)`. **Block 2:** `deck = rng.shuffle(deck)` replaces the deck with `None`; call `rng.shuffle(deck)` on its own line or use `rng.permutation(deck)`. **Block 3:** the generator is created inside the loop with the same seed, so all three weeks are `[226 432 396 331 329 457 225]` and the check prints 1 different week; create it once above the loop. **Block 4:** `ValueError: Probabilities do not sum to 1`: the four values add up to 0.9; `[0.2, 0.2, 0.4, 0.2]` makes Boise twice as likely as each of the others.

LAB 1.5-A: LABS/MODULE-03/L1_5-LINEAR-ALGEBRA/PAYROLL_RECEIPT_STARTER.PY → PAYROLL_RECEIPT_SOLUTION.PY

Reference output: the week-1 staff table (first row `[4. 5. 6. 3.]`, Saturday `[ 8. 7. 10. 5.]`); `234 rated products | 17050 reviews`; plain average rating 4.138, weighted 4.146; `payroll per day: [3172. 2816. 2820. 2996. 4036. 5276. 4396.] (7,)`; shape `(7, 2)`; `week total, current and proposed: [25512. 27252.]`; first column equals Part 2: True; `payroll per store: [5880. 5984. 8832. 4816.] | total 25512.0`; `staff-days per store: [35. 34. 48. 28.]`; `hourly rates (lead, seller, trainee): [22. 19. 17.]`; check True.

The solution's key lines:

```
weighted = rating @ reviews / reviews.sum()     # (234,) @ (234,) -> one number
payroll_per_day = week @ day_cost               # (7, 4) @ (4,)   -> (7,)
both = week @ pay_scales                        # (7, 4) @ (4, 2) -> (7, 2)
per_store = (week * day_cost).sum(axis=0)       # * keeps (7, 4); then collapse the day
rates = np.linalg.solve(hours, payroll)         # (3, 3) and (3,) -> (3,)
```

Why: each part is the receipt from a different side. Part 4 is deliberately a `*`: "per store" needs each store's column weighted by its own cost and then the days collapsed, which is a broadcast and an `axis=0` sum, not a matrix product; `day_cost @ week` fails because the inner numbers are 4 and 7. The check in Part 5 puts the answer back into the original equation. The daily costs, pay scales and the pop-up stall's hours are made-up planning figures; the staff counts and ratings are the course data.

Common wrong results: a weighted "average" in the tens of thousands (not divided by `reviews.sum()`); a `(7, 4)` table for Part 2 (`*`); `[25512. 27252.]` reported as "per day" (that is the week total, the `(7, 2)` table summed over the days); rates that fail the check because `payroll` and `hours` were swapped in `solve`.

LAB 1.5-B: LABS/MODULE-03/L1_5-LINEAR-ALGEBRA/BASKET_MODEL_STARTER.PY → BASKET_MODEL_SOLUTION.PY

Reference output: `724 days | mean transactions 77.8 | mean revenue 6889.75`; `A1 shape: (724, 2)`; `revenue = 117.23 + 87.10 * transactions`; `transactions only: RMSE $631.35, R squared 0.945`; `average dollars per purchase (revenue / transactions): 88.77`; intercept 89.82, per

purchase 87.52, sale day -172.09; `with sale days: RMSE $630.73, R squared 0.945`; quiet day $5,341.19, busy ordinary day $8,842.11, sale day $13,046.16, each give or take about $631.

The solution's key lines:

```
A1 = np.column_stack([np.ones(x.size), x])
coef1, *_ = np.linalg.lstsq(A1, y)
print(np.mean(y / x).round(2))                    # a second route to the slope
A2 = np.column_stack([np.ones(x.size), x, is_promo])
coef2, *_ = np.linalg.lstsq(A2, y)
forecast = scenarios @ coef2
```

Why: the slope ($87.10 per additional purchase) and the directly computed basket value ($88.77) are two routes to nearly the same number; they differ a little because the fitted line also has an intercept. The fit is tight (R squared 0.945) because revenue is, almost by definition, purchases times what a purchase is worth. The sale-day coefficient is small and negative and the RMSE barely moves: once you know how many purchases there were, knowing it was a sale day adds nothing. That is an honest "this column did not earn its place", and it agrees with Session 2's finding that sale days bring more buyers, not bigger baskets.

Common wrong results: `[nan nan]` (the mask was dropped and the missing Flagstaff revenue got in); a single coefficient near 88 (no column of ones); the sale-day coefficient read as "sales lose $172" (it is within the noise: compare the two RMSE figures); forecasts computed one at a time by hand instead of with one `@` (not wrong, but ask for the matrix).

HANDOUT: LABS/MODULE-03/L1_5-LINEAR-ALGEBRA/DEBUG_LINALG.PY (DELIBERATELY BROKEN)

Block 1: `orders * prices` gives a `(3, 3)` table of line totals; `orders @ prices` gives `[756.35 348.95 356.3 ]`. **Block 2:** `price_lists @ units_sold` is `(3, 2) @ (3,)` and raises the matmul error ("size 3 is different from 2"); `price_lists.T @ units_sold` gives `[1461.6 1242.41]`. **Block 3:** one Boise day has no revenue, so the fit returns `[nan nan]`; filter with `boise & ~np.isnan(revenue)` and the slope is 21.39. **Block 4:** the matrix has no column of ones, so there is one coefficient, 20.95, and no intercept; `np.column_stack([np.ones(keep.sum()), visitors[keep]])` gives about -238.85 and 21.39.

THE LESSON PAGES' OWN QUICK QUIZZES

Each lesson page ends with a five-question Quick Quiz (three options each). Learners do them at home; these are the answers with the page's own explanation.

LESSON 1.4 · RANDOM NUMBER GENERATION

1 **Question 1.** What does this print?

```
r1 = np.random.default_rng(7)
r2 = np.random.default_rng(7)
print(np.array_equal(r1.integers(0, 100, size=3), r2.integers(0, 100, size=3)))
```

Answer: C) `True`. Two generators created with the same seed start at the same point of the same sequence, so the same request returns the same numbers. That is what makes random results reproducible.

2 **Question 2.** Which values can `rng.integers(1, 7, size=5)` produce?
Answer: B) 1, 2, 3, 4, 5 or 6. The lower end is included and the upper end is left out, the same convention as range(1, 7). This is the correct way to roll a six-sided die.

3 **Question 3.** `skus` is an array of 242 product codes. Which line picks 5 different products at random?
Answer: A) `rng.choice(skus, size=5, replace=False)`. replace=False means a drawn item is not put back, so the five results are guaranteed to be distinct.

4 **Question 4.** You create `rng = np.random.default_rng(42)` in one notebook cell. In the next cell you run `print(rng.random())` three times and see three different numbers. Why?
Answer: B) The generator moves forward through its sequence with every draw; the seed only fixes where it starts. Each draw advances the generator, so the next request returns the next numbers. Re-creating the generator with the same seed starts the sequence again.

5 **Question 5.** A simulation of 100 days estimates a probability as 0.07. The same simulation with 1,000,000 days gives 0.0449. What is the best reading?
Answer: A) Trust 0.0449 more: estimates from more simulations vary less. This is the law of large numbers. With 100 days each single day moves the estimate by a whole percentage point, while a million days pin it down to about three decimal places.

LESSON 1.5 · LINEAR ALGEBRA ESSENTIALS

1 **Question 1.** What does this print?

```
q1u = np.array([1, 2, 3])
q1v = np.array([4, 5, 6])
print(q1u @ q1v)
```

Answer: B) `32`. The dot product multiplies matching positions and adds the results: 1 times 4, plus 2 times 5, plus 3 times 6. Two vectors in, one number out.

2 **Question 2.** `X` has shape `(5, 3)` and `W` has shape `(3, 2)`. What is the shape of `X @ W`?
Answer: A) `(5, 2)`. The inner numbers, 3 and 3, match, so the multiplication works, and the result keeps the outer numbers: the rows of the left matrix and the columns of the right one.

3 **Question 3.** Why is `np.linalg.solve(A, b)` preferred to `np.linalg.inv(A) @ b`?
Answer: C) Because it is faster and loses less accuracy, since it never builds the inverse. solve goes straight to the answer. Computing the full inverse first is extra work, and each extra step adds rounding error.

4 **Question 4.** A least-squares fit uses the matrix `np.column_stack([np.ones(n), x])`. What is the column of ones for?
Answer: A) It gives the model an intercept, so the line is not forced through zero. The coefficient that

multiplies the ones is added to every prediction, which is exactly what an intercept does. Without it the line must pass through the origin.

5 **Question 5.** What does this print?

```
q5 = np.array([3.0, 4.0])
print(np.linalg.norm(q5))
```

Answer: B) `5.0`. The norm is the square root of the sum of squares: 9 plus 16 is 25, and the square root of 25 is 5. It is the length of the vector, by the Pythagorean theorem.

WARM-UP ANSWERS

W1

`big = revenue > 15000`. What does `big.sum()` tell you? And `big.mean()`?

How many store-days are above $15,000, and what fraction of all store-days that is.

W2

Write a mask for "Boise and a sale day", given the parallel arrays `stores` and `promo`.

`(stores == "Boise") & (promo == 1)`: `&`, with parentheses around each comparison.

W3

`table` has shape `(224, 4)`: days by stores. What is the shape of `table.sum(axis=1)` and what does it hold?

`(224,)`: one total per day. The axis you name (the 4 stores) disappears.

W4

State the two broadcasting rules.

Line the shapes up from the right, a missing entry counting as 1. Each pair of lengths must be equal, or one of them 1; the result takes the larger.

W5

How do you divide every row of that table by its own row total?

`table / table.sum(axis=1, keepdims=True)`: the totals must stand as a column of shape `(224, 1)`.

EXIT TICKET

E1

You run a cell that draws from `rng` twice and get different numbers. Is the code broken? What makes a random result reproducible?

No. The generator moves forward with every draw. Reproducible means the same code, run from the top (from where the seeded generator is created), gives the same result.

E2

`orders` has shape `(3, 3)` (orders by products) and `prices` has shape `(3,)`. What is the difference between `orders * prices` and `orders @ prices`?

`*` multiplies position by position and keeps the shape: a `(3, 3)` table of line totals. `@` multiplies and adds: three order totals, shape `(3,)`.

E3

A fit of revenue on visitors gives a slope of 21.39. Write the sentence a manager should hear, and one caution.

Each additional visitor goes with about $21 more revenue. It describes a pattern across days: it is not a promise about one more visitor, and a forecast should be quoted with its typical miss (the RMSE).

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

WHAT YOU SEE OR HEAR	WHAT IT REVEALS	WHAT TO DO
Q1 answered A; "my numbers changed" in Lab A	The seed is thought to pin every draw, not just the starting point	The two-cell experiment: generator and draw in one cell (same every time), then split (different every time).
Q2 answered A; a largest roll of 5 in the handout	`integers` assumed to include the upper end, unlike	"Like `range` and slicing." Then the habit: check `.min()` and `.max()` of a big batch.

WHAT YOU SEE OR HEAR	WHAT IT REVEALS	WHAT TO DO
	`range`	
Q3 written as a shuffled array; `deck = rng.shuffle(deck)`	Every function is expected to return its result	Print the return value. Pair `permutation` (returns a copy) with `shuffle` (in place, returns `None`) as a contrast to remember.
Q4 answered B	`replace=False` seen as optional decoration	Draw 50 from 60 with replacement and count the distinct values with `np.unique(...).size`.
Q5 answered A or D; "run more simulations" as the fix for a model	Precision and truth are one idea in the learner's head	The gift-bag pair again: same 100,000 runs, 4% and 28%. "Which of these would more runs change?"
Q7 answered C; a `(7, 4)` table where `(7,)` was wanted in Lab A	`*` and `@` both read as "multiply"	"Position by position, or multiply and add?" Then print both shapes side by side.
Q8 answered A or D; trial and error with `.T` until something runs	The shape rule is not yet a tool the learner applies before running	Write `(n, k) @ (k, m)` and circle the inner pair. No keyboard until the result shape is written down.
Q10 answered B or D	A prediction is not yet seen as a row of feature values times the coefficients	Have them build the scenario row `[1, 400, 1]` and name what each entry multiplies.
Exit ticket 3: "one more visitor earns $21.39"; Exercise 4 skipped	A coefficient is read as a cause	The staffing trap, together: fit staff alone, ask for the recommendation, then add visitors.
Q12 answered B	Tools from different jobs are blurred: the column of ones belongs to least squares	Ask what each tool is FOR: ones give an intercept; standardizing gives a fair distance.

INSTRUCTOR KIT · ANSWER KEY

Session 4: DataFrames and Loading Data

Answers with reasoning for the 12 self-check items and the predict-then-run boxes, the real output of every lab solution with notes on what to check while you circulate, the answers to the two lesson pages' own Quick Quizzes, and the warm-up and exit-ticket answers.

Session 4 | 12 self-check items | 5 lab pairs | Confidential

HOW THIS KEY IS ORGANIZED

Self-check numbering matches the workbook: Q1 to Q6 are on Lesson 2.1, Q7 to Q12 on Lesson 2.2. Each item repeats the four options with the correct one marked and says why each wrong option is wrong. Every 'what does this print' answer and every lab output below was produced by running the code against the course data. The lesson pages' own five-question Quick Quizzes (three options each) are answered at the end of each lesson's part.

2.1 · DATAFRAMES AND SERIES: SELF-CHECK

Q1

What does this print?

```
s = pd.Series([5, 6, 7], index=["x", "y", "z"])
t = pd.Series([1, 1], index=["z", "x"])
print((s + t).tolist())
```

A) `[6, 7, 7]`

B) `[6.0, 7.0, nan]`

C) `[6.0, nan, 8.0]`

D) A `ValueError`, because the lengths differ

Why: Series arithmetic aligns on labels: `x` gets 5 + 1, `z` gets 7 + 1, and `y` has no partner, so it is NaN. The NaN makes the whole result float.

A is wrong: That adds by position and treats the missing partner as nothing; pandas matches labels.

B is wrong: That adds by position (5 + 1, 6 + 1) and puts the gap last. The gap belongs to the label `y`, in the middle.

D is wrong: Two NumPy arrays of lengths 3 and 2 would raise an error. Two Series align and fill with NaN.

Q2

What does this print?

```
df = pd.DataFrame({"n": [10, 20, 30, 40]}, index=["a", "b", "c", "d"])
print(len(df.loc["a":"c"]), len(df.iloc[0:2]), df.iloc[-1, 0])
```

`3 2 40`

Why: A `.loc` slice includes its end label, so `a`, `b`, `c` are three rows. An `.iloc` slice leaves out the stop, so positions 0 and 1 are two rows. `.iloc[-1, 0]` is the last row, first column: 40.

Q3

In a table of 242 products, `products.info()` shows the line `rating 234 non-null float64`. What does that line tell you?

A) 8 products have no rating

B) There are 234 different ratings

C) The ratings are stored as text

D) 8 products have a rating of 0

Why: Non-Null Count is the number of values that are present. 242 rows minus 234 present leaves 8 missing ratings.
B is wrong: The number of distinct values comes from `.nunique()`, not from `info()`.
C is wrong: The dtype on that line is `float64`: numbers. Text would say `str`.
D is wrong: A rating of 0 is a value and would be counted as non-null. Missing is not zero.

Q4

Which line correctly counts the products that are in the Tents category or cost less than $20?

A) `(products["category"] == "Tents" | products["price"] < 20).sum()`

B) `(products["category"] == "Tents" or products["price"] < 20).sum()`

C) `products[("category" == "Tents") | ("price" < 20)].sum()`

D) `((products["category"] == "Tents") | (products["price"] < 20)).sum()`

> **Why:** Each condition sits in its own parentheses, `|` combines them element by element, and `.sum()` of the mask counts the True rows.
> **A** is wrong: Without parentheses `|` is evaluated before `==` and `<`, so Python tries `"Tents" | products["price"]` first and raises an error.
> **B** is wrong: `or` wants a single True or False on each side; a Series is 242 of them (ValueError: ambiguous truth value).
> **C** is wrong: It compares the text `"category"` with `"Tents"`, not the column. The column must be written `products["category"]`.

Q5

What does the last line print? (The second line also produces a warning.)

```
df = pd.DataFrame({"price": [10, 50, 90]})
df[df["price"] > 40]["price"] = 0
print(df["price"].tolist())
```

`[10, 50, 90]`

> **Why:** Chained assignment. `df[df["price"] > 40]` creates a temporary table that behaves as a copy; the 0 is written into that and thrown away. `df` is unchanged and pandas warns with `ChainedAssignmentError`. The working form is `df.loc[df["price"] > 40, "price"] = 0`.

Q6

After `top = products.sort_values("price", ascending=False)`, what is `top.index[0]`?

A) 0, because sorting renumbers the rows

B) The original row label of the most expensive product

C) The highest price

D) 241, the last position

> **Why:** `sort_values` reorders the rows and their labels travel with them. The first row of `top` still carries the label it had in `products` (19 in the course data).
> **A** is wrong: Rows are renumbered only if you add `.reset_index(drop=True)`.
> **C** is wrong: The index holds labels, not values. The highest price is `top["price"].iloc[0]`.
> **D** is wrong: 241 is the label of the last row of `products`; it has nothing to do with the sort.

2.1 · PREDICT, THEN RUN

P1

Labels travel with their values.

```
s = pd.Series([3, 1, 2], index=["c", "a", "b"])
print(s.sort_values().index.tolist(), s.iloc[0], s.loc["a"])
```

```
['a', 'b', 'c'] 3 1
```

> **Why:** Sorting by value puts 1, 2, 3 in order, and their labels `a`, `b`, `c` come along. `s` itself is unchanged (`sort_values` returned a new Series), so `.iloc[0]` is still the first value typed, 3, and `.loc["a"]` is the value labeled `a`, 1.

P2

A piece of a table, changed.

```
df = pd.DataFrame({"x": [1, 2, 3, 4]})
part = df[df["x"] > 2]
part["x"] = 0
print(df["x"].sum(), part["x"].sum(), part.index.tolist())
```

```
10 0 [2, 3]
```

Why: `part` is an independent table from the moment it is created (copy-on-write), so setting its column to 0 leaves `df` at 1 + 2 + 3 + 4 = 10. `part` kept the row labels of the rows it came from: 2 and 3.

P3

Looking for missing values.

```
r = pd.Series([4.5, np.nan, 3.0])
print((r == np.nan).sum(), r.isna().sum(), r.mean())
```

```
0 1 3.75
```

Why: `== np.nan` is False everywhere because NaN is not equal to anything, so it counts 0. `.isna()` finds the one missing value. `.mean()` skips missing values: (4.5 + 3.0) / 2 = 3.75.

2.1 · LAB REFERENCE SOLUTIONS

`LABS/MODULE-04/L2_1-DATAFRAMES-SERIES/BUSIEST_DAYS_SOLUTION.PY`

Question: Which were the five biggest store-days by revenue, were they promo days, and how often does a store pass $15,000 in a day?

How the solution works: `sort_values("revenue", ascending=False)` with a list of four column names gives the top-five table (`nlargest(5, "revenue")` is equally good). Two masks, `big` and `promo`, are built once and combined three ways: `big.sum()`, `(big & promo).sum()` and `(big & ~promo).sum()`. `idxmax()` returns the row label, which goes straight into `.loc`.

```
shape: (2903, 8)
date                 str
store                str
visitors           int64
transactions       int64
units              int64
revenue          float64
staff_on_duty      int64
promo_day          int64
columns with missing values: {'revenue': 6}
            date  store   revenue  promo_day
971   2024-08-31  Boise  27677.10          1
1331  2024-11-30  Boise  27199.33          1
2781  2025-11-30  Boise  25457.62          1
2777  2025-11-29  Boise  25419.88          1
1335  2024-12-01  Boise  25160.28          1
days above $15,000: 101
  of those, promo days: 36
  of those, ordinary days: 65
share of all promo days that pass $15,000: 37.5 %
best day    : label 971, 2024-08-31, Boise, $27,677.10
quietest day: label 115, 2024-01-29, Flagstaff, $2,093.79
```

- All five biggest days are Boise on sale weekends (Labor Day weekend 2024, then the Black Friday weekends).
- 101 store-days pass $15,000: 36 promo days and 65 ordinary days. The two parts must add up to the first number; if they do not, a pair of parentheses is missing.
- Only 37.5% of promo store-days pass $15,000, so 'big day' and 'promo day' are not the same thing. A good discussion point.
- A count of 0 for 'ordinary days' usually means `~` was applied to the combined mask instead of to `promo`.
- The one column with missing values is `revenue` (6). Learners who skip TODO 1 will not notice that `sort_values` put those rows last.

- `LABS/MODULE-04/L2_1-DATAFRAMES-SERIES/RECOMMEND_GROUPS_SOLUTION.PY`

Question: Split the 0-to-10 'would you recommend us?' answers into promoters, passives and detractors, and compare Asheville shoppers with everyone.

How the solution works: The column starts as "no answer" and three `.loc` lines overwrite it. A comparison with NaN is False, so the 54 blanks are never overwritten. The score is the difference of two mask means among the rows that answered. The solution wraps that in a small function so it can be reused for the Asheville piece; learners may simply repeat the three lines.

```
(1800, 12)
group
passive      637
promoter     556
detractor    553
no answer     54
score, all shoppers: 0.2
Asheville responses: 202 | answered: 197
score, Asheville   : -14.7
rows of survey that say 'checked': 0
rows of asheville that say 'checked': 202
```

- Group counts: passive 637, promoter 556, detractor 553, no answer 54 (total 1,800).
- Score for all shoppers 0.2; for Asheville -14.7 (202 responses, 197 answered). Asheville is clearly worse: the same store that stands out later in the course for staff satisfaction.
- The denominator is the people who answered (1,746 overall, 197 in Asheville), not all rows. Dividing by all 202 Asheville rows gives -14.4 instead of -14.7. Ask: 'What is your denominator?'
- The last two lines must read 0 and 202: changing the Asheville piece never changes `survey`. No warning appears, because this is not chained assignment.
- If the counts are wrong, read the three masks aloud: `>= 9`, `.between(7, 8)` (both ends included) and `<= 6`. A common slip is `< 6`, which leaves the 241 sixes as 'no answer'.

`LABS/MODULE-04/L2_1-DATAFRAMES-SERIES/DEBUG_FILTERS.PY`

- Bug 1: `products["Price"]` raises `KeyError: 'Price'`. The column is `price` (names are case-sensitive). Prints 82.95.
- Bug 2: `and` between two masks raises the 'truth value of a Series is ambiguous' ValueError. Use `&` with parentheses. Prints 14.
- Bug 3: no parentheses around the conditions, so `&` is evaluated first and raises a TypeError. Wrap each condition. Prints 7.
- Bug 4 (silent): `== np.nan` is never True. Use `.isna().sum()`. Prints 8, not 0.
- Bug 5 (silent): chained assignment; a `ChainedAssignmentError` warning and 0 products on sale. Use `products.loc[products["stock_status"] == "low_stock", "on_sale"] = True`. Prints 29.

2.1 · THE LESSON PAGE'S QUICK QUIZ

These are the five questions at the end of the lesson page (three options each), with the lesson's own explanation.

2.1 Q1

What does this print?

```
qa = pd.Series([10, 20, 30], index=["a", "b", "c"])
qb = pd.Series([1, 2, 3], index=["b", "c", "d"])
print((qa + qb).tolist())
```

A) [11, 22, 33]

B) [nan, 21.0, 32.0, nan]

C) [10.0, 21.0, 32.0, 3.0]

> **Why:** Series arithmetic aligns on labels. Only b and c exist in both, giving 21 and 32. Labels a and d exist in one Series only, so the result there is NaN, and the NaNs make the whole result float.

2.1 Q2

Which expression returns a DataFrame rather than a Series?

A) products[["price"]]

B) products["price"]

C) products.iloc[0]

> **Why:** A list of column names inside the brackets always returns a DataFrame, even when the list holds a single name.

2.1 Q3

What does this print?

```
qdf = pd.DataFrame({"n": [1, 2, 3, 4, 5]}, index=[10, 20, 30, 40, 50])
print(len(qdf.loc[10:30]), len(qdf.iloc[0:2]))
```

A) 2 2

B) 3 3

C) 3 2

> **Why:** loc slices by label and includes both ends: labels 10, 20 and 30 make three rows. iloc slices by position and excludes the stop: positions 0 and 1 make two rows.

2.1 Q4

What does this print?

```
qorig = pd.DataFrame({"x": [1, 2, 3]})
qpart = qorig[qorig["x"] > 1]
qpart["x"] = 0
print(qorig["x"].tolist())
```

A) [1, 2, 3]

B) [1, 0, 0]

C) [0, 0, 0]

> **Why:** Under copy-on-write qpart behaves as an independent copy from the moment it is created, so writing to it changes qpart only. qorig keeps 1, 2, 3.

2.1 Q5

You want the products that cost more than $100 and are out of stock. Which line is correct?

A) products[products["price"] > 100 and products["stock_status"] == "out_of_stock"]

B) products[products["price"] > 100 & products["stock_status"] == "out_of_stock"]

C) products[(products["price"] > 100) & (products["stock_status"] == "out_of_stock")]

> **Why:** Each condition sits in its own parentheses and they are joined with the element-by-element and operator, so the result is one mask with a True or False per row.

2.2 · LOADING DATA: SELF-CHECK

Q7

What does this print?

```
f = io.StringIO("id,qty\n1,4\n2,\n3,6\n")
print(pd.read_csv(f, dtype={"qty": "Int64"})["qty"].tolist())
```

A) `[4.0, nan, 6.0]`

B) `[4, <NA>, 6]`

C) `[4, 0, 6]`

D) A `ValueError: Integer column has NA values`

Why: `"Int64"` with a capital I is the nullable integer: the whole numbers stay whole and the gap is shown as `<NA>`.

A is wrong: That is what you get with no `dtype`: the gap forces the column to float64.

C is wrong: pandas never invents a 0 for a missing value.

D is wrong: That error comes from lowercase `"int64"`, the NumPy type, which cannot hold a missing value.

Q8

A text export starts with two lines of notes, then the header `store;week;revenue`, then rows such as `Boise;2024-W01;55.400,22`. Which call loads `revenue` as a number?

A) `pd.read_csv(f, sep=";", skiprows=2, decimal=",", thousands=".")`

B) `pd.read_csv(f, sep=";", header=None)`

C) `pd.read_csv(f, skiprows=2, decimal=",")`

D) `pd.read_csv(f, sep=";", skiprows=3, decimal=",", thousands=".")`

Why: Four facts in the text, four options: semicolons, two note lines above the header, a decimal comma and a dot for thousands.

B is wrong: `header=None` says the file has no header at all, so the notes and the header row land in the data and every column is text.

C is wrong: Without `sep=";"` pandas splits on commas, so each line is cut at the decimal comma.

D is wrong: `skiprows=3` skips the header line too, so the first data row becomes the column names.

Q9

What does this print?

```
recs = [{"id": 1, "tags": [{"t": "a"}, {"t": "b"}]},
        {"id": 2, "tags": [{"t": "c"}]}]
print(pd.json_normalize(recs, record_path="tags", meta=["id"]).shape)
```

`(3, 2)`

> **Why:** `record_path="tags"` makes one row per item of each inner list: 2 + 1 = 3 rows. The columns are `t` (from the items) and `id` (copied from the parent by `meta`).

Q10

You load `orders.csv` with no options. `orders["order_date"].dt.month` raises `AttributeError: Can only use .dt accessor with datetimelike values`. What is the right fix?

A) Add `index_col="order_date"` to the load

B) Convert the column with `.astype("int64")`

C) Add `encoding="latin-1"` to the load

D) Load with `parse_dates=["order_date"]`, or convert with `pd.to_datetime()`

> **Why:** The column is `str`: pandas does not guess dates. `parse_dates` converts it while loading; `pd.to_datetime` converts it afterward. Then `.dt` works.
> **A** is wrong: That moves the column into the index but leaves it as text.
> **B** is wrong: A date such as `2024-01-01` is not a whole number; the conversion fails.
> **C** is wrong: The encoding is about characters, not types. The file already reads correctly.

Q11

What does this print?

```
df = pd.DataFrame({"a": [1, 2]})
df.to_csv("q11.csv")
print(pd.read_csv("q11.csv").columns.tolist())
```

`['Unnamed: 0', 'a']`

> **Why:** Without `index=False`, `to_csv` writes the index as an extra first column with no name. Read back, it becomes a column called `Unnamed: 0`.

Q12

A CSV file is larger than your computer's memory. You need the sum of one of its columns. Which approach is the right one?

A) Load the whole file, then delete the columns you do not need

B) Read the first 1,000 rows with `nrows` and multiply up

C) Read it with `usecols` and `chunksize`, keeping a running total

D) Open it in a spreadsheet program and save it as Excel first

> **Why:** `usecols` loads only the column you need and `chunksize` hands you one piece at a time, so only one chunk is ever in memory. Sums add up across chunks exactly.
> **A** is wrong: The load itself is what fails: the file does not fit.
> **B** is wrong: That is an estimate from the first rows, not the sum, and the first rows are rarely typical.
> **D** is wrong: A spreadsheet program holds the file in memory too, and has a row limit.

2.2 · PREDICT, THEN RUN

P1

A code that looks like a number.

```
text = "sku,zip\nA1,02134\nB2,97701\n"
a = pd.read_csv(io.StringIO(text))
b = pd.read_csv(io.StringIO(text), dtype={"zip": "str"})
print(a["zip"].tolist(), b["zip"].tolist())
```

```
[2134, 97701] ['02134', '97701']
```

> **Why:** Read as a number, `02134` loses its leading zero for good. `dtype={"zip": "str"}` keeps it as text. The test from the lesson: would it make sense to add two of them? If not, it is text.

P2

A key that is not there.

```
recs = [{"id": 1, "geo": {"city": "Bend"}}, {"id": 2}]
flat = pd.json_normalize(recs)
print(flat.columns.tolist(), flat["geo.city"].isna().sum())
```

```
['id', 'geo.city'] 1
```

Why: The inner dictionary becomes a dotted column, `geo.city`. The second record has no `geo` key, so its cell is NaN: one missing value. Nothing fails and no record is dropped.

P3

A title above the header.

```
text = "Sales report,draft\nday,sales\nMon,10\nTue,12\n"
a = pd.read_csv(io.StringIO(text))
b = pd.read_csv(io.StringIO(text), skiprows=1)
print(a.columns.tolist(), a.shape, b.columns.tolist(), b["sales"].sum())
```

```
['Sales report', 'draft'] (3, 2) ['day', 'sales'] 22
```

Why: Without `skiprows`, pandas takes the first line as the header, so the columns are called `Sales report` and `draft`, the real header `day,sales` lands in the data as a row (three rows instead of two), and no error is raised. Skipping the one title line puts `day,sales` in the header position, and `sales` is a number column that adds up to 22.

2.2 · LAB REFERENCE SOLUTIONS

`LABS/MODULE-04/L2_2-LOADING-DATA/TILL_EXPORT_SOLUTION.PY`

Question: Load the till system's one-week export (`till_export_week08.txt`) so every column has the right dtype, then prove the load is right against `daily_sales.csv`.

How the solution works: Each option answers one thing visible in the first six lines of the file: semicolons (`sep=";"`), three lines of notes (`skiprows=3`), amounts such as `3.536,91` (`decimal=","`, `thousands="."`), the word `n/a` (`na_values`), and dates such as `17.02.2025` (`parse_dates` with `date_format="%d.%m.%Y"`). The proof is a comparison with the same week of the clean file: revenue with `np.isclose`, visitors with `==`.

```
Pinecrest Outfitters - till export, all stores
Week 8 of 2025 (17.02.2025 to 23.02.2025), amounts in USD
Exported by the till system; n/a = register total not received
date;store;visitors;transactions;units;revenue
17.02.2025;Asheville;143;37;82;3.536,91
17.02.2025;Bend;214;48;108;4.970,90
no options: (31, 2) ['Pinecrest Outfitters - till export', ' all stores']
date            datetime64[us]
store                      str
visitors                 int64
transactions             int64
units                    int64
revenue                float64
shape: (28, 6)
        date store  visitors  transactions  units  revenue
9 2025-02-19  Bend       210            60    124      NaN
export revenue : 163755.71
clean revenue  : 163755.71
totals match: True
visitors match: True | 7649
```

- With no options the load gives shape `(31, 2)`: the title line became the header and the commas inside it split it in two. A good moment to say 'the code ran, and the table is nonsense'.
- Correct dtypes: `date` datetime64[us], `store` str, three int64 columns, `revenue` float64. Shape `(28, 6)`.
- One missing revenue: Bend on 2025-02-19 (a genuine gap in the course data).
- Both totals are 163,755.71 and the visitors total is 7,649.
- `revenue` still `str`? `thousands` or `decimal` is missing, or `na_values` is. 29 rows with a first row of text? `skiprows` is 2, not 3.
- Without `date_format` pandas still parses these dates correctly here, but prints a `UserWarning` about day-first dates. The lab insists on saying the format: do not let the reader guess.

- `LABS/MODULE-04/L2_2-LOADING-DATA/PRODUCT_PAGES_SOLUTION.PY`

Question: In November 2025, which product pages were viewed most, and do visitors stay longer on product pages than on other pages?

How the solution works: `record_path="pages"` makes one row per page view (9,348) and `meta` copies `session_id` and `converted` onto each. Two types need repair: `at` is text, and `converted` arrives through `meta` as dtype `object`, so it is cast with `.astype(bool)`. Product pages are the paths that start with `/product/`; removing that prefix leaves the SKU. The result is written to `OUTPUT` as Parquet and CSV and read back.

```
['site', 'period', 'note', 'sessions'] 2400
page views: (9348, 5)
path             str
seconds        int64
at               str
session_id       str
converted     object
after the fix: datetime64[us] bool
product-page views: 2793 in 1610 sessions
sku
SNW-1016    22
PCK-1026    21
CLM-1019    20
ACC-1010    19
KIT-1025    19
mean seconds, product pages: 46.2
mean seconds, all pages    : 48.1
Parquet: datetime64[us] | equal to the original: True
CSV    : str | equal to the original: False
```

- 9,348 page views; 2,793 of them are product pages, in 1,610 sessions.
- Most viewed: SNW-1016 (22), PCK-1026 (21), CLM-1019 (20), then ACC-1010 and KIT-1025 (19 each). The counts are close together: there is no runaway favorite, and that is the honest answer.
- Mean time: 46.2 seconds on product pages against 48.1 on all pages. Visitors do not stay longer on product pages.
- Parquet reads back with `datetime64[us]` and `equals` is True; the CSV reads back with `str` and `equals` is False.
- The `converted` column as `object` is a real surprise: `meta` columns are not type-inferred. It is the lab's 'check the translation' moment.
- Learners who pass `record_path="cart.items"` get a KeyError: a nested path is a list, `["cart", "items"]` (that is the lesson's bonus challenge).

- `LABS/MODULE-04/L2_2-LOADING-DATA/TARGETS_VS_ACTUALS_SOLUTION.PY`

Question: Which of the four stores beat its 2025 revenue target, and by how much?

How the solution works: The Monthly Sales sheet has its header on the row at position 3, so `header=3` (the same as `skiprows=3`); the Targets sheet is tidy. A four-turn loop over the store names sums one column of `monthly` and one filtered column of `targets`, collecting a dictionary per store. `pd.DataFrame(rows)` turns the list into a table, a new column gives the percentage, and `to_excel` with `index=False` writes it.

```
['Monthly Sales', 'Targets']
                                                    0       1       2
0  Pinecrest Outfitters - 2025 revenue by month (...     NaN     NaN
1  Stores: point-of-sale totals. Online: complete...     NaN     NaN
2                                                NaN     NaN     NaN
3                                              Month   Boise    Bend
4                                                Jan  242847  169817
(12, 6) (16, 3)
['Month', 'Boise', 'Bend', 'Flagstaff', 'Asheville', 'Online']
       store   actual   target  pct_of_target
0  Flagstaff  2717996  2555000          106.4
1       Bend  2569347  2621000           98.0
2      Boise  3846407  3963000           97.1
3  Asheville  2017700  2220000           90.9
stores at or above target: 1
read back: (4, 4) ['store', 'actual', 'target', 'pct_of_target']
```

- Flagstaff 106.4% (the only store at or above target), Bend 98.0%, Boise 97.1%, Asheville 90.9%.
- `monthly.shape` is `(12, 6)` and its columns are `Month, Boise, Bend, Flagstaff, Asheville, Online`. Any `Unnamed` means the header option is wrong.
- The read-back shape is `(4, 4)`. A shape of `(4, 5)` with an `Unnamed: 0` column means `index=False` was forgotten.
- This lab is homework or a stretch task in the room; it mirrors the lesson's Exercise 2 but for all four stores and with a written result.

2.2 · THE LESSON PAGE'S QUICK QUIZ

These are the five questions at the end of the lesson page (three options each), with the lesson's own explanation.

2.2 Q1

The second row of this small file has an empty amount. What does the code print?

```
qtext = io.StringIO("id,amount\n1,10\n2,\n3,30\n")
qdf = pd.read_csv(qtext)
print(qdf["amount"].dtype)
```

A) int64

B) float64

C) str

> **Why:** The empty cell becomes NaN, and NumPy int64 cannot hold NaN, so pandas reads the column as float64. Asking for the nullable Int64 dtype keeps it as whole numbers.

2.2 Q2

A worksheet has a title in row 1, a note in row 2, a blank row 3, and the column names in row 4. Which call reads it correctly?

A) pd.read_excel(path, nrows=4)

B) pd.read_excel(path, header=4)

C) pd.read_excel(path, skiprows=3)

> **Why:** Skipping the first three rows makes row 4 the first row pandas sees, and it uses that row as the header. header=3 would do the same.

2.2 Q3

In a list of JSON records, some records have a utm dictionary and others have no utm key at all. What does pd.json_normalize(records) do with the records that lack it?

A) Keeps them, with NaN in the utm.* columns

B) Drops those records from the result

C) Raises a KeyError for the first record without the key

> **Why:** json_normalize builds the columns from every key it finds in any record and fills NaN where a record does not have that key. In this lesson 916 sessions had NaN in the utm columns.

2.2 Q4

You save a table with df.to_csv("out.csv"). A colleague reads it and finds an extra first column named Unnamed: 0. What happened?

A) The file's header row was written twice

B) The index was written as a column; index=False prevents it

C) The file was saved in the wrong encoding

> **Why:** to_csv writes the index as the first column unless told not to, and the default index has no name. Reading the file back turns it into a column called Unnamed: 0. Write with index=False.

2.2 Q5

A table with a datetime column and a category column is written to CSV and to Parquet. Read back with no options, only the Parquet version still has those dtypes. Why?

A) Parquet compresses the data, and CSV does not

B) read_parquet is better at guessing types from the values

C) A Parquet file stores each column's type with the data; a CSV is only text

> **Why:** A Parquet file stores a schema, the type of every column, alongside the values, so nothing has to be guessed. A CSV file is plain text and every type must be inferred or supplied again on each load.

WARM-UP ANSWERS

1. You write `rng = np.random.default_rng(42)`. What is `rng`, and what does the 42 do? **Answer:** A random number generator. The seed fixes where its sequence starts, so the same code run from the top gives the same numbers.
2. Can `rng.integers(1, 7, size=5)` ever return a 7? **Answer:** No. The upper end is left out, so the values are 1 to 6 (a die).
3. What does `replace=False` guarantee in `rng.choice(skus, size=10, replace=False)`? **Answer:** Ten distinct items: nothing is picked twice.
4. For two 1-D arrays of length 3, what is the difference between `a * b` and `a @ b`? **Answer:** `a * b` is position by position and gives three numbers; `a @ b` is the dot product, one number: a weighted sum.
5. A matrix of shape `(726, 3)` is multiplied with `@` by a vector of shape `(3,)`. What shape is the result, and what must match? **Answer:** `(726,)`. The inner numbers must match (3 and 3): one dot product per row.

EXIT TICKET ANSWERS

1. You run `df[df["price"] > 100]["price"] = 0`. What happens to `df`, and what should you write instead? **Answer:** Nothing happens to `df` (pandas warns with `ChainedAssignmentError`): the first brackets made a temporary copy. Write `df.loc[df["price"] > 100, "price"] = 0`.

2. On a table with the default index, how many rows do `df.loc[0:3]` and `df.iloc[0:3]` return, and why are they different? **Answer:** 4 and 3. `.loc` slices by label and includes the end; `.iloc` slices by position and leaves out the stop.

3. You save a table that has a date column to `out.csv` and to `out.parquet`. Read back with no options, which file gives the date back as a date, and why? **Answer:** The Parquet file. It stores each column's dtype with the data; a CSV is only text, so the date comes back as `str` unless you pass `parse_dates` again.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q2 answered `2 2 40` or `3 3 40`.** The learner has one slicing rule, not two. Send them back to the one-line experiment: `.loc` includes the end label, `.iloc` leaves out the stop position.
- **Q5 answered `[10, 0, 0]`.** They believe a filtered piece is a window onto the table (the NumPy view model from Lesson 1.1). Re-run the tents demonstration and have them say the copy-on-write rule in their own words.
- **Q4 answered A (no parentheses).** They know to use `|` but not that it binds tighter than `==` and `<`. Have them add the parentheses and explain what Python tried to compute first.
- **Exit ticket 1 answered 'it sets them to 0'.** The warning was not noticed. Show the cell again in a notebook and ask them to find the word `ChainedAssignmentError`.
- **Q7 answered A (`[4.0, nan, 6.0]`).** They have learned 'gaps make floats' but not that `Int64` is the cure. That is half right; point at the capital I.
- **Q9 answered `(2, 2)`.** They expect one row per record. Draw the tree: `record_path` makes one row per item of the inner list, so 2 + 1 = 3 rows.
- **Q11 answered `['a']`.** They do not yet see the index as something that gets written. Have them open the CSV in a text editor and look at the first column.
- **Lab `till_export`: 29 rows and a text `revenue`.** The header was skipped or counted as a note. Ask which line holds the column names, then recount `skiprows`.
- **Lab `recommend_groups`: a different score for Asheville.** Usually the denominator: all 202 responses instead of the 197 who answered. Ask what `notna()` is for.

INSTRUCTOR KIT · ANSWER KEY

Session 5: Cleaning and Grouping

Answers with reasoning for the 12 self-check items and the predict-then-run boxes, the real output of every lab solution with what to check while circulating, the fixes for the 'find the bug' handout, the two lesson pages' own Quick Quiz answers, and the warm-up and exit-ticket answers.

Session 5 | 12 self-check items | 4 lab pairs | Confidential

HOW THIS KEY IS ORGANIZED

Self-check numbering matches the workbook: Q1 to Q6 are on Lesson 2.3, Q7 to Q12 on Lesson 2.4. Each item repeats the four options with the correct one marked and says why each wrong option is wrong. Every printed output below comes from running the code against the course data. The lesson pages' own five-question Quick Quizzes (three options each) are answered at the end of each lesson's part.

2.3 · CLEANING DATA: SELF-CHECK

Q1

An audit shows that the `Store` column has 31 distinct values. Pinecrest has four stores and an online shop. What is the right first move?

A) Delete the rows whose store name is unusual

B) Print `value_counts()` to see the variants, then strip, fix the case, and map what is left

C) Convert the column to the `category` dtype

D) Fill the unusual names with the most common store

Why: Look before you fix. `value_counts()` shows which variants exist; whitespace and case are mechanical, and the remaining misspellings need a mapping dictionary that you write from what you see.
A is wrong: Those rows are real orders with a badly typed store name. Deleting them deletes revenue.
C is wrong: A category column with 31 categories is still 31 stores. The dtype does not merge spellings.
D is wrong: That would move orders from Boise, Bend and the rest to Online: an invented value, not a repair.

Q2

What does this print?

```
d = pd.Series(["2024-03-05", "03/05/2024", "March 5, 2024", "2024-02-30"])
iso = pd.to_datetime(d, format="%Y-%m-%d", errors="coerce")
us = pd.to_datetime(d, format="%m/%d/%Y", errors="coerce")
print(iso.notna().sum(), iso.fillna(us).notna().sum())
```

`1 2`

Why: The ISO pass understands only the first value: the next two are in other formats and February 30th does not exist, so all three become NaT. Filling from the month/day/year pass recovers the second value. The written-out date needs a third pass, and the impossible date stays missing.

Q3

What does this print?

```
s = pd.Series(["none", "TRAIL15", "-", "trail15"])
p = s.str.upper()
print(p.where(~p.isin(["NONE", "-"])).tolist())
```

A) `['NONE', 'TRAIL15', '-', 'TRAIL15']`

B) `['TRAIL15', 'TRAIL15']`

C) `[nan, 'TRAIL15', nan, 'TRAIL15']`

D) `[nan, 'TRAIL15', nan, 'trail15']`

Why: `.str.upper()` standardizes the case, and `.where(condition)` keeps a value where the condition is true and puts a missing value where it is false. The two placeholders become real missing values; the length stays 4.

A is wrong: That is `p` before `.where()`: the placeholders are still there, counted as promo codes.

B is wrong: `.where()` never removes rows; it replaces values. The result has four items.

D is wrong: The last code was uppercased before the test, so it is `TRAIL15`.

Q4

In `orders_messy.csv` one order total is $4,990.00, while the largest genuine order is $4,893.00. What justified dividing the $4,990.00 by 100?

A) It is above the 99th percentile of order totals

B) It is an online order, and online totals are less reliable

C) It is a round number, which suggests a typing error

D) Its item lines add up to $34.95, so the recorded total is over 100 times what was bought

> **Why:** An outlier is a question, not a verdict. The independent evidence is in `order_items.csv`: the order's items total $34.95, and the recorded total is about 143 times that. The genuine $4,893 order matches its items.
> **A** is wrong: So is the genuine $4,893 bulk order. Being extreme is not evidence of an error.
> **B** is wrong: Nothing in the data says online totals are less reliable; the twelve largest genuine orders are all online.
> **C** is wrong: Plenty of real totals are round numbers. That is a hunch, not evidence.

Q5

What does this print?

```
df = pd.DataFrame({"total": [10.0, None, 50.0]})
print(df["total"].mean(), df["total"].fillna(0).mean())
```

`30.0 20.0`

> **Why:** `.mean()` skips the missing value: (10 + 50) / 2 = 30. Filling the gap with 0 claims that the customer paid nothing and divides by three: 60 / 3 = 20. A fill is a claim about the data.

Q6

`trial` has 406 missing payment methods. What happens when you run `trial["payment_method"].fillna("unknown", inplace=True)`?

A) `trial` is unchanged, and pandas warns with `ChainedAssignmentError`

B) The 406 gaps in `trial` are filled with `unknown`

C) A `TypeError`, because text cannot be used to fill a column

D) The gaps are filled, but only in the first 406 rows

> **Why:** `trial["payment_method"]` is a new, derived object. Under copy-on-write, filling it 'in place' changes that temporary object and never reaches `trial`. Assign the result back: `trial["payment_method"] = trial["payment_method"].fillna("unknown")`.
> **B** is wrong: That is what older pandas did, and what old tutorials promise. Under copy-on-write it no longer happens.
> **C** is wrong: Filling a text column with text is fine; the problem is where the result goes.
> **D** is wrong: `fillna` fills by position of the gaps, not the first rows; and here nothing in `trial` is filled at all.

2.3 · PREDICT, THEN RUN

P1

Case without whitespace.

```
s = pd.Series([" Bend", "bend ", "BEND", "Bend OR"])
print(s.str.title().nunique(), s.str.strip().str.title().nunique())
```

> 4 2

> **Why:** Title case alone leaves four different values, because two of them still carry a space and `Bend OR` becomes `Bend Or`. Stripping first merges the three spellings of Bend. Two remain: `Bend` and `Bend Or`, and the second needs a mapping, which is a decision, not a mechanical step.

P2

A detector for values that are not numbers.

```
s = pd.Series(["$1,200.50", "80", "free", None])
n = pd.to_numeric(s.str.replace("$", "").str.replace(",", ""), errors="coerce")
print(n.tolist(), n.isna().sum())
```

```
[1200.5, 80.0, nan, nan] 2
```

Why: After the symbols are removed, two values convert. `free` cannot, so `coerce` makes it NaN; the empty value was already missing. Two NaNs, for two different reasons: one was a word that needs a decision (free means 0), the other was never there.

P3

Three ways to lose rows.

```
df = pd.DataFrame({"id": [1, 2, 2, 3], "promo": [None, "A", "A", None],
                   "amt": [5.0, 7.0, 7.0, None]})
print(len(df.drop_duplicates()), len(df.dropna()), len(df.dropna(subset=["amt"])))
```

```
3 2 3
```

Why: One exact duplicate goes, leaving 3. `dropna()` with no arguments drops every row with a gap in any column: only the two rows with a promo code survive. Naming the column that matters keeps the 3 rows that have an amount.

2.3 · LAB REFERENCE SOLUTIONS

`LABS/MODULE-05/L2_3-CLEANING-DATA/REGISTER_LOG_SOLUTION.PY`

Question: Clean the Asheville manager's hand-typed log of May 2025 (`asheville_may_log.csv`) and score it against the same 31 days of `daily_sales.csv`.

How the solution works: Structure first: strip and lowercase the headers, rename three of them, save the TOTAL row's number, drop that row and the 3 exact duplicates (31 rows). Text: strip and title-case the store name, map the one misspelling; reduce yes/Yes/Y/YES and no/No/N to their first letter and map to 1/0. Numbers: remove `$` and `,`, coerce (the word `pending` becomes NaN); revenue per transaction exposes one day that is 100 times too large (9,551 per sale against a median of 90), fixed with one `.loc` assignment; the count columns become `Int64`. Dates: three formats, one pass each, combined with `fillna`. Two values stay blank. The stretch step repairs one of them with a firm rule: the TOTAL row minus the sum of the known days.

```
['Date ', 'Store', ' Visitors', 'Transactions', 'Units', 'Revenue ($)', 'Staff', 'Promo
                dtype  missing  distinct
Date              str        0        32
Store             str        1         5
 Visitors     float64        2        26
Transactions  float64        1        24
Units         float64        1        27
Revenue ($)       str        0        32
Staff         float64        1         8
Promo?            str        1         7
rows: 35 | exact duplicate rows: 3
after structure: (31, 8) | stated total: 201379.71
{'Asheville': 16, 'asheville': 5, ' Asheville': 4, 'ASHEVILLE': 4, 'Ashville': 2}
{'no': 21, 'No': 4, 'N': 2, 'yes': 1, 'Yes': 1, 'Y': 1, 'YES': 1}
['Asheville'] {0: 27, 1: 4}
not a number: [['05/09/2025', 'pending']]
{'min': 73.24, '50%': 90.06, 'max': 9551.47}
lost decimal point: [['2025-05-17', 93.0, 888287.0]]
dates per format: 17 10 4
                       dtype  missing  distinct
date           datetime64[us]        0        31
store                     str        0         1
visitors                Int64        1        26
transactions            Int64        0        24
units                   Int64        0        27
revenue               float64        1        30
staff_on_duty           Int64        0         8
promo_day               int64        0         2
still missing: {'visitors': 1, 'revenue': 1}
cells compared: 217 | still blank: 2 | wrong: 0
the pending day, recovered from the TOTAL row: ['2025-05-09'] 3668.5
cells compared: 217 | still blank: 1 | wrong: 0
```

- Audit before: 35 rows, 3 exact duplicates, `Store` with 5 distinct spellings, `Promo?` with 7, `Revenue ($)` as text, four count columns as float (the TOTAL row's empty cells did that).
- After structure: `(31, 8)` and a stated total of 201,379.71.
- Not a number: `pending` on 05/09/2025. Lost decimal point: 2025-05-17, 888,287.00 for 93 transactions. The genuine promo-day figure of $16,558.00 (May 24) must NOT be touched: 96.83 per sale is ordinary.
- Dates per format: 17 ISO, 10 month/day/year, 4 written out. No date is missing.
- Scorecard: `cells compared: 217 | still blank: 2 | wrong: 0`. The two blanks are the `pending` revenue and one `n/a` visitor count, which `read_csv` turned into NaN by itself.
- Stretch: 201,379.71 minus the known revenue is 3,668.50, the true figure for May 9. Scorecard then says 1 blank. The visitor count cannot be recovered from this file: it stays blank.

- `wrong: 1` nearly always means the lost decimal was not repaired, or a threshold on revenue alone caught May 24 as well. `wrong: 31` in one column means `promo_day` is still text.

- `LABS/MODULE-05/L2_3-CLEANING-DATA/TOTAL_ROW_CHECK_SOLUTION.PY`

Question: If someone simply added up `Order Total` in `orders_messy.csv`, how far off would they be, and which flaw moves the sum the most?

How the solution works: The TOTAL row's 3,950,701.45 is the sum of all true order totals (it equals the sum of `order_total` in `orders.csv`). The lab converts the column, takes the careless sum, then applies three repairs in turn and records the sum after each: dropping the 212 duplicates, making the 15 negative totals positive, and dividing the 8 lost-decimal totals by 100 (found with the item-subtotal ratio from the lesson). What is left is explained to the cent by the 40 blank totals.

```
                     step        sum     change  off_by_pct
             careless sum 4126451.89        NaN        4.45
       drop 212 duplicates 4083821.73  -42630.16        3.37
15 negatives made positive 4090475.89    6654.16        3.54
     8 lost decimal points 3943079.74 -147396.15       -0.19
stated total (TOTAL row): 3,950,701.45
gap after all repairs   : 7,621.71
blank totals: 40 | their true sum in orders.csv: 7,621.71
the gap is exactly the blank totals: True
```

- Careless sum 4,126,451.89: 4.45% too high.
- Dropping duplicates: -42,630.16. Negatives made positive: +6,654.16. Lost decimals: -147,396.15, the largest effect by far, from only 8 rows.
- After all repairs 3,943,079.74, which is 7,621.71 below the stated total. The 40 orders with a blank total add up to exactly 7,621.71 in `orders.csv`.
- Discussion: eight rows did three times the damage of 212 duplicates. Counting how many rows a flaw touches says little about how much it matters.
- If `after_decimals` is unchanged, the ratio was computed before `order_id` was converted to `int64`, so `.map()` found no matches.

2.3 · THE LESSON PAGE'S QUICK QUIZ

These are the five questions at the end of the lesson page (three options each), with the lesson's own explanation.

2.3 Q1

What does this print?

```
qs = pd.Series(["12", "7", "n/a", "3.5"])
print(pd.to_numeric(qs, errors="coerce").tolist())
```

A) [12.0, 7.0, nan, 3.5]

B) Nothing: it raises a ValueError at 'n/a'

C) [12.0, 7.0, 0.0, 3.5]

> **Why:** errors='coerce' converts every value that can be a number and turns the rest into NaN instead of raising an error. The NaN makes the result float.

2.3 Q2

A column holds " boise", "BOISE " and "Boise". After .str.title() alone, how many distinct values does it have?

A) 1

B) 2

C) 3

> **Why:** Title case does not remove whitespace, so a leading space, a trailing space and no space are still three different texts. Strip first, then fix the case.

2.3 Q3

Forty of 20,302 orders have no recorded total, and nothing in the file lets you work it out. You need the average order value. What is the most defensible approach?

A) Fill the 40 gaps with 0 so that every row has a number

B) Leave them missing, compute the mean of the known totals, and report that 40 were excluded

C) Call dropna() on the table first, then take the mean

> **Why:** mean() skips missing values, so the average is computed from the 20,262 known totals, and stating that 40 were excluded lets a reader judge the result. No value is invented.

2.3 Q4

What does this print?

```
qd = pd.DataFrame({"id": [1, 1, 2, 3, 3, 3], "v": ["a", "a", "b", "c", "c", "d"]})
print(len(qd.drop_duplicates()), len(qd.drop_duplicates(subset=["id"])))
```

A) 3 3

B) 4 4

C) 4 3

> **Why:** Comparing whole rows leaves four distinct rows: 1 a, 2 b, 3 c and 3 d. Comparing only id keeps the first row of each id, which leaves three, and silently discards the 3 d row.

2.3 Q5

One order total is $55,690, far above the rest. What should you do first?

A) Look for independent evidence about that order, such as its item lines

B) Delete every order above a chosen threshold

C) Replace it with the median order total

> **Why:** An extreme value is a question. Independent evidence, here the item lines for that order, tells you whether it is a real bulk order or an entry error, and what the right value is.

2.4 · GROUPBY OPERATIONS: SELF-CHECK

Q7

What does this print?

```
df = pd.DataFrame({"store": ["a", "a", "b"], "amt": [10, 30, 5]})
r = df.groupby("store")["amt"].agg(["count", "mean"])
print(r.shape, r.loc["a", "mean"])
```

A) `(3, 2) 20.0`

B) `(2, 2) 40.0`

C) `(2,) 20.0`

D) `(2, 2) 20.0`

> **Why:** Two groups give two rows, and two function names give two columns. The index is the store, so `r.loc["a", "mean"]` is the mean of 10 and 30.
> **A** is wrong: Aggregation returns one row per group, not one per original row.
> **B** is wrong: 40 is the sum for store `a`; the column asked for is the mean.
> **C** is wrong: A list of two functions makes a DataFrame with two columns, not a Series.

Q8

What does this print?

```
df = pd.DataFrame({"k": ["x", "x", "y"], "v": [1, 3, 10]})
df["share"] = df["v"] / df.groupby("k")["v"].transform("sum")
print(df["share"].tolist())
```

`[0.25, 0.75, 1.0]`

> **Why:** `transform("sum")` returns one value per original row: 4, 4, 10 (each row's group total). Dividing each value by its own group's total gives each row's share of its group.

Q9

`orders` has a True/False column `returned`. Which expression gives the fraction of each channel's orders that were returned?

A) `orders.groupby("channel")["returned"].mean()`

B) `orders.groupby("channel")["returned"].count()`

C) `orders.groupby("channel")["returned"].size()`

D) `orders.groupby("channel")["returned"].nunique()`

> **Why:** True counts as 1 and False as 0, so the mean of a boolean column within each group is the rate.
> **B** is wrong: `count` is the number of non-missing values: every order, returned or not.
> **C** is wrong: `size` is the number of rows in each group.
> **D** is wrong: `nunique` is the number of distinct values: 2 for each channel (True and False).

Q10

`orders.groupby("promo_code").size().sum()` gives 2,148, but `orders` has 20,302 rows. Why?

A) `size()` skips rows whose order total is missing

B) `groupby` removes duplicate rows before it counts

C) Rows whose key is missing are dropped by default; `dropna=False` keeps them as a group

D) `promo_code` is text and must be a category before grouping

> **Why:** 18,154 orders have no promo code. By default `groupby` silently leaves out every row whose key is missing. `dropna=False` adds a group for them, and the sizes then add up to 20,302.
> **A** is wrong: `size()` counts rows whatever they contain; it does not look at `order_total`.
> **B** is wrong: `groupby` never removes duplicates.
> **D** is wrong: Text keys group perfectly well. The dtype is not the issue.

Q11

What does this print?

```
df = pd.DataFrame({"g": ["a", "a", "b"], "x": [1.0, None, 3.0]})
print(df.groupby("g")["x"].size().tolist(), df.groupby("g")["x"].count().tolist())
```

`[2, 1] [1, 1]`

> **Why:** `size` counts rows: two in group `a`, one in group `b`. `count` counts non-missing values of `x`: group `a` has one missing, so 1 and 1.

Q12

You want the original order rows of every customer who placed 25 or more orders. Which call returns them?

A) `orders.groupby("customer_id")["order_id"].count() >= 25`

B) `orders.groupby("customer_id").filter(lambda g: len(g) >= 25)`

C) `orders.groupby("customer_id").agg(n=("order_id", "count"))`

D) `orders.groupby("customer_id")["order_id"].transform("count")`

> **Why:** `filter` hands each group to the function and keeps all rows of the groups for which it returns True: fewer of the same rows.
>
> **A** is wrong: That is a True/False Series with one value per customer, not order rows.
>
> **C** is wrong: That is a summary with one row per customer.
>
> **D** is wrong: That is one count per order row: the first half of the transform-then-mask pattern. You would still need `orders[... >= 25]`.

2.4 · PREDICT, THEN RUN

P1

An average of averages.

```
df = pd.DataFrame({"team": ["a", "b", "a", "b", "a"], "pts": [2, 10, 4, 20, 6]})
means = df.groupby("team")["pts"].mean()
print(means.tolist(), means.mean(), df["pts"].mean())
```

```
[4.0, 15.0] 9.5 8.4
```

Why: Team `a` averages 4 over three rows and team `b` averages 15 over two rows. The mean of those two means, 9.5, weights the teams equally; the mean of all five rows, 8.4, weights every row equally. They differ because the groups are different sizes.

P2

A key with gaps.

```
df = pd.DataFrame({"code": ["X", None, "X", None], "amt": [1, 2, 3, 4]})
print(len(df.groupby("code")), len(df.groupby("code", dropna=False)),
      df.groupby("code")["amt"].sum().sum())
```

```
1 2 4
```

Why: By default the two rows with no code are dropped, so there is one group (`X`) and the sums add up to 1 + 3 = 4, not 10. With `dropna=False` the missing key becomes a second group.

P3

Three shapes.

```
df = pd.DataFrame({"k": ["a", "a", "b"], "v": [1, 2, 3]})
a = df.groupby("k")["v"].sum()
t = df.groupby("k")["v"].transform("sum")
f = df.groupby("k").filter(lambda g: len(g) > 1)
print(len(a), len(t), len(f))
```

```
2 3 2
```

Why: `agg` (here `sum`) returns one row per group: 2. `transform` returns one value per original row: 3. `filter` returns the original rows of the groups that pass the test: only group `a` has more than one row, so 2 rows.

2.4 · LAB REFERENCE SOLUTIONS

`LABS/MODULE-05/L2_4-GROUPBY-OPERATIONS/STORE_DAYS_SOLUTION.PY`

Question: For each store: days, revenue and conversion; how much a promo day brings in; whether the lesson's growth claim (+19.7% Flagstaff, -5.0% Asheville) holds; and how many days were at least twice the store's own average.

How the solution works: One named aggregation gives the store table; conversion is the sum of transactions divided by the sum of visitors, computed after the aggregation. Two keys (`store` , `promo_day`) and `unstack` give ordinary and promo-day means side by side, and a ratio column gives the lift. A computed year key reproduces the growth figures the lesson quotes. `transform("mean")` puts each store's average on every row so that each day can be compared with its own store.

```
(2903, 8)
           days  days_with_revenue     revenue  visitors  transactions  conversion
store
Asheville   727                726  4140977.47    199851         46918       0.235
Bend        724                721  5045467.37    242447         57190       0.236
Boise       727                726  7364698.20    352716         83251       0.236
Flagstaff   725                724  4988176.84    238135         56366       0.237
groups add up: 2903 == 2903
promo_day        0         1  lift
store
Asheville  5541.10  10463.59  1.89
Bend       6776.66  13422.32  1.98
Boise      9830.07  19332.90  1.97
Flagstaff  6694.55  12583.02  1.88
ordinary and promo day means: [7211.74, 13950.46]
average of those two averages: 10581.1
true average store-day       : 7435.04
year            2024       2025  growth_pct
store
Flagstaff  2270180.0  2717997.0        19.7
Boise      3518292.0  3846406.0         9.3
Bend       2476118.0  2569350.0         3.8
Asheville  2123278.0  2017699.0        -5.0
store
Asheville    12
Bend         15
Boise        15
Flagstaff    21
dtype: int64
standout days: 63 | share that were promo days: 0.619
           date      store   revenue  store_avg  vs_store_avg
970  2024-08-31       Bend  20935.21    6997.87          2.99
2023 2025-05-24  Asheville  16558.00    5703.83          2.90
2775 2025-11-29  Asheville  15828.25    5703.83          2.78
```

- Store table: Asheville 727 days and 726 with revenue, Bend 724 / 721, Boise 727 / 726, Flagstaff 725 / 724. `size` counts rows, `count` skips the 6 missing revenues. The days add up to 2,903.
- Conversion is 23.5% to 23.7% in every store: no store converts visitors noticeably better.
- Promo days bring in 1.88 to 1.98 times an ordinary day's revenue in every store.
- The trap: ordinary days average 7,211.74 and promo days 13,950.46; the average of those two is 10,581.10, but the true average store-day is 7,435.04, because only 96 of 2,903 store-days are promo days.
- Growth, 2024 to 2025: Flagstaff +19.7%, Boise +9.3%, Bend +3.8%, Asheville -5.0%. The lesson's claim holds.
- Standout days (at least twice the store's own average): Flagstaff 21, Bend 15, Boise 15, Asheville 12; 63 in all, and 61.9% of them were promo days. The biggest relative day is Bend on 2024-08-31, 2.99 times its average.

- A learner whose conversion column is a mean of daily ratios gets nearly the same numbers here; ask them which one answers 'what share of all visitors bought?' (the ratio of sums).

- `LABS/MODULE-05/L2_4-GROUPBY-OPERATIONS/BASKET_SIZES_SOLUTION.PY`

Question: How big is a typical basket, are the best sellers by units the best sellers by revenue, what share of its order is each line, and which orders have six or more lines?

How the solution works: Grouping by `order_id` gives one row per order (20,302) with lines, units and subtotal; grouping by `sku` gives one row per product (240) with units, revenue and distinct orders. `transform("sum")` puts each order's subtotal on every one of its lines so a share can be computed. `filter` with `len(g) >= 6` and the transform-then-mask version return the same 1,106 lines.

```
(45490, 6)
orders: 20302 | lines add up: True
       lines  units  subtotal
mean    2.24   3.21    195.23
50%     2.00   3.00    147.80
max     8.00  48.00   4893.00
{1: 5890, 2: 7244, 3: 4559, 4: 1841, 5: 593, 6: 128, 7: 38, 8: 9}
products sold: 240
          units    revenue  orders
sku
ACC-1015   3475  159029.25    2620
KIT-1008   3124   24771.00    1391
ACC-1017   2479   34513.50    1394
          units    revenue  orders
sku
ACC-1015   3475  159029.25    2620
APL-1001    420  117579.00     420
PCK-1001    784  107368.80     784
   order_id       sku  line_total  order_subtotal  share_of_order
0    100001  SLP-1021      371.95           434.9           0.855
1    100001  KIT-1014       62.95           434.9           0.145
2    100002  CLM-1012       32.95           278.9           0.118
3    100002  SNW-1022      245.95           278.9           0.882
lines that are a whole order: 5890
filter       : 1106 lines in 175 orders
transform+mask: 1106 lines in 175 orders
same rows: True
```

- 20,302 orders; the lines add up to 45,490. Mean 2.24 lines and 3.21 units per order; the median basket has 2 lines; the largest has 8 lines, and one order has 48 units (a bulk order).
- 240 SKUs were sold (the catalog has 242, but 8 never sold and 6 discontinued SKUs appear here).
- By units: ACC-1015 (3,475), KIT-1008 (3,124), ACC-1017 (2,479). By revenue: ACC-1015 ($159,029.25), APL-1001 ($117,579.00), PCK-1001 ($107,368.80). Only the first product is on both lists: a cheap item can sell in volume and

earn little.

- 5,890 lines are a whole order by themselves (share exactly 1): the single-line orders.
- Six or more lines: 1,106 lines in 175 orders, by both methods, and the two results have the same index.
- `lines=("sku", "size")` is equally correct; `("sku", "nunique")` also works here only because SKUs are distinct within an order.

`LABS/MODULE-05/L2_4-GROUPBY-OPERATIONS/DEBUG_GROUPBY.PY`

- Bug 1: `orders.groupby("store").mean()` raises `TypeError: dtype 'str' does not support operation 'mean'`. Select the column: `orders.groupby("store")["order_total"].mean()`.
- Bug 2: `summary["store"]` raises `KeyError: 'store'`. The key is the index: `summary.index.tolist()` or `summary.reset_index()["store"].tolist()`.
- Bug 3 (quiet): `["promo_code"].count()` counts only orders with a promo code (card 1,271). Use `.size()`: card 12,392.
- Bug 4 (quiet): rows with no promo code are dropped, so the groups sum to 2,148. Add `dropna=False`: 20,302 of 20,302.
- Bug 5 (quiet): the mean of five store means is 182.66. The company average is `orders["order_total"].mean()`: 194.6.

2.4 · THE LESSON PAGE'S QUICK QUIZ

These are the five questions at the end of the lesson page (three options each), with the lesson's own explanation.

2.4 Q1

What does this print?

```
qdf = pd.DataFrame({"team": ["a", "a", "b", "b", "b"], "pts": [1, 3, 2, 4, 6]})
qres = qdf.groupby("team")["pts"].transform("mean")
print(len(qres), qres.tolist())
```

A) 2 [2.0, 4.0]

B) 5 [2.0, 2.0, 4.0, 4.0, 4.0]

C) 5 [1.0, 3.0, 2.0, 4.0, 6.0]

> **Why:** transform returns one value for every original row: each row gets the mean of its own group. Team a averages 2 and team b averages 4, repeated for their two and three rows.

2.4 Q2

You want one row per store, with a column orders (the number of orders) and a column revenue (the sum of order_total). Which call produces it?

A) orders.groupby("store").agg({"order_total": {"revenue": "sum"}})

B) orders.groupby("store")["order_total"].transform("sum")

C) orders.groupby("store").agg(orders=("order_id", "count"), revenue=("order_total", "sum"))

> **Why:** Named aggregation: each keyword is the new column name, and its tuple gives the source column and the function. The result has one row per store and exactly these two columns.

2.4 Q3

Two of the five rows have no code. What does this print?

```
qd2 = pd.DataFrame({"code": ["X", None, "X", None, "Y"], "amt": [1, 2, 3, 4, 5]})
print(qd2.groupby("code")["amt"].sum().sum())
```

A) 9

B) 15

C) nan

> **Why:** By default groupby drops rows whose key is missing. Only X (1 + 3) and Y (5) form groups, so the group sums total 9, not 15. dropna=False would keep the missing key as a group.

2.4 Q4

The five stores' average order totals are $174.90, $172.91, $177.22, $183.61 and $204.65. Their mean is $182.66, but the average over all orders is $194.60. Why do the two differ?

A) Rounding errors build up when averages are averaged

B) Missing order totals were skipped in one calculation and not in the other

C) The groups are very different sizes, and a mean of means weights each group equally

> **Why:** Averaging the five store averages gives each store equal weight, but the online shop has 12,854 orders and Asheville 1,413. The true overall average weights every order equally, so the large online group pulls it up.

2.4 Q5

Which statement about orders.groupby("payment_method")["promo_code"] is true?

A) .size() counts all orders in each group; .count() counts only those with a promo code

B) .size() and .count() always return the same numbers

C) .count() returns the number of distinct promo codes in each group

> **Why:** size counts the rows in each group whatever they contain. count counts only the non-missing values of the selected column, so it reports how many orders in each group used a promo code.

WARM-UP ANSWERS

1. What three things make up a Series? **Answer:** An array of values, an index of labels, and an optional name.
2. On a table with the default index, how many rows do `df.loc[0:3]` and `df.iloc[0:3]` return? **Answer:** 4 and 3: a label slice includes its end, a position slice leaves out the stop.
3. `df[df["price"] > 100]["price"] = 0` changes nothing. What do you write instead? **Answer:** `df.loc[df["price"] > 100, "price"] = 0`: one `.loc` on the table you want to change.
4. A column of birth years loads as `float64` and prints `1989.0`. Why, and what is the fix? **Answer:** Some years are missing and NumPy's `int64` cannot hold a missing value. Load it as `"Int64"` (capital I).
5. After `pd.read_excel(...)` the columns are called `Unnamed: 1`, `Unnamed: 2`. What happened, and what do you check after every load? **Answer:** The header is not on the row pandas read as the header (`skiprows` / `header`). After every load: `shape`, `dtypes`, `head()`.

EXIT TICKET ANSWERS

1. In what order do you repair a damaged table, and why does structure come first? **Answer:** Structure (headers, rows that are not data, duplicates), then text, numbers, dates, missing values. Fixing the shape first makes later steps simpler and counts honest (for example `astype` fails while a TOTAL row is present).
2. Forty of 20,302 order totals are blank and nothing in the file gives their value. You need the average order value. What do you do, and what do you say? **Answer:** Leave them missing, take the mean of the known totals, and report that 40 orders were excluded. Filling with 0 or with the median would invent data.
3. One row per group; one value per original row; fewer of the same rows. Name the groupby tool for each. **Answer:** `agg` (or a built-in such as `sum`); `transform`; `filter` (or a `transform` and a mask).

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q2 answered `2 2` or `1 3`.** They expect one format string to understand another format, or they expect the impossible date to be parsed. Have them print `iso` and `us` side by side: each pass understands its own format only, and February 30th stays NaT.
- **Q5 answered `30.0 30.0`.** They think `fillna(0)` is neutral. Ask what three customers paid in the second calculation: 10, 0 and 50.
- **Q6 answered B ('the gaps are filled').** The copy-on-write rule from Session 4 has not transferred to `inplace=True`. Show the cell and the warning; the fix is to assign the result back.
- **Q4 answered A ('above the 99th percentile').** They treat 'extreme' as 'wrong'. Remind them that the $4,893 bulk order is extreme and true.
- **Q8 answered `[4, 4, 10]` or a two-item list.** They are thinking of `agg`. Draw the two shapes: `transform` returns one value per original row.
- **Q11 answered `[2, 1] [2, 1]`.** `size` and `count` are the same to them. Ask what `count` does with the missing value in group `a`.
- **Exit ticket 2 answered 'fill with the mean'.** It leaves the mean unchanged, which feels safe, but it invents 40 perfectly average orders and shrinks the spread. With 40 gaps in 20,302 rows, leaving them missing costs nothing.
- **Lab `register_log`: `wrong: 1`.** The lost decimal was missed, or a threshold on revenue caught the genuine promo day. Ask for the evidence in the row (revenue per transaction).
- **Lab `store_days`: 726 days for Asheville.** `count` on `revenue` was used for the number of days. Ask what happens to a day with no revenue figure.

INSTRUCTOR KIT · ANSWER KEY

Session 6: Merging and Reshaping

Answers with reasoning for the 12 self-check items and the predict-then-run boxes, the real output of every lab solution with what to check while circulating, the fixes for the 'find the bug' handout, the two lesson pages' own Quick Quiz answers, and the warm-up and exit-ticket answers.

Session 6 | 12 self-check items | 5 lab pairs | Confidential

HOW THIS KEY IS ORGANIZED

Self-check numbering matches the workbook: Q1 to Q6 are on Lesson 2.5, Q7 to Q12 on Lesson 2.6. Each item repeats the four options with the correct one marked and says why each wrong option is wrong. Every printed output below comes from running the code against the course data. The lesson pages' own five-question Quick Quizzes (three options each) are answered at the end of each lesson's part.

2.5 · MERGING AND JOINING: SELF-CHECK

Q1

What does this print (the lengths of an inner, a left and an outer merge)?

```
a = pd.DataFrame({"k": [1, 2, 3], "x": ["p", "q", "r"]})
b = pd.DataFrame({"k": [2, 3, 3, 4], "y": [10, 20, 30, 40]})
print(len(a.merge(b, on="k")), len(a.merge(b, on="k", how="left")),
      len(a.merge(b, on="k", how="outer")))
```

A) `2 3 4`

B) `3 3 5`

C) `3 4 5`

D) `3 4 4`

Why: Key 2 matches once and key 3 matches twice: 3 rows in the inner join. The left join also keeps key 1 with no partner: 4 rows. The outer join adds key 4 from the right: 5 rows.

A is wrong: That counts matching keys, not rows. Key 3 appears twice on the right, so it produces two rows.

B is wrong: A left join keeps the unmatched left row (key 1), so it has one more row than the inner join.

D is wrong: The outer join also keeps key 4, which exists only in the right table.

Q2

You want to add each customer's loyalty tier to `orders` without losing any order, and you want the merge to fail loudly if `customers` ever contains a duplicated id. Which call does both?

A) `orders.merge(customers, on="customer_id")`

B) `orders.merge(customers, on="customer_id", how="left", validate="many_to_one")`

C) `orders.merge(customers, on="customer_id", how="inner", validate="one_to_one")`

D) `pd.concat([orders, customers])`

> **Why:** `how="left"` keeps every order, and `validate="many_to_one"` checks that `customer_id` is unique in the right-hand table, raising `MergeError` if it is not.
> **A** is wrong: The default is an inner join: 739 orders with no customer record disappear, and nothing checks for duplicate ids.
> **C** is wrong: An inner join loses the unmatched orders, and `one_to_one` fails at once because a customer has many orders.
> **D** is wrong: `concat` stacks the two tables on top of each other; it does not match rows by key.

Q3

What does this print?

```
o = pd.DataFrame({"order": [1, 2], "total": [100, 50]})
i = pd.DataFrame({"order": [1, 1, 1, 2], "sku": ["a", "b", "c", "d"]})
m = o.merge(i, on="order")
print(m["total"].sum(), m.drop_duplicates("order")["total"].sum())
```

`350 150`

> **Why:** The merged table has one row per line, so order 1's total of 100 appears three times: 100 + 100 + 100 + 50 = 350. Going back to one row per order before summing gives the true 150.

Q4

`orders.merge(customers, on="customer_id", how="outer", indicator=True)` shows 739 rows marked `left_only`. What are they?

A) Orders whose `customer_id` has no row in `customers`

B) Customers who have never placed an order

C) Orders that matched more than one customer

D) Duplicate orders that the merge removed

> **Why:** `left_only` means the key was found only in the left table, the one before `.merge`: orders with no customer record (the closed accounts).
> **B** is wrong: Those are the `right_only` rows: keys found only in `customers` (1,058 of them).
> **C** is wrong: Rows that match are marked `both`, however many partners they have.
> **D** is wrong: A merge never removes duplicates, and the indicator does not report them.

Q5

What does this print?

```
a = pd.DataFrame({"store": ["Boise"], "rev": [100]})
b = pd.DataFrame({"store": ["Bend"], "rev": [80], "visitors": [300]})
c = pd.concat([a, b], ignore_index=True)
print(c.shape, c["visitors"].isna().sum())
```

`(2, 3) 1`

> **Why:** `concat` stacks the rows and matches columns by name. The first table has no `visitors` column, so its row gets NaN there: 2 rows, 3 columns, 1 missing value.

Q6

`daily` has the columns `date` and `store`; `weather` has `date` and `city` (the same four place names). Each has one row per place per day. Which call attaches the weather to every sales row?

A) `daily.merge(weather, on=["date", "store"])`

B) `daily.merge(weather, on="date", how="left")`

C) `pd.concat([daily, weather], ignore_index=True)`

D) `daily.merge(weather, left_on=["date", "store"], right_on=["date", "city"], how="left", validate="one_to_one")`

> **Why:** The key is the pair (date, place), and the place column has a different name in each table, so the key columns are named for each side with `left_on` and `right_on`.
> **A** is wrong: `on=` needs the same names in both tables; `weather` has no `store` column, so this raises `KeyError: 'store'`.
> **B** is wrong: Matching on the date alone pairs every store's sales with all four cities' weather: four times the rows.
> **C** is wrong: `concat` stacks the tables into one long table with half-empty columns; it does not match rows.

2.5 · PREDICT, THEN RUN

P1

An outer join with an indicator.

```
left = pd.DataFrame({"k": [1, 2, 2], "a": ["x", "y", "z"]})
right = pd.DataFrame({"k": [2, 2, 3], "b": [10, 20, 30]})
m = left.merge(right, on="k", how="outer", indicator=True)
print(len(m), (m["_merge"] == "both").sum(), (m["_merge"] == "left_only").sum())
```

```
6 4 1
```

> **Why:** Key 2 appears twice on each side, so it makes 2 × 2 = 4 rows marked `both`: a many-to-many match. Key 1 exists only on the left (1 row) and key 3 only on the right (1 row). 4 + 1 + 1 = 6.

P2

Missing keys.

```
a = pd.DataFrame({"k": [1.0, np.nan], "a": ["x", "y"]})
b = pd.DataFrame({"k": [np.nan, np.nan], "b": ["p", "q"]})
print(len(a.merge(b, on="k")), len(a.merge(b, on="k", how="left")))
```

```
2 3
```

Why: In a pandas merge a missing key matches every other missing key (unlike SQL). The one left row with NaN matches both right rows: 2 rows in the inner join. The left join adds the unmatched key 1: 3 rows. Remove or fill missing keys before merging on a column that has them.

P3

An average after a merge.

```
o = pd.DataFrame({"order": [1, 2], "total": [300, 100]})
i = pd.DataFrame({"order": [1, 1, 1, 2], "qty": [1, 1, 1, 1]})
print(o["total"].mean(), o.merge(i, on="order")["total"].mean())
```

```
200.0 250.0
```

Why: The two orders average 200. After the merge order 1 appears three times (once per line), so the mean becomes (300 + 300 + 300 + 100) / 4 = 250. Orders with more lines are counted more often. Compute order-level statistics on the orders table.

2.5 · LAB REFERENCE SOLUTIONS

`LABS/MODULE-06/L2_5-MERGING-JOINING/CUSTOMER_PROFILES_SOLUTION.PY`

Question: Who are the five biggest spenders by name, how many account holders never ordered (and in which tier), and which orders belong to nobody in `customers.csv`?

How the solution works: Orders are first reduced to one row per customer id (3,942 rows), so both keys are unique and the merge onto `customers` is one-to-one: 4,850 rows in, 4,850 out. `indicator=True` marks the 1,058 accounts with no orders as `left_only`; their order count and spend are filled with 0, which is the honest value for a count and a sum. A second, outer merge of the two key columns audits the other side: 150 ids have orders but no account.

```
20302 orders | 4850 customer accounts
summary rows: 3942 | key unique in summary: True | in customers: True
4850 -> 4850
{'both': 3792, 'left_only': 1058, 'right_only': 0}
     first_name  last_name state loyalty_tier  orders    spend
4378       Rosa  Blackwood    GA         Gold      39  8305.39
4339        Zoe       Diaz    WA         Gold      41  7729.68
1608       Rosa     Ibarra    AZ         Gold      36  6868.83
1688     Joanna      Amato    ID         Gold      14  6792.90
505        Liam    Paredes    OR         Gold      29  6630.48
              accounts  never_ordered  avg_spend
loyalty_tier
Basic             2822          0.375    203.254
Gold               734          0.000   2515.087
Silver            1294          0.000   1063.965
accounts add up: True
{'both': 3792, 'left_only': 1058, 'right_only': 150}
ids with orders but no account: 150 | their orders: 739 | their spend: $154,273.80
spend that reached the profiles table: $3,796,427.65 of $3,950,701.45
```

- Summary: 3,942 rows, key unique on both sides. Profiles: `4850 -> 4850`, with 3,792 `both` and 1,058 `left_only`.
- Top five spenders: Rosa Blackwood (GA, 39 orders, $8,305.39), Zoe Diaz (WA, 41, $7,729.68), Rosa Ibarra (AZ, 36, $6,868.83), Joanna Amato (ID, 14, $6,792.90), Liam Paredes (OR, 29, $6,630.48). All Gold.
- By tier: 37.5% of the 2,822 Basic accounts never ordered; Gold and Silver have 0%, as they must (a tier above Basic is earned by spending). Average spend per account: Basic $203, Silver $1,064, Gold $2,515.
- Audit: 150 ids with orders but no account, 739 orders, $154,273.80: the closed accounts from the lesson. So only $3,796,427.65 of the $3,950,701.45 of order value can be attached to a named customer.
- 4,850 becoming 3,792 means the default inner join was used. More than 4,850 means the orders were merged without aggregating first (one row per order).
- Average spend here counts every order status. If someone's figures differ slightly, ask whether they filtered to completed orders: a fair choice, as long as they say so.
- Learners who kept `output/customer_summary.csv` from the 2.4 project can load it instead of recomputing the summary; its `spend` counts completed orders only, so the dollar figures will be lower.

- `LABS/MODULE-06/L2_5-MERGING-JOINING/ORDERS_WEATHER_SOLUTION.PY`

Question: On rainy days, how many orders do signed-in customers place in each store, compared with dry days?

How the solution works: In-store orders are counted per `(order_date, store)`: 2,588 store-days have at least one. The weather table has a row for every one of the 2,924 store-days, so it goes on the left: the 336 store-days with no account-holder order survive as `left_only` and are filled with 0 orders. Then a groupby on city and a rainy flag compares the averages.

```
(20302, 11) (2924, 5)
store-days with at least one order: 2588 of 2924
2924 -> 2924
{'both': 2588, 'left_only': 336, 'right_only': 0}
store-days with no account-holder order, by city: {'Asheville': 111, 'Bend': 97, 'Boise
rainy      False  True  change_pct
city
Asheville   1.99  1.75       -12.0
Bend        2.38  2.02       -15.3
Boise       3.65  2.56       -29.9
Flagstaff   2.40  2.01       -16.3
average orders per store-day (every day counted): 2.55
average over days that had an order row only    : 2.88
orders in the merged table: 7448 | in-store orders: 7448
```

- `2924 -> 2924`; 2,588 `both`, 336 `left_only` (Asheville 111, Bend 97, Flagstaff 91, Boise 37).
- Orders per store-day, dry against rainy: Asheville 1.99 → 1.75 (-12.0%), Bend 2.38 → 2.02 (-15.3%), Boise 3.65 → 2.56 (-29.9%), Flagstaff 2.40 → 2.01 (-16.3%). Fewer orders on rainy days in every store.
- Step 5 is the point of the lab: 2.55 orders per store-day when every day is counted, 2.88 when the days with no orders have silently dropped out. Starting from `daily_orders` (or using an inner join) gives the second, inflated figure.
- The merged table still holds all 7,448 in-store orders: a quick reconciliation.
- `KeyError: 'store'` in the scratch attempt is expected: the weather table calls the column `city`.
- These are account-holder orders only; the lesson's Exercise 3 asks the same kind of question of all visitors with `daily_sales.csv`. Same direction, different basis.

`LABS/MODULE-06/L2_5-MERGING-JOINING/DEBUG_MERGES.PY`

- Bug 1: `ValueError: You are trying to merge on str and int64 columns for key 'order_id'`. Do not convert the key to text (or convert it back with `.astype("int64")`). Prints 45490.
- Bug 2: `KeyError: 'store'`. Use `left_on=["date", "store"], right_on=["date", "city"]` (or rename `city`). Prints 2903.
- Bug 3: `MergeError: Merge keys are not unique in left dataset; not a one-to-many merge`. Lines to products is `validate="many_to_one"`. Prints 45490.
- Bug 4 (quiet): the default inner join drops the 739 orders of closed accounts, so the total prints 3,796,427.65. Add `how="left"`: 3950701.45.
- Bug 5 (quiet): after merging orders with their lines, each order total is repeated once per line, so the mean prints 242.67. Compute it on `orders`: 194.6.

2.5 · THE LESSON PAGE'S QUICK QUIZ

These are the five questions at the end of the lesson page (three options each), with the lesson's own explanation.

2.5 Q1

What does this print (the lengths of an inner, a left and an outer merge)?

```
ql = pd.DataFrame({"k": [1, 2, 2, 3], "a": ["w", "x", "y", "z"]})
qr = pd.DataFrame({"k": [2, 2, 4], "b": ["p", "q", "r"]})
print(len(ql.merge(qr, on="k")), len(ql.merge(qr, on="k", how="left")), len(ql.merg
```

A) 2 4 5

B) 4 4 5

C) 4 6 7

Why: Key 2 is repeated on both sides, so it makes 2 x 2 = 4 rows: that is the inner join. A left join adds the unmatched left keys 1 and 3 for 6. An outer join also adds right key 4 for 7.

2.5 Q2

You merge orders (20,302 rows) with customers using the default settings and get 19,563 rows. What happened?

A) The default inner join dropped 739 orders whose customer has no row in customers

B) pandas removed 739 duplicate orders during the merge

C) 739 customers had no orders, so they were left out

Why: The default is an inner join, which keeps only keys present in both tables. 739 orders have a customer_id with no row in customers, so they were dropped without any message.

2.5 Q3

After orders.merge(items, on="order_id"), the sum of order_total is $11.0 million, though the orders table sums to $4.0 million. Why?

A) Some orders had no matching item lines

B) Each order's total is repeated on every one of its item lines, so it is counted several times

C) The item lines include revenue that the orders table left out

> **Why:** The merge is one-to-many: each order row is repeated once per item line, and its order_total with it. Summing the repeated column counts an order with three lines three times.

2.5 Q4

What does validate="many_to_one" check in orders.merge(customers, on="customer_id", validate="many_to_one")?

A) That every order finds a matching customer

B) That customer_id is unique in orders

C) That customer_id is unique in customers

> **Why:** many_to_one allows repeated keys on the left and requires each key to appear at most once on the right. If customers had a duplicated id the merge would raise a MergeError instead of silently duplicating orders.

2.5 Q5

You have twelve monthly files with identical columns and want one table of all their rows. Which tool fits?

A) pd.concat(list_of_tables, ignore_index=True)

B) Eleven calls to merge on the date column

C) pd.merge_asof on the date column

> **Why:** concat stacks tables on top of each other, matching columns by name. Collect the twelve DataFrames in a list and call it once, with ignore_index=True to renumber the rows.

2.6 · PIVOT TABLES AND RESHAPING: SELF-CHECK

Q7

What does this print?

```
t = pd.DataFrame({"store": ["a", "a", "b"], "year": [2024, 2024, 2025], "amt": [10,
p = t.pivot_table(index="store", columns="year", values="amt")
print(p.loc["a", 2024], p.shape)
```

A) `20.0 (2, 2)`

B) `40 (2, 2)`

C) `20.0 (3, 2)`

D) A `ValueError: Index contains duplicate entries`

Why: No `aggfunc` was given, so the cells hold the mean: store `a` in 2024 is the mean of 10 and 30. Two stores by two years makes a 2 by 2 grid (two of its cells are empty).

B is wrong: 40 is the sum. The default aggregation of `pivot_table` is the mean.

C is wrong: The grid has one row per distinct store (2), not one per source row.

D is wrong: That error comes from `pivot()`, which cannot combine rows. `pivot_table()` combines them.

Q8

What does this print?

```
w = pd.DataFrame({"month": ["Jan", "Feb"], "Boise": [5, 6], "Bend": [3, 4], "Online
tidy = w.melt(id_vars="month", var_name="store", value_name="rev")
print(tidy.shape, tidy["rev"].sum())
```

`(6, 3) 21`

Why: Two rows times three store columns gives six rows. The tidy table has three columns: `month`, `store` and `rev`. Melting moves values without changing them, so they still add up to 21.

Q9

In `pd.crosstab(orders["channel"], orders["status"], normalize="index")`, the cell in row `online`, column `returned` is 0.069. What does it mean?

A) Of all returned orders, 6.9% were online

B) 6.9% of all orders are online orders that were returned

C) Online orders are 6.9% more likely to be returned than store orders

D) Of online orders, 6.9% were returned

> **Why:** `normalize="index"` divides each cell by its row total, so every row adds up to 1: the cell is a share of the online orders.
> **A** is wrong: That is the column percentage, `normalize="columns"` (77.9% on the course data).
> **B** is wrong: That is `normalize="all"`: a share of the grand total.
> **C** is wrong: The cell is a share, not a comparison. The comparison is 6.9% against 3.4% for stores.

Q10

A pivot that counts order lines by category and month, with `margins=True`, shows a grand total of 44,881. The source table has 45,490 rows. What is the most likely explanation?

A) `fill_value=0` removed the lines in empty cells

B) `margins=True` leaves the largest category out of the total

C) 609 lines have a missing category (or month), and rows with a missing key are dropped silently

D) `pivot_table` removes duplicate rows before counting

> **Why:** A pivot, like a groupby, leaves out rows whose row or column key is missing. On the course data 609 lines use discontinued SKUs that have no category after a left merge. Label them and pivot again.
> **A** is wrong: `fill_value` only changes how empty cells are displayed; it removes nothing.
> **B** is wrong: Margins are computed from all the rows that reached the pivot.
> **D** is wrong: `pivot_table` never removes duplicates; it aggregates them.

Q11

What does this print?

```
idx = pd.MultiIndex.from_product([["a", "b"], [2024, 2025]])
s = pd.Series([1, 2, 3, 4], index=idx)
u = s.unstack()
print(u.shape, u.loc["b", 2024])
```

`(2, 2) 3`

Why: `unstack()` takes the innermost index level (the year) and swings it up into the columns: two rows (`a`, `b`) by two columns (2024, 2025). The value for `b` in 2024 was the third one, 3.

Q12

`orders.pivot(index="store", columns="status", values="order_total")` **raises** `ValueError: Index contains duplicate entries, cannot reshape`**. What is the right response?**

A) Call `orders.drop_duplicates()` first, then `pivot()` again

B) Decide how to combine the rows and use `pivot_table(..., aggfunc="sum")`

C) Sort the table by store and status, then `pivot()` again

D) Convert `store` and `status` to the `category` dtype

Why: Thousands of orders share each store and status, and `pivot()` only rearranges: it cannot put several rows in one cell. `pivot_table()` with an explicit `aggfunc` says how to combine them.

A is wrong: The orders are not duplicate rows (each has its own id). And dropping on store and status would throw away almost all of them.

C is wrong: Sorting does not change how many rows land in each cell.

D is wrong: The dtype of the keys is not the problem; the number of rows per cell is.

2.6 · PREDICT, THEN RUN

P1

An empty cell.

```
t = pd.DataFrame({"pay": ["card", "cash", "card"], "ch": ["web", "shop", "shop"],
                  "n": [1, 1, 1]})
p = t.pivot_table(index="pay", columns="ch", values="n", aggfunc="count")
print(p.shape, p.isna().sum().sum(), p.loc["card", "web"])
```

```
(2, 2) 1 1.0
```

Why: Two payment methods by two channels is a 2 by 2 grid. Nobody paid cash on the web, so that one cell is NaN, and the NaN turns the whole `web` column into floats: the count for card on the web prints as 1.0. `fill_value=0` would make it an honest 0 and keep the counts whole.

P2

Labels after a pivot.

```
t = pd.DataFrame({"month": ["Jan", "Feb", "Mar", "Apr"], "store": ["a"] * 4,
                  "rev": [1, 2, 3, 4]})
print(t.pivot(index="month", columns="store", values="rev").index.tolist())
```

```
['Apr', 'Feb', 'Jan', 'Mar']
```

Why: Pivot methods sort their row and column labels, and text sorts alphabetically: Apr, Feb, Jan, Mar. No error is raised. Pivot on the month number, or `reindex` with the labels in calendar order.

P3

A margin of means.

```
t = pd.DataFrame({"store": ["a", "a", "a", "b"], "amt": [10, 10, 10, 50]})
p = t.pivot_table(index="store", values="amt", aggfunc="mean", margins=True)
print(p["amt"].tolist(), t.groupby("store")["amt"].mean().mean())
```

```
[10.0, 50.0, 20.0] 30.0
```

Why: The store means are 10 and 50. The margin `All` is the mean of all four rows, (10 + 10 + 10 + 50) / 4 = 20, not the mean of the two store means, 30. Store `a` has three times as many rows, so it counts three times as much.

2.6 · LAB REFERENCE SOLUTIONS

`LABS/MODULE-06/L2_6-PIVOT-TABLES/PAYMENT_RETURNS_SOLUTION.PY`

Question: Are orders paid one way returned more often than orders paid another way?

How the solution works: A count pivot (payment method by store, `fill_value=0`, margins) shows how many orders are behind each cell and reconciles to 20,302. The rate pivot is the mean of the boolean `returned` column times 100, with no `fill_value`, because the cash/Online cell has no orders and 0% would be a false statement. The two crosstabs normalize the same counts by row and by column. A last pivot by channel shows what the `All` column hides.

```
(20302, 12)
store           Asheville  Bend  Boise  Flagstaff  Online  Total
payment_method
card                  835  1050   1533       1021    7953  12392
cash                  293   370    577        374       0   1614
digital_wallet        181   203    325        209    3876   4794
gift_card             104   102    159        112    1025   1502
Total                1413  1725   2594       1716   12854  20302
corner equals the number of orders: True
store           Asheville  Bend  Boise  Flagstaff  Online  All
payment_method
card                  3.5   3.6    3.7        3.7     6.9  5.7
cash                  2.4   4.9    2.3        2.7     NaN  3.0
digital_wallet        5.5   3.9    3.7        1.9     7.1  6.4
gift_card             4.8   1.0    1.3        1.8     6.6  5.2
All                   3.6   3.8    3.2        3.1     6.9  5.6
gift cards in Bend: 1.0% of 102 orders = 1 returned order
status          cancelled  completed  returned
payment_method
card                  2.0       92.3       5.7
cash                  0.5       96.5       3.0
digital_wallet        2.4       91.1       6.4
gift_card             1.9       92.9       5.2
status          cancelled  completed  returned
payment_method
card                 61.5       61.0      61.9
cash                  2.0        8.3       4.2
digital_wallet       29.5       23.3      27.0
gift_card             7.1        7.4       6.8
Of digital-wallet orders, 6.4% were returned.
Of returned orders, 27.0% were paid by digital wallet.
channel         online  store  All
payment_method
card               6.9    3.6  5.7
cash               NaN    3.0  3.0
digital_wallet     7.1    3.7  6.4
gift_card          6.6    2.1  5.2
All                6.9    3.4  5.6
```

- The count grid's corner is 20,302. Cash/Online is 0 (a count), and NaN in the rate grid (no orders to have a rate).
- Rates in the `All` column: card 5.7%, digital wallet 6.4%, gift card 5.2%, cash 3.0%. It looks as if cash buyers return least.
- The lowest store cell is gift cards in Bend: 1.0% of 102 orders, which is 1 returned order. A cell built from 102 rows should not be read as a finding.

- 'Of digital-wallet orders, 6.4% were returned.' 'Of returned orders, 27.0% were paid by digital wallet.' Both true; different questions.
- By channel: online about 7% for every method (card 6.9, digital wallet 7.1, gift card 6.6) and in stores 2.1% to 3.7%. The differences in the `All` column are mostly the mix of channels: cash is used only in stores, where returns are rarer. Payment method itself explains little.
- A learner who adds `fill_value=0` to the rate pivot gets a 0.0 in cash/Online. Ask: 'Zero percent of what?'

- `LABS/MODULE-06/L2_6-PIVOT-TABLES/SURVEY_BY_AGE_SOLUTION.PY`

Question: Which survey question does each age group score lowest, and how many answers are behind each average?

How the solution works: `melt` turns the five satisfaction columns into two (`item`, `score`): 1,800 responses times 5 questions is 9,000 rows. Because each (age group, question) pair now has hundreds of rows, `pivot()` would raise the duplicate-entries error; `pivot_table` combines them, once with `mean` and once with `count`. `idxmin(axis=1)` reads across each row. `stack()` pushes the question labels back into the index, and `rename` plus `reset_index` makes a tidy table of 30 rows.

```
(1800, 12)
tidy rows: 9000 = 1800 responses x 5 questions
item       sat_overall  sat_price  sat_selection  sat_staff  sat_website
age_group
18-24             3.95       2.80           4.06       4.05         3.69
25-34             3.88       3.32           3.99       4.02         3.67
35-44             3.99       3.38           4.07       3.94         3.67
45-54             3.95       3.38           4.08       4.01         3.66
55-64             3.85       3.44           3.99       4.04         3.72
65+               3.92       3.33           4.05       4.05         3.04
item       sat_overall  sat_price  sat_selection  sat_staff  sat_website
age_group
18-24              189        186            183        132          144
25-34              272        273            272        183          208
35-44              312        309            305        206          233
45-54              327        332            330        232          242
55-64              270        273            273        180          210
65+                377        380            384        262          291
repeated (age_group, item) pairs: 8970 -> pivot() cannot be used, pivot_table() must c
{'18-24': 'sat_price', '25-34': 'sat_price', '35-44': 'sat_price', '45-54': 'sat_price
saved and read back: (30, 3) ['age_group', 'item', 'mean_score']
age_group        item  mean_score
    18-24   sat_price        2.80
      65+ sat_website        3.04
```

- 9,000 tidy rows; both grids are 6 by 5.

- Every age group scores price lowest, except 65+, whose lowest is the website (3.04). The 18-24 group's price score, 2.80, is far below every other group's (3.32 to 3.44).
- The count grid shows fewer answers for `sat_staff` and `sat_website` in every age group (for example 132 and 144 of 192 in the 18-24 group): online shoppers skip the staff question and store shoppers the website question.
- The saved file has shape `(30, 3)`: `age_group`, `item`, `mean_score`. Its two smallest rows are 18-24 / sat_price (2.80) and 65+ / sat_website (3.04).
- In this pandas `stack()` keeps empty cells; there are none here, so 30 rows either way.
- `melt` without `value_vars` also melts `customer_id`, `recommend` and the text columns into `score`: a 'mean' that will not compute. Ask what identifies a row and what is being measured.

- `LABS/MODULE-06/L2_6-PIVOT-TABLES/WEATHER_GRID_SOLUTION.PY`

Question: On how many days was each city the warmest, what is the average high by month, and what goes wrong when months are names?

How the solution works: `store_weather.csv` has exactly one row per (date, city), so plain `pivot()` rearranges it into 731 rows by 4 columns with no empty cell. `idxmax(axis=1)` names the warmest column on each date. For the monthly grid several days land in each cell, so `pivot_table` with `aggfunc="mean"` is needed. Pivoting on `%b` month names shows the alphabetical sort; `reindex` with the twelve names restores calendar order. `melt` returns to 2,924 rows.

```
(2924, 5) | repeated (date, city) pairs: 0
wide: (731, 4) | empty cells: 0
city        Asheville  Bend  Boise  Flagstaff
date
2024-01-01         49    37     37         38
2024-01-02         50    39     44         39
2024-01-03         52    39     30         47
{'Asheville': 338, 'Boise': 227, 'Flagstaff': 106, 'Bend': 60} | total: 731
city   Asheville  Bend  Boise  Flagstaff
month
1           47.9  37.3   29.7       36.2
4           59.3  56.2   59.0       58.6
7           82.8  82.2   92.0       83.2
10          68.4  60.7   66.9       62.3
as pivoted: ['Apr', 'Aug', 'Dec', 'Feb', 'Jan', 'Jul', 'Jun', 'Mar', 'May', 'Nov', 'Oct
reindexed : ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov
city        Asheville  Bend  Boise  Flagstaff
month_name
Jan              47.9  37.3   29.7       36.2
Jul              82.8  82.2   92.0       83.2
tidy rows: 2924 | same as the file: True
```

- Wide grid `(731, 4)`, 0 empty cells (the weather table is complete, unlike daily sales).
- Warmest city: Asheville on 338 days, Boise 227, Flagstaff 106, Bend 60; the counts add up to 731. On a tied day `idxmax` names the first column, so ties favor Asheville.
- Average highs: January 29.7 (Boise) to 47.9 (Asheville); July 82.2 (Bend) to 92.0 (Boise). Boise has the coldest winters and the hottest summers of the four.
- As pivoted, the month names read Apr, Aug, Dec, Feb, ...; after `reindex` they read Jan to Dec.
- The melt returns 2,924 rows, the size of the file.

2.6 · THE LESSON PAGE'S QUICK QUIZ

These are the five questions at the end of the lesson page (three options each), with the lesson's own explanation.

2.6 Q1

A colleague writes orders.pivot_table(index="store", columns="channel", values="order_total") and titles the result "Revenue by store and channel". What is wrong?

A) The call fails, because index and columns must be lists

B) The cells are average order values, because the default aggfunc is the mean

C) The cells are counts of orders, because nothing was summed

> **Why:** With no aggfunc, pivot_table uses the mean. The cells are average order values, so calling the table revenue is wrong. Writing aggfunc='sum' gives revenue.

2.6 Q2

What is in the Bend row, 2024 column of the table this code prints?

```
toy = pd.DataFrame({"store": ["Boise", "Boise", "Bend"],
                    "year": [2024, 2025, 2025],
                    "sales": [10, 20, 30]})
print(toy.pivot_table(index="store", columns="year", values="sales", aggfunc="sum"))
```

A) NaN, because no row has that store and year

B) 0, because the sum of no rows is zero

C) Nothing: the code raises an error because the grid is incomplete

> **Why:** No row has store Bend and year 2024, so there is nothing to sum and the cell is NaN. Adding fill_value=0 would show 0 instead.

2.6 Q3

df.pivot(index="date", columns="store", values="revenue") raises ValueError: Index contains duplicate entries, cannot reshape. What does that tell you?

A) Some revenue values are missing

B) The date column is text and must be converted first

C) At least one date and store pair appears on more than one row

Why: pivot only rearranges, so each date and store pair may appear once. Duplicates mean either the data has repeated rows to investigate, or you need pivot_table with an aggfunc to combine them.

2.6 Q4

A wide table has 12 rows and the columns Month, Boise, Bend, Flagstaff, Asheville, Online. How many rows does table.melt(id_vars="Month") have?

A) 60

B) 72

C) 12

Why: Month is kept as the identifier and the other 5 columns are stacked, so each of the 12 months appears once per store: 12 x 5 = 60 rows, with columns Month, variable and value.

2.6 Q5

A pivot of average order value by store with margins=True shows store means of 175.10, 172.47, 177.21, 182.39 and 203.20, and an All value of 193.17. The plain average of the five store means is 182.07. Why is All different?

A) It is a rounding error that grows with the number of rows

B) The margin leaves out the rows with missing values, so it uses fewer orders

C) All is the mean of every order, so the store with the most orders counts the most

Why: Margins apply the aggfunc to all the underlying rows. The store with the 203.20 mean has far more orders than the others, so it pulls the overall mean toward itself.

WARM-UP ANSWERS

1. In what order do you repair a damaged table? **Answer:** Structure (headers, stray rows, duplicates), then text, numbers, dates, and last the missing values.

2. What does `pd.to_numeric(s, errors="coerce")` do with the value `free`, and why is that useful? **Answer:** It turns it into NaN instead of raising an error, so it works as a detector: values that became NaN but were not empty are the ones to look at.

3. `orders.dropna()` kept 2,085 of 20,302 rows. Why, and what should you write? **Answer:** It drops every row with a gap in any column, and most orders have no promo code. Name the columns that matter: `dropna(subset=[...])`.

4. You need each order's total next to its store's average order. Which groupby tool gives one value per original row? **Answer:** `transform` (for example `orders.groupby("store")["order_total"].transform("mean")`).

5. Grouped by `promo_code`, the group sizes add up to 2,148, but there are 20,302 orders. What happened? **Answer:** Rows whose key is missing are dropped by default. `dropna=False` keeps them as a group.

EXIT TICKET ANSWERS

1. You merge `orders` (20,302 rows) with `customers` using the default settings and get 19,563 rows. What happened, and what would you write instead? **Answer:** The default inner join dropped the 739 orders whose `customer_id` has no row in `customers`. Write `orders.merge(customers, on="customer_id", how="left", validate="many_to_one")` and count the rows.

2. After `orders.merge(items, on="order_id")`, the sum of `order_total` is $11.0 million instead of $4.0 million. Why? **Answer:** The result has one row per order line, so each order's total is repeated on every one of its lines and counted several times. Sum order-level columns on the orders table (or after `drop_duplicates("order_id")`).

3. A count pivot with `margins=True` has a grand total of 44,881, but the source table has 45,490 rows. What does that tell you, and what do you do? **Answer:** 609 rows have a missing value in the row or column key and were dropped silently. Find them, give them an honest label (for example `fillna("Discontinued")`), and pivot again until the corner matches the source.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 answered `2 3 4`.** They count keys, not rows: key 3 appears twice on the right, so it makes two rows. Have them write out the rows of the inner join by hand.
- **Q3 answered `150 150`.** They do not yet see that the merged table repeats order-level values. Ask what one row of `m` is, then have them print `m`.

- **Q4 answered B ('customers who have never placed an order').** Left and right are swapped in their head. `left_only` means the key exists only in the table before `.merge`: here, orders.
- **Q2 answered A (the default merge).** They have the mechanics but not the habit. Ask how many rows that call returns on the course data (19,563) and where the other 739 went.
- **Q7 answered B (`40 (2, 2)`).** The spreadsheet habit: a pivot sums. In pandas the default is the mean.
- **Q8 answered `(2, 4) 21` or `(6, 2) 21`.** `melt` is still fuzzy: each store column becomes rows, and the result has the id column plus two new ones. Melt a three-row table by hand.
- **Q9 answered A ('of all returned orders, 6.9% were online').** Row and column percentages are confused. Ask which total the cell was divided by: with `normalize="index"`, its row.
- **Exit ticket 3 answered 'the margins are wrong'.** They trust the source less than the grid. The margin is computed correctly from the rows that reached the pivot; the question is which rows did not.
- **Lab `customer_profiles`: 3,792 rows after the merge.** The default inner join. Ask what the row count should have been, and what `how=` says.
- **Lab `payment_returns`: 0.0 in the cash/Online rate cell.** `fill_value=0` was copied from the count pivot. Ask: zero percent of what?

INSTRUCTOR KIT · ANSWER KEY

Session 7: Time Series and Windows

Answers with reasons for the twelve self-check items (all four options shown, with why each wrong one is wrong), the Predict, then run boxes, reference notes and real output for all four labs, the lesson pages' own Quick Quiz answers, the warm-up and exit ticket, and a table of common wrong answers with what each one tells you about the learner.

Session 7 | Lessons 2.7-2.8 | 12 self-check items | Confidential

NUMBERING MATCHES THE WORKBOOK

Self-check items carry the same numbers as the participant workbook, and the wording is generated from one source, so question and option text match exactly. Every output block is the real output of the code above it or of the named `*_solution.py`, run on the course data with pandas 3.0. `python3 labs/check_labs.py module-07` runs all eight lab files.

2.7 · TIME SERIES: SELF-CHECK

Q1

After `raw = pd.read_csv("data/daily_sales.csv")`, the line `raw["date"].dt.month` raises `AttributeError: Can only use .dt accessor with datetimelike values`. What is the cause?

A) Some dates are missing from the file

B) The column was loaded as text; pass `parse_dates=["date"]` or convert with `pd.to_datetime`

C) `.dt` only works on the index, not on a column

D) `month` is a method and needs parentheses

Why: With no instructions `read_csv` keeps dates as text, and text has no `.dt` accessor. Parse at load time, then check `df.dtypes`.

A is wrong: Missing days do not change the dtype of the days that are present.

C is wrong: `.dt` is for columns (Series). A DatetimeIndex has the same properties directly, without `.dt`.

D is wrong: `.dt.month` is a property. The error is raised at `.dt`, before `month` is even looked up.

Q2

What does this print?

```
s = pd.Series(range(10),
              index=pd.date_range("2025-01-28", periods=10, freq="D"))
print(len(s.loc["2025-01"]), len(s.loc["2025-01-30":"2025-02-02"]))
```

`4 4`

> **Why:** `"2025-01"` selects every row in January: the 28th, 29th, 30th and 31st. The string range includes **both** ends: January 30 and 31, February 1 and 2.

Q3

`wide` has one row per date and 727 rows between 2024-01-01 and 2025-12-31, a span of 731 days. What does `wide.asfreq("D")` return?

A) 727 rows; the four gaps are interpolated

B) 731 rows; the four new rows are filled with 0

C) An error, because the index has missing dates

D) 731 rows; the four new rows hold `NaN`

> **Why:** `asfreq("D")` gives the table one row for every calendar day. Rows that did not exist appear as `NaN`: pandas shows you the gap and leaves the decision to you.
> **A** is wrong: pandas never estimates values unless you call `interpolate()` or `ffill()` yourself.
> **B** is wrong: Zero is a claim (the store sold nothing). pandas does not make it for you; you fill closed days with 0 on purpose.
> **C** is wrong: Missing dates are exactly what `asfreq` is for. It needs a DatetimeIndex, not a complete one.

Q4

What does this print?

```
s = pd.Series([10.0, np.nan, 30.0],
              index=pd.to_datetime(["2025-05-01", "2025-05-02", "2025-05-03"]))
print(s.resample("ME").agg(["sum", "count"]).values.tolist())
```

A) `[[40.0, 2.0]]`

B) `[[nan, 3.0]]`

C) `[[40.0, 3.0]]`

D) `[[nan, 2.0]]`

Why: `sum` skips the missing value and adds 10 + 30. `count` counts only real values, so it says 2, which is how you find out that the total of 40 is built from two days and not three.
B is wrong: pandas aggregations skip `NaN` by default, and `count` never counts a missing value.
C is wrong: `count` is the number of non-missing values, not the number of rows in the bucket.
D is wrong: The count is right, but the sum is not `NaN`: `sum` skips missing values (that is the danger).

Q5

Bend has no rows at all for three days in August 2024. On that rows-only Series, what does `bend_rows.diff(7)` compare the first Saturday after the gap with?

A) The previous Saturday, seven days earlier

B) Nothing: the result is `NaN` because the window crosses a gap

C) The value seven rows earlier, which is ten days back and not a Saturday

D) Nothing: pandas raises an error on an irregular index

Why: `shift` and `diff` count rows, not days. With three rows missing, seven rows back is ten days back. No error and no warning: complete the calendar with `asfreq("D")` first.
A is wrong: That is what you want, and what you get only on a complete calendar.
B is wrong: `NaN` is what the **completed** calendar gives for the days after a gap, because a week earlier is unknown there. Rows-only data has no `NaN` to pass on.
D is wrong: pandas is happy to shift any index. The mistake is silent.

Q6

What does this print?

```
m = pd.Series([200, 220, 264],
              index=pd.date_range("2025-01-31", periods=3, freq="ME"))
print((m.pct_change() * 100).round(1).tolist())
```

`[nan, 10.0, 20.0]`

Why: `pct_change()` is the change from the previous row divided by the previous row: 20/200 = 10% and 44/220 = 20%. January has no previous row, so it is `NaN`.

2.7 · PREDICT, THEN RUN: ANSWERS

Predict 2.7.1

How many rows does each selection return? (Two numbers.)

```
days = pd.Series(1, index=pd.date_range("2024-01-01", "2024-12-31", freq="D"))
print(len(days.loc["2024-02"]), len(days.loc["2024-12-30":"2024-12-31"]))
```

```
29 2
```

Why: 2024 is a leap year, so February has 29 days. The string range includes both ends: December 30 and December 31.

Predict 2.7.2

One Sunday is missing from this small table. What are the two weekly sums, and the two counts?

```
s = pd.Series([5, 5, 5, 5, 5, 5, 9, 9, 9, 9, 9, 9, 9],
              index=pd.date_range("2025-03-03", periods=14, freq="D").delete(6))
print(s.resample("W").agg(["sum", "count"]).values.tolist())
```

```
[[30, 6], [63, 7]]
```

Why: March 3, 2025 is a Monday, and the first Sunday (March 9) was deleted. The first week has six rows of 5 (sum 30, count 6); the second has seven rows of 9 (sum 63, count 7). The sum alone would suggest the second week was twice as good; per recorded day it went from 5 to 9.

Predict 2.7.3

What does `diff()` print for the last row, and why is that surprising?

```
s = pd.Series([100, 104, 120],
              index=pd.to_datetime(["2025-06-01", "2025-06-02", "2025-06-06"]))
print(s.diff().tolist())
print(s.asfreq("D").diff().tolist())
```

```
[nan, 4.0, 16.0]
[nan, 4.0, nan, nan, nan, nan]
```

Why: On the rows-only Series, `diff()` subtracts the previous **row**: 120 - 104 = 16, although that row is four days earlier. After `asfreq("D")` the three missing days exist as `NaN`, and every difference that touches one of them is `NaN`: the honest answer to "how much did it change since yesterday?"

2.7 · LAB REFERENCE SOLUTIONS

LAB 2.7-A · AN HONEST WEEKLY CALENDAR: LABS/MODULE-07/L2_7-TIME-SERIES/UNITS_CALENDAR_SOLUTION.PY

What it practices: the order of operations on a new column (`units`) and a new bucket (weeks). `units` has no blank values, so after zeroing the four closed days the only unknowns are the five failed-export days (Bend 3, Flagstaff 2).

Step 1. The calendar has 731 days and the wide table 727 rows. `calendar.difference(units.index)` is the four dates on which no store has a row. The reverse, `units.index.difference(calendar)`, is empty and is the usual slip.

Step 4. Two weeks have fewer than 7 days, for two different reasons. The week ending 2024-08-18 has 4 days because the export failed on August 12 to 14. The week ending 2026-01-04 has 3 days because the data stops on Wednesday, December 31: a `"W"` bucket runs Monday to Sunday and is labeled with its Sunday, even when that Sunday is past the end of the data.

Step 5. Take the maximum only over complete weeks. Here the best week overall is also complete (ends 2025-08-31, 1,767 units: Labor Day weekend), so a learner who skips the filter gets the same answer by luck. Ask what the code would have done if the best week had been a partial one.

Step 6. -96 and +80 are real. -542 is the 4-day week compared with a 7-day week, and +443 is the next full week compared with the 4-day week. One incomplete bucket spoils two changes. An honest table shows `NaN` or a footnote for both.

```
rows: 727 | index type: DatetimeIndex
days in the calendar: 731
closed on: ['2024-11-28', '2024-12-25', '2025-11-27', '2025-12-25']
missing days per store after asfreq:
store
Asheville    4
Bend         7
Boise        4
Flagstaff    6
still unknown after zeroing the closed days:
store
Asheville    0
Bend         3
Boise        0
Flagstaff    2
Bend weeks built from fewer than 7 days:
              sum  count
date
2024-08-18  831.0      4
2026-01-04  373.0      3
best complete week: ends 2025-08-31 (Sunday), 1767 units
week-over-week change in units, August 2024:
date
2024-08-04    -96.0
2024-08-11     80.0
2024-08-18   -542.0
2024-08-25    443.0
Freq: W-SUN
The -542 and the +443 are the gap, not the customers: the week ending 2024-08-18 has or
```

LAB 2.7-B · IS THE ONLINE RETURN RATE GETTING WORSE?: LABS/MODULE-07/L2_7-TIME-SERIES/RETURNS_TREND_SOLUTION.PY

What it practices: `resample(..., on=)` on a table whose dates are a column; the mean of a True/False column as a rate; `diff(12)` for year over year in percentage points; and reading a noisy series without inventing a story.

The answer is "no trend". The yearly rate went from 7.11% to 6.76%. Seven months of 2025 were worse than a year earlier and five were better, with changes between -2.80 and +2.84 points. A month has only 24 to 51 returns, so a handful of orders moves the monthly rate by a point or more.

Accept any conclusion that reports the yearly rates and says the monthly differences are noise. **Push back** on "April 2025 was much worse" (+2.84 points, on 46 returns) or "returns improved in May and June" presented as findings: ask how many returns that difference is.

Gaps: `orders.csv` has orders on every one of the 731 days, so every monthly `count` is a full month. The `count` column here is the number of orders, which is the denominator of the rate.

```
online orders: 12854 | returned: 890
            rate_pct  sum  count
order_date
2024-01-31      5.87   24    409
2024-02-29      7.07   28    396
2024-03-31      7.19   33    459
2024-04-30      5.92   27    456
2024-05-31      9.06   50    552
2024-06-30      8.71   42    482
2024-07-31      5.76   28    486
2024-08-31      7.38   40    542
2024-09-30      6.21   29    467
2024-10-31      7.25   35    483
2024-11-30      8.12   41    505
2024-12-31      6.47   46    711
2025-01-31      7.61   36    473
2025-02-28      7.11   33    464
2025-03-31      5.74   31    540
2025-04-30      8.76   46    525
2025-05-31      6.26   38    607
2025-06-30      6.05   36    595
2025-07-31      6.28   37    589
2025-08-31      5.87   37    630
2025-09-30      6.34   35    552
2025-10-31      8.10   46    568
2025-11-30      6.90   41    594
2025-12-31      6.63   51    769
highest: 2024-05 at 9.06% (50 of 552 orders)
lowest:  2025-03 at 5.74% (31 of 540 orders)
by quarter (%):
order_date
2024-03-31    6.72
2024-06-30    7.99
2024-09-30    6.49
2024-12-31    7.18
2025-03-31    6.77
2025-06-30    6.95
2025-09-30    6.15
2025-12-31    7.15
Freq: QE-DEC
by year (%):
order_date
2024-12-31    7.11
2025-12-31    6.76
Freq: YE-DEC
2025 minus the same month of 2024, in percentage points:
order_date
2025-01-31    1.74
```

```
2025-02-28    0.04
2025-03-31   -1.45
2025-04-30    2.84
2025-05-31   -2.80
2025-06-30   -2.66
2025-07-31    0.52
2025-08-31   -1.51
2025-09-30    0.13
2025-10-31    0.85
2025-11-30   -1.22
2025-12-31    0.16
Freq: ME
months of 2025 with a higher rate than a year earlier: 7 of 12
Conclusion: no trend. Monthly rates bounce between about 5.7% and 9.1% because each mor
24 to 51 returns. 2025 was worse in 7 months and better in 5, and the yearly rate went
from 7.11% to 6.76%. 'Returns feel worse' is not supported by the data.
```

2.7 · THE LESSON PAGE'S OWN QUICK QUIZ

These five questions are on the lesson page (three options each). Letters and reasons are read from the lesson.

#	QUESTION (SHORT FORM)	ANSWER	REASON
Q1	Rows returned by `s.loc["2025-03-30":"2025-04-02"]` on a complete daily index	C	Label slices include both ends: March 30, 31, April 1, 2. Four rows.
Q2	`resample("ME").sum()` for a month with three `NaN` days	A	Sums skip missing values, so the total is built from the days that exist, with no warning. Print `count` beside it.
Q3	`q3.diff().iloc[-1]` when the previous row is three days earlier	B	`diff` works on row positions: 150 minus 110 is 40.0, whatever the dates are.
Q4	Growth against the same month a year earlier, monthly rows	A	The same month last year is 12 rows back: `monthly.pct_change(12)`.
Q5	`NaN` on December 25 (closed) and on August 12 (export failed)	C	A closed day is a real zero; a failed export is unknown. Fill the first with 0, leave or openly estimate the second.

2.8 · WINDOW FUNCTIONS: SELF-CHECK

Q7

What does this print?

```
s = pd.Series([4, 8, 6, 10])
print(s.rolling(3, min_periods=2).mean().tolist())
```

A) `[nan, nan, 6.0, 8.0]`

B) `[4.0, 6.0, 6.0, 8.0]`

C) `[nan, 6.0, 6.0, 8.0]`

D) `[6.0, 6.0, 8.0, nan]`

> **Why:** The window is this row and the two before it, and it answers as soon as 2 values are present. Row 1 has only one value (`NaN`); row 2 is the mean of 4 and 8; then the full windows give 6 and 8.
> **A** is wrong: That is the strict default, where all 3 values must be present.
> **B** is wrong: That is `min_periods=1` , where one value is enough.
> **D** is wrong: A trailing window looks backward, so the gap is at the start, not the end.

Q8

What does this print?

```
s = pd.Series([5, 3, 9, 4, 9])
print((s == s.cummax()).tolist())
```

`[True, False, True, False, True]`

> **Why:** `cummax()` is the best value so far: 5, 5, 9, 9, 9. A row equals it when it sets or ties the record. The last 9 ties the record, so it is `True` as well.

Q9

A long table has columns `store`, `date`, `revenue`, sorted by date and then store. Which code adds each store's own 7-day average, lined up with the right rows?

A) `df["avg"] = df["revenue"].rolling(7).mean()`

B) `df["avg"] = df.groupby("store")["revenue"].rolling(7).mean()`

C) `df["avg"] = df.groupby("date")["revenue"].transform(lambda s: s.rolling(7).mean())`

D) Sort by store and date, then `df["avg"] = df.groupby("store")["revenue"].transform(lambda s: s.rolling(7).mean())`

> **Why:** Sort, group, window. `transform` returns one value per row with the original index, so the new column lines up.
> **A** is wrong: Seven consecutive rows are different stores on neighboring days: a number with no meaning.
> **B** is wrong: The result has a two-level index (store, row label), and assigning it back raises `TypeError: incompatible index of inserted column with frame index`.
> **C** is wrong: Grouping by date puts four stores of one day in each group: every result is `NaN`, and it is the wrong grouping anyway.

Q10

What does this print?

```
s = pd.Series([30, 10, 30, 20])
print(s.rank(ascending=False, method="dense").tolist())
```

A) `[1.0, 3.0, 1.0, 2.0]`

B) `[1.5, 4.0, 1.5, 3.0]`

C) `[1.0, 4.0, 1.0, 3.0]`

D) `[3.0, 1.0, 3.0, 2.0]`

> **Why:** With `ascending=False` the largest value gets rank 1. The two 30s tie for first. `"dense"` leaves no gap after a tie, so 20 is rank 2 and 10 is rank 3.
> **B** is wrong: That is the default `method="average"`: the tied values share positions 1 and 2.
> **C** is wrong: That is `method="min"` (sports ranking): after two firsts the next rank is 3.
> **D** is wrong: That is a dense rank with the default `ascending=True`, where the smallest value is rank 1.

Q11

To flag unusual days, an analyst computes `lift = rev / rev.rolling(28).mean()` and flags days where `lift >= 1.5`. What is wrong with the baseline?

A) A 28-day window is too long to react to a sale

B) Each day is inside its own baseline, so every spike is understated; use `rev.shift(1).rolling(28).mean()`

C) A rolling mean cannot be used as a divisor

D) The first 27 rows are `NaN`, so the division fails

> **Why:** The window ends on the day being judged, so part of the spike is already in the average it is compared with. Shift first, then roll: the baseline then holds only what was known that morning.
> **A** is wrong: Window length is a judgment call, not an error. The leak is the problem at any length.
> **C** is wrong: Dividing two aligned Series is ordinary pandas.
> **D** is wrong: `NaN` rows give `NaN` lifts and are simply not flagged; nothing fails.

Q12

What does this print?

```
s = pd.Series([1, 2, 3, 4, 5])
print(s.shift(1).rolling(2).sum().tolist())
```

`[nan, nan, 3.0, 5.0, 7.0]`

> **Why:** `shift(1)` gives `NaN, 1, 2, 3, 4`. A 2-row sum needs two real values: the first two rows have none or one, then 1+2, 2+3, 3+4. Each result uses only the rows **before** its own.

2.8 · PREDICT, THEN RUN: ANSWERS

Predict 2.8.1

How many `NaN` values, and what is the last number?

```
s = pd.Series([2, 4, 6, 8, 10, 12])
r = s.rolling(4).mean()
print(int(r.isna().sum()), r.iloc[-1])
```

```
3 9.0
```

Why: A window of 4 is full for the first time on the 4th row, so the first 3 results are `NaN`. The last window holds 6, 8, 10 and 12: mean 9.0.

Predict 2.8.2

Two groups stacked in one column. What does each line print?

```
t = pd.DataFrame({"g": ["a", "a", "a", "b", "b", "b"],
                  "v": [1, 2, 3, 10, 20, 30]})
print(t["v"].cumsum().tolist())
print(t.groupby("g")["v"].cumsum().tolist())
```

```
[1, 3, 6, 16, 36, 66]
[1, 3, 6, 10, 30, 60]
```

Why: The plain `cumsum` runs straight through both groups: 6 + 10 = 16 carries group a's total into group b. The grouped version restarts at the first row of each group. This is the long-table trap in its smallest form, and it is how year to date works.

Predict 2.8.3

Which of the three columns could you have computed on the morning of each row? Predict the last row of each.

```
s = pd.Series([10, 10, 10, 40, 10])
print(s.rolling(3).mean().round(1).tolist())
print(s.rolling(3, center=True).mean().round(1).tolist())
print(s.shift(1).rolling(3).mean().round(1).tolist())
```

```
[nan, nan, 10.0, 20.0, 20.0]
[nan, 10.0, 20.0, 20.0, nan]
[nan, nan, nan, 10.0, 20.0]
```

Why: Only the third. The trailing mean already contains the 40 on the day of the spike (20.0). The centered mean sees the spike one row early (20.0 on the third row) and has no value on the last row. The shifted version is still 10.0 on the day of the spike and only reacts the day after.

2.8 · LAB REFERENCE SOLUTIONS

LAB 2.8-A · FOOT TRAFFIC: WHEN DID FLAGSTAFF PASS BEND?: LABS/MODULE-07/L2_8-WINDOW-FUNCTIONS/FOOT_TRAFFIC_SOLUTION.PY

What it practices: `rolling().mean()` and its `NaN` edge, `min_periods`, comparing two smoothed series, `cummax()` for records, and `rolling().sum()` with `idxmax()` for a best stretch. Everything is on the wide calendar, so no grouping is needed.

Step 2. With `min_periods=21` the first 28-day value appears on row 21 (2024-01-21), and the five unknown visitor days (Bend 3, Flagstaff 2) cost nothing: each affected window still has 25 or 26 known days.

Step 3 has two defensible answers and one better one. Flagstaff's 28-day average is first above Bend's on 2024-12-25, but the lead changed hands several times after that. Bend was last ahead on 2025-06-03, so the answer to "since when has Flagstaff been ahead?" is 2025-06-04. Flagstaff led on 324 days in all. The `known` mask in the solution keeps rows where both averages exist; without it, the `NaN` rows at the start count as "not ahead", which happens to give the same date here.

Step 4. 18 record days. A tie with the previous record would also be `True` (see self-check Q8); there are none in this column.

Step 5. The label that `idxmax()` returns is the **last** day of the 7; subtract 6 days for the start. Three of the four stores peak around Labor Day or Memorial Day sale weekends; Flagstaff peaks in the week before Christmas 2025, which fits its growth.

```
(731, 4) | unknown store-days: 5
Boise, 7-day average, first 8 days:
date
2024-01-01      NaN
2024-01-02      NaN
2024-01-03      NaN
2024-01-04      NaN
2024-01-05      NaN
2024-01-06      NaN
2024-01-07    380.4
2024-01-08    366.3
Freq: D
28-day average visitors:
store       Asheville   Bend  Boise  Flagstaff
date
2024-06-30      319.0  388.0  530.0      350.0
2024-12-31      304.0  358.0  523.0      367.0
2025-06-30      300.0  384.0  577.0      412.0
2025-12-31      292.0  367.0  575.0      420.0
Flagstaff first ahead of Bend on: 2024-12-25
days with Flagstaff ahead: 324
Bend last ahead on: 2025-06-03 | Flagstaff ahead for good since: 2025-06-04
Flagstaff record days: 18
date
2024-08-31    762.0
2025-05-24    816.0
2025-08-30    853.0
Asheville  2024-05-24 to 2024-05-30  2,809 visitors
Bend       2025-08-29 to 2025-09-04  3,431 visitors
Boise      2025-08-27 to 2025-09-02  4,812 visitors
Flagstaff  2025-12-15 to 2025-12-21  3,492 visitors
```

LAB 2.8-B · THE ONLINE SHOP: MILESTONES, RANKS AND A FAIR BASELINE: LABS/MODULE-07/L2_8-WINDOW-FUNCTIONS/ONLINE_MILESTONES_SOLUTION.PY

What it practices: a running total that restarts per group, ranking within groups, `cummax()`, and the leakage idea from the project on a new series.

Step 1. 2024 passed $1,000,000 on December 11; 2025 on October 16, about eight weeks earlier in the year. Full-year totals: 1,077,800.75 and 1,280,183.70 (+18.8%).

Step 2. December is rank 1 and February rank 12 in both years. In between, the order changes: May was 2nd in 2024 and 5th in 2025. `rank` returns floats; the solution converts to `int` only because there are no ties.

Step 4. The record day, 2025-06-24, is a Tuesday with $8,960.15. Against the mean of the four Tuesdays before it (4,066.72) it is 2.20 times normal. With the day inside its own 4-Tuesday window the "baseline" rises to 5,583.56 and the ratio falls to 1.60. Across two years, 25 days reach 2 times an honest baseline and only 2 reach 2 times a leaky one: the leak hides exactly the days the baseline exists to find.

A question worth asking: why was that Tuesday so large? `done[done["order_date"] == "2025-06-24"]` shows one order of $4,184.90, one of the twelve bulk outfitter orders described in `data/README.md`. A real order, not an error.

```
days: 731 | days with no revenue: 0 | total: 2357984.45
2024: passed $1,000,000 on 2024-12-11 | full year 1,077,800.75
2025: passed $1,000,000 on 2025-10-16 | full year 1,280,183.70
             revenue  rank_in_year
order_date
2024-01      73567.0            11
2024-02      70712.0            12
2024-03      80882.0            10
2024-04      86564.0             8
2024-05      99581.0             2
2024-06      97009.0             3
2024-07      90041.0             6
2024-08      92585.0             4
2024-09      90564.0             5
2024-10      82443.0             9
2024-11      88500.0             7
2024-12     125352.0             1
2025-01      86269.0            11
2025-02      86043.0            12
2025-03      96183.0             9
2025-04      92205.0            10
2025-05     114708.0             5
2025-06     127372.0             2
2025-07     116606.0             4
2025-08     125729.0             3
2025-09     101342.0             7
2025-10     105098.0             6
2025-11      98475.0             8
2025-12     130155.0             1
record days: 13
order_date
2025-05-26    8115.83
2025-06-11    8925.85
2025-06-24    8960.15
2025-06-24 (Tuesday): revenue 8,960.15
  mean of the 4 Tuesdays BEFORE it: 4,066.72 -> 2.20 times normal
  without shift(1):                 5,583.56 -> 1.60 times 'normal' (the day is in its
days at least 2 times their baseline: honest 25 | leaky 2
```

2.8 · THE LESSON PAGE'S OWN QUICK QUIZ

These five questions are on the lesson page (three options each). Letters and reasons are read from the lesson.

#	QUESTION (SHORT FORM)	ANSWER	REASON
Q1	`NaN` count in `s.rolling(7).mean()` for 100 complete daily values	B	The trailing window is full for the first time on row 7, so 6 `NaN`, all at the start.

#	QUESTION (SHORT FORM)	ANSWER	REASON
Q2	`pd.Series([1, 2, 3, 4]).rolling(2).sum().tolist()`	C	`[nan, 3.0, 5.0, 7.0]`: each value is the row plus the one before; the first row has none.
Q3	`df["revenue"].rolling(7).mean()` on a table sorted by date, then store	A	Seven consecutive rows mix different stores. Sort by store and date, then `groupby` with `transform`.
Q4	A feature for predicting today's revenue, safe from leakage	C	`rev.shift(1).rolling(7).mean()`: the window covers yesterday and the six days before.
Q5	`pd.Series([50, 80, 80, 95]).rank(ascending=False, method="min")`	A	`[4.0, 2.0, 2.0, 1.0]`: 95 is first, the two 80s both get 2, the next rank is 4.

WARM-UP ANSWERS

1. What is the default `how=` of `merge`, and what happens to a row whose key has no match? **Answer:** `inner`. The row is dropped with no warning. In Lesson 2.5 that lost 739 orders whose customer has no row in `customers.csv`. Use `how="left"` when you are adding columns to a main table.
2. After a merge the result has **more** rows than the left table. What does that tell you? **Answer:** The key repeats on the other side (one-to-many or many-to-many), so left rows were duplicated. Count rows before and after, and state the relationship with `validate=`.
3. What does `pivot_table` compute in each cell if you leave out `aggfunc`? **Answer:** The mean. A table titled "revenue" built that way shows average order values. Always write `aggfunc`.
4. `df.pivot(index="date", columns="store", values="revenue")` raises `ValueError: Index contains duplicate entries, cannot reshape`. What does the error tell you about the data? **Answer:** At least one date and store pair appears on more than one row. `pivot` only rearranges; choose how to combine the repeats and use `pivot_table(..., aggfunc=...)`.
5. A sheet has one row per month and one column per store. Which method turns it into tidy rows of month, store, revenue? **Answer:** `melt(id_vars="Month", var_name="store", value_name="revenue")`. Twelve rows and five store columns become 60 rows.

EXIT TICKET ANSWERS

1. You resample daily revenue to monthly totals with `.sum()`. What do you print beside every total, and why? **Answer:** The `count` of days with a value (for example `.agg(["sum", "count"])`). Sums skip `NaN` silently, so a month with missing days looks like a weak month.

2. Write an expression for a 7-day average of the Series `rev` that is safe to use for judging or predicting today. **Answer:** `rev.shift(1).rolling(7).mean()` (or `rev.rolling(7, closed="left").mean()`). Shift first, so today is not inside its own baseline.

3. List the order of operations for a time series, and name the step people skip most often. **Answer:** Parse the dates; put them in the index; complete the calendar with `asfreq`; decide what each gap means; only then resample, shift or compare. The skipped step is completing the calendar.

Reading the tickets: a ticket that says "mean" or "nothing" for question 1 needs the Bend August example again. For question 2, `rev.rolling(7).mean().shift(1)` is also correct (the same numbers); `rev.rolling(7).mean()` and anything with `center=True` are the leak, and those learners get a two-minute recap at the start of Session 8, where leakage returns as the cutoff rule. For question 3, accept any wording that puts the calendar before the arithmetic.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

WRONG ANSWER OR SYMPTOM	WHAT IT REVEALS	WHAT TO DO
Q2 answered `3 3` or `4 3`	Treats a date-string range like a Python slice that stops one short.	Re-run demo step 5 and count the rows of `.loc["2024-12-30":"2025-01-02"]` aloud.
Q3 answered B (filled with 0)	Believes pandas decides what a gap means.	Show `wide.asfreq("D").isna().sum()` again. Zero is something you write, on purpose, for closed days only.
Q4 answered C (`count` is 3)	Reads `count` as "rows in the bucket".	`count` is non-missing values; `size` is rows. That difference is the whole point of printing it.
Q5 answered A or B	Thinks `diff(7)` knows about dates.	The lesson's rows-only table: the Saturday compared with a Wednesday. `shift` and `diff` count rows.
`Q7 answered A`	Has not taken in what `min_periods` changes.	Have them run it with `min_periods` 3, 2 and 1 and line up the three outputs.
Q9 answered A or B	Windowing a long table without grouping, or grouping without aligning.	The quick test: first rows of each group `NaN`? Then `transform`.
Q11 answered D, or exit ticket question 2 answered `rev.rolling(7).mean()`	Leakage has not landed: the window is seen as "the past" although it ends on today.	Demo step 11 again, one row only: May 23, 2025, 11,292 against 10,408. Ask which one was known at opening time.

WRONG ANSWER OR SYMPTOM	WHAT IT REVEALS	WHAT TO DO
Lab 2.7-A: reports the week ending 2024-08-18 as Bend's worst week	Reads a sum without its count.	Point at the `count` column in their own output: 4 days.
Lab 2.7-A: `closed_days` is empty	Typed `units.index.difference(calendar)`.	Ask which of the two has all 731 days; the difference starts from the complete one.
Lab 2.8-A: reports 2024-12-25 as the day Flagstaff passed Bend, with no caveat	Takes the first crossing as the answer although the lines cross several times.	Plot or print `ahead` around June 2025; ask for the last date Bend led.
Lab 2.7-B: "returns got worse in April 2025"	Reads noise as a finding.	Ask how many returned orders make up +2.84 points on 525 orders (about 15).

INSTRUCTOR KIT · ANSWER KEY

Session 8: Features and First Charts

Answers with reasons for the twelve self-check items (all four options shown, with why each wrong one is wrong), the Predict, then run boxes, reference notes, real output and reference charts for all four labs, the lesson pages' own Quick Quiz answers, the warm-up and exit ticket, and a table of common wrong answers with what each one tells you about the learner.

Session 8 | Lessons 2.9-3.1 | 12 self-check items | Confidential

NUMBERING MATCHES THE WORKBOOK

Self-check items carry the same numbers as the participant workbook, and the wording is generated from one source, so question and option text match exactly. Every output block is the real output of the code above it or of the named `*_solution.py`, run on the course data with pandas 3.0 and Matplotlib 3.11. The small charts are what the two chart-lab solutions save. `python3 labs/check_labs.py module-08` runs all eight lab files.

2.9 · FEATURE ENGINEERING BASICS: SELF-CHECK

Q1

Marketing asks which customers will order again, so one row of the feature table is a customer. Which of these is not a valid column for that table as it stands?

A) Days from the customer's signup to the cutoff date

B) The `order_total` of each of the customer's orders

C) The number of orders the customer placed before the cutoff

D) The share of the customer's orders that were placed online

Why: A customer has many orders, so "the order total" is many values, not one fact about the customer. Aggregate it first (a sum, a mean, a maximum) and it becomes a customer feature.

A is wrong: One number per customer: a fine feature (tenure).

C is wrong: One number per customer, and it respects the cutoff.

D is wrong: The mean of a True/False column per customer: one number each.

Q2

What does this print?

```
ages = pd.Series([18, 24, 25, 40])
bands = pd.cut(ages, bins=[17, 24, 34, 120], labels=["18-24", "25-34", "35+"])
print(bands.tolist())
```

`['18-24', '18-24', '25-34', '35+']`

> **Why:** By default each bin includes its right edge and excludes its left: (17, 24], (24, 34], (34, 120]. So 24 is still in the first band and 25 starts the second. That is why the first edge is 17, not 18.

Q3

What does this print?

```
x = pd.Series([0, 10, 20, 30])
print(pd.cut(x, bins=[0, 10, 20]).isna().sum())
```

A) `0`

B) `1`

C) `2`

D) `4`

> **Why:** The bins are (0, 10] and (10, 20]. The value 0 is not in (0, 10] because the left edge is excluded, and 30 is beyond the last edge. Both become `NaN`, with no warning.
> **A** is wrong: Two values fall outside the bins; `cut` does not stretch the edges to fit.
> **B** is wrong: 30 is outside, and so is 0: the left edge of the first bin is excluded unless you pass `include_lowest=True`.
> **D** is wrong: 10 and 20 sit exactly on right edges, which are included.

Q4

`customers` has 4,850 rows. `per_customer`, built with `orders.groupby("customer_id").agg(...)`, has 3,942 rows. What does `customers.merge(per_customer, on="customer_id")` return with the default settings?

A) 4,850 rows, with `NaN` for customers who never ordered

B) 3,942 rows, one for every customer id in the orders

C) 8,792 rows, every row of both tables

D) 3,792 rows: every customer who never ordered is gone

> **Why:** The default is an inner join, which keeps only ids found in both tables. The 1,058 customers with no orders vanish (and so do the 150 order ids with no account). For a feature table, use `how="left"` from the table that defines the unit, and check that the row count did not change.
> **A** is wrong: That is what `how="left"` gives, and it is what you want.
> **B** is wrong: 150 of those ids belong to closed accounts with no row in `customers`, so they cannot match either.
> **C** is wrong: A merge matches rows; it does not stack the two tables.

Q5

A method needs numbers. You have `loyalty_tier` (Basic, Silver, Gold) and `acquisition_channel` (five values with no order). Which encoding is appropriate?

A) Ordered integer codes for the tier; one yes/no column per channel with `pd.get_dummies`

B) Integer codes 0 to 4 for the channel, in alphabetical order; one yes/no column per tier

C) Integer codes for both, because they take the least space

D) Standardize both columns to mean 0 and standard deviation 1

> **Why:** The tier is ordinal: 0, 1, 2 says Gold is above Silver is above Basic, which is true. The channel is nominal: numbering it would tell a model that one channel is "twice" another, so each channel gets its own yes/no column.
> **B** is wrong: This has it backwards: it invents an order for the channels and throws away the real order of the tiers.
> **C** is wrong: Small is not the goal. Codes on a nominal column state an order that does not exist.
> **D** is wrong: Scaling works on numbers. Text has to be encoded first, and scaling does not fix a made-up order.

Q6

You build features as of the cutoff 2025-07-01 to predict who orders in the following six months. A colleague's `n_orders` counts all of each customer's orders in the file. What happens?

A) An error: pandas refuses to aggregate rows dated after the cutoff

B) Nothing important: more data makes a better feature

C) The feature looks more predictive than it is, because orders after the cutoff are in both the feature and the target

D) The feature becomes useless, because the extra rows add noise

> **Why:** This is the "rows from the future" leak. Anyone who ordered after the cutoff has a higher count because of it, so the count partly contains the answer. It will look excellent in testing and fail on a real today. Filter to `order_date < cutoff` before aggregating.
> **A** is wrong: Leaks never raise an error. That is what makes them dangerous.
> **B** is wrong: On a real cutoff date those rows do not exist yet, so the feature cannot be computed as tested.
> **D** is wrong: The opposite: it becomes suspiciously good. "Too good" is the warning sign.

2.9 · PREDICT, THEN RUN: ANSWERS

Predict 2.9.1

The same three values, binned twice. What does each line print?

```
v = pd.Series([10, 20, 30])
edges, names = [0, 10, 20, 30], ["a", "b", "c"]
print(pd.cut(v, bins=edges, labels=names).tolist())
print(pd.cut(v, bins=edges, labels=names, right=False).tolist())
```

```
['a', 'b', 'c']
['b', 'c', nan]
```

> **Why:** By default each bin includes its right edge: 10 is in (0, 10], 20 in (10, 20], 30 in (20, 30]. With `right=False` each bin includes its **left** edge instead, so 10 moves to [10, 20), 20 to [20, 30), and 30 is no longer inside any bin: it becomes `NaN` with no warning.

Predict 2.9.2

Three customers, one of whom never ordered. What are the two lists?

```
people = pd.DataFrame({"id": [1, 2, 3]})
bought = pd.DataFrame({"id": [1, 1, 3], "total": [10.0, 20.0, 7.0]})
per = bought.groupby("id").agg(n=("total", "count"), avg=("total", "mean"))
t = people.merge(per, on="id", how="left")
t["n"] = t["n"].fillna(0).astype(int)
print(t["n"].tolist(), t["avg"].tolist())
```

```
[2, 0, 1] [15.0, nan, 7.0]
```

Why: The left merge keeps customer 2 with `NaN` in both new columns. The count is filled with 0, which is a fact. The average is left missing: there is no average of zero orders.

Predict 2.9.3

What are the column names, and what does each column add up to?

```
d = pd.get_dummies(pd.Series(["web", "store", "web"]), dtype=int)
print(d.columns.tolist(), d.sum().tolist(), d.sum(axis=1).tolist())
```

```
['store', 'web'] [1, 2] [1, 1, 1]
```

Why: One column per category, in alphabetical order: `store`, then `web`. Column sums count the rows in each category (1 and 2). Every row has exactly one 1, so each row sums to 1.

2.9 · LAB REFERENCE SOLUTIONS

LAB 2.9-A · WHICH ORDERS COME BACK?: LABS/MODULE-08/L2_9-FEATURE-ENGINEERING/ORDER_FEATURES_SOLUTION.PY

What it practices: choosing a unit of analysis (an order), flags, a per-entity aggregate joined back with a left merge, `qcut` and `cut`, checking each feature against the target with rate and size, and an interaction feature.

Step 2. The category of a line is read from the SKU prefix (`FTW`), not by merging with `products.csv`. That is deliberate: 609 order lines use six discontinued SKUs that are not in the catalog, and a merge would give them no category (Lesson 2.5). `("is_footwear", "any")` is True when at least one line of the order is footwear.

Step 3. 20,302 rows before and after, and no order without lines. `validate="one_to_one"` would raise if `per_order` had two rows for one order.

Step 5. What separates: channel (online 6.9%, store 3.4%), footwear (10.7% against 4.2%), order value (top half about 6.5% to 6.8%, bottom half 4.4% to 4.9%) and number of lines (4.3% for one line, rising to 7.4% for five or more). What does not: a promo code (5.4% against 5.7%) and the weekend (5.4% against 5.7%).

Step 6 is the finding. Online orders containing footwear: 14.4% returned (2,854 orders). The other three combinations: 3.3%, 3.8% and 4.8%. In a store, people try shoes on; online they cannot. Neither `is_online` nor `has_footwear` shows this alone. Features can be combined, and the combination came from thinking about how the business works.

Step 7. `status` is what `returned` was computed from. Kept as a feature it predicts the target perfectly and means nothing. More generally: nothing recorded after the order shipped may be a feature for "will it be returned".

```
orders: (20302, 11) | order lines: (45490, 5)
per_order rows: 20302
rows before and after the merge: 20302 20302 | orders with no lines: 0
missing after binning: 0 0
overall return rate: 0.0563
            mean   size
is_online
False      0.034   7448
True       0.069  12854
             mean   size
used_promo
False       0.057  18154
True        0.054   2148
             mean   size
is_weekend
False       0.057  13690
True        0.054   6612
               mean   size
has_footwear
False          0.042  15960
True           0.107   4342
             mean  size
value_band
low         0.044  5091
mid         0.049  5062
high        0.068  5074
top         0.065  5075
             mean  size
lines_band
1           0.043  5890
2           0.053  7244
3-4         0.069  6400
5+          0.074   768
                  mean   size
online_footwear
False            0.042  17448
True             0.144   2854
                         mean   size
is_online has_footwear
False     False         0.033   5960
          True          0.038   1488
True      False         0.048  10000
          True          0.144   2854
TODO 7: `status` IS the answer. returned is computed from it, so a table that keeps sta
has the target written into it. Drop it (and anything else recorded after the order sh
```

LAB 2.9-B · HOW BUSY WILL THE STORE BE TOMORROW?: LABS/MODULE-08/L2_9-FEATURE-ENGINEERING/STORE_DAY_FEATURES_SOLUTION.PY

What it practices: a ratio within a group (`transform`), bins with open outer edges, a window feature built on the completed calendar (Lesson 2.7 meets 2.8), and a leak audit where the cutoff is a time of day.

Step 4. Relative traffic: cold days 0.82 of normal, hot days 1.15; rainy 0.82 against 1.03; promotion days 1.70; weekends 1.35 against 0.86.

Step 5. On 104 rows, seven rows back is not seven days back, because 21 store-days have no row. The first examples are the days after Thanksgiving 2024, where the rows-only lag is off by one day and compares a Friday with a Thursday. No error. The calendar version (`asfreq("D")` , then `shift(7)`) is correct, and it is `NaN` where last week is unknown.

Step 6. `transactions` , `units` and `revenue` correlate with visitors at 0.94 to 0.96 because they are counted during the day being predicted: leaks. The calendar lag (0.86) is honest. `promo_day` is planned in advance. `high_f` is the **measured** high: on the evening before, only a forecast exists. `staff_on_duty` (0.93) is the interesting one: it is fixed in advance, so it passes the cutoff rule, but it encodes a manager's own forecast of demand. Whether to use it depends on what the prediction is for.

```
rows before and after: 2903 2903
           visitors  rel_visitors
store
Asheville    274.90           1.0
Bend         334.87           1.0
Boise        485.17           1.0
Flagstaff    328.46           1.0
temp_band
cold     432
cool     932
warm    1012
hot      527
missing temp_band: 0
            mean  size
temp_band
cold       0.824   432
cool       0.937   932
warm       1.054  1012
hot        1.153   527
        mean  size
rainy
False  1.027  2514
True   0.823   389
            mean  size
promo_day
0          0.976  2807
1          1.699    96
             mean  size
is_weekend
False       0.858  2071
True        1.353   832
rows where 7 rows back is not 7 days back: 104
      date     store  visitors  lag7_rows  lag7_calendar
2024-11-29 Asheville       390      212.0          290.0
2024-11-30 Asheville       651      290.0          299.0
2024-12-01 Asheville       507      299.0          374.0
lag7_calendar  r =  0.86   feature: last week is in the past
staff_on_duty  r =  0.93   feature: the schedule is fixed in advance (but it encodes sc
promo_day      r =  0.32   feature: the sale calendar is planned
high_f         r =  0.27   only as a FORECAST made the evening before, not the measured
transactions   r =  0.96   LEAK: counted during the day being predicted
units          r =  0.96   LEAK: counted during the day being predicted
revenue        r =  0.94   LEAK: counted during the day being predicted
```

2.9 · THE LESSON PAGE'S OWN QUICK QUIZ

These five questions are on the lesson page (three options each). Letters and reasons are read from the lesson.

#	QUESTION (SHORT FORM)	ANSWER	REASON
Q1	Five groups of equal size by total spend	A	`pd.qcut(spend, q=5)`: `qcut` chooses its edges from the quantiles, so each bin holds about the same number of rows.
Q2	Which bin does 10 fall into with `bins=[0, 10, 20]`?	C	The first, (0, 10]: bins include their right edge and exclude their left.
Q3	Encoding `acquisition_channel` (five values, no order)	B	One yes/no column per channel (one-hot), so no channel is treated as larger than another.
Q4	Which feature breaks the cutoff rule at 2025-07-01?	B	The loyalty tier recorded on 2025-12-31: it reflects spending in the very period being predicted.
Q5	`NaN` in `n_orders` and `avg_order` after a left merge	C	Fill `n_orders` with 0 (a fact) and leave `avg_order` missing (an average of nothing does not exist).

3.1 · LINE, SCATTER AND BAR CHARTS: SELF-CHECK

Q7

Question: "Do products with more reviews tend to have higher ratings?" Which chart answers it?

A) A line chart of rating against review count

B) A scatter plot: one dot per product, review count against rating

C) A bar chart with one bar per product

D) A histogram of the ratings

> **Why:** Two numeric columns, and the question is whether they move together: one dot per row, nothing joined, because the products have no order.
> **A** is wrong: A line joins points in order and suggests one product leads to the next.
> **C** is wrong: 242 bars compare products one by one; they do not show a relationship between two measures.
> **D** is wrong: A histogram shows one column's distribution. The review counts are not in it.

Q8

What does this print?

```
fig, ax = plt.subplots()
bars = ax.bar(["a", "b", "a", "b", "a"], [1, 5, 3, 2, 2])
print(len(bars), len(fig.axes))
plt.close(fig)
```

`5 1`

> **Why:** `ax.bar` draws one bar per value it is given and never aggregates: five bars, piled up at two x positions. The figure still has one Axes.

Q9

A tutorial writes `plt.title("Revenue")`. You are holding the chart in a variable `ax`. What do you write?

A) `ax.set_title("Revenue")`

B) `ax.title("Revenue")`

C) `fig.set_title("Revenue")`

D) `ax.title = "Revenue"`

> **Why:** On an Axes the labeling methods start with `set_`: `set_title`, `set_xlabel`, `set_ylabel`, `set_xlim`.
> **B** is wrong: `ax.title` is the title object itself; calling it raises `TypeError: 'Text' object is not callable`.
> **C** is wrong: A Figure has no `set_title`; the title of one chart belongs to its Axes.
> **D** is wrong: This replaces the title object with a string and draws nothing.

Q10

A histogram of 20,302 order totals with `bins=40` shows almost everything in three bars, on an axis that runs to $5,000. What is the best fix?

A) `bins=400`, so that the bars are narrower

B) Delete the large orders from the DataFrame before plotting

C) `range=(0, 1000)`, and say in the title how many orders are not shown

D) Switch to a bar chart with one bar per order

> **Why:** A few very large orders stretch the range. `range=` puts the 40 bins where the orders are, and the title tells the reader what was left out.
> **A** is wrong: The axis still runs to $5,000, with most of it empty.
> **B** is wrong: The bulk orders are real. Removing rows changes the data; choosing a range only changes the view.
> **D** is wrong: 20,302 bars is not a distribution, and `ax.bar` does not count anything.

Q11

What does this print?

```
fig, ax = plt.subplots()
counts, edges, patches = ax.hist([1, 2, 2, 3, 3, 3, 9], bins=4, range=(0, 8))
print(counts.tolist(), len(edges))
plt.close(fig)
```

`[1.0, 5.0, 0.0, 0.0] 5`

> **Why:** Four bins over 0 to 8 are each 2 wide: 0 to 2, 2 to 4, 4 to 6, 6 to 8. The value 1 is in the first; 2, 2, 3, 3, 3 are in the second. The 9 is outside the range and is silently left out. Four bins have five edges.

Q12

A scatter plot of 2,903 store-days shows a solid blob in the lower left: you cannot tell 5 overlapping dots from 500. Which change fixes it?

A) A larger `figsize`

B) `linewidth=0.8` and `linestyle="--"`

C) `bins=40` and `range=(0, 1000)`

D) A smaller `s` and an `alpha` below 1

> **Why:** This is overplotting. Smaller dots overlap less, and see-through dots make dense regions darker, so density becomes visible.
> **A** is wrong: A bigger figure spreads the same solid dots; the dense region is still a blob.
> **B** is wrong: Those style a line. A scatter has none.
> **C** is wrong: Those belong to `ax.hist`.

3.1 · PREDICT, THEN RUN: ANSWERS

Predict 3.1.1

One figure, a loop that plots three columns. What are the two numbers?

```
t = pd.DataFrame({"a": [1, 2], "b": [2, 1], "c": [3, 3]})
fig, ax = plt.subplots()
for col in t.columns:
    ax.plot(t.index, t[col], label=col)
print(len(fig.axes), len(ax.lines))
plt.close(fig)
```

```
1 3
```

> **Why:** `plt.subplots()` made one Axes, and every `ax.plot` call adds a line to that same Axes: one chart, three lines.

Predict 3.1.2

No error. How many bars, and how tall is the tallest thing you can see?

```
fig, ax = plt.subplots()
bars = ax.bar(["x", "y", "x"], [5, 2, 9])
print(len(bars), max(bar.get_height() for bar in bars))
plt.close(fig)
```

```
3 9
```

Why: Three bars, one per value, with two of them piled at "x". You would see a bar of height 9 at "x": the larger of 5 and 9, not their sum of 14.

Predict 3.1.3

Three dates given as text, out of order. Where does Matplotlib put them on the x axis?

```
fig, ax = plt.subplots()
ax.plot(["2025-01-03", "2025-01-01", "2025-01-02"], [3, 1, 2])
print(list(ax.get_xticks()))
plt.close(fig)
```

```
[0, 1, 2]
```

Why: Text values are categories. Matplotlib places them at positions 0, 1, 2 in the order they arrive, so January 3 comes first on the axis. It does not know they are dates, so it cannot sort or space them. Parse dates before plotting.

3.1 · LAB REFERENCE SOLUTIONS

LAB 3.1-A · FOUR QUESTIONS, FOUR CHARTS: LABS/MODULE-08/L3_1-LINE-SCATTER-BAR/FOUR_QUESTIONS_SOLUTION.PY

What it practices: choosing the chart from the question; the five-line pattern four times; aggregating before a line and a bar and not before a scatter or a histogram; `bar_label`, `alpha`, `axvline`; saving with `fig.savefig`.

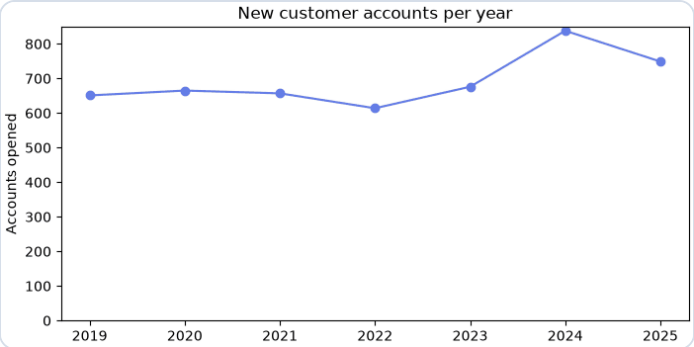
Question 1: a line. Seven yearly counts, 614 to 838, best year 2024. The reference solution sets `ax.set_ylim(bottom=0)` so the rise in 2024 is not exaggerated. Bars are acceptable for seven years; ask what the learner would do with 84 months.

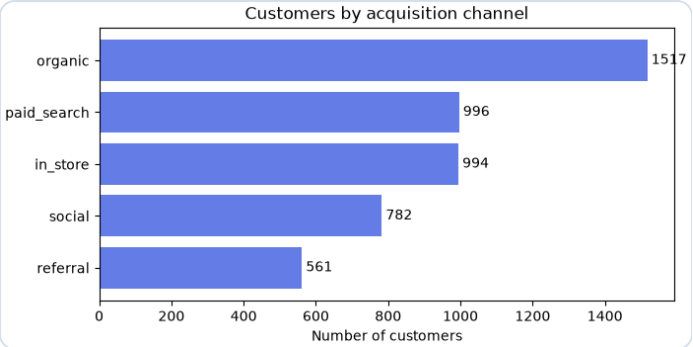
Question 2: horizontal bars. `value_counts().sort_values()` is smallest first, and `barh` draws the first row at the bottom, so the longest bar (organic, 1,517) ends up on top.

Question 3: a scatter. 228 dots, not 242: `weight_kg` is missing for 14 products and Matplotlib leaves those rows out silently. The reference title says so. Correlation 0.69: heavier products do cost more, with a wide spread. Accept "tend to"; push back on "because".

Question 4: a histogram. 30 bins, each $20.28 wide; median $82.95; 21 products above $300 form a long right tail. No `range=` is needed here: the tail is part of the answer and 242 rows leave no bar empty of meaning. A learner who did a `groupby` first has confused the histogram with a bar chart.

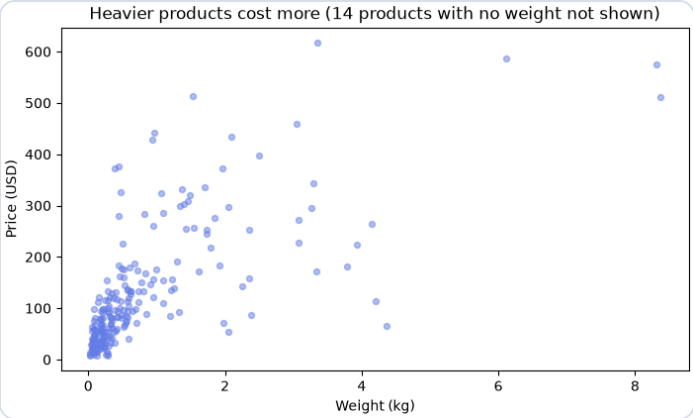
```
customers: (4850, 12) | products: (242, 13)
signup_date
2019    651
2020    665
2021    657
2022    614
2023    676
2024    838
2025    749
Q1 line: 1 line with 7 points; best year 2024 with 838
Q2 bars: 5 bars; top bar is organic with 1517
Q3 scatter: 228 dots, 14 products without a weight; correlation 0.69
Q4 histogram: 30 bins, each 20.28 dollars wide; median 82.95 | products above $300: 21
saved: ['q1_signups.png', 'q2_channels.png', 'q3_weight_price.png', 'q4_prices.png']
```



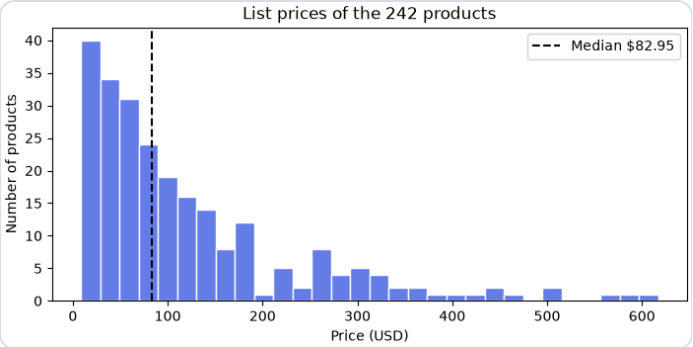


q1_signups.png

q2_channels.png



q3_weight_price.png



q4_prices.png

LAB 3.1-B · CHART REPAIR: LABS/MODULE-08/L3_1-LINE-SCATTER-BAR/CHART_REPAIR_SOLUTION.PY

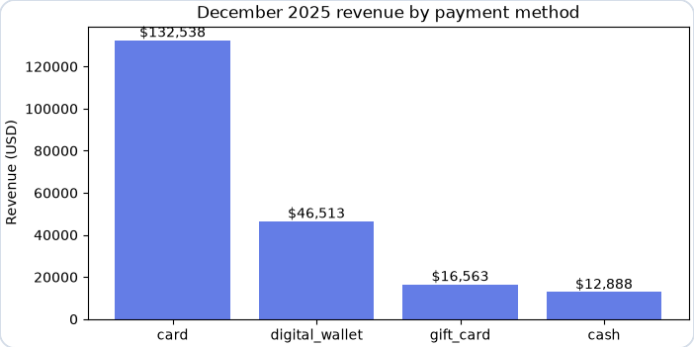
What it practices: recognizing the three silent chart mistakes from the lesson on new data, and fixing the cause in pandas, not the symptom in Matplotlib.

Chart 1. `ax.bar` drew one bar per order. The visible heights were each method's largest single order (card $3,779.45, gift card $840.60, cash $702.25, digital wallet $640.80), which even ranks the methods wrongly. After `groupby(...).sum()`: card $132,538, digital wallet $46,513, gift card $16,563, cash $12,888.

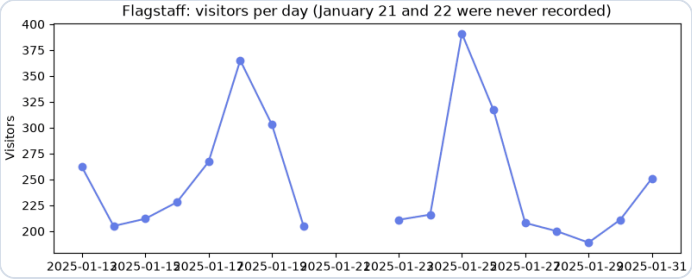
Chart 2. Flagstaff's export failed on January 21 and 22, 2025, so those days have no rows and the line ran straight across them. `set_index("date")["visitors"].asfreq("D")` puts the days back as `NaN`, and a line breaks at `NaN`. The reader can now see that data is missing.

Chart 3. The file was read without `parse_dates`, so the dates are text and Matplotlib treats each as a category: 731 ticks printed on top of each other. The fix is at `read_csv`, not in tick settings. With real dates there are 9 ticks.

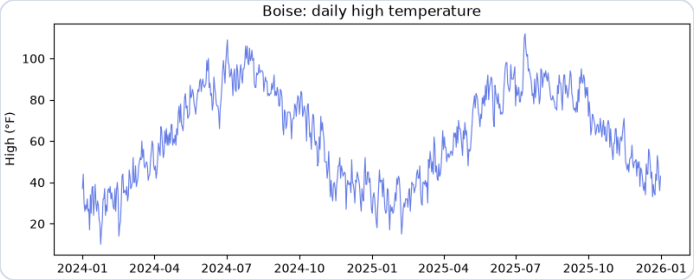
```
Chart 1: was 1145 bars (largest single order per method: {'card': 3779.45, 'cash': 702.
         now 4 bars: {'card': 132538.28, 'digital_wallet': 46513.22, 'gift_card': 16563
Chart 2: was 17 rows; now 19 days with 2 NaN on ['2025-01-21', '2025-01-22']
Chart 3: dtype is now datetime64[us] and the x axis has 9 ticks (it had 731)
saved: ['fixed_1_payment.png', 'fixed_2_visitors.png', 'fixed_3_temperature.png']
```



fixed_1_payment.png



fixed_2_visitors.png



fixed_3_temperature.png

3.1 · THE LESSON PAGE'S OWN QUICK QUIZ

These five questions are on the lesson page (three options each). Letters and reasons are read from the lesson.

#	QUESTION (SHORT FORM)	ANSWER	REASON
Q1	Typical order total and how much the totals vary	B	A histogram: it bins one numeric column and counts the rows in each bin.
Q2	`print(len(fig_q.axes), len(ax_q.lines))` after two `plot` calls	C	`1 2`: one Axes; each `ax.plot` call adds a line to it.
Q3	Visible height of `ax.bar(daily["store"], daily["revenue"])`	A	Each store's single highest daily revenue: 2,903 bars pile up and only the tallest shows.
Q4	Figure and Axes	C	A Figure is the whole canvas; an Axes is one chart on it, with its own x and y axis.
Q5	A line plotted over three days with no rows	A	A straight segment across the gap. Restore the calendar with `asfreq` so the days hold `NaN` and the line breaks.

WARM-UP ANSWERS

1. You resample daily revenue to monthly totals. What do you print beside every total? **Answer:** The `count` of days with a value. Sums skip `NaN` silently, so a month with missing days looks like a weak month.
2. What does `wide.asfreq("D")` do to a table with dates in the index? **Answer:** It gives every calendar day a row. Days that had no row appear, filled with `NaN`, and you then decide what each gap means.
3. On a gap-free daily Series, how many `NaN` values does `s.rolling(7).mean()` have, and where? **Answer:** Six, at the start. A 7-row window is full for the first time on row 7.
4. Write a 7-day average of `rev` that is safe for judging today. **Answer:** `rev.shift(1).rolling(7).mean()`. Shift first, so today is not inside its own baseline.
5. A long table holds four stores stacked in one column. What three things do you do, in order, to get each store's own 7-day average? **Answer:** Sort (by store, then date), group (by store), window (inside `transform`). The first rows of each group should come out `NaN`.

EXIT TICKET ANSWERS

1. Before you build any feature, what is the first thing you decide? **Answer:** The unit of analysis: what one row of the table represents (a customer, an order, a store-day). Every feature must be one fact about that unit.
2. State the cutoff rule in one sentence. **Answer:** For every feature: could I have computed this value on the cutoff date, using only what existed then? If not, it leaks.
3. Name the chart for each question: (a) how did it change over time, (b) do two numbers move together, (c) which group is bigger, (d) what values are typical and how spread out are they? **Answer:** (a) line, `ax.plot`; (b) scatter, `ax.scatter`; (c) bar, `ax.bar` or `ax.barh`, after aggregating; (d) histogram, `ax.hist`, on the raw rows.

Reading the tickets: for question 1, "load the data" or "choose the features" means the unit of analysis has not landed; start Session 9 by asking what one row is in the nine-panel project. For question 2, accept any wording with "on that date" and "only what existed then". For question 3, the common slip is (c) and (d) swapped: bar and histogram look alike and answer different questions. Those learners should redo self-check Q8 and Q11.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

WRONG ANSWER OR SYMPTOM	WHAT IT REVEALS	WHAT TO DO
Q2 answered `['18-24', '25-34', '25-34', '35+']`	Believes bins include their left edge, as Python ranges do.	Read (17, 24] aloud as "above 17, up to and including 24". Predict 2.9.1 shows both conventions.

WRONG ANSWER OR SYMPTOM	WHAT IT REVEALS	WHAT TO DO
Q3 answered B (one `NaN`)	Sees the value beyond the last edge but not the excluded left edge.	`pd.cut(pd.Series([0]), bins=[0, 10])` on the projector; then `include_lowest=True`.
`Q4 answered A`	Assumes a merge keeps the left table.	The default is inner. Count rows before and after every merge; this is Session 6's first lesson again.
Q5 answered B or C	Ordinal and nominal are not yet separate ideas.	Ask: "Is Gold more than Silver? Is referral more than social?" The first has an answer.
Q6 answered B, or exit ticket question 2 left vague	The cutoff rule is not yet operational.	Demo step 12 again: 698 customers with zero orders in the whole file, target rate 0.0. "How did the column know?"
Q8 answered `2 1`	Thinks `ax.bar` groups equal labels.	`len(bars)` on the projector. `ax.bar` never aggregates.
Q11 answered with a count for the 9	Does not know that `range=` drops values outside it.	That is why the lesson's title says how many orders are not shown.
Lab 2.9-A: 45,490 rows after the merge	Merged the order lines, not the per-order summary: the unit of analysis changed from order to line.	"What is one row now?" Aggregate first, then merge, with `validate="one_to_one"`.
Lab 2.9-A: stops at "footwear is returned more"	Reads features one at a time.	Ask for the same table split by channel. In stores footwear makes no difference.
Lab 3.1-A: a line chart for the acquisition channels	Chose the chart from habit, not from the question.	"Does anything flow from organic to paid search?" Unordered groups get bars.
Lab 3.1-A: no files in `output/`, or a script that hangs	Called `plt.show()` and never reached `savefig`, or saved after closing.	Save with `fig.savefig(...)` right after drawing; no `plt.show()` in a lab file.
Lab 3.1-B: fixed Chart 3 by rotating or thinning the tick labels	Treated the symptom.	`boise["date"].dtype` is `str`. Fix the cause at `read_csv`.

INSTRUCTOR KIT · ANSWER KEY

Session 9: Composing and Customizing Figures

Answers with reasons for the twelve self-check items (all four options shown, with why each wrong one is wrong), the Predict, then run boxes, reference notes, real output and reference charts for all four labs, the lesson pages' own Quick Quiz answers, the warm-up and exit ticket, and a table of common wrong answers with what each one tells you about the learner.

Session 9 | Lessons 3.2-3.3 | 12 self-check items | Confidential

NUMBERING MATCHES THE WORKBOOK

Self-check items carry the same numbers as the participant workbook, and the wording is generated from one source, so question and option text match exactly. Every output block is the real output of the code above it or of the named `*_solution.py`, run on the course data with pandas 3.0 and Matplotlib 3.11, and every small chart is what the lab solution saves. `python3 labs/check_labs.py module-09` runs all eight lab files.

3.2 · SUBPLOTS AND COMPOSITION: SELF-CHECK

Q1

What does this print?

```
fig, axs = plt.subplots(2, 3)
print(axs.shape, len(fig.axes), type(axs[1, 2]).__name__)
plt.close(fig)
```

`(2, 3) 6 Axes`

Why: Rows first, then columns: a 2 by 3 NumPy array holding six Axes. `axs[1, 2]` is row 1, column 2, the bottom-right panel, and it is one Axes.

Q2

After `fig, axs = plt.subplots(1, 4)`, the line `axs[0, 2].set_title("Boise")` fails. Why?

A) A grid must have at least two rows before panels can be titled

B) A one-row grid comes back as a 1-D array, so it takes one index: `axs[2]`

C) `set_title` only works on the figure: `fig.set_title`

D) The panels are numbered from 1, so the third panel is `axs[1, 3]`

> **Why:** `plt.subplots` squeezes away dimensions of length one. One row of four is an array of shape `(4,)`, and two indexes raise `IndexError: too many indices for array`. Use `squeeze=False` when you want two indexes every time.
> **A** is wrong: Any grid can be titled; the problem is the index, not the shape.
> **C** is wrong: A Figure has `suptitle`; each panel's title is still `ax.set_title`.
> **D** is wrong: Python indexes from 0; the third panel is position 2.

Q3

Four panels show monthly revenue for four stores. When should you pass `sharey=True`?

A) Always: panels without a shared scale are wrong

B) Never: sharing squashes the smaller stores

C) When the reader should compare sizes across panels; separate scales show each shape but hide the differences in size

D) Only when the panels are in one row

> **Why:** Sharing is a decision about the question. A shared y axis answers "which is bigger?"; separate axes answer "what shape does each have?". When panels do not share a scale, say so in the title or caption.
> **A** is wrong: Separate scales are right when shape is the point, as long as the reader is told.
> **B** is wrong: Squashing is a cost worth paying when size is the point.
> **D** is wrong: Sharing works for any grid shape, and `sharey="row"` shares by row.

Q4

What does this print?

```
fig, axd = plt.subplot_mosaic("AAB;CCB")
print(sorted(axd), len(fig.axes))
plt.close(fig)
```

`['A', 'B', 'C'] 3`

> **Why:** The string is a picture: one letter per cell, `;` between rows. A spans two cells of the top row, C two cells of the bottom row, and B the right column of both rows. Three names, three panels, returned as a dictionary.

Q5

Which chart is an honest use of a second y axis?

A) Revenue on the left and visitors on the right, made with `twinx()`

B) Revenue on the left and staff on duty on the right

C) Boise's revenue on the left and Asheville's revenue on a separate right-hand scale

D) Temperature in °F on the left and the same values in °C on the right, with `secondary_yaxis`

> **Why:** When both sides measure the same quantity, a formula ties the two scales together and nobody gets to choose where the lines sit. With two different measures, the scales are arbitrary and the picture can be made to say anything.
> **A** is wrong: Two measures, two arbitrary scales: their crossings and overlaps mean nothing. Stack the panels or plot a ratio.
> **B** is wrong: The same problem, with a causal story ready to be read into it (Lesson 4.4 takes this pair apart).
> **C** is wrong: Two scales for the same unit hide the fact that one store is nearly twice the other.

Q6

Seven categories are drawn on a 3 by 3 grid with `for ax, name in zip(axs.flat, categories):`. What does the reader see in the last two cells?

A) Two empty frames with ticks from 0 to 1, until you remove them with `ax.remove()`

B) Nothing: `zip` deletes the unused panels

C) Copies of the last category

D) An `IndexError`, because the grid has more panels than names

> **Why:** `zip` stops at the shorter input and quietly leaves the extra Axes untouched. They look like charts that failed to load. Remove them: `for ax in axs.flat[len(categories):]: ax.remove()`.
> **B** is wrong: `zip` only stops looping; it does not touch the figure.
> **C** is wrong: Nothing is drawn on the leftover panels at all.
> **D** is wrong: `zip` never raises for unequal lengths.

3.2 · PREDICT, THEN RUN: ANSWERS

Predict 3.2.1

Three grids. What type or shape does each give back?

```
a = plt.subplots(1, 1)[1]
b = plt.subplots(2, 1)[1]
c = plt.subplots(2, 1, squeeze=False)[1]
print(type(a).__name__, b.shape, c.shape)
plt.close("all")
```

```
Axes (2,) (2, 1)
```

> **Why:** A 1 by 1 grid is squeezed to a single Axes. One column of two is squeezed to a 1-D array of shape (2,). `squeeze=False` keeps the full 2-D shape (2, 1).

Predict 3.2.2

Two panels with very different data and `sharey=True`. Same y range or not?

```
fig, axs = plt.subplots(1, 2, sharey=True)
axs[0].plot([0, 1], [0, 10])
axs[1].plot([0, 1], [0, 2])
print(axs[0].get_ylim() == axs[1].get_ylim())
plt.close(fig)
```

```
True
```

Why: With a shared y axis both panels use one range that fits all the data, here the range of the first panel. The second line now sits low in its panel, which is the honest picture: it is the smaller one.

Predict 3.2.3

A period in the layout picture. How many panels, with which names?

```
fig, axd = plt.subplot_mosaic([["main", "side"],
                               ["main", "."]])
print(list(axd), len(fig.axes))
plt.close(fig)
```

```
['main', 'side'] 2
```

Why: "main" covers both rows of the left column, so it is one tall panel. "." leaves its cell empty: no panel is made for it. Two names, two Axes.

3.2 · LAB REFERENCE SOLUTIONS

LAB 3.2-A · FOOT TRAFFIC ON ONE PAGE: LABS/MODULE-09/L3_2-SUBPLOTS-COMPOSITION/TRAFFIC_PANELS_SOLUTION.PY

What it practices: a grid and its array, `zip(axs.flat, ...)`, separate against shared scales, `suptitle` and `supylabel`, a panel-drawing function for small multiples, and `subplot_mosaic` with named panels.

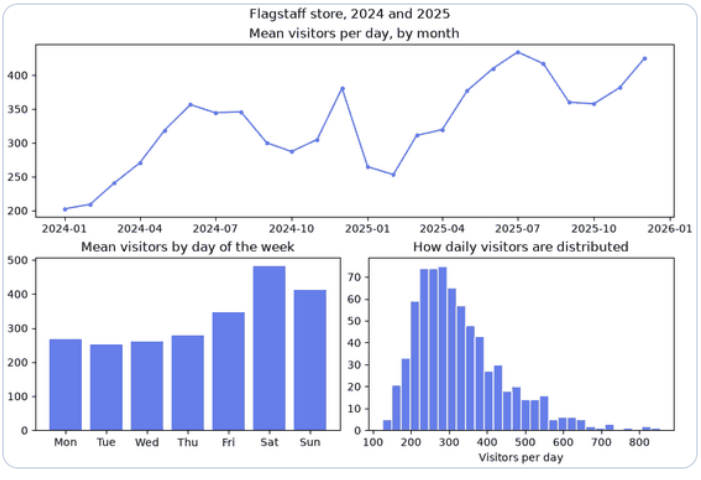
The question's answer: comparing mean daily visitors in 2025 with 2024, Flagstaff +21.0%, Boise +9.2%, Bend +2.5%, Asheville -7.5%. The small multiples show it best: Flagstaff's colored line climbs through the gray ones.

Step 1 against Step 2. Own scales: y ranges of roughly (203, 341), (238, 418), (303, 629) and (191, 446), so every line fills its panel and the stores look alike. Shared: one range, and Boise sits well above the rest. A learner who leaves the Step 1 title without saying "each panel has its own scale" has missed the point of the step.

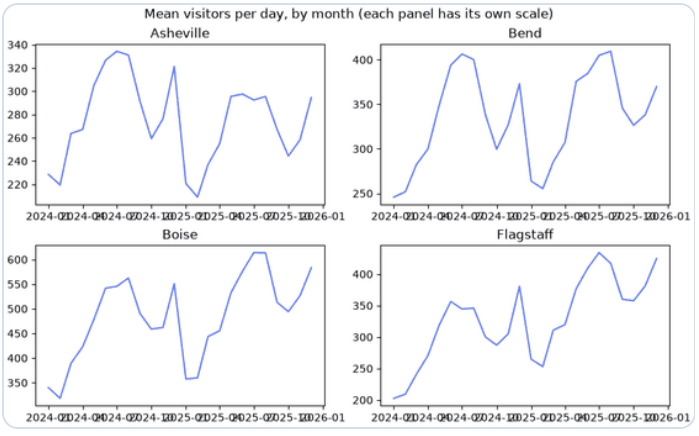
Step 3. Draw the gray lines first and the colored line last; otherwise the gray paints over it. Each panel holds 4 lines.

Step 4. 725 Flagstaff rows (two failed export days). The busiest weekday is Saturday with about 482 visitors; the quietest weekday averages about 252. The histogram needs no `dropna()` here because visitors have no blank values, unlike revenue in the lesson's Boise version.

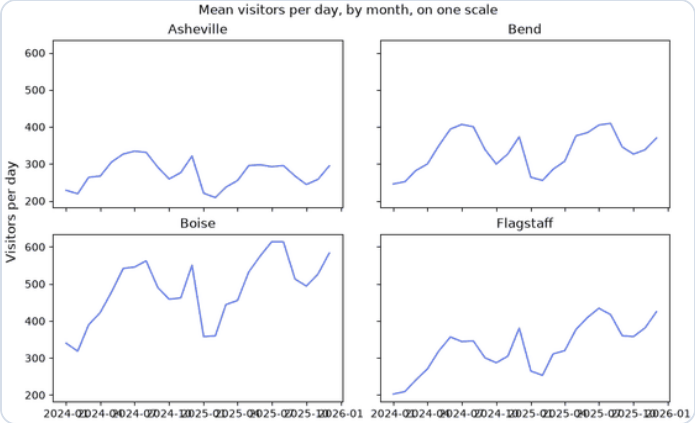
```
monthly mean visitors: (24, 4) ['Asheville', 'Bend', 'Boise', 'Flagstaff']
change in mean daily visitors, 2025 vs 2024 (%): {'Asheville': -7.5, 'Bend': 2.5, 'Bois
Step 1 y ranges: [(203, 341), (238, 418), (303, 629), (191, 446)]
Step 2 same y range in every panel: True
Step 3 lines per panel: [4, 4, 4, 4]
Flagstaff rows: 725
Step 4 panels: ['trend', 'week', 'spread'] | busiest weekday: Sat 482 | quietest: 252
saved: ['flagstaff_summary.png', 'grid_own_scales.png', 'grid_shared.png', 'small_mult:
```



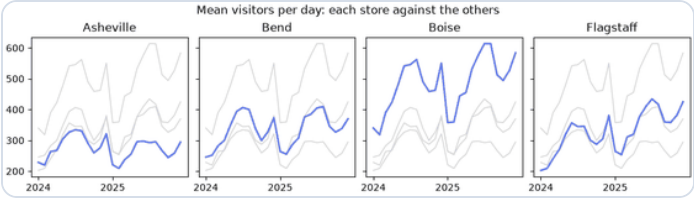
flagstaff_summary.png



grid_own_scales.png



grid_shared.png



small_multiples.png

LAB 3.2-B · TWO MEASURES, HONESTLY: LABS/MODULE-09/L3_2-SUBPLOTS-COMPOSITION/HONEST_AXES_SOLUTION.PY

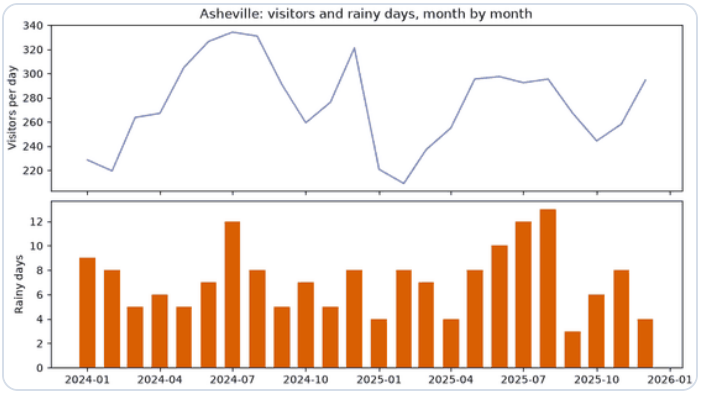
What it practices: `twinx()` and why it misleads, stacked panels with `sharex=True`, computing the relationship instead of drawing two lines, and `secondary_yaxis` with a pair of conversion functions.

Step 1. Same data, right axis 0 to 20 in one panel and 0 to 60 in the other: in the first the rainy-day line swings across the visitors line, in the second it lies flat along the bottom. The figure has 4 Axes (each panel plus its twin).

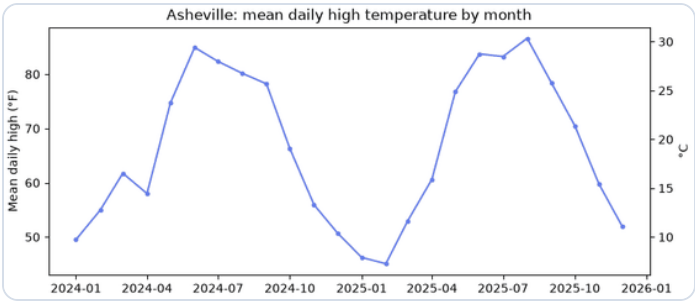
Step 3 is the discussion. Month by month the correlation of rainy days and mean visitors is only 0.28: the season dominates (summer brings both more rain and more visitors in Asheville). Day by day the effect is clear: 286.8 visitors on dry days against 236.4 on rainy days, -17.6%, the same number as the Lesson 3.1 project. The level of aggregation changes the answer; a twin-axis chart of monthly totals would have hidden the real effect.

Step 4. The warmest month is August 2025 at 86.6 °F (30.3 °C). Nobody chose the right-hand scale: a formula did. That is the test for an honest second axis.

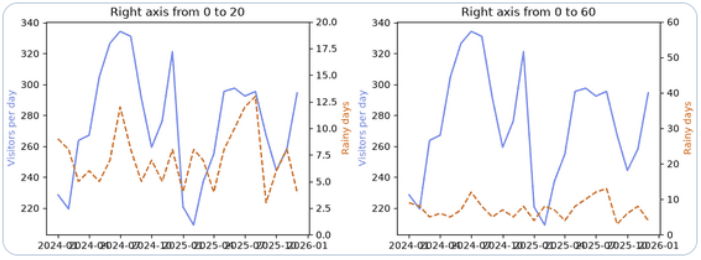
```
(24, 3) | rainy days per month from 3 to 13
Step 1: same data, two right-axis ranges; figure has 4 Axes
Step 3: monthly correlation of rainy days and visitors: 0.28
        mean visitors on dry days: 286.8 | on rainy days: 236.4 (-17.6%)
Step 4: warmest month 2025-08, 86.6 °F = 30.3 °C
saved: ['stacked.png', 'temperature_two_units.png', 'twin_two_stories.png']
```



stacked.png



temperature_two_units.png



twin_two_stories.png

3.2 · THE LESSON PAGE'S OWN QUICK QUIZ

These five questions are on the lesson page (three options each). Letters and reasons are read from the lesson.

#	QUESTION (SHORT FORM)	ANSWER	REASON
Q1	`plt.subplots(3, 2)`: shape and type of `axs[2, 0]`	A	`(3, 2) Axes`: rows then columns; indexing with a row and a column gives one Axes.
Q2	Boise and Asheville lines reach the same height in a 2 by 2 grid	C	Each panel chose its own y scale; `sharey=True` would make heights comparable.
Q3	`subplot_mosaic([["top", "top"], ["left", "right"]])`: how many panels?	B	`3 ['left', 'right', 'top']`: "top" covers both cells of the first row.
Q4	Twin-axis lines cross in March	B	The crossing depends only on how the two scales were chosen; stacked panels or a ratio would be more honest.
Q5	`plt.title(store)` inside the loop over a 2 by 2 grid	A	Only the last panel gets a title, showing the last store's name.

3.3 · CUSTOMIZING PLOTS: SELF-CHECK

Q7

A chart shows the monthly online return rate for 2024 and 2025; both lines wobble between about 5.7% and 9.1%. Which title serves a reader best?

A) Online return rate by month

B) Online returns are rising

C) The online return rate held near 7% in both years

D) Why customers send things back

> **Why:** It states what the chart shows, in words a reader can check against the lines. What is plotted (monthly rate, online orders, 2024 to 2025) goes in the subtitle.
> **A** is wrong: A description, not a finding: it spends the most-read text on the chart without saying anything.
> **B** is wrong: A claim the data does not support (the yearly rate went from 7.11% to 6.76%). A finding title can be wrong; check it with a number.
> **D** is wrong: The chart shows how often, not why. Causes stay out of a title unless they have been tested.

Q8

What does this print?

```
x = 1234567
print(f"${x / 1000:,.0f}K", "{x:.1%}".format(x=0.0712))
```

`$1,235K 7.1%`

> **Why:** The first divides by 1,000, rounds to no decimals and adds thousands separators: the `FuncFormatter` recipe for thousands of dollars. The second multiplies by 100 and adds the percent sign. Matplotlib fills the field named `x` with each tick value.

Q9

A heatmap shows each store's revenue change against the same month last year, from -12.6% to +29.0%. Which color choice fits?

A) A qualitative palette such as Okabe-Ito, one color per cell

B) A sequential map such as viridis, from the lowest change to the highest

C) A diverging map such as RdBu, with the limits set symmetric around zero

D) A rainbow map such as jet, to show as many levels as possible

Why: Zero is a meaningful midpoint: shrinking and growing are different stories. A diverging map gives the two sides different hues and puts the neutral color at zero, which needs `vmin=-limit, vmax=limit`.
A is wrong: Qualitative colors are for unordered groups; these numbers have an order.
B is wrong: Viridis would color the small positive changes much like the negative ones, so the reader cannot see where zero is.
D is wrong: Jet's brightness rises and falls, inventing boundaries that are not in the data, and it fails color-blind readers.

Q10

A line chart has four series whose ends are well apart. Why replace the legend with direct labels?

A) The reader no longer has to look a color up in a box, and the chart works even when colors are hard to tell apart

B) Matplotlib raises a warning when a legend has more than three entries

C) Direct labels make the figure render faster

D) Legends cannot be used with gray lines

Why: A legend is a lookup table: find a line, note its color, find it in the box, read the name, come back. A label at the end of each line removes the trip, and it does not depend on color at all.
B is wrong: There is no such warning. Legends are fine when lines cannot be labeled at their ends.
C is wrong: Speed has nothing to do with it; this is about the reader.
D is wrong: A legend works with any colors; with several gray lines it just becomes useless, which is the reason to label directly.

Q11

What does this print?

```
before = plt.rcParams["axes.spines.top"]
with plt.rc_context({"axes.spines.top": False}):
    fig, ax = plt.subplots()
inside = ax.spines["top"].get_visible()
plt.close(fig)
print(before, inside, plt.rcParams["axes.spines.top"])
```

`True False True`

> **Why:** The figure was created inside the `with` block, so it was built with the style: no top spine. When the block ends the setting goes back to the default, `True`, for everything drawn afterward.

Q12

Which edit makes a bar chart of 2025 store revenue misleading?

A) Removing the top and right spines

B) Starting the y axis at $1.5 million to "show the differences better"

C) Adding a faint horizontal grid behind the bars

D) Writing each bar's value on it with `bar_label`

> **Why:** A bar's length is its value. With the axis cut at $1.5 million, Boise's bar is drawn about four and a half times as tall as Asheville's, although Boise sells less than twice as much. Bars start at zero.
>
> **A** is wrong: Removing ink that carries no information is clean-up, not distortion.
>
> **C** is wrong: A faint grid helps the reader estimate values and stays out of the way.
>
> **D** is wrong: Direct value labels add information and remove a trip to the axis.

3.3 · PREDICT, THEN RUN: ANSWERS

Predict 3.3.1

Two format strings. What does each produce?

```
print("{x:,.0f}".format(x=1234.6), "${x:,.2f}".format(x=7))
```

```
1,235 $7.00
```

Why: `,` adds thousands separators and `.0f` rounds to no decimals: 1,235. `.2f` always shows two decimals, so 7 becomes $7.00. Matplotlib does exactly this with each tick value.

Predict 3.3.2

Which sides of the frame are still visible?

```
fig, ax = plt.subplots()
ax.spines[["top", "right"]].set_visible(False)
print([side for side in ax.spines if ax.spines[side].get_visible()])
plt.close(fig)
```

```
['left', 'bottom']
```

Why: An Axes has four spines: left, right, bottom, top. Hiding two leaves the left and bottom ones, the usual clean frame.

Predict 3.3.3

Three positions and three labels. What do the tick labels say?

```
fig, ax = plt.subplots()
ax.plot([0, 1], [0, 1])
ax.set_xticks([0, 0.5, 1], labels=["start", "middle", "end"])
print([t.get_text() for t in ax.get_xticklabels()])
plt.close(fig)
```

```
['start', 'middle', 'end']
```

Why: `set_xticks` with `labels=` fixes the positions and the text together, one label per position. With only two labels for three positions it would raise a `ValueError` instead.

3.3 · LAB REFERENCE SOLUTIONS

LAB 3.3-A · FROM FIRST DRAFT TO FINISHED CHART: LABS/MODULE-09/L3_3-CUSTOMIZING-PLOTS/FINISH_CHART_SOLUTION.PY

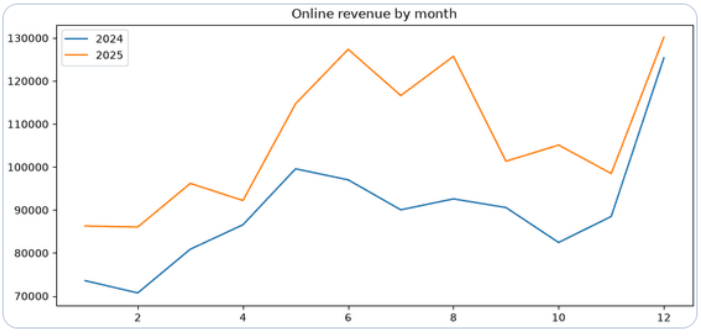
What it practices: checking a finding with numbers, a headline with a subtitle, `FuncFormatter`, `set_xticks` with labels, gray plus one, direct labels, an annotation with an arrow, and clean-up, all as small functions that take an `ax`.

Step 1 decides the headline. 2025 beat 2024 in all 12 months; the largest gap is August (+$33,144), the smallest December (+$4,803); the year went from $1,077,801 to $1,280,184 (+18.8%, the Online growth figure from Lesson 2.4). "Online revenue beat 2024 in every month of 2025" is supported; "online revenue soared" is an adjective, not a finding.

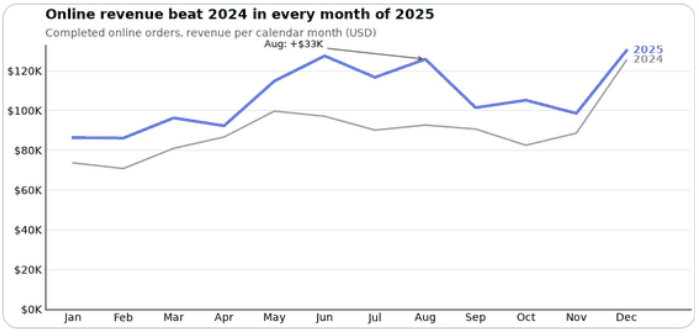
Step 3. The x values are month numbers, so this is not a date axis: `ax.set_xticks(range(1, 13), labels=MONTHS)` sets positions and labels together. A learner who reaches for `mdates` here will get odd results; ask what type the x values are.

Steps 4 and 5. Expected checks: y labels `$0K, $20K, $40K, $60K`; visible spines `['left', 'bottom']`; `ax.get_legend()` returns `None`. The annotation position takes a few tries; that is normal.

```
year       2024     2025
month
1       73567.0  86269.0
2       70712.0  86043.0
3       80882.0  96183.0
months in which 2025 beat 2024: 12 of 12
largest gap: Aug, $33,144 | smallest gap: Dec, $4,803
full year: $1,077,801 -> $1,280,184 (18.8%)
y tick labels: ['$0K', '$20K', '$40K', '$60K']
x tick labels: ['Jan', 'Feb', 'Mar', 'Apr'] ...
visible spines: ['left', 'bottom']
legend: None
saved: ['draft.png', 'finished.png']
```



draft.png



finished.png

LAB 3.3-B · GIVE COLOR A JOB: LABS/MODULE-09/L3_3-CUSTOMIZING-PLOTS/COLOR_JOBS_SOLUTION.PY

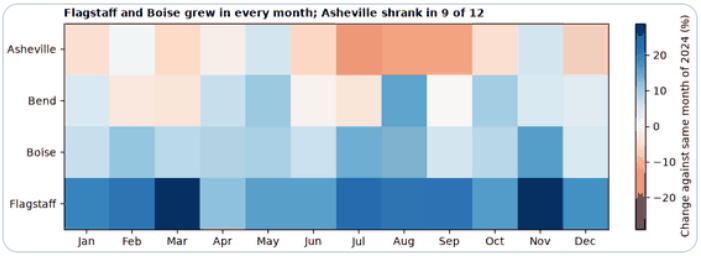
What it practices: choosing a palette type from the data (sequential against diverging), `ax.imshow` with `aspect="auto"`, a colorbar as the legend of a colormap, tick labels for a grid, and centering a diverging map.

Step 1. Cells run from 214 to 588 mean visitors per day; Boise's row is brightest in every month, and summer and December are bright for every store.

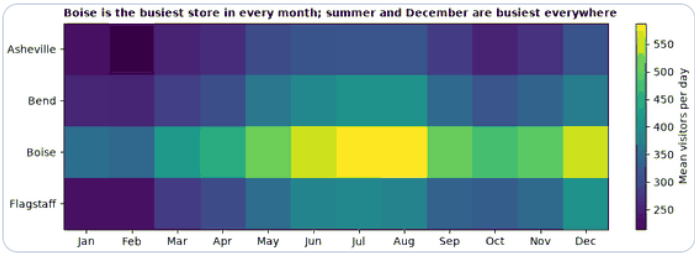
Step 2. Changes run from -12.6% to +29.0%. With symmetric limits (-29.0 to 29.0) white means "no change". With the data's own limits the neutral middle of the map would sit at about +8%, so small gains would look like losses. That is the one line of the lab that matters.

Step 3. Months that shrank against 2024: Asheville 9, Bend 5, Boise 0, Flagstaff 0. The diverging map makes Asheville's row and Flagstaff's row read as opposites at a glance, which is the Session 7 finding again in a new form.

```
visitors table: (4, 12) | change table: (4, 12)
Step 1: lowest cell 214, highest cell 588
Step 2: changes from -12.6% to 29.0%; color limits -29.0 to 29.0
Step 3: months that shrank against 2024: {'Asheville': 9, 'Bend': 5, 'Boise': 0, 'Flags
saved: ['heat_change.png', 'heat_visitors.png']
```



heat_change.png



heat_visitors.png

3.3 · THE LESSON PAGE'S OWN QUICK QUIZ

These five questions are on the lesson page (three options each). Letters and reasons are read from the lesson.

#	QUESTION (SHORT FORM)	ANSWER	REASON
Q1	Best title for dry against rainy visitors	B	"Rainy days bring about a fifth fewer visitors at every store" states the finding.
Q2	`"${x:,.0f}".format(x=1234567.8)`	A	`$1,234,568`: commas, no decimals, a dollar sign.
Q3	Why three stores gray and Flagstaff colored	C	Color directs attention to the story; gray keeps the context.

#	QUESTION (SHORT FORM)	ANSWER	REASON
Q4	Red and green lines that differ only in color	A	A color-blind-safe pair plus a second cue such as line style or direct labels.
Q5	`rc_context({"font.size": 16})`: before, inside, after	C	`10.0 16.0 10.0`: the setting applies only inside the block.

WARM-UP ANSWERS

1. Before you build a feature table, what is the first thing you decide? **Answer:** The unit of analysis: what one row represents. Every feature must be one fact about that unit.
2. After a left merge of per-customer summaries, which `NaN` values become 0 and which stay missing? **Answer:** Counts and sums become 0 (true statements). Averages, shares and recency stay missing; add a flag such as `has_ordered`.
3. State the cutoff rule. **Answer:** For every feature: could I have computed this value on the cutoff date, using only what existed then? If not, it leaks.
4. What is the difference between a Figure and an Axes? **Answer:** The Figure is the whole canvas, the thing that gets saved. An Axes is one chart on it, with its own x and y axis.
5. `ax.bar(daily["store"], daily["revenue"])` draws four bars and no error. What are their heights? **Answer:** Each store's single highest day: 2,903 bars pile up at four positions and the tallest shows. `ax.bar` never aggregates; group first.

EXIT TICKET ANSWERS

1. What does `plt.subplots(2, 2)` give you in `axs`, and what is the safest way to loop over the panels? **Answer:** A 2-D NumPy array of four Axes. Loop with `for ax, item in zip(axs.flat, items):`, which works for every grid shape.
2. Why is a twin-axis chart of two different measures risky, and what do you draw instead? **Answer:** The two scales are arbitrary, so where the lines sit, cross or overlap can be changed without changing the data. Stack two panels with `sharex=True`, or plot the ratio itself.
3. Rewrite the title "Monthly online revenue, 2024 and 2025" as a finding, and say what you checked first. **Answer:** For example "Online revenue beat 2024 in every month of 2025", after checking that the 2025 value is higher in all 12 months (it is; the smallest gap is December, $4,803).

Reading the tickets: question 1 answered with "a list" or with `for ax in axs` (which loops over rows of a 2-D array) needs the shape table again at the start of Session 10. For question 2 accept "the scales can be chosen" in any words;

"it looks cluttered" misses the point. For question 3 the check matters more than the wording: a ticket with a headline and no check is the learner to talk to first next time.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

WRONG ANSWER OR SYMPTOM	WHAT IT REVEALS	WHAT TO DO
Q1 answered `(3, 2)` or `6 6`	Columns and rows swapped, or `len(fig.axes)` confused with the array size.	Draw the grid with indexes in each cell; rows first, as in NumPy.
`Q2 answered D`	Thinks panels are numbered from 1 (the old `plt.subplot(2, 2, 1)` style).	Arrays index from 0; `plt.subplots` returns an array.
Q3 answered A or B	Treats sharing as a rule rather than a choice for the question.	Ask: "Is this figure about size or about shape?"
`Q5 answered A`	The twin-axis demonstration has not landed.	Show demo step 8 again: same data, two right-axis ranges, two stories.
`Q7 answered B`	Writes the story first and checks later.	Lab 2.7-B: the yearly online return rate fell from 7.11% to 6.76%.
`Q9 answered B`	Knows "sequential for numbers" but not the role of a meaningful midpoint.	Lab 3.3-B Step 2: where does white fall with uncentered limits?
Q11 answered `True True True`	Thinks the style is applied when the figure is drawn, not when it is created.	Create the figure inside the block; anything made outside keeps the defaults.
Lab 3.2-A: the gray lines cover the colored one	Drawing order not yet understood.	The last call is drawn on top; or `zorder=3`.
Lab 3.2-A: `axs[0].plot` on the 2 by 2 grid	Indexes a row instead of a panel.	`print(axs.shape)`; then `axs[0, 0]` or the loop.
Lab 3.3-A: headline written before TODO 1	Finding titles treated as decoration.	Ask for the number that proves each word of the headline.
Lab 3.3-A: `mdates.MonthLocator` on the month-number axis	Did not check the type of the x values.	The x values are 1 to 12, not dates: `set_xticks(range(1, 13), labels=MONTHS)`.

WRONG ANSWER OR SYMPTOM	WHAT IT REVEALS	WHAT TO DO
Lab 3.3-B: diverging map with `vmin` / `vmax` from the data	Does not see why the midpoint must be zero.	Find the color of a +5% cell with and without symmetric limits.

INSTRUCTOR KIT · ANSWER KEY

Session 10: Saving Figures; Center and Spread

Self-check answers with the reasoning for every option, the predict-then-run answers, reference notes for the four labs and two debug handouts, the lesson pages' own quiz answers, and the warm-up and exit-ticket answers. Every output block is the real output of the code shown.

Session 10 | 12 self-check items | Lessons 3.4-4.1 | Confidential

HOW THIS KEY IS ORGANIZED

The 12 self-check items are generated from the same list as the workbook and the facilitator guide, so the numbering, wording and option letters match exactly. Each item shows all four options with the correct one marked, then why it is right and why each other option is wrong. Lab outputs below are the real output of the `*_solution.py` files in `labs/module-10/`; `python3 labs/check_labs.py module-10` runs them all.

3.4 · SAVING FIGURES: SELF-CHECK

Q1

A figure is created with `figsize=(9, 5)` and saved with `fig.savefig("sales.png", dpi=120)`. How many pixels is the image?

A) 900 wide by 500 high

B) 1,080 wide by 600 high

C) 600 wide by 1,080 high

D) 129 wide by 125 high

Why: pixels = inches x dpi, one dimension at a time: 9 x 120 = 1,080 wide and 5 x 120 = 600 high.
A is wrong: That is the size at the default 100 dpi; the call asks for 120.
C is wrong: `figsize` is (width, height), so the 9 belongs to the width.
D is wrong: dpi multiplies the inches. It is not added to them.

Q2

What does this print?

```
fig, ax = plt.subplots(figsize=(4, 3))
fig.savefig("q2.png", dpi=50)
print(plt.imread("q2.png").shape[:2])
```

`(150, 200)`

> **Why:** `plt.imread` returns an array shaped (rows, columns, channels), so the height comes first: 3 x 50 = 150 rows and 4 x 50 = 200 columns. A common wrong answer is `(200, 150)`: width first, the way `figsize` is written.

Q3

Two PNG files are both 1,600 by 800 pixels. File A came from `figsize=(16, 8)` at `dpi=100`, file B from `figsize=(8, 4)` at `dpi=200`. Which statement is true?

A) The two images are identical, because the pixel counts are identical

B) B is blurrier, because its sheet was smaller

C) A holds more detail, because its figure was larger

D) In B the text, lines and markers are twice as large relative to the chart

> **Why:** Text and line widths are measured in points (1/72 inch), so they take up twice the share of an 8-inch sheet that they take of a 16-inch sheet. The dpi then scales the whole sheet to the same 1,600 pixels.
> **A** is wrong: Same pixel count, different drawing: the sheet size decided how big the text is on the sheet.
> **B** is wrong: Both files have exactly 1,600 by 800 pixels; neither is blurrier.
> **C** is wrong: Detail in a PNG is its pixel count, and the counts are equal.

Q4

A scatter plot of 200,000 points must go into a printed report as a PDF. The file is huge and slow to open. What is the best fix?

A) Pass `rasterized=True` to the scatter layer and keep saving as PDF

B) Save as JPEG instead, because JPEG files are small

C) Lower the `dpi` of the PDF to 50

D) Add `bbox_inches="tight"` to the save call

> **Why:** A vector file stores one drawing instruction per dot. Rasterizing only the heavy layer turns the dots into an embedded image, while the axes, title and labels stay sharp vector text.
> **B** is wrong: JPEG smears sharp edges and text; the lesson says never to use it for charts.
> **C** is wrong: Vector marks have no dpi, so the 200,000 instructions are all still there.
> **D** is wrong: That changes the saved area, not the number of marks in the file.

Q5

A chart is saved with `transparent=True` and pasted onto a dark slide. The title and tick labels cannot be seen. Why?

A) Transparent PNG files cannot contain text

B) The slide program removed the labels when it resized the image

C) Only the background became transparent; the text is still black, on a dark slide

D) The dpi was too low for the labels to be drawn

> **Why:** `transparent=True` removes the figure's background and nothing else. Black labels on a dark page disappear. Draw the chart in a dark style, or save with a solid `facecolor="white"`.
> **A** is wrong: The text is in the file; it is simply the same color as what is behind it.
> **B** is wrong: Resizing does not delete parts of an image.
> **D** is wrong: A low dpi gives fewer pixels, not missing labels.

Q6

What does this print?

```
plt.close("all")
first, ax = plt.subplots()
second, ax = plt.subplots()
first.savefig("q6.png")
plt.close(first)
print(len(plt.get_fignums()))
```

`1`

Why: Saving a figure does not close it. Two figures were created and one was closed, so pyplot is still holding `second`. In a loop that makes hundreds of charts this is how memory fills up: create, draw, save, close.

4.1 · DESCRIPTIVE STATISTICS I: CENTER AND SPREAD: SELF-CHECK

Q7

What does this print?

```
print(pd.Series([10, 20, 30, 40, 400]).median())
```

A) `100.0`

B) `30.0`

C) `25.0`

D) `400`

Why: The median is the middle value of the sorted list. With five values that is the third one, 30. The 400 could be 4,000,000 and the answer would not change.

A is wrong: That is the mean (500 / 5): the balance point, dragged up by the 400.

C is wrong: That is the mean of the two values next to the middle of the first four; there are five values, so there is a single middle.

D is wrong: That is the maximum.

Q8

`orders["payment_method"]` **holds the words card, digital_wallet, cash and gift_card. Which summary of its center makes sense?**

A) The mean

B) The median

C) The mode

D) The standard deviation

> **Why:** Categories cannot be added or put in order, so the only center is the most common value: `.mode()` (card, with 12,392 of the 20,302 orders).
> **A** is wrong: `.mean()` on text raises `TypeError: Cannot perform reduction 'mean' with string dtype`.
> **B** is wrong: A median needs an order from smallest to largest; payment methods have none.
> **D** is wrong: That is a measure of spread, and it needs numbers.

Q9

What does this print?

```
x = np.array([1, 2, 3, 4, 5])
print(np.var(x), pd.Series(x).var())
```

`2.0 2.5`

> **Why:** The squared deviations from the mean (3) are 4, 1, 0, 1, 4: a sum of 10. NumPy divides by n (10 / 5 = 2.0, `ddof=0`); pandas divides by n - 1 (10 / 4 = 2.5, `ddof=1`).

Q10

Pinecrest's order totals have a mean of $194.60, a median of $146.85 and a mode of $53.90. Finance wants a revenue forecast for 1,000 orders next month. Which number do you multiply by 1,000?

A) The mean, $194.60

B) The median, $146.85

C) The mode, $53.90

D) The midpoint of the mean and the median

Why: Only the mean multiplies back to a total: mean x count = sum. The median describes a typical order, and it would understate revenue by about a quarter here.
B is wrong: 20,302 orders x $146.85 is about $2.98 million; the real total is $3.95 million.
C is wrong: The mode of a measured quantity is mostly an accident: $53.90 is 114 of 20,302 orders.
D is wrong: An invented number that is neither the typical order nor the total divided by the count.

Q11

One order in a column of 20,000 order totals was typed as $1,000,000 by mistake. Which pair of summaries barely changes?

A) Mean and standard deviation

B) Range and standard deviation

C) Mean and range

D) Median and interquartile range

Why: The median and the IQR depend only on the order of the values near the middle. One extreme row is just "one more value above the middle". They are robust.
A is wrong: Both are built from every value's distance, so one huge value has huge leverage: in the lesson the SD went from $178.76 to $7,019.02.
B is wrong: The range IS the two extremes, and the SD squares the distance to the bad row.
C is wrong: The mean rose by $49 in the lesson's stress test, and the range became $999,993.62.

Q12

What does this print?

```
revenue = pd.Series([100.0, np.nan, 300.0])
print(revenue.mean(), np.mean(revenue.to_numpy()), revenue.count())
```

A) `200.0 nan 2`

B) `133.33 nan 3`

C) `200.0 200.0 3`

D) `nan nan 2`

Why: pandas skips missing values (`skipna=True`), so its mean is (100 + 300) / 2. NumPy's `np.mean` does not skip: one `nan` makes the answer `nan`, with no error. `.count()` counts the values that are present.
B is wrong: pandas does not treat the missing value as 0, and `.count()` leaves it out.
C is wrong: `np.mean` returns `nan`; `np.nanmean` is the NumPy function that skips.
D is wrong: The pandas method skips the missing value instead of spreading it.

PREDICT, THEN RUN: ANSWERS

Predict 3.4-1

What does this print? (Careful: which number comes first?)

```
fig, ax = plt.subplots(figsize=(5, 2))
fig.savefig("charts/p1.png", dpi=300)
plt.close(fig)
print(plt.imread("charts/p1.png").shape[:2])
```

```
(600, 1500)
```

Why: `shape` is (rows, columns): the height comes first. 2 inches x 300 dpi = 600 rows, and 5 x 300 = 1,500 columns.

Predict 3.4-2

What two numbers does this print?

```
for i in range(3):
    fig, ax = plt.subplots()
    fig.savefig(f"charts/p2_{i}.png")
print(len(plt.get_fignums()))
plt.close("all")
print(len(plt.get_fignums()))
```

```
3
0
```

Why: Saving does not close a figure, so after the loop pyplot is still holding all three. `plt.close("all")` releases them. In your own loops, close each figure right after saving it.

Predict 4.1-1

Three numbers: the mean, the median and the mode. Predict all three.

```
s = pd.Series([3, 5, 5, 8, 50])
print(s.mean(), s.median(), s.mode()[0])
```

```
14.2 5.0 5
```

Why: The sum is 71, so the mean is 14.2: larger than four of the five values, because the 50 has leverage. The middle value of the sorted list is 5, and 5 is also the only value that appears twice.

Predict 4.1-2

Same three numbers, two libraries. What does each print?

```
x = np.array([2, 4, 9])
print(np.var(x), pd.Series(x).var())
```

```
8.666666666666666 13.0
```

Why: The mean is 5 and the squared deviations are 9, 1 and 16: a sum of 26. NumPy divides by n = 3 (8.67); pandas divides by n - 1 = 2 (13.0). With only three values the two differ by a third.

LAB REFERENCE SOLUTIONS

LAB 3.4-A · `LABS/MODULE-10/L3_4-SAVING-FIGURES/PIXEL_BUDGET_SOLUTION.PY`

One drawing function, `weekday_chart(figsize)`, is called with a different sheet size for each destination. The email file is 7 x 100 by 3.5 x 100 pixels. The two newsletter files reach 1,400 by 700 in two ways: 14 x 7 inches at `dpi=100` and 7 x 3.5 inches at `dpi=200`. The answer to TODO 3 is the point of the lab: the pixel counts are identical, but on the smaller sheet the 12-point title is twice the share of the page, so after scaling it is about 33 pixels tall instead of 17. The PDF and SVG have no pixel size at all. Accept any wording that says "text size is set in points on the sheet; the sheet size, not the dpi, decides how big it looks".

```
mean visitors by weekday: [276, 259, 269, 283, 350, 492, 412]
email: (700, 350) 20 KB
bend_news_big_sheet.png      (1400, 700)
bend_news_small_sheet.png    (1400, 700)
Same pixels. On the 7 x 3.5 inch sheet the text is twice as large relative
to the chart: a 12-point title is 12/72 inch on any sheet, and the dpi then
scales the whole sheet.
title height in pixels: big sheet 16.7 | small sheet 33.3
bend_poster.pdf   19.4 KB (vector: no pixel size to check)
bend_poster.svg   35.1 KB (vector: no pixel size to check)
5 files in OUTPUT:
  bend_email.png
  bend_news_big_sheet.png
  bend_news_small_sheet.png
  bend_poster.pdf
  bend_poster.svg
open figures: 0
```

Check quickly while circulating: the two `(1400, 700)` lines, and `open figures: 0` on the last line. A learner who sees `open figures: 3` saved correctly and forgot `plt.close(fig)`. File sizes in KB may differ slightly from machine to machine; pixel sizes may not.

LAB 3.4-B · `LABS/MODULE-10/L3_4-SAVING-FIGURES/CATEGORY_PRICE_PACK_SOLUTION.PY`

TODO 1 has more than one right answer for the web preset (8 x 4 at 120, or 9.6 x 4.8 at 100, give 960 x 480); the solution uses `(8, 4)` at 120. For the slide, `(9.6, 5.4)` at 200 gives 1,920 x 1,080 with readable text; `(19.2, 10.8)` at 100 gives the right pixels and tiny text, which is the mistake to talk about. TODO 2 is one line: `fig.savefig(path, dpi=preset["dpi"], facecolor="white")`. The loop is create, draw, save, close; `PdfPages` gathers the three print-size figures into one file. The manifest then verifies the pack without opening a file by hand.

```
          count  median
category
Apparel      35   66.95
Footwear     28   74.95
Tents        24  298.45
price_ranges.pdf has 3 pages
              file      pixels  over_5_kb
     tents_web.png   960 x 480       True
   tents_slide.png 1920 x 1080       True
  footwear_web.png   960 x 480       True
footwear_slide.png 1920 x 1080       True
   apparel_web.png   960 x 480       True
 apparel_slide.png 1920 x 1080       True
  price_ranges.pdf                   True
files: 7 | web 960 x 480: True | slides 1920 x 1080: True | open figures: 0
```

Common wrong results: every size reads `640 x 480` (the presets were not changed from the Matplotlib default); `FileNotFoundError` (the `folder.mkdir` line was deleted); a manifest with 6 files (the PDF path was not appended to `saved`); `open figures: 9` (no `plt.close`).

DEBUG HANDOUT · `LABS/MODULE-10/L3_4-SAVING-FIGURES/DEBUG_BLANK_SLIDE.PY`

As shipped it prints `saved size: 660 x 500`, `slide is blank: True`, `open figures: 2`. Three faults in one line: (1) `plt.savefig()` saved the **current** figure, the empty `scratch`, so the file is blank: write `chart.savefig(...)`; (2) `bbox_inches="tight"` crops or grows the saved area, so the size is not the size asked for: remove it (the figure already uses `layout="constrained"`); (3) a 19.2 x 10.8 inch sheet at 100 dpi has the right pixels and text far too small for a slide: use `figsize=(9.6, 5.4)` with `dpi=200`, which keeps 1,920 x 1,080. Finally `plt.close("all")` at the end. The fixed script prints `slide is 1920 x 1080: True`, `slide is blank: False`, `open figures: 0`.

LAB 4.1-A · `LABS/MODULE-10/L4_1-CENTER-AND-SPREAD/ORDERS_PER_BUYER_SOLUTION.PY`

`orders.groupby("customer_id").size()` gives one number per buyer. The three centers disagree in the order that right-skewed counts always produce: mode (1) below median (4) below mean (5.15). "Our customers order about five times" quotes the mean; 65% of buyers are below it. The mean is still the right number for one job: mean x buyers gives back the 20,302 orders exactly, while median x buyers gives 15,768. Step 4 shows that this is not about one extreme buyer: removing the heaviest buyer (41 orders) moves the mean by 0.01 and the median not at all. The tail is long, not a single point.

```
buyers: 3942 | orders: 20302
mean  : 5.15
median: 4.0
mode  : 1 (719 buyers ordered exactly 1 time)
share of buyers below the mean: 0.651
largest five counts: [36, 37, 37, 39, 41]
mean   x buyers: 20302
median x buyers: 15768
real total     : 20302
without the top buyer: mean 5.14 | median 4.0
A typical buyer placed 4 orders in two years (the median).
The most common count is 1.
The mean is 5.15 because a long tail of frequent buyers pulls it up:
65% of buyers are below it. Use the mean only for totals
(mean x buyers = orders).
```

Common wrong results: `mode` printed as a whole Series (forgot `[0]`); a share of `0.5` (compared with the median instead of the mean); a mean of 4.19 (counted orders per row of `customers.csv` after a merge, which adds the 1,058 accounts that never ordered: a different, also interesting, question).

LAB 4.1-B · `LABS/MODULE-10/L4_1-CENTER-AND-SPREAD/VISITOR_SPREAD_SOLUTION.PY`

Step 1 is the lesson's hand calculation on new numbers: deviations that sum to zero, a sum of squares of 2,554.8, then 510.96 (divide by 5) or 638.7 (divide by 4). Step 2 matches them to the libraries: `np.var` is the divide-by-n version, pandas `.var()` the n - 1 version, and `np.std(five, ddof=1)` equals the pandas SD. Step 3 uses `groupby().apply()` with a function that returns a Series, then `.unstack()` to get one row per store. Step 4 is a deliberate twist: Asheville has the smallest SD in visitors (89.5) and Boise the largest (161.8), but Boise is also the biggest store. Relative to its own mean, Flagstaff varies most (0.357). Which answer is "the steadiest" depends on whether the staffing question is about people or about percentages; a good answer says which. Step 5: one bad row triples Bend's SD (109.1 to 302.8) and multiplies the range by 12, while the median and the IQR do not move.

```
rows: 2903 | visitors present: 2903
five days         : [200 220 235 253 263]  mean: 234.2
deviations        : [-34.2 -14.2   0.8  18.8  28.8]  sum: 0.0
sum of squares    : 2554.8
variance, / n     : 510.96000000000004
variance, / (n-1) : 638.7
np.var(five)             : 510.96000000000004
pd.Series(five).var()    : 638.7
np.std(five)             : 22.6
pd.Series(five).std()    : 25.27
np.std(five, ddof=1)     : 25.27
            mean  median  range     sd    iqr
store
Asheville  274.9   252.0  578.0   89.5  104.0
Bend       334.9   310.0  653.0  109.1  131.2
Boise      485.2   444.0  987.0  161.8  196.0
Flagstaff  328.5   304.0  728.0  117.4  140.0
            mean     sd  sd_over_mean
store
Asheville  274.9   89.5         0.326
Bend       334.9  109.1         0.326
Boise      485.2  161.8         0.333
Flagstaff  328.5  117.4         0.357
smallest SD: Asheville | largest SD: Boise
largest SD relative to its mean: Flagstaff
clean  mean  334.9  median 310.0  range   653  SD  109.1  IQR 131.25
broken mean  344.8  median 310.0  range  7808  SD  302.8  IQR 131.25
```

Common wrong results: an SD of 22.6 for the five days where 25.27 was expected (NumPy's default); a table that is all `NaN` (TODO 3 still returns the placeholders); a range of `nan` for a store (used `revenue`, which has 6 missing values, instead of `visitors`, which has none).

DEBUG HANDOUT · `LABS/MODULE-10/L4_1-CENTER-AND-SPREAD/DEBUG_AVERAGE_OF_AVERAGES.PY`

Finding 1 averages four group means as if the groups were the same size ($190.56). Card has 12,392 orders and gift_card 1,502, so the right figure comes from the rows: `orders["order_total"].mean()` = $194.60, or `np.average(by_method, weights=orders.groupby("payment_method")["order_total"].count())`. **Finding 2** prints `nan` because `np.mean` does not skip the 6 missing revenue values: use `daily["revenue"].mean()` or `np.nanmean(revenue)` = $7,435.04, and say that it is based on 2,897 of 2,903 rows. **Finding 3**: analyst A used NumPy (`ddof=0`, 37.13) and analyst B pandas (`ddof=1`, 40.1). Seven days standing in for the store in general is a sample, so report 40.1 and write `ddof` explicitly in the code.

THE LESSON PAGES' OWN QUICK QUIZZES

Each lesson page ends with a five-question Quick Quiz (three options each). Learners do them at home; these are the correct options and the lesson's own reasons.

Lesson 3.4: Q1 C, Q2 B, Q3 A, Q4 B, Q5 A

Lesson 3.4 · Quiz 1

A figure is created with `figsize=(8, 4)` and saved with `fig.savefig("chart.png", dpi=150)`. How large is the image?

C) 1,200 by 600 pixels

> **Why:** Pixels are inches times dpi: 8 times 150 is 1200 wide and 4 times 150 is 600 high.

Lesson 3.4 · Quiz 2

What does this print?

```
quiz_fig, _ = revenue_chart("Bend", figsize=(6, 3))
quiz_fig.savefig(out / "quiz.png", dpi=200)
plt.close(quiz_fig)
print(plt.imread(out / "quiz.png").shape)
```

B) `(600, 1200, 4)`

> **Why:** The array shape is rows, columns, channels. The image is 3 inches high at 200 dpi, so 600 rows, and 6 inches wide, so 1200 columns, with 4 channels: red, green, blue and alpha.

Lesson 3.4 · Quiz 3

A line chart is going into a printed annual report, and the designer may need to resize it during layout. Which format is the best choice?

A) PDF or SVG, because vector files stay sharp at any size

> **Why:** A vector file stores drawing instructions instead of pixels, so it stays sharp at any size the designer chooses, and its text remains real text.

Lesson 3.4 · Quiz 4

A chart saved from `figsize=(16, 9)` at `dpi=120` fills a slide exactly (1,920 by 1,080), but nobody can read the labels. What is the best fix?

B) Use `figsize=(8, 4.5)` at `dpi=240`

> **Why:** Text is sized in points, a fixed fraction of an inch, so on a smaller sheet it takes up more of the chart. Halving the figure size and doubling the dpi keeps 1920 by 1080 pixels and makes the text twice as large.

Lesson 3.4 · Quiz 5

A script saves 300 charts in a loop and gets slower and slower until it runs out of memory. What is missing?

A) `plt.close(fig)` after each `savefig`

> **Why:** pyplot keeps every figure it creates until it is closed. Closing each figure after saving it releases its memory, so the loop uses the same small amount for every chart.

Lesson 4.1: Q1 B, Q2 A, Q3 C, Q4 A, Q5 C

Lesson 4.1 · Quiz 1

Pinecrest's order totals have a mean of $194.60 and a median of $146.85. What is the best explanation?

B) A long tail of large orders pulls the mean up, while the median stays with the bulk of the orders

> **Why:** The distribution is right-skewed. Large orders have leverage on the mean, the balance point, but each of them counts as just one value above the middle for the median.

Lesson 4.1 · Quiz 2

What does this print?

```
ratings = np.array([2, 4, 4, 4, 5, 5, 7, 9])
print(np.std(ratings))
```

A) `2.0`, because NumPy divides the sum of squared deviations by `n`

> **Why:** The mean is 5 and the squared deviations add up to 32. NumPy divides by n, which is 8, giving a variance of 4 and a standard deviation of 2.0. pandas would divide by 7 and print about 2.14.

Lesson 4.1 · Quiz 3

A column of delivery times is strongly right-skewed and may contain a few data-entry errors. Which pair of summaries describes it most faithfully?

C) Median and interquartile range

> **Why:** The median and the IQR depend only on the order of the values near the middle, so a long tail or a few bad rows barely move them.

Lesson 4.1 · Quiz 4

What does this print?

```
pay = pd.Series(["card", "cash", "card", "gift_card", "cash"])
print(pay.mode().tolist())
```

A) `['card', 'cash']`

> **Why:** card and cash both appear twice, so there is a tie, and mode() returns a Series containing every winner, sorted.

Lesson 4.1 · Quiz 5

Store A took 10 orders with a mean of $100. Store B took 90 orders with a mean of $200. What is the mean of all 100 orders?

C) $190

> **Why:** Total dollars are 10 x 100 + 90 x 200 = 19,000 over 100 orders, so the mean is 190. Group means must be weighted by group size, never averaged directly.

WARM-UP AND EXIT TICKET ANSWERS

WARM-UP

1. `fig, axs = plt.subplots(2, 2)`: what kind of object is `axs`, and what loop visits every panel whatever the grid shape? **Answer:** A 2-D NumPy array of Axes. `for ax, group in zip(axs.flat, groups):` works for every grid shape.
2. What does `sharey=True` buy you in a grid of panels, and what does it cost? **Answer:** Heights become comparable across panels. The cost: small series are flattened, so shape is harder to see. Choose on purpose and tell the reader.
3. Why is a twin-axis chart with two different measures risky, and what should you draw instead? **Answer:** The two scales are arbitrary, so the chart can be made to say almost anything. Use stacked panels with a shared x axis, or plot a ratio.
4. What should a finished chart's title say? And the subtitle? **Answer:** The title states the finding. The subtitle says what is plotted, the units and the period.
5. You have a dictionary of style settings. How do you apply it to one block of plotting code without changing the rest of the session? **Answer:** `with plt.rc_context(style_dict):` around the block.

EXIT TICKET

1. A figure has `figsize=(6, 4)` and is saved with `dpi=150`. How many pixels wide and high is the PNG? **Answer:** 900 wide by 600 high (6 x 150 and 4 x 150).
2. A column of order totals is strongly right-skewed. Which measure of center and which measure of spread do you report for a typical order? **Answer:** The median and the IQR. (The mean is for totals.)
3. `np.std(x)` and `pd.Series(x).std()` give different answers for the same small list. Why? **Answer:** NumPy divides by n (`ddof=0`); pandas divides by n - 1 (`ddof=1`).

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

WHAT YOU SEE OR HEAR	WHAT IT REVEALS	WHAT TO DO
"I raised the dpi and the text is still tiny."	They think of dpi as zoom on the text. They have one knob in their head, not two.	Back to the sheet and the scanner. Halve the `figsize`, double the dpi, compare the two files and their `imread` shapes.
Self-check Q2 answered `(200, 150)`.	They read `shape` as (width, height), the order used by `figsize`.	"Rows first, like a DataFrame." Have them print `image.shape` for a tall, narrow figure.
A blank PNG and no error.	`plt.savefig()` saved pyplot's current figure, not theirs; or they saved after `plt.show()` in a script.	Ask them to point at the figure's name in the save line. Rewrite it as `fig.savefig(...)`.
Q3 answered "identical, the pixel counts are the same".	They equate an image with its pixel dimensions and have not connected points to inches.	Open the lesson's side-by-side figure or the lab's two newsletter files. Ask what an 11-point label means on each sheet.
"The median is $147, so 20,302 orders bring in about $2.98 million."	They treat every center as interchangeable; they have not met the idea that only the mean multiplies back to a total.	Compute `totals.sum()` and both products. The mean is for totals, the median for typical.
"The mean is wrong because of outliers; delete the bulk orders."	They believe skew is caused by a handful of errors.	Rerun the demo's bulk check: removing the twelve moves the mean by $2. The whole tail does it, and the bulk orders are real.
Q9 answered `2.5 2.5` or `2.0 2.0`.	They do not know the two libraries have different defaults.	Have them run `np.std(x)`, `pd.Series(x).std()` and `np.std(x, ddof=1)` on five numbers and say which two match.

WHAT YOU SEE OR HEAR	WHAT IT REVEALS	WHAT TO DO
Q12 answered `nan nan 2`.	They expect pandas to behave like NumPy with missing values (or the reverse).	Print `revenue.count()` next to `len(revenue)`, then `revenue.mean()` and `np.mean(revenue.to_numpy())`.
Exit ticket 2 answered "mean and SD".	Reporting habit: mean and SD are what they have always seen.	Ask for the histogram and for the share of values below the mean. If it is far from half, lead with median and IQR.

INSTRUCTOR KIT · ANSWER KEY

Session 11: Shape, Outliers and Distributions

Self-check answers with the reasoning for every option, the predict-then-run answers, reference notes for the four labs and two debug handouts, the lesson pages' own quiz answers, and the warm-up and exit-ticket answers. Every output block is the real output of the code shown.

Session 11 | 12 self-check items | Lessons 4.2–4.3 | Confidential

HOW THIS KEY IS ORGANIZED

The 12 self-check items are generated from the same list as the workbook and the facilitator guide, so the numbering, wording and option letters match exactly. Each item shows all four options with the correct one marked, then why it is right and why each other option is wrong. Lab outputs below are the real output of the `*_solution.py` files in `labs/module-11/`; `python3 labs/check_labs.py module-11` runs them all. The two 4.3 labs are seeded, so their random results are the same on every run of the whole file.

4.2 · DESCRIPTIVE STATISTICS II: SHAPE, POSITION AND OUTLIERS: SELF-CHECK

Q1

Pinecrest's order totals have a skewness of 5.48. After `np.log(totals)` the skewness is -0.41. What does that tell you?

A) The logarithm deleted the twelve bulk orders

B) The original totals contained errors that the logarithm repaired

C) On the log scale the pile is roughly symmetric, so methods that prefer symmetric data will behave better there

D) The mean of the logs is the log of the mean

Why: A logarithm turns equal ratios into equal distances, which pulls in a long right tail. Between -0.5 and 0.5 counts as roughly symmetric. No row was removed or changed.

A is wrong: Every one of the 20,302 orders is still there; a transform changes the scale, not the rows.

B is wrong: The totals are correct. Skew is a shape, not an error.

D is wrong: That is false in general, and it is not what skewness measures.

Q2

What does this print?

```
print(pd.Series([10, 20, 30, 40]).quantile(0.5))
```

`25.0`

Why: The 0.5 quantile is the median. With four values the cut falls between 20 and 30, and pandas interpolates halfway by default. `quantile` takes a fraction from 0 to 1; `quantile(50)` raises a ValueError.

Q3

A $12,000 day has a z-score of 3.05 at the Asheville store and 0.50 at the Boise store. What does that mean?

A) The day is far above Asheville's own average, measured in Asheville's SDs, and only a little above Boise's

B) Asheville took 3.05 times as much revenue as Boise that day

C) The day was in the top 3.05% of Asheville's days

D) z-scores from different stores cannot be compared

Why: z = (value - mean) / SD, using each store's own mean and SD. $12,000 is about three SDs above Asheville's mean of $5,704 and half an SD above Boise's mean of $10,144: exceptional in one store, routine in the other.
B is wrong: A z-score is a distance in SDs, not a ratio of revenues. The revenue is $12,000 in both cases.
C is wrong: A z-score is not a percentage. Turning it into "how rare" needs the shape of the distribution (in the data, 1.5% of Asheville's days reach $12,000).
D is wrong: Comparing across different scales is exactly what z-scores are for: the units cancel.

Q4

On the order totals the IQR rule flags 848 orders and the rule "|z| above 3" flags 217. Why does the z-score rule flag so many fewer?

A) The z-score rule only looks at the smallest values

B) The IQR rule counts duplicated orders twice

C) z-scores cannot be computed for skewed data

D) The mean and SD are inflated by the same large orders being judged, which pushes the z-score line outward; the quartiles are not

> **Why:** The z rule is built on the mean and SD, which are not robust: the tail stretches them, so the cutoff lands near $732. The IQR fences are built on quartiles, which the tail cannot move, so the fence stays at $544.
> **A** is wrong: Both rules look at both sides; here neither can flag a small order (the lower fence is negative).
> **B** is wrong: There are no duplicates in `orders.csv`, and the rule does not count rows twice.
> **C** is wrong: A z-score is always a valid distance. What fails on skewed data is reading it as "how rare".

Q5

What does this print?

```
s = pd.Series([1, 2, 3, 4, 5, 6, 7, 8, 9, 50])
q1, q3 = s.quantile(0.25), s.quantile(0.75)
upper = q3 + 1.5 * (q3 - q1)
print(upper, (s > upper).sum())
```

`14.5 1`

> **Why:** Q1 is 3.25 and Q3 is 7.75, so the IQR is 4.5 and the upper fence is 7.75 + 6.75 = 14.5. Only the 50 lies beyond it. A box plot of this Series would end its upper whisker at 9, the largest value inside the fence, and draw the 50 as a separate point.

Q6

In the survey, Online shoppers' mean `sat_staff` is 4.07, computed from 248 of the 824 Online respondents. What is the honest way to report it?

A) Report 4.07 as the staff satisfaction of Online shoppers

B) Report it with its n and say who answered: 70% of Online shoppers skipped the question, so it describes the minority who dealt with staff

C) Fill the 576 missing answers with 4.07 so that the n is 824

D) Leave the Online row out of the table without comment

> **Why:** The answers are not missing at random: most online shoppers never met the staff. A grouped statistic always carries its n, and here the n changes what the number means.
> **A** is wrong: It hides that the figure rests on 30% of the group, who may differ from the rest.
> **C** is wrong: Inventing 576 answers makes the n look solid and adds no information.
> **D** is wrong: Silently dropping a group hides the finding about who did not answer.

4.3 · PROBABILITY DISTRIBUTIONS: SELF-CHECK

Q7

You want to model the number of returned orders in a day: events scattered through time at a steady average rate, with no fixed number of tries. Which distribution do you try first, and what do you check?

A) Uniform; check that the histogram is flat

B) Normal; check that the 68-95-99.7 rule holds

C) Binomial with n = 731; check that p is the same every day

D) Poisson; check that the mean and the variance are about equal

> **Why:** Counts of events in a fixed interval at a steady rate are Poisson, whose mean and variance are both the rate. For daily returns the data gives a mean of 1.564 and a variance of 1.526.
> **A** is wrong: Zero, one and two returns are far more common than six or seven; nothing about the counts is flat.
> **B** is wrong: The normal is continuous and symmetric. These are small counts that cannot go below zero.
> **C** is wrong: 731 is the number of days in the data, not a number of tries within one day.

Q8

What does this print?

```
print(stats.norm(loc=20, scale=3).cdf(20))
```

`0.5`

> **Why:** `cdf(x)` is the probability of x or less. A normal distribution is symmetric about its mean, so half of it lies at or below 20, whatever the SD.

Q9

`batch = stats.binom(n=25, p=0.07)` models returns in a batch of 25 orders. Which expression is the probability of 3 or more returns?

A) `batch.sf(2)`

B) `batch.sf(3)`

C) `batch.pmf(3)`

D) `batch.cdf(3)`

> **Why:** `sf(k)` is the probability of more than k. For a count, "3 or more" is "more than 2".
> **B** is wrong: `sf(3)` is more than 3: 4 or more. This off-by-one slip is the most common binomial error.
> **C** is wrong: `pmf(3)` is exactly 3.
> **D** is wrong: `cdf(3)` is 3 or fewer.

Q10

The standard deviation of order totals is about $178.76. What is the standard error of the mean of a random sample of 400 orders?

A) About $178.76

B) About $8.94

C) About $0.45

D) About $3,575

Why: SE = SD / sqrt(n) = 178.76 / 20, about $8.94. The SD describes how much single orders vary; the SE describes how much a sample mean would vary from sample to sample.
A is wrong: That is the SD. It does not shrink as you collect more data; the SE does.
C is wrong: That divides by n (400) instead of by the square root of n.
D is wrong: That multiplies by the square root of n.

Q11

From 200 orders an auditor gets a 95% confidence interval for the mean order total that is about $44 wide. Roughly how many orders are needed for an interval half as wide?

A) 100

B) 400

C) 800

D) 20,000

Why: The width is about 2 x t x SD / sqrt(n). To halve it, the square root of n must double, so n must be four times as large: 800. Precision is expensive.
A is wrong: A smaller sample gives a wider interval.
B is wrong: Doubling n shrinks the width by a factor of about 1.41, not 2.
D is wrong: That would shrink the width by a factor of 10, far more than asked.

Q12

What is true of the two lists this prints?

```
for _ in range(2):
    rng = np.random.default_rng(7)
    print(rng.integers(1, 7, size=3).tolist())
```

A) They are identical: each pass builds a new generator from the same seed, which replays the same numbers

B) They differ, because a generator never repeats itself

C) The second list continues where the first one stopped

D) The code raises an error, because a seed can be used only once

> **Why:** A seed fixes the whole sequence. Re-creating the generator inside the loop restarts that sequence every time, so a simulation written this way repeats one experiment over and over. Create the generator once, at the top.
> **B** is wrong: A seeded generator is reproducible by design.
> **C** is wrong: That is what would happen if ONE generator were created before the loop.
> **D** is wrong: A seed can be reused as often as you like.

PREDICT, THEN RUN: ANSWERS

Predict 4.2-1

One number. Careful: is the answer one of the values in the list?

```
print(pd.Series([10, 20, 30, 40, 50]).quantile(0.25))
```

```
20.0
```

> **Why:** The 0.25 quantile sits a quarter of the way along the sorted values. With five values that is exactly the second one, 20. (With four values, `[10, 20, 30, 40]`, the cut falls between two values and pandas interpolates: 17.5.)

Predict 4.2-2

Three z-scores. Predict the list.

```
s = pd.Series([2, 4, 6])
z = (s - s.mean()) / s.std()
print(z.tolist())
```

```
[-1.0, 0.0, 1.0]
```

Why: The mean is 4. The sample SD (pandas divides by n - 1) is the square root of (4 + 0 + 4) / 2, which is 2. So the values are one SD below, exactly at, and one SD above the mean.

Predict 4.3-1

Two numbers. No calculator needed.

```
model = stats.norm(loc=50, scale=10)
print(model.cdf(50), model.ppf(0.5))
```

```
0.5 50.0
```

Why: A normal distribution is symmetric about its mean, so half of it is at or below 50: `cdf(50)` is 0.5. `ppf` is the reverse of `cdf`: the value with half the distribution below it is 50.

Predict 4.3-2

Ten coin flips. Three numbers: predict each.

```
flips = stats.binom(n=10, p=0.5)
print(flips.mean(), flips.cdf(10), flips.sf(10))
```

```
5.0 1.0 0.0
```

Why: The expected count is n x p = 5. `cdf(10)` is the probability of 10 or fewer heads: certain. `sf(10)` is the probability of more than 10 heads in 10 flips: impossible.

LAB REFERENCE SOLUTIONS

LAB 4.2-A · `LABS/MODULE-11/L4_2-SHAPE-POSITION-OUTLIERS/CATALOG_WEIGHTS_SOLUTION.PY`

The lesson's whole outlier workflow on a new column. Shape first: the mean (0.793 kg) is more than twice the median (0.35 kg) and the skewness is 3.46; on the log scale it is 0.18. Position: 90% of weighed products are at or below about 2 kg. Note the `dropna()` before computing that share: comparing a missing value with 2 gives False, so `(weight <= 2).mean()` would silently count the 14 unweighed products as "not at or below 2 kg" and print 84.7% instead of 89.9%. Then the two rules: the IQR fences (-0.872 to 1.908 kg; the lower fence is negative, so nothing can be "too light") flag 26 products, and `abs(z) > 3` flags 4. Step 4 is the point: the 26 are mostly tents (11), and the 4 are a cooler and three 6-person family tents. They are real products, and they are exactly the ones a shipping team must plan for. Nothing is removed; the answer reports the median with its n and says so.

```
rows: 242 | weights present: 228 | missing: 14
mean: 0.793 | median: 0.35
skew: 3.46 | skew of log(weight): 0.18
0.10    0.087
0.25    0.170
0.50    0.350
0.75    0.865
0.90    2.001
0.95    3.203
share of weighed products at or below 2 kg: 89.9%
IQR fences: -0.872 to 1.908 kg
flagged by the IQR rule: 26 | flagged by |z| > 3: 4
category
Tents           11
Snow             5
Camp Kitchen     4
Climbing         3
Packs            2
Sleeping         1

                                  name     category  weight_kg
                  Quillback Canyon Cooler Camp Kitchen       4.37
Tallgrass Timberline 6-Person Family Tent        Tents       6.11
 Juniper Vane Meadow 6-Person Family Tent        Tents       8.31
     Quillback Talus 6-Person Family Tent        Tents       8.37
               n  rows  median    q3
category
Accessories   31    32    0.12  0.32
Apparel       33    35    0.22  0.39
Camp Kitchen  26    29    0.26  0.57
Snow          18    22    0.33  1.92
Climbing      23    24    0.35  0.50
Footwear      26    28    0.48  0.59
Sleeping      23    24    0.74  1.32
Packs         24    24    0.84  1.41
Tents         24    24    1.56  3.13
Typical product weight: 0.35 kg (median of 228 weighed products;
14 have no weight recorded). Middle half: 0.17 to 0.86 kg.
Nothing removed: the 26 flagged products are real (mostly tents). They need
their own box size, not deletion.
```

Check quickly while circulating: the line `flagged by the IQR rule: 26 | flagged by |z| > 3: 4`.
Common wrong results: 84.7% in Step 2 (missing values counted as False); 0 flagged by both rules (the placeholder Series were not replaced); a "typical weight" of 0.79 kg (the mean, on a column with skewness 3.46); a final sentence that says the tents were removed.

LAB 4.2-B · `LABS/MODULE-11/L4_2-SHAPE-POSITION-OUTLIERS/PRICE_SATISFACTION_SOLUTION.PY`

The overall distribution first (41% neutral, 41.5% satisfied, 17.5% dissatisfied), then named aggregation by age group with two small functions. The medians are useless for comparison (3.0 for five of the six groups), which is the lesson's point about ordinal data; the shares carry the finding. Among the 186 customers aged 18 to 24 who answered, 19.4% are satisfied with prices and 38.2% are dissatisfied; in every other group 40 to 51% are satisfied and 14 to 17% dissatisfied. Step 3 is a contrast with the lesson's staff question: here each group skipped at about 2 to 3%, so nothing suggests the answers are missing for a reason connected to age. The chart is saved to `OUTPUT`; the lab never calls `plt.show()`. Whether the gap could be chance is a Session 12 question; the description, with its n, comes first.

```
responses: 1800 | answered the price question: 1753
           count  percent
sat_price
1.0           35      2.0
2.0          271     15.5
3.0          719     41.0
4.0          574     32.7
5.0          154      8.8
median: 3.0 | mean (a rough index only): 3.31
              n  rows  median  satisfied_pct  dissatisfied_pct
age_group
18-24       186   192     3.0           19.4              38.2
25-34       273   280     3.0           39.6              14.3
35-44       309   318     3.0           44.7              15.2
45-54       332   339     3.0           42.8              13.6
55-64       273   279     4.0           51.3              15.0
65+         380   392     3.0           43.2              16.6
percent who skipped the question, by age group:
age_group
18-24    3.1
25-34    2.5
35-44    2.8
45-54    2.1
55-64    2.2
65+      3.1
saved: price_satisfaction_by_age.png (440, 990)
Among the 186 customers aged 18-24 who answered, 19% are
satisfied with prices and 38% are dissatisfied; among the other
1567 it is 44% and 15%.
```

Common wrong results: `dissatisfied_pct` all `NaN` (TODO 2 still returns `np.nan`); a table with `mean` as the headline number (accept it only beside the shares, to one decimal); `n` equal to `rows` (used `"size"` twice, so the missing answers are hidden); no file saved (TODO 5: the `fig.savefig` line must come before `plt.close(fig)`).

DEBUG HANDOUT · `LABS/MODULE-11/L4_2-SHAPE-POSITION-OUTLIERS/DEBUG_SUMMARY_REPORT.PY`

Result 1 passes a fraction to a function that wants 0 to 100: `np.percentile(totals, 0.90)` is the 0.9th percentile, $13.50. Use `np.percentile(totals, 90)` or `totals.quantile(0.90)`: $405.40. **Result 2** draws an empty box because `daily["revenue"]` has six missing values and Matplotlib does not skip them (the median line is at `nan`): pass `daily["revenue"].dropna()`. **Result 3** describes two identifier columns; a "mean order id" is arithmetic on labels: select only `order_total`. **Result 4** prints group means with no n: add `count` and `size` (`.agg(["mean", "count", "size"])`) and the Online row turns out to rest on 248 of 824 respondents. Better still, report the share satisfied for an ordinal item.

LAB 4.3-A · LABS/MODULE-11/L4_3-PROBABILITY-DISTRIBUTIONS/CONVERSION_MODEL_SOLUTION.PY

Describe, freeze, check, then use. Conversion (transactions divided by visitors) has a mean of 0.2354, an SD of 0.0271 and a skewness of 0.05: one even mound, so a normal model is worth trying. `stats.norm(loc=mu, scale=sigma)` freezes it. The 68-95-99.7 check agrees with the data to about one percentage point (69.4%, 95.6%, 99.5%). The managers' questions map onto the four methods: below 20% is `cdf(0.20)`, above 27% is `sf(0.27)`, a bad day (the 5th percentile) is `ppf(0.05)`; each is printed beside the same quantity from the data. `model.rvs(size=365, random_state=rng)` draws a simulated year from the one generator created at the top. Because the generator is seeded and used once, the count of simulated bad days is the same on every run of the file.

```
store-days: 2903 | missing: 0
mean: 0.2354 | SD: 0.0271 | skew: 0.05
within 1 SD:  data 69.4%   model 68.3%
within 2 SD:  data 95.6%   model 95.4%
within 3 SD:  data 99.5%   model 99.7%
P(conversion below 20%): model 0.096   data 0.090
P(conversion above 27%): model 0.101   data 0.108
5th percentile (a bad day)   : model 0.1908   data 0.1920
95th percentile (a great day): model 0.2801   data 0.2804
simulated days below 20% in a year: 34
the model expects                 : 34.9
On a typical day about 23.5% of visitors buy. Below 19.1% is a bad day
(1 day in 20); above 28.0% is a great one. The normal model matches the
data to within about one percentage point for every question asked.
```

Common wrong results: a "within 1 SD" model share far from 68% (passed `sigma ** 2` as `scale`); `ppf(20)` returning `nan` (`ppf` takes a probability, `cdf` takes a value); P(above 27%) of about 0.9 (used `cdf` where `sf` was needed); a different simulated count on every run (created a new unseeded generator, or ran the cell twice in a notebook: both worth a conversation, neither wrong arithmetic).

LAB 4.3-B · LABS/MODULE-11/L4_3-PROBABILITY-DISTRIBUTIONS/LINES_PER_ORDER_CI_SOLUTION.PY

The lesson's inference steps on a count. The population is every order's number of product lines (mean 2.241, SD 1.116, skew 0.91). The intern's sample of 150 has a mean of 2.093: the best single guess, and not the truth. SE = s / sqrt(n) = 0.0821, which `stats.sem` confirms, and `stats.t.interval` gives 1.931 to 2.256, which contains the true mean (only just: a reminder that the truth can sit anywhere inside an interval, and outside it for about 1

sample in 20). Step 4 is the lesson's Common Mistake 6 with a twist: line counts are whole numbers, so the only orders "inside" the interval are those with exactly 2 lines, 35.7% of them. The interval is a statement about the mean, not about where orders fall. Step 5 repeats the study 5,000 times: 95.2% of the intervals contain the true mean, and the SD of the 5,000 sample means (0.0904) matches sigma / sqrt(n) (0.0911), which is the central limit theorem doing its work. Step 6: four times the sample halves the width (0.358 to 0.179 lines), because the SE divides by the square root of n.

```
orders: 20302 | mean lines: 2.241 | SD: 1.116 | skew: 0.91
sample mean: 2.093 | sample SD: 1.006
SE by hand: 0.0821 | scipy.stats.sem: 0.0821
95% CI: 1.931 to 2.256
contains the true mean? True
share of individual orders inside the interval: 35.7%
(only the orders with exactly 2 lines; the interval describes the mean, not orders)
intervals that contain the true mean: 95.2% of 5,000
SD of the 5,000 sample means: 0.0904 | sigma / sqrt(n): 0.0911
n = 150: median width of the 95% CI = 0.358 lines
n = 600: median width of the 95% CI = 0.179 lines
```

Common wrong results: an SE of 1.0 (reported the SD); an interval of about 0.1 to 4.1 (used `scale=s` instead of `scale=se`: that is a range for single orders, not for the mean); coverage of 100% or 0% (compared the intervals with the sample mean instead of `population.mean()`); `ddof` left at 0 in `many.std(axis=1)` (the coverage drops slightly). Different numbers from the key in Steps 5 and 6 mean a cell was run twice or out of order; the conclusions are the same.

DEBUG HANDOUT · `LABS/MODULE-11/L4_3-PROBABILITY-DISTRIBUTIONS/DEBUG_DELIVERY_MODEL.PY`

Answer 1 passes the variance as `scale`: `stats.norm(mu, sigma)` gives 0.025, close to the data's 0.019. **Answer 2** reads `uniform(10, 20)` as (low, high); the second argument is the width, so the model runs from 10 to 30. `stats.uniform(loc=10, scale=10).sf(18)` is 0.2. **Answer 3** is off by one: "5 or more" is `batch.sf(4)` = 0.072, not `sf(5)` = 0.024. **Answer 4** re-creates the generator with the same seed on every pass, so all 1,000 "batches" are the same draw: create `rng` once, before the loop (or draw all at once with `size=1_000`).

THE LESSON PAGES' OWN QUICK QUIZZES

Each lesson page ends with a five-question Quick Quiz (three options each). Learners do them at home; these are the correct options and the lesson's own reasons.

Lesson 4.2: Q1 C, Q2 A, Q3 B, Q4 B, Q5 A

Lesson 4.2 · Quiz 1

A column has a skewness of 2.4. Which description fits best?

C) Most values are low, with a long tail of large values on the right

> **Why:** Positive skewness means the long tail is on the right. Most values are small, a few are much larger, and those few pull the mean above the median. Median and IQR are the safer summary.

Lesson 4.2 · Quiz 2

A store's visitor count on one day has a z-score of -2. What does that mean?

A) The count was two standard deviations below that store's average

> **Why:** A z-score is a distance from the mean measured in standard deviations, with the sign giving the direction. Minus 2 is two standard deviations below that store's mean.

Lesson 4.2 · Quiz 3

In a standard box plot, where does the upper whisker end?

B) At the largest data value that is no more than 1.5 IQRs above Q3

> **Why:** The fence is at Q3 + 1.5 x IQR, but the whisker is drawn only as far as the last actual data point inside that fence. Values beyond it are drawn as individual points.

Lesson 4.2 · Quiz 4

The IQR rule flags 4% of Pinecrest's orders. You check a sample and they are genuine large purchases. What should you do for a report on total revenue?

B) Keep them, because they are real and they are revenue

> **Why:** A flag is a prompt to investigate. These values are real and belong to the question, so they stay. If you also describe a typical order, use the median and IQR so the tail does not distort it.

Lesson 4.2 · Quiz 5

What does this print?

```
answers = pd.Series([5, 5, 4, None, 1])
print(answers.mean(), answers.count())
```

A) `3.75 4`

Why: pandas skips the missing answer: 15 divided by the 4 answers present is 3.75, and count() reports 4. The mean describes the people who answered, not all five who were asked.

Lesson 4.3: Q1 A, Q2 C, Q3 B, Q4 C, Q5 B

Lesson 4.3 · Quiz 1

A warehouse inspects 50 parcels. Each has the same 3% chance of being damaged, independently of the others. Which distribution models the number of damaged parcels?

A) Binomial, with n = 50 and p = 0.03

Why: A fixed number of independent tries (50), each with the same probability (0.03), counting successes: that is the definition of a binomial distribution with n = 50 and p = 0.03.

Lesson 4.3 · Quiz 2

Which value is closest to what this prints?

```
scores = stats.norm(loc=100, scale=15)
print(round(scores.cdf(130), 3))
```

C) 0.977

Why: 130 is 2 SDs above the mean. About 95 percent of a normal lies within 2 SDs, leaving 2.5 percent in each tail, so about 97.7 percent lies below 130.

Lesson 4.3 · Quiz 3

What does `stats.norm(loc=100, scale=15).ppf(0.975)` return?

B) The value that 97.5% of the distribution lies below, about 129.4

Why: ppf is the reverse of cdf: give it a probability and it returns the value with that much probability below it. Here that is 100 + 1.96 x 15, about 129.4.

Lesson 4.3 · Quiz 4

Order totals are strongly right-skewed. You repeatedly draw random samples of 200 orders and record each sample's mean. What does the central limit theorem say about those means?

C) The means are approximately normal, centered on the population mean, with SD about σ/√200

> **Why:** The distribution of the sample mean approaches a normal curve centered on the population mean, with a standard deviation of sigma divided by the square root of n, whatever the shape of the population.

Lesson 4.3 · Quiz 5

From a random sample, you compute a 95% confidence interval for the mean order total: $178 to $222. Which statement is correct?

B) The method that produced this interval captures the true mean in about 95% of samples

> **Why:** The 95 percent describes the procedure. Across many repeated samples, about 95 percent of intervals built this way contain the true mean. It is a statement about the mean, not about individual orders.

WARM-UP AND EXIT TICKET ANSWERS

WARM-UP

1. A figure with `figsize=(10, 5)` is saved with `dpi=200`. How many pixels wide and high is the PNG? **Answer:** 2,000 wide by 1,000 high: pixels = inches x dpi.
2. The labels on your slide image are too small. Which two settings do you change, and what stays the same? **Answer:** A smaller `figsize` and a higher `dpi`, so the pixel size stays the same. Raising the dpi alone does not make text bigger.
3. Pinecrest's mean order is $194.60 and the median is $146.85. Why do they differ, and which describes a typical order? **Answer:** The totals are right-skewed: a long tail of large orders pulls the mean up. The median describes a typical order; the mean is for totals.
4. `np.std(x)` and `pd.Series(x).std()`: which divides by n and which by n - 1? **Answer:** NumPy divides by n (`ddof=0`); pandas divides by n - 1 (`ddof=1`).
5. What is the IQR, and why prefer it to the SD for a skewed column? **Answer:** Q3 minus Q1, the width of the middle half of the data. It is robust: extreme values cannot inflate it.

EXIT TICKET

1. The IQR rule flags 91 store-days as unusually high. What do you do before removing any of them? **Answer:** Investigate them: are they provable errors, and are they part of the question? (In the lesson's project all 91 are weekends, promotions or December days: real, and kept.)

2. You have a frozen distribution called `model`. Which method returns the value that 95% of the distribution lies below? **Answer:** `model.ppf(0.95)`. (`cdf` goes from a value to a probability; `ppf` goes back.)

3. A 95% confidence interval for the mean order total is $162 to $204. Does that mean 95% of orders are between $162 and $204? If not, what does the 95% describe? **Answer:** No. It is a statement about the mean, not about single orders. The 95% describes the method: about 95 of 100 intervals built this way contain the true mean.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

WHAT YOU SEE OR HEAR	WHAT IT REVEALS	WHAT TO DO
"The rule flagged them, so I deleted them."	They treat an outlier rule as an error detector.	Ask the two questions (provable error? part of the question?). Have them compute the share of revenue or the kinds of products they just removed.
Self-check Q3 answered "top 3.05%".	They read a z-score as a percentile.	Compute both for the same day: `z` and `(revenue >= 12000).mean()`. A z-score is a distance; rarity needs the shape.
`np.percentile(totals, 0.9)` used for the 90th percentile.	They carry the pandas 0-to-1 scale into NumPy.	Print it next to `totals.quantile(0.9)` and next to `describe()`. $13.50 cannot be the 90th percentile of orders with a median of $147.
A table of Likert means with no counts.	They treat 1-to-5 answers as measurements, and a mean as self-explanatory.	Add `count` and `size`. Ask for the share satisfied. Show the Online row: 248 answers from 824 respondents.
"The data becomes normal when the sample is big enough."	A misreading of the central limit theorem.	Plot 20,302 order totals: as skewed as ever. What becomes normal is the distribution of the sample mean.
Q9 answered `batch.sf(3)`.	They read `sf(k)` as "k or more".	Have them print `1 - batch.cdf(2)`, `batch.sf(2)` and `batch.sf(3)` and say each in words.
Q10 answered $178.76 or $0.45.	They confuse SD with SE, or divide by n instead of its square root.	Rerun the central limit table: the SD of the means matches sigma / sqrt(n) at every n.
"95% of orders are in the confidence interval." / "There is a 95% chance the true mean is in it."	They read the interval as a range for the data, or attach the probability to one interval instead of to the method.	Compute the share of orders inside the interval (small). Rerun the coverage cell: the 95% is a percentage of intervals.

WHAT YOU SEE OR HEAR	WHAT IT REVEALS	WHAT TO DO
Different random numbers from the guide, and worry.	They do not yet see a generator as a sequence.	Restart the kernel and run all cells once, in order: the numbers match. Then run one cell twice and watch what follows change.

INSTRUCTOR KIT · ANSWER KEY

Session 12: Correlation and Hypothesis Tests

Self-check answers with the reasoning for every option, the predict-then-run answers, reference notes for the four labs and two debug handouts, the lesson pages' own quiz answers, and the warm-up and exit-ticket answers. Every output block is the real output of the code shown.

Session 12 | 12 self-check items | Lessons 4.4–4.5 | Confidential

HOW THIS KEY IS ORGANIZED

The 12 self-check items are generated from the same list as the workbook and the facilitator guide, so the numbering, wording and option letters match exactly. Each item shows all four options with the correct one marked, then why it is right and why each other option is wrong. Lab outputs below are the real output of the `*_solution.py` files in `labs/module-12/`; `python3 labs/check_labs.py module-12` runs them all. Items 1 to 6 (Lesson 4.4) involve no hypothesis tests, as in the lesson.

4.4 · CORRELATION ANALYSIS: SELF-CHECK

Q1

Across the 234 products that have sold, the number of reviews and the units sold have r = 0.46. Which reading is soundest?

A) Asking customers for more reviews will raise sales

B) Products with more reviews tend to have sold more; the arrow may run from sales to reviews, and the data alone cannot say

C) 46% of a product's sales are explained by its reviews

D) The correlation is too weak to mean anything

Why: A correlation describes how two columns move together. Every sale is a chance to leave a review, so the arrow can run backward; only an experiment (asking for reviews on some products and not others) could show that reviews cause sales.

A is wrong: That reads the correlation as a cause, in one chosen direction.

C is wrong: r is not a percentage. The share of variance a line accounts for is r squared: about 20% here.

D is wrong: 0.46 is a moderate, clearly visible association. Weak is not the same as meaningless.

Q2

What does this print?

```
x = pd.Series([1, 2, 3, 4])
y = pd.Series([10, 8, 6, 4])
print(round(x.corr(y), 2))
```

`-1.0`

> **Why:** Every step up in x is exactly a step of 2 down in y: all points lie on one falling straight line, so r is -1. r measures how tightly points hug a line, not how steep it is (a slope of -2 still gives -1).

Q3

To check whether staffing and revenue are linked beyond the calendar, which calculation compares like with like?

A) Correlate the z-scores of staff and revenue instead of the raw values

B) Use Spearman's correlation instead of Pearson's

C) Subtract each group's mean (same store, month, weekday and promotion status) from both columns with `groupby().transform("mean")`, then correlate the deviations

D) Remove the outlier days first, then recompute r

> **Why:** Deviations from comparable days remove what the calendar explains, so what is left compares days that a manager expected to be alike. In the lesson r falls from 0.88 to 0.14.
> **A** is wrong: r is already built from z-scores; standardizing does not change it.
> **B** is wrong: Spearman handles curves and outliers. It does not remove a confounder.
> **D** is wrong: The busy days are real and are exactly where the confounder acts; dropping them does not compare like with like.

Q4

For which pair of columns is Spearman clearly the better choice than Pearson?

A) Visitors and revenue, which form a straight, tight band

B) Revenue in dollars and the same revenue in cents

C) Two columns that are identical

D) 1-to-5 overall satisfaction and the 0-to-10 recommend score

> **Why:** Survey answers are ordinal: ordered, without guaranteed equal steps. Spearman works on ranks, so it needs only the order. (The lesson gets 0.63 for this pair, from 1,699 people who answered both.)
> **A** is wrong: A straight band without extreme points is exactly Pearson's home ground.
> **B** is wrong: A change of units does not change r at all, by either method.
> **C** is wrong: Both methods give exactly 1.

Q5

A line fitted to two columns with r = 0.5. What does this print?

```
r = 0.5
print(f"the line accounts for {r ** 2:.0%} of the variance")
```

`the line accounts for 25% of the variance`

> **Why:** The share of variance a straight line accounts for is r squared. r = 0.5 sounds like "halfway" and accounts for a quarter; r = 0.3 leaves more than 90% unexplained.

Q6

A regression of revenue on staff on duty gives a slope of $1,154 per extra staff member. A manager plans to add two people every Tuesday to gain about $2,300 a day. What is the problem?

A) A slope describes how revenue differs across days with different staffing, not what changing the staffing would do; here it is mostly the calendar acting on both

B) The slope must first be multiplied by r

C) The intercept was left out of the calculation

D) Regression cannot be used when one column holds whole numbers

Why: Staff are scheduled from expected demand, which also drives revenue. Among comparable days the correlation falls to 0.14, and holding visitors constant to 0.07. The slope is a correct description and a poor forecast of an intervention.
B is wrong: A slope is already in dollars per staff member; multiplying by r has no meaning.
C is wrong: For a change of two staff the intercept cancels: only the slope matters. The problem is causal, not arithmetic.
D is wrong: Whole-number predictors are fine for a least-squares line.

4.5 · HYPOTHESIS TESTING: SELF-CHECK

Q7

Shuffling the store labels 10,000 times gives a p-value of 0.030 for Asheville's lower recommend score. What does 0.030 measure?

A) In a world where the store makes no difference, about 3% of shuffles produced a gap at least as large as the real one

B) There is a 3% probability that the null hypothesis is true

C) There is a 97% probability that Asheville really is worse

D) The gap is 3% of the average score

Why: A p-value is computed by assuming the null is true: it describes how often chance alone gives data at least this extreme. It is not the probability of any hypothesis.
B is wrong: The p-value assumes the null; it cannot also be the probability that the null is true.
C is wrong: The same mistake turned around. A test does not give the probability that a hypothesis is true.
D is wrong: A p-value is not a size. The gap is 0.36 points; its effect size is d = -0.17.

Q8

You want to compare 1-to-5 staff-satisfaction ratings at Asheville (193 answers) with the other stores (754 answers). Which test fits best?

A) A Welch t-test on the two means

B) A paired t-test

C) A one-sample t-test against 4

D) A Mann-Whitney U test

Why: Ratings are ordinal, so a rank-based test for two independent groups is the honest choice. Report the probability of superiority, U / (n1 x n2), as the effect size (0.37 in the lesson).
A is wrong: A mean assumes the step from 1 to 2 equals the step from 4 to 5; for ratings that is doubtful.
B is wrong: The groups are different people; nothing pairs one Asheville answer with one other-store answer.
C is wrong: That compares one group with a target, not two groups with each other.

Q9

What does this print?

```
print(stats.ttest_ind([1, 2, 3], [1, 2, np.nan], equal_var=False).pvalue)
```

`nan`

Why: SciPy's tests do not skip missing values the way pandas `.mean()` does. One `nan` makes the result `nan`, with no error. Fix: `.dropna()` first, or pass `nan_policy="omit"`.

Q10

A Welch t-test reports a 95% confidence interval for the difference in means of -0.68 to -0.03. Without computing it, what do you know about the two-sided p-value at alpha = 0.05?

A) It is above 0.05

B) It is exactly 0.05

C) It is below 0.05

D) It cannot be known from the interval

Why: A 95% interval that excludes 0 and a two-sided p below 0.05 are the same fact seen from two sides (the lesson's test gives p = 0.030). The interval adds what p cannot: the gap could be anywhere from tiny to about two thirds of a point.

A is wrong: That would go with an interval that contains 0.

B is wrong: That would put 0 exactly on an end of the interval.

D is wrong: For the same test and confidence level, the interval and the p-value always agree on this question.

Q11

A pilot with 10 Asheville answers and 50 others gives p = 0.82 for a staff-rating gap that the full survey shows clearly (p near 3 in a billion). How should the pilot's result be worded?

A) The stores' staff ratings are the same

B) We could not detect a difference with this sample; it is too small to say much

C) The pilot proves the null hypothesis

D) The full survey must contain an error

Why: With ten answers the test has very little power, so it missed a real difference: a Type II error. A large p-value means the data are compatible with no difference, and may be just as compatible with a real one.

A is wrong: "Not significant" means "not detected", never "the same".

C is wrong: A test never proves the null; it can only fail to reject it.

D is wrong: Small samples disagreeing with large ones is exactly what low power predicts.

Q12

Five independent tests, all of true null hypotheses, each at alpha = 0.05. What does this print?

```
print(f"{1 - 0.95 ** 5:.0%}")
```

23%

Why: Each test has a 95% chance of no false alarm, so all five stay quiet with probability 0.95 to the fifth power, about 0.77. With 30 tests the chance of at least one false alarm is 79%. Count every test you ran, and correct with Holm.

PREDICT, THEN RUN: ANSWERS

Predict 4.4-1

Two correlations. Predict both before you run.

```
u = pd.Series([-2, -1, 0, 1, 2])
print(u.corr(u ** 2), u.corr(u * 3 + 1))
```

```
0.0 1.0
```

Why: u squared is completely determined by u, but the relationship is a symmetric curve with no straight-line trend, so r is 0. `u * 3 + 1` is a straight rising line, so r is exactly 1, whatever the slope.

Predict 4.4-2

Pearson, then Spearman. Which one is 1?

```
a = pd.Series([1, 2, 3, 4, 5])
b = pd.Series([1, 4, 9, 16, 100])
print(round(a.corr(b), 2), round(a.corr(b, method="spearman"), 2))
```

```
0.8 1.0
```

Why: b goes up every time a goes up, so the ranks match perfectly: Spearman is 1. The values do not lie on a straight line (the last jump is huge), so Pearson is lower, 0.8.

Predict 4.5-1

Two identical groups. What are t and p?

```
res = stats.ttest_ind([1, 2, 3], [1, 2, 3], equal_var=False)
print(res.statistic, res.pvalue)
```

```
0.0 1.0
```

Why: The difference in means is exactly 0, so t is 0, and every possible result is at least as extreme as this one: p = 1. Data that match the null perfectly are not surprising at all under it.

Predict 4.5-2

A table in which both rows have the same proportions. Predict the statistic, the p-value and the expected counts.

```
table = pd.DataFrame([[30, 10], [30, 10]])
chi = stats.chi2_contingency(table)
print(chi.statistic, chi.pvalue, chi.expected_freq.tolist())
```

```
0.0 1.0 [[30.0, 10.0], [30.0, 10.0]]
```

Why: The expected counts are what independence predicts, and here the real counts are exactly those, so the distance (the chi-square statistic) is 0 and p is 1.

LAB REFERENCE SOLUTIONS

LAB 4.4-A · `LABS/MODULE-12/L4_4-CORRELATION-ANALYSIS/REVIEWS_AND_SALES_SOLUTION.PY`

Look, measure, then question the direction. The scatter plots come first and are saved to `OUTPUT` (the lab never calls `plt.show()`). Reviews and units sold: Pearson 0.456, Spearman 0.442, a moderate association that is roughly linear (the two methods agree). Star rating and units sold: essentially nothing (0.095 and 0.012). Price and units: Pearson -0.319 against Spearman -0.600, the lesson's curved relationship again. The Spearman matrix and heatmap (scale fixed at -1 to 1 so 0 is the neutral color) show one more thing: price is related to both reviews (-0.27) and units (-0.60), so it is a candidate confounder. Holding price constant leaves the reviews-units link at 0.424: price does not explain it away. The direction is still open: every sale is a chance for a review. The honest answer for marketing says so, and names what would settle it (an experiment). No p-values anywhere: that is Lesson 4.5's question.

```
products that have sold: 234
review_count vs units sold: Pearson  0.456   Spearman  0.442
rating       vs units sold: Pearson  0.095   Spearman  0.012
price        vs units sold: Pearson -0.319   Spearman -0.600
              price  rating  review_count  units_sold
price          1.00   -0.01         -0.27       -0.60
rating        -0.01    1.00          0.06        0.01
review_count  -0.27    0.06          1.00        0.44
units_sold    -0.60    0.01          0.44        1.00
reviews and units sold, controlling for price: 0.424
Products with more reviews have sold more units (r about 0.46), and the link
survives holding price constant (0.42). But the arrow can run backward:
every sale is a chance for a review. These data cannot say that more reviews
would cause more sales; a test (reviews requested for some products, not others)
could. Star ratings show no link with units sold (Spearman about 0.01).
```

Common wrong results: a `ValueError` about strings (called `catalog.corr()` on the whole table: select numeric columns); a heatmap whose colors shift between runs (no `vmin=-1, vmax=1`); an answer that says "reviews cause sales" or "46% of sales".

LAB 4.4-B · `LABS/MODULE-12/L4_4-CORRELATION-ANALYSIS/STAFFING_RULE_SOLUTION.PY`

The regression recovers the managers' rule of thumb: about 1.6 staff for every 100 visitors, plus a base of under one person, with r = 0.925 (r squared 0.856). For a 400-visitor day the line predicts 7.2 staff, give or take about one (the SD of the residuals, 0.95). Fitted store by store, the slopes are 1.55 to 1.63: one rule for the whole chain, with slightly different bases. Step 4 shows that regression is not symmetric: predicting visitors from staff gives 53.9 visitors per staff member, not 1 / 0.0159 = 63.0, although r is the same both ways. Step 5 ties it to the lesson: the slope describes scheduling, and "more staff bring more visitors" is the arrow pointing the wrong way.

```
store-days: 2903 | visitors 125 to 1164 | staff 2 to 20
slope    : 0.0159 staff per visitor
         = 1.59 staff per 100 visitors
intercept: 0.84
r        : 0.925 | r squared: 0.856
predicted staff for a 400-visitor day: 7.2
typical miss (SD of the residuals)   : 0.95 staff
Asheville  slope per 100 visitors  1.63   intercept  0.86
Bend       slope per 100 visitors  1.56   intercept  0.66
Boise      slope per 100 visitors  1.55   intercept  1.08
Flagstaff  slope per 100 visitors  1.63   intercept  0.76
visitors predicted from staff: slope 53.9 visitors per staff member
1 / (staff per visitor)      : 63.0  (not the same line)
r both ways: 0.925 0.925
The slope describes how staffing differs across days with different traffic:
about 1.6 more staff per 100 extra visitors, the same in all four stores.
It does not say that adding staff brings visitors: managers set staff FROM
expected visitors. And it says nothing about a 3,000-visitor day (outside 125-1164).
```

Common wrong results: a slope of 54 or 63 in Step 1 (arguments reversed: `linregress(predictor, target)`, predictor first); "staff per visitor 0.0159" read as "almost nothing" (say it per 100 visitors); a prediction for 3,000 visitors reported without comment (outside the data's 125 to 1,164).

DEBUG HANDOUT · `LABS/MODULE-12/L4_4-CORRELATION-ANALYSIS/DEBUG_CORRELATION_CLAIMS.PY`

Claim 1: `nan` comes from NumPy and the six missing revenues, not from a missing relationship: `daily["visitors"].corr(daily["revenue"])` gives 0.94. **Claim 2**: r is not a slope; the slope from `stats.linregress` is about $21 per visitor. **Claim 3**: r is not a percentage; the line accounts for r squared, about 81%, and "explains" is still a description, not a cause (staffing follows expected traffic). **Claim 4**: r = 0 means no *linear* relationship; u squared is completely determined by u. Only a scatter plot can show there is no relationship.

LAB 4.5-A · `LABS/MODULE-12/L4_5-HYPOTHESIS-TESTING/RETURNED_ORDER_SIZE_SOLUTION.PY`

Hypotheses first, in a comment (H0: equal means; H1: different, two-sided; alpha 0.05). Step 1 justifies the method: very unequal group sizes and spreads (so Welch), strong skew, but groups of 1,143 and 18,762 orders (so the central limit theorem makes the means close to normal). Welch: returned orders average $25.54 more (95% CI $15.29 to $35.79; t = 4.89, p = 1.1e-06). Cohen's d is 0.14: small, the two groups overlap almost completely. The permutation test reports 0.0002, the smallest value 9,999 shuffles can give: no shuffle produced a gap as large. Step 5 challenges the result with a confounder: online orders are larger (mean $204.65 against $177.25) and are returned more often. Within online orders alone the gap is still $23.85 (CI $12.29 to $35.42), so it is not only a channel effect. The report states "real and small" and keeps "observational" as its caveat.

```
returned  n  1143  mean  218.72  median  179.85  SD 171.00  skew  1.6
completed n 18762  mean  193.17  median  145.90  SD 179.60  skew  5.7
difference 25.54, 95% CI 15.29 to 35.79
t = 4.89, df = 1300, p = 1.1e-06
Cohen's d = 0.14
permutation p-value = 0.0002  (9,999 shuffles: 0.0002 is the smallest possible)
channel
online    204.65
store     177.25
online only: difference 23.85, 95% CI 12.29 to 35.42
             p = 5.5e-05, d = 0.12
Returned orders average about $25 more than completed ones (95% CI $15 to $36;
Welch p = 1.1e-06; permutation test agrees). The effect is small (d = 0.14):
the two groups overlap almost completely. Within online orders alone the gap
is about $24, so it is not only a channel difference. Observational data.
```

Common wrong results: a p-value of 2.9e-06 (forgot `equal_var=False`: Student's test); a report that says "returned orders are much bigger" (d = 0.14); "there is a 0.0001% chance the null is true" (the classic misreading: ask what p is computed assuming).

LAB 4.5-B · `LABS/MODULE-12/L4_5-HYPOTHESIS-TESTING/FOOTWEAR_RETURNS_SOLUTION.PY`

Lesson 3.3's chart, now tested. 4,342 orders contain at least one footwear line. Online: 14.4% of footwear orders were returned against 4.8% of the rest, three times as often; chi-square 313.8 with 1 degree of freedom, p = 3.3e-70; the smallest expected count is 197.6, far above 5, so the approximation is sound; Cramér's V is 0.157, which undersells a tripled return rate, so the report leads with the rates and their ratio. Stores: 3.8% against 3.3%, p = 0.43: no difference detected, which is worded as "not detected", not "none". Two tests, so Holm: online 6.6e-70, store 0.43.

```
orders with footwear: 4342 of 20302
returned  False  True
footwear
False      9520    480
True       2444    410
returned: with footwear 14.4%, without 4.8%, ratio 3.0
chi-square = 313.8, df = 1, p = 3.3e-70
smallest expected count: 197.6 | Cramer's V: 0.157
store: with footwear 3.8%, without 3.3%
       chi-square = 0.63, p = 0.43
smallest expected count: 50.5
Holm-adjusted p-values: online 6.6e-70 | store 0.43
Online orders that contain footwear were returned 14.4% of the time, against 4.8%
for other online orders: three times as often (chi-square p far below 0.001 after
Holm; V = 0.16). In stores the rates are 3.8% and 3.3% (p = 0.43): no difference
was detected. That is not proof of none: the store gap could be small and real.
```

Common wrong results: a `TypeError` from `chi2_contingency` (passed the raw columns instead of the `crosstab` table); a pooled test over both channels that blends two different stories; "in stores footwear makes no difference" (not supported by a non-significant test).

DEBUG HANDOUT · `LABS/MODULE-12/L4_5-HYPOTHESIS-TESTING/DEBUG_TEST_REPORT.PY`

Conclusion 1: `nan` comes from missing values in `ash` (no `dropna()`), not from the data showing nothing: with `.dropna()` p is 0.030. **Conclusion 2**: a p-value is not the probability that the null is true; say "if the stores had the same mean, a gap this large would appear in about 3% of samples". **Conclusion 3**: p = 0.57 means no difference was detected; the 95% CI (about -0.51 to 0.28) shows the data are also compatible with a real gap of a third of a point either way. **Conclusion 4**: significant is not big: the shortfall is 0.14 points and d = -0.07. **Conclusion 5**: choosing `alternative="less"` after seeing the means halves p (0.030 becomes 0.015) by sleight of hand; use the two-sided test unless a direction was fixed in advance.

THE LESSON PAGES' OWN QUICK QUIZZES

Each lesson page ends with a five-question Quick Quiz (three options each). Learners do them at home; these are the correct options and the lesson's own reasons.

Lesson 4.4: Q1 B, Q2 C, Q3 A, Q4 B, Q5 A

Lesson 4.4 · Quiz 1

Across Pinecrest's store-days, staff on duty and revenue have r = 0.88. Staff are scheduled in advance from expected demand. What is the soundest conclusion?

B) Staffing and revenue rise and fall together, largely because both follow expected demand

> **Why:** Expected demand drives both the schedule and the visitors who produce revenue, so it is a confounder. Among comparable days the correlation falls to about 0.14.

Lesson 4.4 · Quiz 2

For price and units sold, Pearson's r is -0.32 and Spearman's is -0.60. What does the gap suggest?

C) Units fall consistently as price rises, but along a curve, not a straight line

> **Why:** Spearman works on ranks and detects any consistently falling pattern. Pearson only credits a straight line. A much stronger Spearman points to a monotonic relationship that is curved or distorted by extreme values.

Lesson 4.4 · Quiz 3

You compute the correlation between daily visitors and daily revenue in dollars. Then you convert revenue to cents and compute it again. What happens to r?

A) Nothing: r is unchanged

> **Why:** r is built from z-scores, and multiplying a column by a positive constant does not change its z-scores. Correlation has no units.

Lesson 4.4 · Quiz 4

Two columns have a Pearson correlation of 0.00. Which statement is justified?

B) There is no straight-line trend; a curved relationship is still possible

> **Why:** Pearson's r measures only straight-line association. A symmetric curve, or groups with opposite trends, can give r of zero while the variables are strongly related. Look at the scatter plot.

Lesson 4.4 · Quiz 5

A regression of revenue on visitors gives a slope of 21.19 and an intercept of -107.93. What is the best reading?

A) Within the range of the data, each additional visitor goes with about $21 more revenue on average

> **Why:** The slope is in units of y per unit of x: about 21 dollars of revenue per additional visitor, on average, across the range of days in the data. The intercept only positions the line.

Lesson 4.5: Q1 B, Q2 A, Q3 C, Q4 A, Q5 C

Lesson 4.5 · Quiz 1

A Welch t-test comparing two stores gives p = 0.03. Which statement is correct?

B) If the stores truly had the same mean, a difference at least this large would appear in about 3% of samples

> **Why:** A p-value describes the data under the null hypothesis: how often chance alone would give a result at least this extreme. It says nothing directly about the probability of either hypothesis.

Lesson 4.5 · Quiz 2

What does this print, and why?

```
q_res = stats.ttest_1samp([5, 6, 7, 8, 9], popmean=7)
print(q_res.pvalue)
```

A) `1.0`, because the sample mean equals the target exactly, so the data are not surprising at all under the null

> **Why:** The sample mean is exactly 7, so the t statistic is 0. Every possible sample is at least that far from the target, so the p-value is 1: the data are as compatible with the null as data can be.

Lesson 4.5 · Quiz 3

You have daily revenue for two stores on the same 700 dates and want to know whether one store earns more. Both stores are busy on the same weekends and slow in the same months. Which test fits best?

C) A paired t-test on the two stores matched by date

> **Why:** Each date gives one value per store, so the data are pairs. Testing the daily differences removes the swings the stores share, which is what made the Bend and Flagstaff gap detectable in the lesson.

Lesson 4.5 · Quiz 4

An analyst compares 20 product categories with the company average, each at $\alpha = 0.05$, and finds one category with $p = 0.04$. What is the most reasonable reading?

A) It could easily be a false alarm: about one such result is expected from 20 tests even if nothing differs

> **Why:** With 20 tests at 0.05 you expect about one false alarm even if no category differs, and the chance of at least one is 64%. After a Holm or Bonferroni correction, $p = 0.04$ would not be significant.

Lesson 4.5 · Quiz 5

A test on 50,000 customers per group reports $p = 0.001$ and Cohen's $d = 0.02$. What should the report say?

C) The difference is very likely real and also very small; whether it matters is a practical judgment

> **Why:** Statistical and practical significance are separate. A huge sample can detect a difference of two hundredths of a standard deviation; whether that matters is a business judgment based on the effect size and its interval.

WARM-UP AND EXIT TICKET ANSWERS

WARM-UP

1. A column has a skewness of 1.5. Which center and spread do you report, and what do you draw first? **Answer:** Median and IQR (strongly skewed, long right tail), and a histogram first: skewness does not show humps.
2. An outlier rule flags 4% of your rows. Which two questions do you ask before removing anything? **Answer:** Is it a provable error? Is it part of the question I am answering? Remove only provable errors, and say what you removed.
3. You have a frozen distribution `model`. Which method gives P(X at most x), and which gives P(X more than x)? **Answer:** `model.cdf(x)` and `model.sf(x)`.
4. In one sentence: what does the central limit theorem say becomes normal? **Answer:** The distribution of the sample mean (not the data): centered on the population mean, with SD sigma / sqrt(n).
5. A 95% confidence interval for the mean order total is $165 to $209. What does the 95% describe? **Answer:** The method: about 95 of 100 intervals built this way contain the true mean. It is not a range for 95% of orders.

◆ EXIT TICKET

1. Staff on duty and revenue correlate at r = 0.88. Give two explanations other than "more staff cause more revenue". **Answer:** A confounder (expected demand, the calendar, drives both) and the reverse arrow (expected revenue decides the schedule). Selection and chance also count as explanations in general.
2. Complete the sentence: "A p-value is the probability of _____, assuming _____." **Answer:** "... a result at least as extreme as the one observed, assuming the null hypothesis is true." It is not the probability that the null is true.
3. Besides the p-value, which two numbers belong in every test report? **Answer:** An effect size (how big?) and a confidence interval (how sure, and how big it could be).

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

WHAT YOU SEE OR HEAR	WHAT IT REVEALS	WHAT TO DO
"r = 0.94, so 94 cents per visitor."	r and the slope are one idea in their head.	Fit the line and print both. r has no units; the slope is in dollars per visitor.
"Schedule more staff to sell more."	A correlation (or a slope) read as the effect of an intervention.	Ask how the data came to be, then run the like-with-like check: 0.88 becomes 0.14.
Self-check Q1 answered C ("46% of sales").	r read as a percentage.	Have them square it (about 20%) and read Common Mistake 4 of 4.4.

WHAT YOU SEE OR HEAR	WHAT IT REVEALS	WHAT TO DO
"p = 0.03, so a 3% chance the stores are the same."	The classic misreading: probability of the hypothesis instead of probability of the data under the null.	Point at the board definition. Have them rewrite the sentence starting "If there were no difference, ...". Show the shuffle histogram again.
"p = 0.57, so Bend and Flagstaff are the same."	"Not significant" heard as "no difference".	Print the confidence interval: it allows a gap of a third of a point either way. Use "we could not detect a difference".
"p = 0.003, a big problem."	Significance confused with size.	Compute the effect size (d = -0.07) and ask whether a manager would act on 0.14 points.
Q9 answered "an error" or a number.	They expect SciPy to skip missing values the way pandas does.	Run it. Then `.dropna()` or `nan_policy="omit"`.
Q10 answered "cannot be known".	They see the interval and the p-value as unrelated.	Show the Welch output: the interval excludes 0 exactly when p is below 0.05 (two-sided, same level).
Q12 answered 5% or 25%.	They think the false-alarm rate stays at alpha however many tests run (or add the rates).	Simulate it: 1 - 0.95 ** k for k = 1, 5, 20, 30.

INSTRUCTOR KIT · ANSWER KEY

Session 13: A/B Testing and the EDA Workflow

Answers with reasoning for every workbook item, reference output for all six labs, the lesson pages' quiz answers, and the rubric, model answer and weak answers for Checkpoint 1.

Session 13 | 12 self-check items | Checkpoint 1 rubric | Confidential

HOW TO USE THIS KEY

- Self-check items repeat the workbook's wording and options exactly; the correct option is highlighted and each wrong option has a one-line reason.
- Every number comes from running the code on the course data with the course environment (pandas 3.0, SciPy 1.18, statsmodels 0.15).
- Checkpoint 1 is at the end: the rubric, the model answer from a real run, and seven common weak answers with what each one reveals.

4.6 · A/B TESTING: SELF-CHECK

Q1

Pinecrest showed the old and the new checkout page during the same four weeks and let a coin flip decide who saw which. What does the coin flip buy?

A) Two groups of exactly the same size, which the z-test requires

B) A smaller sample size than a before/after comparison would need

C) Groups that are alike in everything except the page, including things nobody measured

D) A guarantee that the difference between the pages is not due to chance

Why: Random assignment spreads every kind of visitor evenly across the arms, so the page is the only systematic difference. Chance is still possible; the test judges that.

A is wrong: The arms were 14,079 and 13,921. The test does not need equal sizes; it needs a split consistent with the design.

B is wrong: Randomization does not reduce the sample size. The power calculation sets it.

D is wrong: Randomization removes confounding, not chance. That is why a test and an interval are still needed.

Q2

A colleague sizes a test for a rise from 8.9% to 10.0% like this and gets 129,733 visitors per arm. What went wrong?

```
n = NormalIndPower().solve_power(effect_size=0.011, alpha=0.05, power=0.80)
```

A) `effect_size` expects Cohen's h from `proportion_effectsize(0.100, 0.089)`, not the raw difference in rates

B) `power` should be 0.20, the chance of missing the effect

C) `solve_power` needs `nobs1` as well, or it has nothing to solve

D) Nothing: a one-point lift really needs about 130,000 visitors per arm

Why: `effect_size` is Cohen's h (0.0376 here). Passing 0.011 asks for a much smaller effect than intended, and the answer is twelve times too large. The right call gives 11,092.

B is wrong: Power is the chance of detecting a real effect: 0.80 is the convention.

C is wrong: `solve_power` solves for the one argument you leave out. Giving all four raises `ValueError: need exactly one keyword that is None`.

D is wrong: With the effect size computed correctly the answer is 11,092 per arm.

Q3

Conversion went from 8.86% to 10.03%. What does this print? (First number: percentage points. Second: relative lift.)

```
print(round((0.1003 - 0.0886) * 100, 2), round(0.1003 / 0.0886 - 1, 3))
```

`1.17 0.132`

Why: 1.17 percentage points is the absolute difference. 0.132 (13.2%) is the relative lift: 1.17 is 13.2% of 8.86. Both are correct; always say which one you mean.

Q4

An experiment planned as 50/50 reports 14,400 visitors in treatment and 13,600 in control. `stats.binomtest(14400, 28000, p=0.5).pvalue` is `1.8e-06`. What do you do?

A) Carry on: 51.4% is close enough to half

B) Drop 800 treatment visitors at random so the arms match, then analyze

C) Report the conversion result with a note that the split was uneven

D) Stop and find where the control visitors went: the experiment did not run as planned, so its results cannot be read

> **Why:** A fair coin almost never gives 51.4% in 28,000 flips. This is a sample ratio mismatch: visitors are going missing from one arm, which breaks the comparison. Fix the bug and rerun.
> **A** is wrong: With 28,000 visitors a fair coin stays very close to 50%; p = 0.0000018 says this is not chance.
> **B** is wrong: Trimming hides the symptom. Whatever removed 800 control visitors probably removed a particular kind of visitor.
> **C** is wrong: A failed health check is not a caveat to attach; nothing downstream can be trusted.

Q5

A team looks at its four-week test every Monday and stops the first time p is below 0.05. In the lesson's simulation of experiments with no real effect, how often did that rule declare a winner?

A) About 5%, because alpha is 0.05 at each look

B) About 12%, because each look is another chance to cross the line by luck

C) About 1.25%, because the four looks share the 5%

D) About 50%, because the p-value wanders at random

> **Why:** One planned look kept false alarms near 5% (4.4% in the simulation). Stopping at any of four looks raised it to 12.2%. Peeking is the multiple-comparisons problem in disguise.
> **A** is wrong: 5% holds for ONE look. Four chances to stop add up.
> **C** is wrong: 1.25% is the stricter per-look threshold you would USE to pay for four looks, not the false-alarm rate of looking at 0.05.
> **D** is wrong: The p-value wanders, but not that wildly: the simulated rate was 12.2% for weekly looks and about 27% for daily looks.

Q6

By device: mobile +1.76 points (p = 0.00004), desktop +0.27 points (p = 0.67). The test of the difference between the two lifts gives p = 0.049, and nobody planned this comparison. Which sentence belongs in the memo?

A) The new page works on mobile and does nothing on desktop

B) The new page should be shipped to mobile visitors only

C) The gain appears concentrated on mobile; the evidence that devices differ is borderline and exploratory, and worth a follow-up test

D) The desktop result proves the page has no effect on desktop

> **Why:** "Significant here, not there" is not evidence that here and there differ. The direct test of the difference is on the line (p = 0.049) and unplanned, so it is a lead to confirm.
> **A** is wrong: Desktop's interval runs from about -1.0 to +1.5 points: a lift as large as the overall 1.17 is still inside it.
> **B** is wrong: That builds a strategy on an unplanned, borderline comparison. The confirmed result is the overall lift.
> **D** is wrong: Not significant means not detected. It never proves zero.

4.6 · PREDICT, THEN RUN

P1

Output and reasoning

```
print(round(proportions_ztest([20, 10], [100, 100])[1], 3))
```

> **Answer:** `0.048`. Just below 0.05 (p = 0.048), even though one rate is double the other. With 100 visitors per arm the interval is wide; "twice as good" on small counts is weak evidence.

P2

Output and reasoning

```
h = proportion_effectsize(0.100, 0.089)
n90 = NormalIndPower().solve_power(effect_size=h, alpha=0.05, power=0.90)
print(round(n90))
```

> **Answer:** `14850`. About 14,850 per arm: a third more visitors than the 11,092 needed for 80% power. Power is bought with sample size.

P3

Output and reasoning

```
print(round(stats.binomtest(5100, 10_000, p=0.5).pvalue, 3))
print(round(stats.binomtest(5080, 10_000, p=0.5).pvalue, 3))
```

> **Answer:** `0.047 / 0.112`. 5,100 / 4,900 fails (p = 0.047); 5,080 / 4,920 passes (p = 0.112). A 51% share looks harmless and is already unusual for a fair coin at this size. Many teams use a stricter threshold (0.01 or 0.001) for this check so that it does not cry wolf.

4.6 · WORKED EXAMPLE AND LAB REFERENCE NOTES

Worked example (weekend against weekday). Output as printed in the workbook: weekday +1.03 points (p = 0.012), weekend +1.53 points (p = 0.021); difference +0.50 points, p = 0.52. A learner who writes "it works better on weekends" has compared two estimates without their uncertainty.

`labs/module-13/l4_6-ab-testing/novelty_check_solution.py`. The week column is `(date - first date).dt.days // 7 + 1`. Each week is compared by itself with `==` (the lesson's peeking table used `<=`, cumulative). The halves are compared with the lesson's difference-between-lifts test. Reference output:

```
rows: 28000 | days: 28
week
1    7018
2    7065
3    6960
4    6957

week 1        7,018 visitors  8.20% -> 9.25%  +1.05 pts (95% CI -0.27 to +2.37)  p =
0.1188
week 2        7,065 visitors  9.44% -> 10.20%  +0.76 pts (95% CI -0.63 to +2.15)  p
= 0.2806
week 3        6,960 visitors  9.73% -> 10.53%  +0.80 pts (95% CI -0.61 to +2.22)  p
= 0.2662
week 4        6,957 visitors  8.06% -> 10.14%  +2.08 pts (95% CI +0.73 to +3.43)  p
= 0.0026
weeks whose own p-value is below 0.05: [4]
weeks whose interval contains the overall lift of +1.17 points: [1, 2, 3, 4]

weeks 1-2    14,083 visitors  8.82% -> 9.73%  +0.91 pts (95% CI -0.05 to +1.87)  p =
0.0622
weeks 3-4    13,917 visitors  8.90% -> 10.33%  +1.43 pts (95% CI +0.45 to +2.41)  p
= 0.0041
Late lift minus early lift: +0.52 points (standard error 0.70), p = 0.46

Verdict: every week shows a positive lift (about +0.8 to +2.1 points) and every
weekly interval
contains the overall +1.17. One week of 7,000 visitors is too small to be
significant alone,
so 'only week 4 worked' is a power problem, not a finding. The late lift is not
smaller than the
early one (+1.43 vs +0.91 points, p = 0.46 for the difference):
no sign of a novelty effect in these four weeks. This check was not in the plan:
exploratory.
```

What a good sentence contains: every week positive; every weekly interval contains +1.17; the late lift is not smaller than the early one (p = 0.46); one week alone has too little power; the check was unplanned. **Common wrong answers:** "it only worked in week 4" (reading p-values instead of lifts); using cumulative weeks (reproduces the peeking table: p = 0.1188, 0.0622, 0.0308, 0.0008); "the effect is growing" (the +0.52-point difference between halves has p = 0.46).

`labs/module-13/l4_6-ab-testing/followup_test_plan_solution.py`. Baseline = control-arm mobile rate (6.8%); traffic = 15,462 mobile visitors / 28 days = 552 a day. Reference output:

```
mobile visitors: 15462 in 28 days
baseline 6.8%, about 552 mobile visitors a day

detect 6.8% -> 7.6% (+0.8 points, h = 0.0310):  16,376 per arm, about  59 days
detect 6.8% -> 8.0% (+1.2 points, h = 0.0459):   7,458 per arm, about  27 days
detect 6.8% -> 8.6% (+1.8 points, h = 0.0676):   3,432 per arm, about  12 days

28 days = 7,728 mobile visitors per arm
power of a 28-day test for 6.8% -> 8.0%: 0.81
power of the same test for 6.8% -> 7.6%: 0.49

with the raw difference 0.012 as effect size: 109,012 per arm (wrong: effect_size is
Cohen's h)

Plan: mobile visitors only, 50/50 at random, primary metric conversion per visitor,
two-sided,
alpha 0.05. Run 28 days (four full weeks, about 7,728 per arm): 81% power to detect
6.8% -> 8.0%. A true lift as small as 0.8 points would be caught only 49% of the
time;
say so in the plan. One analysis, after day 28.
```

Points to draw out: planning on the observed 8.6% (12 days) is optimistic, because a result that crossed the significance line tends to overstate the true effect; four full weeks gives 81% power for a 1.2-point lift and only 49% for 0.8 points, and the plan should say so. **Common wrong answers:** passing 0.012 as `effect_size` (109,012 per arm); using the treatment arm's rate as the baseline; forgetting that `n` is per arm when converting to days (half the true duration).

`labs/module-13/l4_6-ab-testing/aa_test_solution.py`. Seeded with `np.random.default_rng(42)`; 2,000 random halvings of the 13,921 control visitors. Reference output:

```
control visitors: 13921 | conversions: 1233 | rate: 8.86%
one split: 8.65% vs 9.06%, p = 0.388

A/A splits with p < 0.05: 4.9%  (the promised false-alarm rate is 5%)
A/A splits with p < 0.01: 0.8%
smallest p-value in 2,000 splits: 0.0021
p-value at the 10th, 30th, 50th, 70th, 90th percentile: [0.1  0.3  0.49 0.7  0.88]

Reading: with no difference at all, about 1 split in 20 is 'significant', and p-
values are spread
evenly from 0 to 1. One significant A/A result is expected now and then; a rate far
above 5% would
mean the assignment or the counting is broken.
Splits with a p-value as small as the real test's 0.0008: 0 of 2,000.
```

The answer to the starter's closing question (a colleague ran ONE A/A test and got p = 0.03; is the tool broken?): no. About 1 in 20 A/A tests does that. A rate well above 5% over many A/A tests would be the warning sign. **Common wrong answer:** shuffling once outside the loop, which gives 2,000 identical p-values.

4.6 · THE LESSON PAGE'S OWN QUIZ

1. **Question 1: C.** Random assignment makes the groups alike in everything except the page, so the page is the only systematic difference. Speed and equal sizes are not the reason.
2. **Question 2: B.** A much smaller number: a bigger effect (8.9% to 13.9%) is easier to detect, so fewer visitors are needed.
3. **Question 3: A.** Far above 5%: each look is another chance to cross the line by luck (about 27% with daily looks).
4. **Question 4: B.** The lifts are similar (+1.09 and +1.26 points); the returning group is half the size, so its estimate is less precise.
5. **Question 5: A.** Revenue per visitor across all randomized visitors: buyers are not a randomized group.

4.7 · AN EDA WORKFLOW: SELF-CHECK

Q7

Someone hands you a table you have never seen. After writing your questions, what is the first fact to establish about it?

A) Which columns correlate most strongly

B) Its grain: what one row represents, and which column identifies a row

C) Whether the numeric columns are normally distributed

D) Which rows are outliers, so they can be removed

> **Why:** Every count and every average depends on what one row is. Structure (grain, keys, types) is step 2 of the routine, straight after the questions.
> **A** is wrong: Pairs of columns are step 5. Without the grain you cannot say what a correlation is a correlation of.
> **C** is wrong: Shape matters in step 4, and only for the columns your questions need.
> **D** is wrong: Unusual rows are investigated in step 3, not removed on sight. Pinecrest's twelve bulk orders are real.

Q8

What does this print?

```
df = pd.DataFrame({"a": [1.0, None, 3.0],
                   "b": [None, 5.0, 6.0],
                   "c": ["x", "y", None]})
print(len(df.dropna()), len(df.dropna(subset=["a"])))
```

`0 2`

> **Why:** Every row has a gap somewhere, so `dropna()` with no arguments keeps nothing. Naming the one column the analysis needs keeps the two rows where `a` is present.

Q9

In the survey, `sat_website` is missing for 46% of store shoppers and 3% of online shoppers. Its mean is 3.54. What is the most accurate description?

A) Pinecrest customers rate the website 3.54

B) The mean is unusable until the gaps are filled with 3.54

C) Store shoppers rate the website 3.54

D) Customers who answered, mostly online shoppers, rate the website 3.54

> **Why:** The values are not missing at random: people skipped the question about the part of Pinecrest they do not use. The column describes those who answered.
>
> **A** is wrong: That claims to speak for everyone, including the many store shoppers who never answered.
>
> **B** is wrong: Filling with the mean invents opinions. Leave missing values missing and say who answered.
>
> **C** is wrong: It is the other way round: store shoppers are the ones who mostly skipped it.

Q10

What does this print?

```
left = pd.DataFrame({"id": [1, 2, 3]})
right = pd.DataFrame({"id": [1, 1, 1, 2], "x": [5, 6, 7, 8]})
print(len(left.merge(right, on="id")),
      len(left.merge(right, on="id", how="left")))
```

`4 5`

Why: `id` 1 matches three rows and `id` 2 matches one: 4 rows in the inner merge, and `id` 3 is dropped. The left merge keeps `id` 3 with a missing `x`: 5 rows. Three rows became four or five without any message, which is why merges get `validate=`.

Q11

You compare online order totals across 22 groups (weekdays, payment methods, months). Three groups have p below 0.05; after a Holm correction one remains. What goes in the findings?

A) The one that survived, with the size of its gap, the number of comparisons, and the result of checking an obvious competing explanation; the other two are leads

B) All three, with their original p-values

C) None: results found by scanning are never reportable

D) Only the group with the largest gap in dollars, whatever its p-value

Why: Scanning is multiple testing. Correct for it, judge the size of the gap, challenge the survivor, and say that 22 comparisons were made. Uncorrected hits are leads for later.

B is wrong: With 22 tests about one false alarm is expected at 0.05. Uncorrected p-values overstate the evidence.

C is wrong: Exploration is how leads are found. A corrected, sanity-checked result can be reported as exploratory.

D is wrong: The largest gap may come from the smallest, noisiest group.

Q12

Which sentence from the survey note is a caveat, not a finding?

A) Only 42% of customers are satisfied with prices

B) Every respondent has placed an order, so customers who never ordered are not heard

C) Asheville's staff rating is 3.62 against about 4.1 at the other stores

D) Asheville is the only store whose revenue fell in 2025

> **Why:** A caveat says who or what the data cannot speak for. This one limits every finding in the note to active customers.
> **A** is wrong: That is a finding: one sentence with a number.
> **C** is wrong: A finding (and one that survived the correction for 55 comparisons).
> **D** is wrong: A finding from a second dataset that points the same way.

4.7 · PREDICT, THEN RUN

P1

Output and reasoning

```
left = pd.DataFrame({"k": [1, 1, 2]})
right = pd.DataFrame({"k": [1, 1, 2], "v": [10, 20, 30]})
print(len(left.merge(right, on="k")))
```

> **Answer:** `5`. 5 rows from 3: the key 1 appears twice on each side, so it matches 2 × 2 = 4 times, plus one row for key 2. A many-to-many merge multiplies. `validate="one_to_one"` or `"many_to_one"` would have raised an error.

P2

Output and reasoning

```
s = pd.Series([4, None, 2])
print(s.mean(), s.fillna(0).mean())
```

> **Answer:** `3.0 2.0`. `mean()` skips the gap: (4 + 2) / 2 = 3.0 is the honest mean. `fillna(0)` counts the missing answer as a zero and gives 2.0.

P3

Output and reasoning

```
print(multipletests([0.01, 0.04, 0.03], method="holm")[1].round(2).tolist())
```

> **Answer:** `[0.03, 0.06, 0.06]`. One. Holm multiplies the smallest p by 3 (0.03), the next by 2 (0.03 × 2 = 0.06) and keeps the last at least as large as the one before (0.06). Three raw hits become one.

4.7 · WORKED EXAMPLE AND LAB REFERENCE NOTES

Worked example (frequent buyers and recommending). 20,302 orders become 3,942 rows, one per customer; the validated merge keeps 1,800 survey rows; Spearman = -0.02; mean recommend 7.23 to 7.48 across the four order-count bands. The point is the grain and the honest null. A learner who merges `survey` with `orders` directly gets 9,230 rows and should be asked "what is one row now?"

`labs/module-13/l4_7-eda-workflow/shipping_rules_solution.py`. The decisive line is `goods = order_total - shipping_cost`. Reference output:

```
rows: 20302 | order_id unique: True
grain: one row per order

missing values: {'channel': 0, 'shipping_cost': 0, 'order_total': 0}
shipping values: [0.0, 6.95, 14.95]
shipping_cost  0.00   6.95   14.95
channel
online          9176   2754    924
store           7448      0      0
Store orders never pay shipping, so questions 2 and 3 are about ONLINE orders only.

shipping_cost
0.00     71.4
6.95     21.4
14.95     7.2

               count   min  median      max
shipping_cost
0.00            9176  75.0  200.90  4893.00
6.95            2754   6.0   44.45    74.96
14.95            924   7.5  152.23  4097.15

shipping_cost  0.00   6.95   14.95
goods_75_plus
False             0   2754    224
True           9176      0    700

shipping_cost  0.00   6.95   14.95
goods
[0, 25)          0.0   90.8    9.2
[25, 50)         0.0   91.8    8.2
[50, 75)         0.0   94.2    5.8
[75, 100)       93.5    0.0    6.5
[100, 150)      92.5    0.0    7.5
[150, 300)      93.0    0.0    7.0
[300, 5000)     92.8    0.0    7.2

FINDINGS
- Store orders never pay shipping. Online: 71.4% ship free, 21.4% pay 6.95, 7.2% pay
14.95.
- Among orders that did not pay 14.95, the rule is exact: goods of 75.00 or more
ship free
  (9,176 of 9,176), goods under 75.00 pay 6.95 (2,754 of 2,754). No exceptions in
two years.
- The 14.95 charge appears at every order size, on about 6% to 9% of orders in each
band.
CAVEATS
- The rule is inferred from the data, not read from a policy. 14.95 looks like an
```

```
option the
  customer chooses (faster delivery?): a hypothesis. No column records the delivery
method.
- 'goods' is the total after any discount: the threshold applies to what the
customer pays.
NEXT QUESTIONS
- Do customers add items to reach 75.00? (Look at the distribution of goods just
above and below.)
```

Mark the note, not the tables. A good note says: stores never pay; among online orders that did not pay 14.95 the rule is exact (goods of 75.00 or more ship free, under 75.00 pay 6.95: 9,176 and 2,754 orders, no exceptions); 14.95 appears at every size on about 7% of orders; and as a caveat that the rule is inferred, and that what 14.95 buys is a guess. **Common wrong answers:** grouping on `order_total` (the threshold appears at 81.95 for paid orders and looks fuzzy); "orders over $75 ship free" with no mention of the 700 large orders that paid 14.95; a note with findings and no caveat.

`labs/module-13/l4_7-eda-workflow/order_size_scan_solution.py`. Mann-Whitney because order totals are skewed (online mean 204.65, median 156.40). Reference output:

```
online orders: 12854 | median order total: 156.4
order_total is right-skewed (mean 204.65 ), so compare medians and use the Mann-
Whitney test.

22 tests, 4 below 0.05 before correction, 1 after
       segment     group    n  median  median_rest       p p_holm
         month      June 1077 169.400       154.95 0.00023  0.005
       weekday  Saturday 1789 146.240       158.75 0.00815  0.171
payment_method gift_card 1025 145.950       156.90 0.04011  0.802
         month     April  981 168.400       155.40 0.04687  0.890
         month  December 1480 149.775       157.40 0.05039  0.907

Share of online orders that contain a tent:
june
False    10.4
True     17.7

Median order total:
june       False   True
has_tent
False     143.80  146.52
True      328.82  336.85
has_tent = False: June against other months, p = 0.14
has_tent = True: June against other months, p = 0.60

FINDINGS
- Scanned 22 groups (7 weekdays, 3 payment methods, 12 months). Four had p < 0.05
before
  correction; one survives Holm: June, median $169.40 against $154.95 in other
months.
- June's gap is mostly product mix: 17.7% of June orders contain a tent against
10.4% in other
  months, and a tent order is about $329 against $144. Within tent orders and within
non-tent
  orders, June is only $3 to $8 higher and neither gap is significant (p = 0.60 and
0.14).
CAVEATS
- Saturday (lower), gift cards (lower) and April (higher) did not survive the
correction:
  leads, not findings.
- Exploratory scan on observational data. 'Has a tent' was chosen after seeing June
stand out.
NEXT QUESTION
- If the banner is about basket size, the lever is what is in the basket, not the
calendar.
```

The lesson in it: four raw hits, one survivor, and the survivor is mostly product mix. June orders contain a tent 17.7% of the time against 10.4%, and tent orders are more than twice the size. Within tent orders and within non-tent

orders the June gap is small and not significant. **Common wrong answers:** reporting Saturday, gift cards or April as findings; "people spend more in June" with no check of what they buy; testing the tent explanation and then calling it proven (it was chosen after seeing the data: also exploratory).

`labs/module-13/l4_7-eda-workflow/customer_overview_solution.py`. Reference output:

```
                            dtype  missing  missing_pct  unique
example
customer_id                 int64        0          0.0    4850
10001
first_name                    str        0          0.0     151
Odessa
last_name                     str        0          0.0     148
Kato
email                         str        0          0.0    4850
odessa.kato@example.co
phone                         str      280          5.8    1104          (303)
555-0175
city                          str        0          0.0      35
Denver
state                         str        0          0.0      14
CO
signup_date          datetime64[us]        0          0.0    2181     2023-08-07
00:00:00
birth_year                 float64      442          9.1      59
1989.0
acquisition_channel           str        0          0.0       5
paid_search
loyalty_tier                  str        0          0.0       3
Gold
marketing_opt_in             bool        0          0.0       2
True

grain: one row per customer account | key customer_id unique: True | email unique:
True

phone missing by acquisition_channel: {'in_store': 6.3, 'organic': 5.5,
'paid_search': 5.3, 'referral': 6.1, 'social': 6.0}  chi-square p = 0.85
phone missing by loyalty_tier: {'Basic': 5.9, 'Gold': 5.2, 'Silver': 5.9}  chi-
square p = 0.75
birth_year missing by acquisition_channel: {'in_store': 8.7, 'organic': 8.9,
'paid_search': 8.5, 'referral': 10.2, 'social': 10.1}  chi-square p = 0.66
birth_year missing by loyalty_tier: {'Basic': 8.7, 'Gold': 10.1, 'Silver': 9.5}
chi-square p = 0.43
Four checks, none below 0.05: no sign that either column is missing for a particular
kind of customer.

_merge
both          19563
left_only       739
right_only        0
orders with no customer row: 739 from 150 customer ids
ids between 10001 and 15000 that are absent from customers.csv: 150
```

```
share online: {False: 63.3, True: 63.3}
share returned: {False: 5.6, True: 5.5}
median order total: {False: 146.7, True: 158.95}
order total, orphan against matched (Mann-Whitney): p = 0.07
first and last orphan order: 2024-01-01 to 2025-12-30

FINDINGS
- One row per account, keyed by customer_id (4,850 rows). phone is missing for 5.8%
and
  birth_year for 9.1%; neither is tied to acquisition channel or loyalty tier.
- 739 orders (3.6%) belong to 150 customer ids with no row in customers.csv: exactly
the 150
  ids missing from the 10001-15000 range (closed accounts). Their orders span both
years and
  have the same channel mix and return rate as everyone else's.
CAVEATS
- 'Not tied to channel or tier' covers the columns checked, not every possible
pattern.
- An inner merge of orders with customers silently drops those 739 orders. Their
median total is
  a little higher ($158.95 against $146.70) but within what chance produces (p =
0.07).
```

Unlike the survey, these gaps show no pattern in the columns checked: a useful contrast, and worth one sentence in a note ("no sign that ... "), not silence. The 739 orphan orders are the 150 closed accounts; an inner merge would have dropped them without a word. **Common wrong answer:** "phone is missing at random" stated as a fact; four checks cannot show that, only fail to find a pattern.

4.7 · THE LESSON PAGE'S OWN QUIZ

1. **Question 1: A.** Questions focus the work and let you tell planned comparisons from patterns found by scanning. They may change, openly.
2. **Question 2: C.** `1 2.0`: only one row is complete in both columns, and `mean()` skips the gaps: (1 + 3) / 2.
3. **Question 3: B.** Customers who answered, mostly store shoppers, rate the staff 4.02.
4. **Question 4: B.** Each response was repeated once per order, so frequent buyers count many times in any average.
5. **Question 5: C.** Look at the size of the gap (0.21) and at what could produce it: here the unhappy 65+ group inside the comparison group.

WARM-UP AND EXIT TICKET

◆ WARM-UP: RETRIEVAL FROM SESSION 12

1. Across store-days, staff on duty and revenue correlate at r = 0.88. Give one reason this does not mean "add staff, sell more". **Answer:** Staff are scheduled from expected demand (weekday, season, promotions), and expected demand also drives revenue: a confounder. Comparing like with like (same store, month, weekday, promo status) the correlation falls to 0.14.

2. For the same two columns Pearson's r is -0.32 and Spearman's is -0.61. What does the gap tell you? **Answer:** The relationship is consistent in direction but curved (or one influential point is distorting Pearson). Spearman works on ranks, so it measures any monotonic trend. Plot it.

3. Finish the sentence: "A p-value is the probability of ... assuming ..." **Answer:** ...a result at least as extreme as the one observed, assuming the null hypothesis is true. It is not the probability that the null is true, and it says nothing about the size of the effect.

4. Which test: (a) the mean of two independent groups of different sizes; (b) whether two categorical columns are related? **Answer:** (a) Welch's t-test, `stats.ttest_ind(x, y, equal_var=False)`. (b) Chi-square on a table of counts, `stats.chi2_contingency(pd.crosstab(a, b))`. Hold on to (b): today's first test is the same test in another form.

5. You run 20 tests at alpha = 0.05 and one comes back with p = 0.04. What do you say about it? **Answer:** About one false alarm is expected from 20 tests when nothing is going on. Count every test, correct (Holm), and call a scanned result exploratory.

EXIT TICKET

1. Name two things you check about an A/B test before you look at a single conversion. **Answer:** Any two of: one row per unit (`visitor_id` is unique); the split is consistent with the design (`stats.binomtest` against 50/50: here p = 0.35); the arms are balanced on things fixed before the page was shown (device, returning). If a check fails: stop, do not adjust.

2. "Significant on mobile, not significant on desktop." What is the right next step, and what word goes in the memo? **Answer:** Test the difference between the two lifts directly (here 1.49 points, p = 0.049) and correct for the slices. The word is "exploratory": the comparison was not planned.

3. Write the first three steps of the EDA routine and say what "grain" means. **Answer:** Questions, structure, quality (then one variable, two variables, segments, findings and caveats). Grain: what one row of the table represents.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **"A p-value is the chance the result is due to chance" (warm-up 3).** The learner has the conditional backward. They will misread peeking ("p was 0.03, so 97% sure") and scans. Have them say the full sentence with "assuming the null is true" once per session until it sticks.
- **"The arms must be equal in size" (Q1, exit 1).** Confuses balance with equality. The check is against what a fair coin produces, not against 50.00%.

- **Mixing points and percent (Q3).** Writing "conversion rose 1.17%" or "13 points". Reveals that the learner computes without saying units; insist on the unit in every spoken answer today.
- **"Not significant, so no effect" (Q6, lab A).** The oldest mistake of Module 4. Ask for the interval: desktop's runs from -0.97 to +1.50 points.
- **"Fill the gaps with the mean" (Q9).** The learner treats missing values as a nuisance to remove, not as information about who answered.
- **A finding offered as a caveat or the reverse (Q12, exit 3).** Usually a sign that the note was written as a diary of steps. The F / C / N marking exercise from the lab debrief fixes it.

CHECKPOINT 1: RUBRIC, MODEL ANSWER AND WEAK ANSWERS

> **THE QUESTION**
>
> Pinecrest's store buyers believe that **December shoppers buy more expensive things**. Finance wants a number before it plans next December's stock. Using `data/daily_sales.csv`: is the average price per unit sold (revenue ÷ units) higher in December than in the rest of the year? By how much, how sure are you, and is it the same in all four stores?

Rubric. Five rows, four levels, 20 points. The levels and row names match the capstone rubric in Lesson 8.1 (Reproducibility is left out: thirty minutes is too short to judge it). "Proficient" in every row (15 points) is a successful checkpoint. Circle one cell per row and write one sentence of advice.

CRITERION	1 · BEGINNING	2 · DEVELOPING	3 · PROFICIENT	4 · EXEMPLARY
Question and plan	No plan. Code starts at once.	A plan exists but the measure or the comparison is decided as the code goes along.	Measure (revenue ÷ units per store-day), comparison (December against other days) and checks are written before the analysis, and the work stays on them.	Also asks in the plan whether "December against the rest" is a fair split, or revises the plan openly when the monthly profile shows it is not.

CRITERION	1 · BEGINNING	2 · DEVELOPING	3 · PROFICIENT	4 · EXEMPLARY
Data handling	File used as loaded. Row counts never looked at.	Some checks (shape, missing values) but the 6 missing revenue values vanish or are filled without comment.	States the grain (one row per store per day), finds the 6 missing revenue values and says what was done with them; the new column is checked (range or describe).	Also checks that a different reasonable choice gives the same answer (median instead of mean, ratio of sums instead of mean of daily ratios, or without promo days).
Analysis	Two averages compared, no measure of uncertainty.	A test is run but only a p-value is reported, or the test does not fit (chi-square on a mean, a paired test on unpaired days).	Difference in dollars with a 95% confidence interval, a suitable test (Welch or Mann-Whitney) and an effect size (Cohen's d or percent). Stores are compared.	Also looks at every month and sees the winter plateau (November to February), or says that neighboring days are not independent and what that does to the p-value.
Figure	No figure, or a default chart with no labels.	Readable and labeled, but the title names the topic ("Price by month") or the chart hides the comparison.	One figure with a title that states the finding, labeled axes with units, and the comparison easy to see.	The figure alone carries the answer: stores and months both visible, a reference level marked.
Communication	Retells the steps. No answer to the question.	An answer, but numbers lack uncertainty or the caveats are generic ("more data is needed").	Leads with the answer and its size; two specific caveats; one next question; plain language.	Also never says more than the data allow: separates "revenue per unit is higher" (shown) from "shoppers buy pricier things" (one explanation; mix against prices cannot be told apart from this file).

Quick guide when time is short: read the note first. A note with the answer, a size, an interval and two specific caveats is at Proficient or better in Analysis and Communication before you open a single cell. Then look for three things in the notebook: a plan above the first code cell, a line that deals with the 6 missing revenue values, and a figure whose title is a sentence.

MODEL ANSWER (FROM A REAL RUN)

The file is `labs/module-13/checkpoint-1/checkpoint1_solution.py`: the plan as comments, then six short cells. It took the author about 25 minutes at a learner's pace. Its complete output:

```
rows: 2903 | one row per store-day: True
missing revenue: 6 | days with zero units: 0
store-days used: 2897 | December: 238 | other: 2659
count    2897.00
mean       41.92
std         2.65
min        33.96
25%        40.07
50%        41.73
75%        43.62
max        52.20

December: mean $44.95 (median $44.91, n = 238)
Other months: mean $41.65 (median $41.52, n = 2659)
Difference: $3.31 per unit (+7.9%), 95% CI $2.97 to $3.65
Welch t-test p = 2.0e-52, Cohen's d = 1.33

december   False   True   gap
store
Asheville  41.59  44.45  2.86
Bend       41.66  45.45  3.79
Boise      41.64  45.14  3.51
Flagstaff  41.69  44.77  3.08
december  False   True   gap
date
2024      41.65  44.76  3.10
2025      41.64  45.15  3.51

date
1     43.55
2     43.23
3     41.07
4     41.19
5     41.29
6     40.99
7     40.99
8     41.10
9     41.02
10    41.27
11    42.56
12    44.95
March to October: mean $41.12. December against those months: $3.84 (+9.3%)

figure saved: checkpoint1_price_per_unit.png

FINDINGS
- Yes. In December a unit sold in the stores brings in $44.95 on average against
$41.65 in the
```

```
  other eleven months: $3.31 more (+7.9%; 95% CI $2.97 to $3.65; Welch p < 0.001;
  Cohen's d = 1.33, a large effect: the middle December day is above 90% of the
other days).
- It is the same story in all four stores (gaps of $2.86 to $3.79) and in both years
  ($3.10 and $3.51).
- December is the peak of a winter season, not a one-month jump: November ($42.56),
January
  ($43.55) and February ($43.23) are also above the March-to-October level of
$41.12.
  Against those months the December gap is $3.84 (+9.3%).
CAVEATS
- Revenue per unit rises if shoppers choose pricier items OR if prices are higher:
this file cannot
  tell the two apart (it has no products). "Buy pricier things" is one reading, not
a proven one.
- The unit is a store-day, and neighboring days are alike, so the p-value overstates
the evidence.
  With an effect this large and this consistent, the conclusion does not depend on
it.
- Only two Decembers (238 store-days); 6 store-days had no revenue and were left
out.
NEXT QUESTION
- Is it the mix? Compare the categories sold in December with other months using
order_items and products.
```

The figure it saves is a line chart of mean revenue per unit by month, one line per store, with the March-to-October average marked and the title "Store shoppers pay about $4 more per unit in December, in every store".

Equally correct variations. Mann-Whitney instead of Welch (the distribution is close to symmetric, so both fit). Medians instead of means ($44.91 against $41.52). The ratio of sums instead of the mean of daily ratios ($45.06 against $41.53, +8.5%): say which one was used. A bootstrap interval. A bar chart of twelve months instead of lines. Excluding the promo days at the start of December changes nothing visible. Any of these with a clear note is Proficient or better.

WHAT SEPARATES PROFICIENT FROM EXEMPLARY HERE

- **The winter plateau.** "The rest of the year" contains November ($42.56), January ($43.55) and February ($43.23). Against March to October ($41.12) the December gap is $3.84 (+9.3%), not $3.31. Seeing this means the learner looked at every month before trusting a two-group split.
- **The wording about cause.** The data show that revenue per unit is higher. "Shoppers buy pricier things" is one explanation; higher winter prices is another; this file cannot separate them. An Exemplary note says so and names the files that could (`order_items`, `products`).
- **Independence.** Store-days close together are alike, so 238 December rows are not 238 independent observations and the p-value is too small. With d = 1.33 in every store and both years the conclusion does not depend on it; saying so is the mark of someone who read Lesson 4.5's caveat about days.

COMMON WEAK ANSWERS AND WHAT THEY REVEAL

- **"December is $44.95, the rest is $41.65, so yes."** Two means and no uncertainty: Analysis at level 1. Ask: "How much would those two numbers move with a different set of days?" The interval ($2.97 to $3.65) answers that in one line.
- **"p = 2.0e-52, so December shoppers definitely buy more expensive things."** A tiny p-value read as a large, causal fact. The p-value says the gap is not chance; the size is the $3.31 (+7.9%); and "buy more expensive things" is one explanation. Higher prices in winter would produce the same column. Communication at level 2.
- **A NaN result, or "the mean is nan".** The 6 missing revenue values were passed to NumPy or SciPy (`stats.ttest_ind` returns nan when the input holds NaN). Lesson 4.5, Common Mistake 1. Data handling at level 1 or 2; the fix is `dropna(subset=[...])` and a sentence saying so.
- **Total December revenue compared with average monthly revenue.** Answers a different question (December is the busiest month) with a different measure. The plan row decides this: the question names revenue ÷ units.
- **A chi-square test, or a paired t-test.** Chi-square is for counts in categories; a paired test needs pairs, and December days have no partner days. Welch's t-test or Mann-Whitney fits two independent groups of a numeric measure.
- **"December is higher than the rest of the year" and nothing about the other winter months.** Correct and still Proficient. Exemplary work plots or tabulates all twelve months and notices that January ($43.55), February ($43.23) and November ($42.56) sit in "the rest" and pull it up: against March to October the gap is $3.84, not $3.31.
- **Twelve separate month-against-rest tests with no correction, then "four months are significant".** A scan presented as planned results. Not wrong to look, but it needs the count of tests and a correction (Lesson 4.5), or one planned comparison plus a descriptive monthly table.

INSTRUCTOR KIT · ANSWER KEY

Session 14: SQL from Python: Queries and Joins

Answers with reasoning for every workbook item, reference output for all six labs, the lesson pages' quiz answers, and the warm-up and exit-ticket answers.

Session 14 | 12 self-check items | Confidential

HOW TO USE THIS KEY

- Self-check items repeat the workbook's wording and options exactly; the correct option is highlighted and each wrong option has a one-line reason.
- Every output comes from running the code against `data/pinecrest.db` with the course environment (SQLite 3.53, SQLAlchemy 2.1, pandas 3.0). Counts do not depend on the SQLite version.
- The four "what does this print" items build a tiny in-memory database, so learners can run them anywhere.

5.1 · DATABASE CONNECTIONS: SELF-CHECK

Q1

A query is written `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT`. Which clause does the engine work on first?

A) `SELECT`, because it is written first

B) `WHERE`, because rows must be filtered before anything else

C) `FROM`, because the engine has to pick the table before it can filter, group or select

D) `ORDER BY`, because sorted rows are faster to filter

> **Why:** The engine runs FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT. Knowing that order explains most beginner errors, such as an aggregate in WHERE.
> **A** is wrong: SELECT is written first but runs fifth, after the groups exist.
> **B** is wrong: WHERE runs second: it needs a table to filter.
> **D** is wrong: ORDER BY runs near the end, on the finished result.

Q2

What does this print?

```
conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE t (x INTEGER, y TEXT)")
conn.executemany("INSERT INTO t VALUES (?, ?)",
                 [(1, "a"), (2, None), (3, "c")])
print(conn.execute("SELECT COUNT(*), COUNT(y), SUM(x) FROM t").fetchone())
```

`(3, 2, 6)`

Why: `COUNT(*)` counts rows (3). `COUNT(y)` counts rows where `y` is not NULL (2: Python's `None` was stored as NULL). `SUM(x)` is 1 + 2 + 3. The row comes back as a tuple.

Q3

This line raises `ProgrammingError: Incorrect number of bindings supplied. The current statement uses 1, and there are 5 supplied.` What is the fix?

```
conn.execute("SELECT COUNT(*) FROM orders WHERE store = ?", "Boise")
```

A) Pass a one-item tuple: `("Boise",)`

B) Write five placeholders: `?????`

C) Put the value in the text with an f-string

D) Use double quotes around the `?`

Why: The parameters must be a sequence of values. A string is a sequence of its characters, so `"Boise"` looked like five values. A one-item tuple needs the trailing comma; a list `["Boise"]` also works.

B is wrong: The statement needs one value. Five placeholders would ask for five different values.

C is wrong: That removes the error and opens the door to SQL injection. Never.

D is wrong: A quoted `?` is the text of a question mark, not a placeholder.

Q4

You want the average order per store for completed orders in 2025, but only for stores whose average is above $190. Which condition belongs in `HAVING`?

A) `status = 'completed'`

B) `AVG(order_total) > 190`

C) `order_date >= '2025-01-01'`

D) `store IN ('Boise', 'Bend')`

> **Why:** WHERE filters rows before the groups exist, so it cannot mention an aggregate. A condition on a group's result goes in HAVING, which runs after GROUP BY.
> **A** is wrong: A condition on each row: WHERE.
> **C** is wrong: A condition on each row: WHERE.
> **D** is wrong: It mentions the grouping column but it is still a row filter, and WHERE is the normal place for it.

Q5

A store name typed into a web form arrives in the variable `store`. Which line passes it to the database safely?

A) `conn.execute(f"SELECT * FROM orders WHERE store = '{store}'")`

B) `conn.execute("SELECT * FROM orders WHERE store = '" + store + "'")`

C) `conn.execute("SELECT * FROM orders WHERE store = '{}'".format(store))`

D) `conn.execute("SELECT * FROM orders WHERE store = ?", (store,))`

> **Why:** With a placeholder the SQL text and the value travel separately, so the value can never be read as part of the command.
> **A** is wrong: An f-string pastes the visitor's text into the command: SQL injection.
> **B** is wrong: String concatenation is the same mistake with a different spelling.
> **C** is wrong: So is `.format()`. All three break on a name like O'Brien even with no attacker.

Q6

What does this print?

```
conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE t (k INTEGER PRIMARY KEY)")
conn.commit()
try:
    with conn:
        conn.execute("INSERT INTO t VALUES (1)")
        conn.execute("INSERT INTO t VALUES (1)")
except sqlite3.IntegrityError:
    pass
print(conn.execute("SELECT COUNT(*) FROM t").fetchone()[0])
```

`0`

> **Why:** Both inserts are in one transaction. The second breaks the primary key, the block raises, and `with conn:` rolls the whole transaction back, including the first insert. All or nothing.

5.1 · PREDICT, THEN RUN

P1

Output and reasoning

```
sql = "SELECT COUNT(*) FROM products WHERE name LIKE '%TENT%'"
print(conn.execute(sql).fetchone()[0])
```

> **Answer:** `18`. In SQLite, `LIKE` ignores case for the letters A to Z, so `'%TENT%'` matches "Tent". It is 18, not 24 (the size of the Tents category), because six products in that category are shelters and tarps with no "tent" in the name. In PostgreSQL `LIKE` is case sensitive and this returns 0; use `ILIKE` there.

P2

Output and reasoning

```
cur = conn.execute("SELECT order_id FROM orders ORDER BY order_id LIMIT 5")
cur.fetchone()
cur.fetchone()
print(len(cur.fetchall()), cur.fetchone())
```

> **Answer:** `3 None`. The cursor moves forward only: two rows were fetched, three were left for `fetchall()`, and after that the cursor is spent, so `fetchone()` returns `None`.

P3

Output and reasoning

```
print(conn.execute("SELECT COUNT(*), COUNT(promo_code) FROM orders").fetchone())
```

> **Answer:** `(20302, 2148)`. `COUNT(*)` counts all rows; `COUNT(promo_code)` counts only the rows where the code is not NULL (20,302 - 18,154).

5.1 · WORKED EXAMPLE AND LAB REFERENCE NOTES

Worked example (`promo_report`). Output as printed in the workbook. Points to draw out: NULL forms its own group (9,162 completed 2025 orders with no code); `HAVING` removed the small groups after grouping; the year reaches the SQL only as two parameter values. A learner who writes `WHERE strftime('%Y', order_date) = '2025'` is also correct; the range form is the habit that later lets an index help (Lesson 5.4).

`labs/module-14/l5_1-database-connections/staff_report_solution.py`. Read-only connection; six steps. Reference output:

```
(0, 'employee_id', 'INTEGER', 0, None, 1)
(1, 'first_name', 'TEXT', 1, None, 0)
(2, 'last_name', 'TEXT', 1, None, 0)
(3, 'title', 'TEXT', 1, None, 0)
(4, 'department', 'TEXT', 1, None, 0)
(5, 'location', 'TEXT', 1, None, 0)
(6, 'manager_id', 'INTEGER', 0, None, 0)
(7, 'hire_date', 'TEXT', 1, None, 0)
(8, 'salary', 'INTEGER', 1, None, 0)
(9, 'email', 'TEXT', 1, None, 0)
rows: 55

['first_name', 'last_name', 'title', 'hire_date']
fetchone: ('Clara', 'Soto', 'Chief Executive Officer', '2016-02-01')
fetchall: [('Bianca', 'Sato', 'Store Manager', '2016-05-08'), ('Tamar', 'Kimura',
'VP of Retail', '2016-09-30')]
fetchone again: None

    employee         title  location  salary
  Helga Shah Store Manager     Boise   77800
  Leo Mendez Store Manager      Bend   76500
 Bianca Sato Store Manager Asheville   73900
Uma Quintero Store Manager Flagstaff   68900

department  people  payroll  avg_salary first_hire
    Retail      36  1841600     51156.0 2016-05-08
Operations      11   753600     68509.0 2018-05-21
E-commerce       6   552500     92083.0 2018-07-23
 Executive       2   273600    136800.0 2016-02-01

 location  people  avg_salary
       HQ      17     99512.0
    Boise      10     43950.0
Asheville       8     45613.0
     Bend       8     45613.0

            count      sum
department
Retail         36  1841600
Operations     11   753600
E-commerce      6   552500
Executive       2   273600
```

Common wrong answers: the "at least 8 people" condition in WHERE (`misuse of aggregate`); expecting the last `fetchone()` to return the first row again; `location != 'HQ'` instead of the list of four stores in Step 3 (it lets the Warehouse in; here the Retail filter hides the slip, which is worth pointing out); `AVG(salary)` shown unrounded.

`labs/module-14/l5_1-database-connections/safe_lookup_solution.py`. Reference output:

```
Boise, quantity <= 0:  unsafe 65 | safe 65
Warehouse, quantity <= 5:  unsafe 39 | safe 39

crafted input, unsafe: 1210 (every row in the table)
crafted input, safe:   0 (no location has that odd name)
rows in inventory:     1210

rows for TNT-1001: 5
ProgrammingError: Incorrect number of bindings supplied. The current statement uses
1, and there are 8 supplied.

Flagstaff, out of stock: 62 SKUs
     sku  quantity  reorder_point last_counted
APL-1012         0              3   2025-12-01
PCK-1009         0              8   2025-12-01
SNW-1008         0              4   2025-12-01
KIT-1007         0              7   2025-12-02
last_counted dtype: datetime64[us]
ValueError: sort_by must be one of ['last_counted', 'quantity', 'sku']
```

The SQL the unsafe function builds from the crafted input is `SELECT COUNT(*) FROM inventory WHERE location = 'nowhere' OR '1'='1' --' AND quantity <= 0`. The quote closes the string early, `OR '1'='1'` is true for every row, and the two dashes turn the rest of the line into a comment. **Common wrong answers:** `(location)` without the comma; quotes around the placeholder (`'?'`); trying a placeholder for the ORDER BY column (it sorts by a constant and silently does nothing); believing the f-string for `sort_by` breaks the rule (it inserts a name checked against a fixed list, never a value).

`labs/module-14/l5_1-database-connections/save_summary_solution.py`. Works on `pinecrest_work.db` in the output folder. Reference output:

```
dialect: sqlite | tables: ['customers', 'employees', 'inventory', 'order_items',
'orders', 'products']

 location  skus  units  below_reorder  out_of_stock
Warehouse   242  14229             49            24
     Bend   242   2230             81            59
Flagstaff   242   2191             89            62
Asheville   242   2140             79            54
    Boise   242   2025             89            65

 location  below_reorder  below_pct
Warehouse             49       20.2
     Bend             81       33.5
Flagstaff             89       36.8
Asheville             79       32.6
    Boise             89       36.8

rows written: 5
highest share below reorder point: [('Boise', 36.8), ('Flagstaff', 36.8)]
tables now: ['customers', 'employees', 'inventory', 'order_items', 'orders',
'products', 'stock_summary']

IntegrityError: UNIQUE constraint failed: inventory.sku, inventory.location
inventory rows before: 1210 | after: 1210 | Popup Stand rows: 0

stock_summary in the shipped database: False | in the working copy: True
```

The Popup Stand row was valid, and it is gone, because it shared a transaction with a row that broke the primary key. **Common wrong answers:** `engine.connect()` for the inserts with no commit (the rows vanish at the end of the block: SQLAlchemy 2 does not autocommit); a bare string instead of `text(...)`; `if_exists="append"` on a rerun, which doubles the table to 10 rows; forgetting `index=False`.

5.1 · THE LESSON PAGE'S OWN QUIZ

1. **Question 1: B.** `GROUP BY store HAVING COUNT(*) > 2000`: a condition on an aggregate goes in HAVING, after GROUP BY. A fails (aggregate in WHERE); C has the clauses in the wrong order.
2. **Question 2: C.** Text typed by the visitor becomes part of the SQL command itself: SQL injection.
3. **Question 3: A.** 280: `COUNT(*)` is 4,850 and `COUNT(phone)` is 4,570, the rows where phone is not NULL.
4. **Question 4: C.** The table as it was before the INSERT: nothing was committed, so the change was never saved to the file.
5. **Question 5: B.** Put the GROUP BY and SUM in the SQL so that only one row per store comes back.

5.2 · COMPLEX QUERIES: SELF-CHECK

Q7

`orders` has 20,302 rows; 739 of them have a `customer_id` that is not in `customers`; 1,058 customers have never ordered. How many rows does `orders JOIN customers ON c.customer_id = o.customer_id` return?

A) 20,302

B) 19,563

C) 21,360

D) 19,244

> **Why:** A plain JOIN is an inner join: only orders that found a customer survive. 20,302 - 739 = 19,563. The 739 rows vanish with no error.
> **A** is wrong: That is the LEFT JOIN count: every order kept, 739 of them with an empty customer side.
> **C** is wrong: That is the FULL OUTER JOIN: 19,563 + 739 + 1,058.
> **D** is wrong: Customers without orders have no order row to lose, so 1,058 is not subtracted.

Q8

What does this print?

```
conn = sqlite3.connect(":memory:")
conn.executescript("""
    CREATE TABLE o (id INTEGER, total REAL);
    CREATE TABLE i (order_id INTEGER, sku TEXT);
    INSERT INTO o VALUES (1, 100), (2, 50);
    INSERT INTO i VALUES (1, 'a'), (1, 'b'), (1, 'c'), (2, 'd');
""")
sql = "SELECT SUM(o.total) FROM o JOIN i ON i.order_id = o.id"
print(conn.execute(sql).fetchone()[0])
```

`350.0`

> **Why:** Order 1 has three lines, so its total of 100 appears three times after the join; order 2 appears once. 300 + 50 = 350, not the true 150. This is the fan-out trap: sum a column at the level where it lives.

Q9

Which query lists the customers who have never placed an order?

A) `SELECT c.* FROM customers AS c LEFT JOIN orders AS o ON o.customer_id = c.customer_id WHERE o.order_id IS NULL`

B) `SELECT c.* FROM customers AS c JOIN orders AS o ON o.customer_id = c.customer_id WHERE o.order_id IS NULL`

C) `SELECT c.* FROM customers AS c LEFT JOIN orders AS o ON o.customer_id = c.customer_id WHERE o.order_id = NULL`

D) `SELECT c.* FROM orders AS o LEFT JOIN customers AS c ON c.customer_id = o.customer_id WHERE c.customer_id IS NULL`

Why: The anti-join: keep every customer with LEFT JOIN, then keep the rows where the order side is empty. It returns the 1,058 customers with no orders.
B is wrong: An inner join has already thrown away customers with no order, so nothing is left to find.
C is wrong: `= NULL` is never true. Test for NULL with `IS NULL`.
D is wrong: That starts from orders: it finds the 739 orders with no customer, the opposite question.

Q10

359 of the 20,302 orders used the code TRAIL15. `SELECT COUNT(*) FROM orders WHERE promo_code <> 'TRAIL15'` returns 1,789, not 19,943. Why?

A) `<>` is not a valid operator in SQLite

B) The query needs `DISTINCT`

C) Text comparison ignores case, so some codes were double counted

D) For the 18,154 orders with no code the comparison with NULL is unknown, and WHERE keeps only rows where the condition is true

Why: NULL means unknown, so `NULL <> 'TRAIL15'` is unknown and the row is skipped. To mean "every order that did not use TRAIL15", add `OR promo_code IS NULL` or compare `COALESCE(promo_code, 'none')`.
A is wrong: `<>` is the standard not-equal operator (`!=` works too).
B is wrong: DISTINCT removes duplicate rows from a result; it does not bring back skipped rows.
C is wrong: Case has nothing to do with it: 359 + 1,789 + 18,154 = 20,302 accounts for every order.

Q11

What does this print?

```
conn = sqlite3.connect(":memory:")
print(conn.execute("SELECT 7 / 2, 7 / 2.0, 100 * 1 / 4").fetchone())
```

`(3, 3.5, 25)`

> **Why:** Dividing one integer by another gives an integer in SQLite (and PostgreSQL): 7 / 2 is 3. One real number makes the arithmetic real: 3.5. `100 * 1 / 4` is all integers: 25. A percentage column that is all zeros is integer division: multiply by `100.0`.

Q12

A recursive CTE walks the employee tree from the CEO down. What makes the recursion stop?

A) The `LIMIT` that every recursive CTE must have

B) It reaches the row whose `manager_id` is NULL

C) A round of the recursive part finds no new rows

D) It has run once for every row in the table

> **Why:** Each round joins the table to the rows found in the round before. When a round adds nobody (the newest employees manage no one), the loop ends and everything found so far is returned.
> **A** is wrong: A LIMIT is a useful guard while developing, but it is not required and is not what ends a healthy recursion.
> **B** is wrong: The row with a NULL manager is the anchor, the START of the walk.
> **D** is wrong: The number of rounds is the depth of the tree (six levels here), not the number of rows.

5.2 · PREDICT, THEN RUN

P1

Output and reasoning

```
sql = "SELECT COUNT(*) FROM employees AS e JOIN employees AS m"
print(conn.execute(sql).fetchone()[0])
```

> **Answer:** `3025`. 55 × 55. SQLite treats a JOIN with no ON as a cross join. No error; PostgreSQL would reject it. On `orders` and `order_items` the same slip would produce over 900 million rows.

P2

Output and reasoning

```
print(conn.execute("SELECT NULL = NULL, NULL IS NULL, COUNT(NULL)").fetchone())
```

> **Answer:** `(None, 1, 0)`. `NULL = NULL` is unknown (shown as `None`), not true. `NULL IS NULL` is 1 (true). `COUNT(NULL)` is 0, because COUNT of a value skips NULL.

P3

Output and reasoning

```
q = "SELECT channel FROM orders"
for op in ["UNION", "UNION ALL"]:
    print(op, conn.execute(f"SELECT COUNT(*) FROM ({q} {op} {q})").fetchone()[0])
```

> **Answer:** `UNION 2 / UNION ALL 40604`. `UNION` removes duplicates, so two copies of a column with two distinct values give 2 rows. `UNION ALL` stacks everything: 20,302 + 20,302.

5.2 · WORKED EXAMPLE AND LAB REFERENCE NOTES

Worked example (return rate by tier). Output as printed in the workbook: the groups add up to 20,302 orders; the inner join would have kept 19,563. Basic 8.0%, Silver 5.4%, Gold 4.7%, closed accounts 5.5%. The point to press: tiers are defined by completed spend, which returns reduce, so "loyal customers return less" is partly built into the definition. A learner who writes "Gold customers are more careful buyers" has skipped Lesson 4.4's question about how the data was generated.

The table of links in the workbook. customers (one) to orders (many): 739 orders without a customer, 1,058 customers without an order. orders (one) to order_items (many): none unmatched. products (one) to order_items (many): 609 lines without a product, 8 products without a line.

`labs/module-14/l5_2-complex-queries/join_row_counts_solution.py`. Reference output:

```
 lines  inner_join  left_join
 45490       44881      45490

 revenue_inner_join  lines
          3602058.1  41221

         brand  lines  item_revenue
     Wrenfield   5107     515811.85
     Quillback   5688     496503.80
     Pinecrest   5864     472441.80
     Tallgrass   5450     457486.65
  Granite Fork   4640     443898.60
  Juniper Vane   5105     433123.35
   Cedar Notch   5334     414783.45
Larkspur Ridge   4033     368008.60
  Discontinued    567      34228.25
all completed lines: 41788 | item revenue: 3636286.35

     sku  lines  units  item_revenue  last_sold
APL-1033     93    143      11432.85 2024-09-23
SNW-1018     37     37      10136.15 2024-09-08
KIT-1016    357    800       6000.00 2024-09-29
PCK-1025     18     18       4823.10 2024-09-21
SNW-1007     73    164       2214.00 2024-09-23
PCK-1015     31     31       1889.45 2024-09-22

       brand  never_sold
Granite Fork           3
   Wrenfield           2
Juniper Vane           1
   Pinecrest           1
   Tallgrass           1
```

The lost revenue is 3,636,286.35 - 3,602,058.10 = 34,228.25, on 567 completed lines (609 lines in all statuses) of six SKUs. **Common wrong answers:** predicting 45,490 for the inner join; reading the unnamed group as "brand missing in the data" instead of "product no longer in the catalog"; writing the anti-join with `WHERE p.sku = NULL` (returns nothing); an inner join to products in Step 4 (no rows: it removed exactly the rows being looked for).

`labs/module-14/l5_2-complex-queries/stock_vs_sales_solution.py`. Reference output:

```
 units_sold_2025  units_on_hand
           30894          22815

 units_sold_2025  units_on_hand  rows_summed
          154470        3693132       111520
Each order line met its SKU's five inventory rows (units sold x 5), and each
inventory
row was repeated once per order line of that SKU (units on hand x 162). No error, no
warning.

 skus  units_sold_2025  units_on_hand
  242            30894          22815

    category  sold_2025  on_hand  weeks_of_cover cover
 Accessories       8305     3884            24.3  thin
Camp Kitchen       6420     3647            29.5    ok
        Snow       2099     1678            41.6    ok
    Footwear       3184     2557            41.8    ok
     Apparel       3225     2643            42.6    ok
    Climbing       2223     2090            48.9 heavy
    Sleeping       2298     2373            53.7 heavy
       Packs       1821     2149            61.4 heavy
       Tents       1319     1794            70.7 heavy

 integer_math  real_math  ratio_as_integers
           38  38.401631                  0
Check: 2025 lines on SKUs that are not in products (they would be missing from
per_sku): 0
```

Why those factors: every SKU has exactly five inventory rows, so each order line is repeated five times (30,894 × 5 = 154,470). Each inventory row is repeated once per 2025 order line of its SKU, about 162 times on average. The fix aggregates each side to one row per SKU before joining; `per_sku` then has 242 rows and both totals are right. The join to `sold` is a LEFT JOIN so that a product with no 2025 sales stays, with 0. **Common wrong answers:** `COUNT(DISTINCT ...)` tricks to "undo" the fan-out (they cannot fix a sum); `52 * on_hand / sold` with integers (38 instead of 38.4 overall); no guard against dividing by zero (`NULLIF(sold, 0)`).

`labs/module-14/l5_2-complex-queries/org_payroll_solution.py`. Reference output:

```
 employee_id      employee            title              reports_to
           2  Tamar Kimura     VP of Retail Chief Executive Officer
           3   Arjun Silva VP of E-commerce Chief Executive Officer
           4 Elise Delgado VP of Operations Chief Executive Officer
          55    Zane Laine       HR Manager Chief Executive Officer

         head  people_including_head  levels_below  payroll
 Tamar Kimura                     36             4  1841600
Elise Delgado                     11             2   753600
  Arjun Silva                      6             2   552500
   Zane Laine                      1             0    88600

 employees  in_a_branch  distinct_people  salary_not_in_a_branch
        55           54               54                  185000

 products  rated_4_or_more  rated_below_4  no_rating  count_rating
      242              164             70          8           234

 avg_weight_known  avg_with_zero_filled  with_weight  all_products
            0.793                 0.747          228           242

 not_in  not_exists  managers
      0          32        23
```

The anchor is the CEO's four direct reports, each carrying its own name as `head`; the recursive part passes `branch.head` down unchanged. 54 of 55 employees are in exactly one branch; the missing salary, 185,000, is the CEO's. The NULL checks: 164 + 70 = 234 rated products, and 8 unrated make 242; `AVG(weight_kg)` uses the 228 known weights while filling with 0 drags the average down; `NOT IN` returns 0 because the list contains the CEO's NULL. **Common wrong answers:** starting the anchor at the CEO (one branch of 55); `UNION` instead of `UNION ALL`; reporting the zero-filled average weight.

5.2 · THE LESSON PAGE'S OWN QUIZ

1. **Question 1: A.** 20,302: a LEFT JOIN keeps every order; the 739 unmatched ones have NULL on the customer side.
2. **Question 2: C.** 0: the list contains a NULL, and NOT IN against a list with a NULL is never true.
3. **Question 3: B.** Each order's total is repeated once for every line on the order, then summed: the fan-out trap.
4. **Question 4: A.** 7: UNION removes duplicates. Four cities are shared; add Online, HQ and Warehouse.
5. **Question 5: C.** In the ON clause. In WHERE the condition would discard the NULL rows and undo the left join.

WARM-UP AND EXIT TICKET

WARM-UP: RETRIEVAL FROM SESSION 13

1. Name three of the six lines of an experiment plan. **Answer:** Any three of: unit, primary metric, guardrail metric, hypotheses (two-sided, alpha), sample size and duration, decision rule.

2. Conversion went from 8.86% to 10.03%. Say the lift two ways. **Answer:** +1.17 percentage points (absolute) and +13% (relative). Both are correct; always say which.

3. Why is "stop the test the first time p is below 0.05" a problem? **Answer:** Each look is another chance to cross the line by luck. With weekly looks the false-alarm rate was about 12% instead of 5%; with daily looks about 27%.

4. What does "grain" mean, and what is the grain of `order_items`? **Answer:** What one row represents. One product line on an order (45,490 rows for 20,302 orders). Today's joins live or die on this.

5. Merging 1,800 survey rows with `orders` on `customer_id` gave 9,230 rows. What happened, and what guards against it? **Answer:** Each response was repeated once per order of that customer. Guard: summarize to one row per customer first, and state the relationship with `validate=`. In SQL the same trap is called fan-out, and you will see it turn $3.7 million into $10.3 million after the break.

EXIT TICKET

1. In one sentence each: what does WHERE filter, and what does HAVING filter? **Answer:** WHERE filters rows before they are grouped (so it cannot mention an aggregate). HAVING filters groups after GROUP BY (so it can).

2. Why do values go into SQL through placeholders and never through an f-string? **Answer:** With a placeholder the SQL text and the value travel separately, so a value can never be read as part of the command. An f-string pastes the value into the command: a quote in the input can change the query (SQL injection), and an honest name like O'Brien breaks it.

3. `orders` has 20,302 rows and 739 have no matching customer. How many rows from `orders JOIN customers`, and how many from `orders LEFT JOIN customers`? **Answer:** 19,563 and 20,302. The inner join drops the unmatched rows silently; the left join keeps them with NULL on the customer side.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **"WHERE and HAVING are the same thing" (Q4, exit 1).** The learner has not connected the clauses to the order they run in. Have them point at the board: where do the groups come into existence?
- **"The f-string is fine if I trust the input" (Q5, exit 2).** They see injection only as an attack. Give the honest failure: a customer called O'Brien.
- **Predicting 20,302 for the inner join (Q7, exit 3).** They treat a join as a lookup that cannot fail. Ask what happened to the order whose customer card is missing.

- **150 for the fan-out item (Q8).** The right total, which means the learner did not picture the joined rows. Have them write the four rows of the join by hand.
- **"NULL is like zero" (Q10, lab C).** Then `<>` should have matched. Insist on the word "unknown".
- **"The recursion stops at the CEO" (Q12).** The learner reads the tree bottom-up. Trace two rounds from the anchor on the board.

INSTRUCTOR KIT · ANSWER KEY

Session 15: SQL Windows and Query Speed

Answers with reasoning for every workbook item, reference output for all five labs, the lesson pages' quiz answers, and the warm-up and exit-ticket answers.

Session 15 | 12 self-check items | Confidential

HOW TO USE THIS KEY

- Self-check items repeat the workbook's wording and options exactly; the correct option is highlighted and each wrong option has a one-line reason.
- Outputs come from running the code on the course data (SQLite 3.53). Plans and counts are stable; any time in milliseconds is from the author's machine and will differ on yours. Nothing in this key depends on a time.

5.3 · SQL WINDOW FUNCTIONS: SELF-CHECK

Q1

Which pandas line gives the same column as `SUM(order_total) OVER (PARTITION BY store)`, one value on every order row?

A) `orders.groupby("store")["order_total"].sum()`

B) `orders.groupby("store")["order_total"].transform("sum")`

C) `orders["order_total"].cumsum()`

D) `orders["order_total"].rolling(5).sum()`

Why: `transform` returns one value per original row, aligned with the rows, exactly like a window function. A window keeps every row; GROUP BY (and `.sum()` after groupby) collapses them.
A is wrong: That returns five numbers, one per store: the GROUP BY version.
C is wrong: That is a running total over all rows, `SUM(...) OVER (ORDER BY ...)`, with no partition.
D is wrong: That is a moving sum of five rows, unrelated to the store.

Q2

What does this print?

```
rows = conn.execute("""
    SELECT v, ROW_NUMBER() OVER (ORDER BY v DESC, id),
           RANK() OVER (ORDER BY v DESC), DENSE_RANK() OVER (ORDER BY v DESC)
    FROM (SELECT 1 AS id, 50 AS v UNION ALL SELECT 2, 70
          UNION ALL SELECT 3, 50 UNION ALL SELECT 4, 40)
    ORDER BY v DESC, id
""").fetchall()
print(rows[-1])
```

`(40, 4, 4, 3)`

> **Why:** Order: 70, 50, 50, 40. ROW_NUMBER never repeats: 4. RANK gives both 50s the rank 2 and skips 3, so 40 is 4. DENSE_RANK does not skip: 70 is 1, the 50s are 2, 40 is 3.

Q3

`SELECT ... ROW_NUMBER() OVER (PARTITION BY category ORDER BY revenue DESC) AS rn FROM ... WHERE rn <= 2` **fails. Why, and what is the fix?**

A) WHERE runs before window functions are computed; compute `rn` in a CTE and filter it in the outer query

B) ROW_NUMBER needs a ROWS BETWEEN frame; add one

C) `rn` is a reserved word; rename it

D) PARTITION BY cannot be combined with ORDER BY; remove the partition

> **Why:** Window functions are computed in the SELECT step, after WHERE. The fix is the recipe: number in a CTE, then filter outside it.
> **B** is wrong: Ranking functions ignore the frame entirely.
> **C** is wrong: `rn` is an ordinary name; the error says "misuse of aliased window function rn".
> **D** is wrong: PARTITION BY with ORDER BY is exactly how you number rows within groups.

Q4

Which frame averages the three months before the current one, without the current month (no leakage)?

A) `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`

B) `ROWS BETWEEN 3 PRECEDING AND 1 PRECEDING`

C) `ROWS BETWEEN 3 PRECEDING AND CURRENT ROW`

D) `ROWS BETWEEN 1 FOLLOWING AND 3 FOLLOWING`

> **Why:** The frame ends one row before the current row, so the current month never sees itself. It is the SQL form of `shift(1).rolling(3, min_periods=1).mean()`.
> **A** is wrong: Three months including the current one: a moving average, not a prior average.
> **C** is wrong: Four rows, including the current one.
> **D** is wrong: The three months AFTER: that looks into the future.

Q5

What does this print? (Two rows share d = 1.)

```
sql = """SELECT d, x, SUM(x) OVER (ORDER BY d)
         FROM (SELECT 1 AS d, 10 AS x UNION ALL SELECT 1, 20
               UNION ALL SELECT 2, 5)
         ORDER BY d, x"""
print([row[2] for row in conn.execute(sql)])
```

`[30, 30, 35]`

> **Why:** The default frame (RANGE ... CURRENT ROW) includes every row that ties with the current one in the window's order, so both d = 1 rows see 10 + 20. With `ORDER BY d, x ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` the answer would be `[10, 30, 35]`.

Q6

A SQL column is `AVG(revenue) OVER (ORDER BY month ROWS BETWEEN 3 PRECEDING AND CURRENT ROW)`. A colleague computes `df["revenue"].rolling(4).mean()`. Where do they disagree?

A) On every row: SQL counts rows, pandas counts days

B) Nowhere: the two are identical

C) Only on the last row, because SQL has no following rows

D) On the first three rows: SQL averages the rows that exist, pandas gives NaN until four rows exist

> **Why:** SQL uses whatever part of the frame exists. pandas `rolling(4)` needs a full window unless you add `min_periods=1`, which reproduces the SQL column.
> **A** is wrong: Both count rows here (ROWS in SQL, an integer window in pandas).
> **B** is wrong: They agree from the fourth row on, not before.
> **C** is wrong: The frame looks backward; the last row has a full frame in both.

5.3 · PREDICT, THEN RUN

P1

Output and reasoning

```
show(MONTHLY + """
    , lagged AS (
        SELECT month, LAG(revenue, 12) OVER (ORDER BY month) AS year_before
        FROM monthly
    )
    SELECT COUNT(*) AS months, COUNT(year_before) AS with_a_year_before
    FROM lagged
""")
```

> **Answer:**
>
> ```
>  months  with_a_year_before
>      24                  12
> ```
>
> LAG(revenue, 12) looks twelve rows back; only the twelve months of 2025 have one. (COUNT of a value skips the NULLs.)

P2

Output and reasoning

```
show("""
    SELECT NTILE(3) OVER (ORDER BY v) AS bucket, v
    FROM (SELECT 1 AS v UNION ALL SELECT 2 UNION ALL SELECT 3 UNION ALL SELECT 4
          UNION ALL SELECT 5 UNION ALL SELECT 6 UNION ALL SELECT 7)
""")
```

Answer:

```
 bucket  v
      1  1
      1  2
      1  3
      2  4
      2  5
      3  6
      3  7
```

Seven rows into three buckets: 3, 2, 2. NTILE gives the extra rows to the first buckets.

P3

Output and reasoning

```
show(MONTHLY + """
    SELECT month, revenue,
           LAST_VALUE(revenue) OVER (ORDER BY month) AS last_value
    FROM monthly
    ORDER BY month
    LIMIT 2
""")
```

Answer:

```
   month   revenue  last_value
2024-01 106287.63   106287.63
2024-02 104687.91   104687.91
```

The default frame ends at the current row, so the "last" value of the frame is the row itself. Extend the frame to `UNBOUNDED FOLLOWING`, or sort the other way and use FIRST_VALUE.

5.3 · WORKED EXAMPLE AND LAB REFERENCE NOTES

Worked example (weakest month per store). Output as printed in the workbook. A learner who writes `ORDER BY revenue DESC` gets each store's best month instead; one who forgets the PARTITION gets one row in total. The point to check: the store average is on the same row because it is also a window.

`labs/module-15/l5_3-sql-window-functions/staff_rankings_solution.py`. Reference output:

```
location       employee  salary  location_avg  vs_avg
   Boise     Helga Shah   77800       43950.0 33850.0
   Boise    Luka Petrov   54900       43950.0 10950.0
   Boise    Mabel Getty   50500       43950.0  6550.0
   Boise     Ben Ellery   39700       43950.0 -4250.0
   Boise   Noor Ekwueme   39000       43950.0 -4950.0
   Boise  Zara Prescott   39000       43950.0 -4950.0
   Boise  Sienna Dubois   35300       43950.0 -8650.0
   Boise   Anton Zeller   34700       43950.0 -9250.0
   Boise     Bodhi Kato   34500       43950.0 -9450.0
   Boise Elise Mahlangu   34100       43950.0 -9850.0

location       employee  salary  pct_of_payroll  rnk  dense_rnk
   Boise     Helga Shah   77800            17.7    1          1
   Boise    Luka Petrov   54900            12.5    2          2
   Boise    Mabel Getty   50500            11.5    3          3
   Boise     Ben Ellery   39700             9.0    4          4
   Boise   Noor Ekwueme   39000             8.9    5          5
   Boise  Zara Prescott   39000             8.9    5          5
   Boise  Sienna Dubois   35300             8.0    7          6
   Boise   Anton Zeller   34700             7.9    8          7
   Boise     Bodhi Kato   34500             7.8    9          8
   Boise Elise Mahlangu   34100             7.8   10          9

 location  rn          employee                  title  salary
Asheville   1       Bianca Sato           Store Manager   73900
Asheville   2 Gabriel Underhill       Assistant Manager   55000
     Bend   1        Leo Mendez           Store Manager   76500
     Bend   2        Zane Novak       Assistant Manager   54800
    Boise   1        Helga Shah           Store Manager   77800
    Boise   2       Luka Petrov       Assistant Manager   54900
Flagstaff   1      Uma Quintero           Store Manager   68900
Flagstaff   2    Darius Pacheco       Assistant Manager   55700
       HQ   1        Clara Soto Chief Executive Officer  185000
       HQ   2      Tamar Kimura            VP of Retail  140700
Warehouse   1    Bianca Ferrand       Warehouse Manager   71100
Warehouse   2    Gideon Rinaldi     Warehouse Associate   42100
rows: 12 (6 locations x 2)

pandas top two match SQL: True
Boise average in pandas: 43950.0
```

Common wrong answers: `rn <= 2` in the same SELECT (`misuse of window function`); RANK used for the top two, which can return three rows on a tie; `ORDER BY salary DESC` with no tiebreaker (the 39,000 pair in Boise could swap places between runs); percentages computed with integer arithmetic.

`labs/module-15/l5_3-sql-window-functions/silver_milestone_solution.py`. Reference output:

```
 customer_id  order_id order_date  order_total  n  spent_so_far
       10034    100002 2024-01-01       278.90  1        278.90
       10034    101642 2024-03-20       159.80  2        438.70
       10034    102130 2024-04-08       115.80  3        554.50
       10034    103801 2024-06-09       136.95  4        691.45
       10034    103897 2024-06-13       283.90  5        975.35
       10034    106224 2024-09-04       303.90  6       1279.25

 customers_reaching_600  avg_order_number  fewest  most
                   2028               3.5       1     9
 order_number  customers
            1         80
            2        424
            3        578
            4        502
            5        286
            6        123
            7         31
            8          3
            9          1

customers in the Silver or Gold tier (customers table): 2028
The two counts agree: the running total reproduces the tier rule exactly.

 which_order  gaps  avg_days_since_previous
     order 2  3223                   118.2
     order 3  2615                   103.8
     order 4  2134                    89.4
     order 5  1703                    78.2
6th or later  6685                    57.2
Caution: later orders come only from customers who keep buying, and frequent buyers
have
short gaps by definition. The table describes WHO reaches a 6th order, not a change
in anyone.
```

The check in Step 3 is the heart of the lab: a running total over completed orders reproduces the documented tier rule exactly (2,028 = 1,294 Silver + 734 Gold). **Common wrong answers:** leaving out the join to `customers` (closed accounts add customers who reach $600 but have no tier, so the counts no longer match); counting all statuses (returns are not spend); the default frame without `order_id` as a tiebreaker (two orders on one day get the same running total, and the milestone can land on the wrong order); reading the gap table as "customers order more often over time" (it is a selection effect: only frequent buyers reach a 6th order).

`labs/module-15/l5_3-sql-window-functions/weekly_trend_solution.py`. Reference output:

```
weeks: 51 (full Monday-to-Sunday weeks of 2025)
week  orders  change  avg_4wk  weeks_in_frame  prior_4wk
  01     113     NaN    113.0               1        NaN
  02      94   -19.0    103.5               2      113.0
  03     110    16.0    105.7               3      103.5
  04     119     9.0    109.0               4      105.7
  05     109   -10.0    108.0               4      109.0

week  orders  prior_4wk  vs_prior_pct
  20     170      126.8          34.1
  34     184      131.0          40.5
  47     171      132.3          29.3
week 20: 2025-05-19 to 2025-05-25
week 34: 2025-08-25 to 2025-08-31
week 47: 2025-11-24 to 2025-11-30
All three hold a sale weekend (May 23-26, Aug 29-Sep 1, Nov 28-Dec 1): the spikes
are the promotions.

avg_4wk   against rolling(4, min_periods=1): largest difference 0.0e+00
prior_4wk against shift(1).rolling(4, min_periods=1): largest difference 0.0e+00
rolling(4) WITHOUT min_periods has 3 NaN rows at the start

 quartile  weeks  fewest  most
        1     13      94   116
        2     13     116   128
        3     13     129   141
        4     12     142   201
```

Week 01 is the first full Monday-to-Sunday week of 2025 (from January 6); the query uses 51 full weeks. The three spikes are the three sale weekends. **Common wrong answers:** including the partial weeks at the start and end of the year (week 00 and a short last week look like collapses); comparing with `avg_4wk` instead of `prior_4wk` (the spike week is inside its own average and hides itself); expecting `rolling(4)` to match without `min_periods=1`.

5.3 · THE LESSON PAGE'S OWN QUIZ

1. **Question 1: B.** 20,302, one per order: a window function keeps every row.
2. **Question 2: A.** (30, 4, 3): RANK skips after the tie (10, 20, 20, 30 gives 1, 2, 2, 4); DENSE_RANK does not (1, 2, 2, 3).
3. **Question 3: C.** Compute ROW_NUMBER() per customer in a CTE (newest first, with a tiebreaker) and keep rn = 1 outside it.
4. **Question 4: C.** They match from the third row on; SQL averages the rows that exist, pandas gives NaN until the window is full.

5 **Question 5: A.** Each store's first month gets the previous store's last month instead of NULL: the partition is missing.

5.4 · QUERY OPTIMIZATION: SELF-CHECK

Q7

Which plan line means the engine answered from the index alone and never opened the table?

A) `SCAN orders`

B) `SEARCH orders USING INDEX idx_orders_store_date (store=?)`

C) `SEARCH orders USING COVERING INDEX idx_orders_store_date_total (store=? AND order_date>? AND order_date<?)`

D) `USE TEMP B-TREE FOR ORDER BY`

> **Why:** COVERING means the index holds every column the query needs, so the table is not touched.
> **A** is wrong: A SCAN reads every row of the table.
> **B** is wrong: A plain index search finds the rows in the index, then fetches each one from the table.
> **D** is wrong: That is a sorting step, not a way of finding rows.

Q8

`orders` has an index on `(store, order_date)` and no other. Which filter can NOT seek in it?

A) `WHERE order_date = '2025-07-04'`

B) `WHERE store = 'Bend'`

C) `WHERE store = 'Bend' AND order_date = '2025-07-04'`

D) `WHERE store = 'Bend' AND order_date >= '2025-07-01'`

> **Why:** The leftmost-prefix rule: dates are sorted only within each store, so there is no single place to jump to for a date alone (before ANALYZE, the plan is a SCAN of the index).
> **B** is wrong: The first column of a composite index works on its own.
> **C** is wrong: Both columns, equality on each: the best case.
> **D** is wrong: Equality on the first column and a range on the second: exactly what the index was built for.

Q9

After `DROP INDEX`, you run the same `EXPLAIN QUERY PLAN` text again on the same connection and it still names the dropped index. What is going on?

A) The DROP did not happen until you commit

B) SQLite keeps dropped indexes until VACUUM

C) Python's sqlite3 reused its cached prepared statement, planned while the index existed; connect with `cached_statements=0` or open a new connection

D) EXPLAIN QUERY PLAN always shows the plan from the first time a query ran

> **Why:** The connection caches prepared statements by their text. Lesson 5.4 opens its connection with `cached_statements=0` so that every plan is made afresh, which matters only in a lesson about plans.
> **A** is wrong: DROP INDEX takes effect on your connection at once; committing makes it permanent for others.
> **B** is wrong: VACUUM reclaims space; the index is gone from the schema as soon as it is dropped.
> **D** is wrong: EXPLAIN plans the statement it is given; the stale answer comes from the cache in Python, not from SQLite.

Q10

What does this print?

```
conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE t (a INTEGER, b TEXT)")
conn.execute("CREATE INDEX idx_t_a ON t (a)")
sql = "EXPLAIN QUERY PLAN SELECT * FROM t WHERE a + 0 = 5"
print(conn.execute(sql).fetchone()[3])
```

`SCAN t`

> **Why:** The index is sorted by `a`, not by `a + 0`. Arithmetic or a function around the column hides it from the index; `WHERE a = 5` would give `SEARCH t USING INDEX idx_t_a (a=?)`. The same happens with `strftime('%Y', order_date) = '2025'`: write a range on the bare column.

Q11

A table receives thousands of inserts a minute and has nine indexes, most added "to be safe". What does each extra index cost?

A) Nothing: indexes only affect reads

B) Wrong query results when two indexes cover the same column

C) A full table scan on every SELECT

D) Slower inserts and a larger file, because every index is a sorted copy that must be kept up to date

> **Why:** The lesson measured it: the same 45,490-row load got slower and the file larger with every index added. Index for the queries that are slow and matter; drop the rest.
> **A** is wrong: Every insert, update and delete must update every index.
> **B** is wrong: Results stay correct; only speed and size change.
> **C** is wrong: Indexes never force a scan; the planner may simply not use them.

Q12

Before adding an index a query took 12 ms; after, 4 ms, each measured once. What should you do before reporting "3 times faster"?

A) Nothing: the numbers speak for themselves

B) Run ANALYZE and report the plan instead of times

C) Repeat each measurement several times, take the best, and check the plan changed from SCAN to SEARCH

D) Measure on a different computer

> **Why:** Single timings vary, and the first run is often slowest because later runs find pages already in memory. Best-of-N timings plus a changed plan are evidence; one pair of numbers is not.
> **A** is wrong: One before and one after can show nothing but the cache warming up.
> **B** is wrong: ANALYZE gives the planner statistics; it does not replace measuring.
> **D** is wrong: Another machine gives different absolute numbers and the same noise problem.

5.4 · PREDICT, THEN RUN

P1

Output and reasoning

```
plan("SELECT * FROM order_items WHERE sku = ?", ("TNT-1001",))
```

Answer:

```
    SCAN order_items
```

No index on `sku`, so a SCAN. The Tuning Report adds `idx_order_items_sku` for exactly this lookup.

P2

Output and reasoning

```
plan("SELECT sku FROM products WHERE name LIKE ?", ("%Tent%",))
```

Answer:

```
    SCAN products
```

A pattern starting with `%` can match anywhere, so sorted order is no help: a SCAN whatever indexes exist.

P3

Output and reasoning

```
plan("SELECT * FROM orders WHERE customer_id = ? ORDER BY order_date", (12673,))
```

Answer:

```
    SEARCH orders USING INDEX idx_orders_customer_id (customer_id=?)
    USE TEMP B-TREE FOR ORDER BY
```

A SEARCH on the customer index, then a TEMP B-TREE to sort that customer's handful of orders: cheap, because only a few rows are sorted.

5.4 · WORKED EXAMPLE AND LAB REFERENCE NOTES

Worked example (product lookup). The plan changes; the time barely does, because `products` has 242 rows. The point is the judgment: an index that changes the plan is not automatically worth keeping. Learners who say

"SEARCH, so keep it" have skipped the measurement and the cost.

`labs/module-15/l5_4-query-optimization/customer_lookup_plans_solution.py`. Reference output (times will differ):

```
state and city
    SCAN customers
city only
    SCAN customers
state and city, newest first
    SCAN customers
    USE TEMP B-TREE FOR ORDER BY

state and city
    SEARCH customers USING INDEX idx_customers_state_city (state=? AND city=?)
city only
    SCAN customers USING COVERING INDEX idx_customers_state_city
state and city, newest first
    SEARCH customers USING INDEX idx_customers_state_city (state=? AND city=?)
    USE TEMP B-TREE FOR ORDER BY

200 lookups: 49.2 ms before, 26.2 ms after (your numbers will differ)
same answer: True | 488 customers in Boise, ID

state and city, newest first, with (state, city, signup_date):
    SEARCH customers USING COVERING INDEX idx_customers_state_city_signup (state=?
AND city=?)

after DROP INDEX:
    SCAN customers
indexes named idx_... left in the copy: 0
```

Points to draw out: city-only is `SCAN customers USING COVERING INDEX` after the composite index, which is still a scan (leftmost prefix); the speedup for state and city is modest because about one customer in ten matches; the third column removes the TEMP B-TREE because, within a state and city, the index is already in sign-up order. **Common wrong answers:** calling the covering-index scan a SEARCH; putting `city` first "because it is more selective" (then a state-only filter cannot seek); re-planning after DROP INDEX on a connection without `cached_statements=0` and reporting that the drop failed.

`labs/module-15/l5_4-query-optimization/n_plus_one_solution.py`. Reference output (times will differ):

```
Gold customers in the list: 300

plan of the per-customer query:
    SCAN orders
300 queries in a loop: 133 ms (your numbers will differ)
one GROUP BY query:    2.6 ms
same answers: True

plan with idx_orders_customer_id:
    SEARCH orders USING INDEX idx_orders_customer_id (customer_id=?)
300 queries in a loop: 3.0 ms
one GROUP BY query:    1.2 ms
same answers: True

       orders   spent
count   300.0   300.0
mean     10.6  2335.2
min       3.0  1502.0
max      31.0  6079.0

The loop was the slow part, not SQL: 300 scans of 20,302 orders. Either fix works;
together
they are best. On a server database each loop query also pays a network round trip,
which no
index can remove, so the one-query version is the habit to keep.
```

The IN list is built from one `?` per id and the ids are passed as parameters, so the Lesson 5.1 rule holds. **Common wrong answers:** pasting the ids into the SQL with `str(tuple(ids))` (works on integers, and is exactly the habit that breaks on text); forgetting to sort before comparing the two results (row order is not guaranteed); concluding "the index makes the loop fine": it does on a local file, not where each query is a network round trip.

5.4 · THE LESSON PAGE'S OWN QUIZ

1. **Question 1: C.** The engine read every row of the table to find the three.
2. **Question 2: A.** A range on the bare column can seek in the index; a function or a leading wildcard cannot.
3. **Question 3: B.** Inserts are slower and the database is larger than it needs to be.
4. **Question 4: B.** Almost every row matches, so there is almost nothing to skip.
5. **Question 5: A.** SCAN inventory: location is the second column of the primary-key index, so there is no place to jump to (leftmost-prefix rule).

WARM-UP AND EXIT TICKET

WARM-UP: RETRIEVAL FROM SESSION 14

1. In one sentence each: what does WHERE filter, and what does HAVING filter? **Answer:** WHERE filters rows before grouping; HAVING filters groups after GROUP BY. (Today adds a third stage: window functions run after both, in the SELECT step.)

2. `orders` has 20,302 rows and 739 have no customer row. How many rows from `JOIN customers`, and from `LEFT JOIN customers`? **Answer:** 19,563 and 20,302.

3. Total revenue came out as $10.3 million instead of $3.7 million after joining `order_items`. What happened, and what is the rule? **Answer:** Fan-out: each order's total was repeated once per line, then summed. Sum a column at the level where it lives; aggregate first (a CTE), then join.

4. Why does `WHERE employee_id NOT IN (SELECT manager_id FROM employees)` return 0 rows? **Answer:** The list contains a NULL (the CEO's `manager_id`), and a comparison with NULL is unknown, so NOT IN is never true. Use `NOT EXISTS` or filter the NULLs out of the list.

5. A value travels into SQL through a placeholder. What can a placeholder NOT stand for, and what do you do then? **Answer:** A table name, a column name or a keyword. Check the name against a fixed list of allowed names in Python, and only then put it in the query text.

EXIT TICKET

1. `orders` has 20,302 rows and 5 stores. How many rows from `SELECT store, AVG(order_total) FROM orders GROUP BY store`, and from `SELECT store, AVG(order_total) OVER (PARTITION BY store) FROM orders`? **Answer:** 5 and 20,302. GROUP BY collapses the rows; a window function keeps every row and attaches the group value to it.

2. Give the three steps of "top N per group". **Answer:** Number the rows with ROW_NUMBER() OVER (PARTITION BY group ORDER BY measure DESC, tiebreaker); put that in a CTE; filter the CTE with WHERE rn <= N. (WHERE cannot see a window function in the same SELECT.)

3. You have just created an index. Name two things you check before saying it helped. **Answer:** Any two of: the plan for the query it was meant for now says SEARCH ... USING INDEX (or COVERING INDEX) instead of SCAN; best-of-several timings before and after; the query returns exactly the same rows; what it costs in file size and write speed.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **"5 rows" for the window query (exit 1, Q1).** The learner still reads OVER as a kind of GROUP BY. Show the two results side by side and count.
- **Filtering in the same SELECT (Q3, exit 2).** The running order from Session 14 has not absorbed the window step. Add it to the board: FROM, WHERE, GROUP BY, HAVING, **windows**, SELECT, ORDER BY, LIMIT.

- `[10, 30, 35]` **for the tie item (Q5).** The answer for an explicit ROWS frame, which means the learner has not met the default frame's RANGE behavior. That is the bug they will ship in a running total.
- **"The index makes it faster" with no plan (exit 3, Q12).** Belief without evidence. Ask for the plan line and a best-of-N time every time.
- `SEARCH` **for the arithmetic item (Q10).** The learner thinks an index is about a column name, not about sorted values. Ask: "Is the index sorted by a, or by a + 0?"

INSTRUCTOR KIT · ANSWER KEY

Session 16: Documents and HTTP

Answers, reasoning and reference output for Session 16: the 12 self-check items, the predict-then-run items, four lab solutions, both lesson quizzes, the warm-up and the exit ticket.

Session 16 | 12 self-check items | 4 lab solutions | Confidential

HOW TO USE THIS KEY

- Self-check answers first (the same 12 items as the workbook, with every option explained), then the predict-then-run items, the lab reference solutions, the lesson pages' own quizzes, and the warm-up and exit ticket.
- Every number here comes from running the reference solutions and demo code on the course data. If a learner's number differs, the difference is a finding: ask for their query before you say it is wrong.
- Correct letters for the multiple-choice items: Q1 B, Q2 D, Q4 A, Q5 C, Q6 B, Q7 D, Q9 A, Q11 C, Q12 B.

5.5 · NOSQL WITH MONGODB: SELF-CHECK

Q1

In the sessions collection, `utm` is absent from direct visits and never set to null. Which query counts exactly the sessions where the field is not there, and would still be right if some documents held `utm: null`?

A) `{"utm": None}`

B) `{"utm": {"$exists": False}}`

C) `{"utm": ""}`

D) `{"utm": {"$size": 0}}`

> **Why:** `$exists` tests whether the field is present at all, which is the precise question.
> **A** is wrong: `None` matches documents where the field is null **and** documents where it is missing, so it stops being exact the day a null appears.
> **C** is wrong: That matches only documents whose `utm` is the empty string. No session has that.
> **D** is wrong: `$size` is for lists (arrays); `utm` is an object, and a missing field has no size.

Q2

What does this print?

```
c = mongomock.MongoClient()["demo"]["notes"]
c.insert_many([{"n": 1, "tags": ["a", "b"]},
               {"n": 2, "tags": ["b"]},
               {"n": 3}])
print(c.count_documents({"tags": "b"}), c.count_documents({"tags": None}))
```

A) `3 0`

B) `2 0`

C) `1 1`

D) `2 1`

> **Why:** A query on a list field matches when **any** element equals the value, so `"b"` matches documents 1 and 2. `None` matches a missing field, so `{"tags": None}` finds document 3.
> **A** is wrong: Document 3 has no `tags` at all, so it cannot contain "b".
> **B** is wrong: `None` in a query also matches documents where the field is missing: that is document 3.
> **C** is wrong: Document 1 counts too: "b" is one of its two tags.

Q3

What does this print?

```
c = mongomock.MongoClient()["demo"]["sales"]
c.insert_many([{"store": "Bend", "units": 2},
               {"store": "Boise", "units": 5},
               {"store": "Bend", "units": 4}])
rows = c.aggregate([{"$group": {"_id": "$store", "units": {"$sum": "$units"}}},
                    {"$sort": {"units": -1}}])
print([(r["_id"], r["units"]) for r in rows])
```

`[('Bend', 6), ('Boise', 5)]`

> **Why:** `$group` makes one output document per `_id` value ("$store" means: the value of the field `store`). `{"$sum": "$units"}` adds the units: Bend 2 + 4 = 6, Boise 5. `$sort` with -1 puts the larger total first.

Q4

Which pipeline stage turns every element of a list into its own document, the way `explode` does in pandas?

A) `$unwind`

B) `$group`

C) `$project`

D) `$match`

Why: `{"$unwind": "$pages"}` turned 2,400 sessions into 9,348 page-view documents, each still carrying its session's other fields.
B is wrong: `$group` goes the other way: many documents in, one per key out.
C is wrong: `$project` chooses or computes fields; the number of documents stays the same.
D is wrong: `$match` filters documents; it never splits one.

Q5

What does this print?

```
c = mongomock.MongoClient()["demo"]["visits"]
c.insert_many([
    {"id": 1, "pages": [{"path": "/cart", "seconds": 10},
                        {"path": "/sale", "seconds": 200}]},
    {"id": 2, "pages": [{"path": "/cart", "seconds": 150}]},
])
loose = {"pages.path": "/cart", "pages.seconds": {"$gte": 100}}
strict = {"pages": {"$elemMatch": {"path": "/cart", "seconds": {"$gte": 100}}}}
print(c.count_documents(loose), c.count_documents(strict))
```

A) `1 1`

B) `2 2`

C) `2 1`

D) `1 2`

Why: Without `$elemMatch` each condition may be met by a **different** element: visit 1 has a /cart page and (another) page of 200 seconds, so it matches the loose query. Only visit 2 spent 100 seconds or more on /cart itself.
A is wrong: The loose query also matches visit 1: its two conditions are satisfied by two different pages.
B is wrong: The strict query needs one page that is /cart **and** at least 100 seconds. Visit 1's /cart page lasted 10.
D is wrong: The counts are the other way round: `$elemMatch` is the stricter question, so it can never match more.

Q6

What does this print? (The collection holds the three sales documents from Q3.)

```
rows = list(c.aggregate([{"$group": {"_id": "store", "n": {"$sum": 1}}}]))
print(rows)
```

A) `[{'n': 2, '_id': 'Bend'}, {'n': 1, '_id': 'Boise'}]`

B) `[{'n': 3, '_id': 'store'}]`

C) `[]`

D) A `KeyError`: there is no field called `store`

> **Why:** Without the leading `$`, `"store"` is just a piece of text, the same for every document, so all three land in one group whose key is the word `store`.
> **A** is wrong: That is what `"_id": "$store"` returns. The dollar sign is what says "the value of this field".
> **C** is wrong: Nothing was filtered out; every document went into the single group.
> **D** is wrong: MongoDB raises no error here. A wrong pipeline gives a wrong answer quietly.

6.1 · HTTP WITH REQUESTS: SELF-CHECK

Q7

A script runs `r = requests.get(url, timeout=10)` and the server answers with status 404. What happens on that line?

A) `requests` raises `HTTPError`

B) `r` is `None`

C) The call is retried until it succeeds or 10 seconds pass

D) A normal response comes back; `r.ok` is `False` and nothing is raised

> **Why:** To `requests` a 404 is a complete conversation. You get a response whose body is an error page. It only becomes an exception when you call `r.raise_for_status()`.
> **A** is wrong: That happens only when you call `raise_for_status()`.
> **B** is wrong: `requests.get` returns a `Response` or raises; it never returns `None`. (Our own `fetch()` helper does.)
> **C** is wrong: `requests` never retries on its own, and `timeout` is not a retry budget.

Q8

What does this print?

```
star = "★".encode("utf-8")
print(len("★"), len(star), star.decode("cp1252") == "★")
```

`1 3 False`

> **Why:** One character, three bytes in UTF-8. Decoding those three bytes with the wrong table (cp1252) gives three junk characters, not the star, so the comparison is `False`. That junk is called mojibake.

Q9

You print `r.text` from a category page and every rating shows `â˜… 4.5`. What is the first thing to check?

A) `r.encoding` and the `Content-Type` header

B) Whether the page needs JavaScript

C) `r.status_code`, because this means the page is a 404

D) Your `timeout` value

> **Why:** Three junk characters in place of one symbol means UTF-8 bytes decoded with a Western European table, usually because the server did not name a charset. Set `r.encoding = "utf-8"` or parse `r.content`.
> **B** is wrong: JavaScript changes which elements exist, not how a character decodes.
> **C** is wrong: The page arrived; a 404 would give you a different page, not garbled stars.
> **D** is wrong: A timeout raises an exception. It cannot change the characters.

Q10

What does this print? (`prepare()` only builds the request; nothing is sent.)

```
params = {"status": "returned", "since": "2025-12-01"}
prepared = requests.Request("GET", "http://127.0.0.1:8000/orders",
                            params=params).prepare()
print(prepared.url)
```

`http://127.0.0.1:8000/orders?status=returned&since=2025-12-01`

> **Why:** `params` becomes the query string: a `?`, then `name=value` pairs joined by `&`, in the order of the dictionary. Nothing here needs encoding, so the values appear as written.

Q11

A site's `robots.txt` says `Disallow: /staff/` and `Crawl-delay: 1`. The staff page has no login. What should your collection script do?

A) Fetch the staff page, but only once

B) Fetch it with a browser's User-Agent so the site cannot tell

C) Skip the staff page, and wait at least one second between its other requests

D) Skip `robots.txt`: it has no legal force

> **Why:** `robots.txt` is the owner's request to programs. Nothing technical enforces it, which is exactly why honoring both lines is your job.
> **A** is wrong: Once is still going where you were asked not to go.
> **B** is wrong: Disguising your script to get around a site's wishes breaks two ground rules at once.
> **D** is wrong: Whatever its legal weight where you live, ignoring it is the fastest way to get blocked, and it is not how professionals work.

Q12

What does this print?

```
resp = requests.Response()
resp.status_code = 503
print(resp.ok)
try:
    resp.raise_for_status()
    print("fine")
except requests.exceptions.HTTPError:
    print("caught")
```

A) `True` then `fine`

B) `False` then `caught`

C) `False` then `fine`

D) `True` then `caught`

> **Why:** `ok` is `True` only below 400, and `raise_for_status()` raises `HTTPError` for every 4xx **and** 5xx status. 503 is a server-side error, and it is still an error.
> **A** is wrong: 503 is not a success code.
> **C** is wrong: `raise_for_status()` does not let 5xx through; only 2xx and 3xx pass quietly.
> **D** is wrong: `ok` and `raise_for_status()` always agree: both treat 400 and above as failure.

PREDICT, THEN RUN: ANSWERS

5.5

1.

```
1653
1653
737
```

`None` matches null and missing, so the first two lines agree: 916 sessions with no `utm` plus 737 with a `utm` that has no `campaign` = 1,653. The third line asks for the 737 alone. Common wrong answer: 916 for line 1 (forgetting the sessions that have `utm` but no campaign).

2.

```
15 0
```

A cursor can be read once. The first list uses it up; the second gets nothing. Common wrong answer: `15 15`. Fix: `docs = list(cursor)` and reuse `docs`.

3.

```
200
2400
1857
```

`$size` tests an exact length: 200 one-page sessions. `"pages.0"` is "element 0 exists", true for all 2,400. The third line counts carts whose item list has length 0: the 1,857 sessions with an empty cart (2,400 − 543 with items).

6.1

1.

```
200 True 2
404 False 0
```

Page 13 exists and holds the last 2 products. Page 14 is a 404: `ok` is False, and its body (the site's error page) contains no product cards. The dangerous part is that the second line raises nothing. Common wrong answer: "the second line is an error".

2.

```
http://127.0.0.1:8000/products?category=Tents&per_page=5&page=2
```

The pairs appear in dictionary order, joined by `&` after a `?`. Common wrong answers: a path (`/products/Tents/5/2`), or the parameters in alphabetical order.

3.

```
text/html; charset=UTF-8
JSONDecodeError: Expecting value: line 1 column 1 (char 0)
```

The content type says HTML, so `.json()` fails with `JSONDecodeError: Expecting value: line 1 column 1 (char 0)`: the first character was the `<` of a page. Common wrong answer: "it returns an empty dictionary".

LAB REFERENCE SOLUTIONS

LAB 5.5-A · SALE_PAGE_AUDIT_SOLUTION.PY

File: `labs/module-16/l5_5-nosql-mongodb/sale_page_audit_solution.py`. Output of a real run:

```
sessions in the collection: 2400
1. saw /sale: 290
2. saw /sale and ordered: 37
   saw /sale, on mobile, cart of $200 or more: 20
3. loose (any page took 60 s): 218
   strict (60 s on /sale itself): 81
4. saw /sale with no utm field: 92
5. three biggest carts:
   {'session_id': 'S00039', 'device': {'type': 'desktop'}, 'cart': {'value': 1265.85}}
   {'session_id': 'S01848', 'device': {'type': 'desktop'}, 'cart': {'value': 1202.85}}
   {'session_id': 'S00806', 'device': {'type': 'mobile'}, 'cart': {'value': 1023.9}}
6. order rate: 12.8% of /sale visitors vs 9.6% of all sessions
```

- **TODO 1** `{"pages.path": "/sale"}`: a condition on a list field is true when any element matches.
- **TODO 2** `{**saw_sale, "converted": True}` and `{**saw_sale, "device.type": "mobile", "cart.value": {"$gte": 200}}`. Pairs in one dictionary are joined by AND.
- **TODO 3** `{"pages": {"$elemMatch": {"path": "/sale", "seconds": {"$gte": 60}}}}`. 81, not 218: in 137 sessions the minute was spent on some other page.
- **TODO 4** `{**saw_sale, "utm": {"$exists": False}}`. Here `{"utm": None}` would give the same 92, because no document stores a null `utm`; `$exists` is still the query that says what you mean.
- **TODO 5** `coll.find(saw_sale, {"_id": 0, "session_id": 1, "device.type": 1, "cart.value": 1}).sort("cart.value", -1).limit(3)`. The projection keeps the nesting: `device` is still an object with one key.
- **The sentence for line 6.** 37 of 290 (12.8%) against 230 of 2,400 (9.6%). It is a difference between groups, not proof of cause: sessions with more page views are more likely both to include /sale and to end in an order. A full-credit answer names that.

Common wrong answers and what they reveal. 0 for TODO 1: the path was written `sale` or `/Sale` (values are exact and case-sensitive). 218 for the strict count: `$elemMatch` was not used. 2,400 or an error for TODO 4: `"$exists"` was written without quotes or placed outside the inner dictionary.

LAB 5.5-B · LANDING_PAGES_SOLUTION.PY

File: `labs/module-16/l5_5-nosql-mongodb/landing_pages_solution.py`. Output of a real run:

```
1. landing pages with at least 80 sessions:
          landing  sessions  orders  order_pct
                /      1301     125        9.6
         /account        96       6        6.2
            /sale        87       5        5.7
/category/apparel        85      10       11.8
    /help/returns        83       7        8.4
          /stores        80       8       10.0
2. top five SKUs by units in carts:
     sku  units  carts
SNW-1016     16      8
KIT-1027     13      9
PCK-1008     12      7
SNW-1022     12      8
ACC-1024     11      6
3. one row per cart line: (734, 4) ['sku', 'qty', 'unit_price', 'session_id']
   pandas says the top SKU is SNW-1016 with 16 units
   total units in all carts: 1106 | lines: 734 | sessions with a cart: 543
```

- **Pipeline 1** is `$addFields` (the landing page is element 0 of the list of paths) → `$group` on `"$landing"` with `{"$sum": 1}` and a `$cond` count → `$match` on the group size (HAVING) → `$sort`. The percentage is computed in pandas because mongomock has no `$round`.
- **Pipeline 2** is `$unwind` on `"$cart.items"` → `$group` on `"$cart.items.sku"` → `$sort` → `$limit`. The second sort key (`"_id": 1`) makes the order repeatable when units tie (PCK-1008 and SNW-1022 both have 12).
- **The check** flattens with `record_path=["cart", "items"]` and `meta=["session_id"]`. 543 sessions with a cart is the same 543 as the lesson's funnel.
- **Reading the result honestly.** `/category/apparel` has the highest rate, from 10 orders in 85 sessions. That is a small sample; a learner who writes "apparel landing pages convert best" without the counts has over-read it.

Common wrong answers. One group of 2,400 in pipeline 1: `"_id": "landing"` without the dollar sign. An empty result in pipeline 2: `{"$unwind": "cart.items"}` without the dollar sign, or `$limit` before `$sort`.

LAB 6.1-A · CATEGORY_CHECK_SOLUTION.PY

File: `labs/module-16/l6_1-http-requests/category_check_solution.py`. Output of a real run (identical online against the practice shop and offline from `_fixtures/`, apart from the first line):

```
1. may fetch category-footwear.html   True
1. may fetch category-snow.html       True
1. may fetch category-skis.html       True
1. may fetch staff/                   False
   crawl delay: 1 second(s)
2. report:
                  page  status                       type  bytes  cards  stars
category-footwear.html     200 text/html; charset=UTF-8  14787     28     25
    category-snow.html     200 text/html; charset=UTF-8  11987     22     22
    category-skis.html     404 text/html; charset=utf-8   2514   <NA>   <NA> http error
3. the 404 page still has a body: 2514 bytes of text/html; charset=utf-8
   product cards counted in it: 0 (a quiet zero, not an error)
4. encoding from the header: UTF-8 | Content-Type: text/html; charset=UTF-8
   11987 bytes became 11939 characters
   as cp1252: stars 0 | junk 'â˜…' 22
   as utf-8 : stars 22 | junk 'â˜…' 0
5. category-footwear.html: 28 products, 25 with a rating star, 3 not rated yet
5. category-snow.html: 22 products, 22 with a rating star, 0 not rated yet
   category-skis.html: no such page (404), so no count is reported for it.
```

- **TODO 1** `robots.can_fetch(USER_AGENT, urljoin(BASE_URL, path))` and `robots.crawl_delay(USER_AGENT)`. `robots.txt` allows `category-skis.html`: it has no opinion about pages that do not exist. Permission and existence are separate questions.
- **TODO 2** `raise_for_status()` inside `try`, the counting in `else`. The 404 row keeps `<NA>` in `cards` and `stars`: no answer, which is different from zero. (`Int64`, capital I, is the pandas integer type that can hold a missing value.)
- **TODO 4** `snow.encoding = "cp1252"` then count; `snow.encoding = "utf-8"` then count. Each star is three bytes; decoded with the wrong table it becomes the three characters `â˜…`. 22 stars, so 22 pieces of junk and 48 fewer characters than bytes in total (the page has other multi-byte characters too).
- **The last sentence.** `category-skis.html` must not get a count: the server said 404, so the body is an error page and anything counted in it is meaningless.
- **On the unzipped offline copy** the `type` column reads `text/html` (no charset), the 404 body is 335 bytes, and step 4 reports `ISO-8859-1` and `11987 bytes became 11987 characters`. The counts in steps 2 and 5 are the same because the lab sets `resp.encoding = "utf-8"` before counting.

Common wrong answers. "Skis: 0 products" (the status was not checked). `stars` of 0 for every page on the offline copy (the encoding line was left out of TODO 2). `can_fetch` given the path instead of the full URL: it still answers, and answers `True` for `staff/`, because the rule is written for `/practice/shop/staff/`.

LAB 6.1-B · API_PRICE_CHECK_SOLUTION.PY

File: `labs/module-16/l6_1-http-requests/api_price_check_solution.py`. Output of a real run against `data/mock_api.py`:

```
1. URL: http://127.0.0.1:8000/products?category=Camp+Kitchen&per_page=100
   Content-Type: application/json; charset=utf-8
   keys: ['page', 'per_page', 'total', 'total_pages', 'next', 'items']
   total: 29 | items on this page: 29 | next: None
2. products: (75, 13)
              products  median_price  max_price
category
Tents               24        298.45     616.95
Snow                22         78.45     255.95
Camp Kitchen        29         39.95     222.95
3. category Skis: status 200 | total 0 | items 0
4. product TNT-9999: status 404 | ok: False
   HTTPError caught; the API's own message: No product with SKU 'TNT-9999'.
5. Highest median price: Tents ($298.45 across 24 products).
```

- **TODO 1** `params={"category": "Camp Kitchen", "per_page": 100}`; `requests` writes the space as `+`. `next` is `None` because 29 products fit on one page of 100. (Following `next` across pages is Session 18.)
- **TODO 2** one request per category, `raise_for_status()`, `pd.DataFrame(r.json()["items"])`, then `pd.concat`. 24 + 22 + 29 = 75 rows, 13 columns (every column of `products.csv`).
- **TODO 3** status 200, total 0, no items. An unknown category is a valid question whose answer is "none". Compare with the MongoDB typo from the first half: an empty result is not an error, so check your spelling against values you know exist.
- **TODO 4** `err.response.json()["error"]["message"]`: the exception carries the response, and the response body explains the failure.

Common wrong answers. `ConnectionError` on the first line: the practice API is not running. A median for `Camp+Kitchen` that is missing: the category was typed into the URL by hand. 20 rows per category instead of 24, 22 and 29: `per_page` was left at its default.

LESSON QUIZ ANSWERS

LESSON 5.5: NOSQL WITH MONGODB

The five-question quiz at the end of `sql_nosql_mongodb.html` (three options each).

ITEM	ANSWER	REASON
Question 1	**B**	Support tickets are self-contained, nested and irregular, and are read whole: what a document holds well.
Question 2	**A**	916. In a query `None` matches null **and** missing; `$exists` tells them apart.

ITEM	ANSWER	REASON
Question 3	**C**	`$elemMatch` makes one list element satisfy both conditions (path and seconds).
Question 4	**A**	After `$group`, `$match` filters groups: the role `HAVING` plays in SQL.
Question 5	**C**	`pd.DataFrame` leaves nested objects as dictionaries in the cells; `json_normalize` flattens them.

LESSON 6.1: HTTP WITH REQUESTS

The five-question quiz at the end of `webscraping_requests.html` (three options each).

ITEM	ANSWER	REASON
Question 1	**B**	A 404 is a complete conversation: a normal response with `.ok` False; nothing is raised until `raise_for_status()`.
Question 2	**A**	`params` builds `?category=Camp+Kitchen&page=2`: the space is encoded as `+`.
Question 3	**C**	`timeout` bounds the wait to connect and the wait between bytes, not the whole download, and it retries nothing.
Question 4	**A**	Mojibake: UTF-8 bytes decoded with the wrong table. Set `r.encoding = "utf-8"` or parse `r.content`.
Question 5	**C**	`robots.txt` is the owner's request to programs; honoring it is the script's job.

WARM-UP AND EXIT TICKET ANSWERS

WARM-UP (RETRIEVAL FROM SESSION 15)

1. How does a window function differ from `GROUP BY`? **Answer:** A window function computes over a set of related rows and **keeps every row**; `GROUP BY` collapses each group to one row.
2. Three orders tie for first place. What do `ROW_NUMBER`, `RANK` and `DENSE_RANK` give the fourth row? **Answer:** `ROW_NUMBER` 4, `RANK` 4, `DENSE_RANK` 2. They differ only in how they treat ties.
3. You want the top 3 products per category. Why does `WHERE rn <= 3` fail in the same query, and what is the fix? **Answer:** `WHERE` runs before window functions are computed. Number the rows in a CTE, then filter in the outer query.

4. In `EXPLAIN QUERY PLAN`, what is the difference between `SCAN` and `SEARCH`? **Answer:** `SCAN` reads the whole table; `SEARCH` jumps to the rows it needs through a primary key or an index.

5. Name one situation in which an index does not help. **Answer:** Any one of: the column is wrapped in a function, the `LIKE` pattern starts with a wildcard, most rows match, the table is small.

EXIT TICKET

1. In a pymongo query, what does `{"utm": None}` match, and which operator asks the precise question? **Answer:** Documents where `utm` is null **and** documents where it is missing. `$exists` tells the two apart.

2. `requests.get()` comes back with status 404. What has happened in your program, and which one line makes the failure loud? **Answer:** Nothing was raised; you hold a normal response whose body is an error page. `r.raise_for_status()`.

3. Name any two of the six ground rules. **Answer:** Any two of: obey `robots.txt`; read the terms; prefer an API; go slowly and identify yourself; never go around a login or block; no personal data.

What wrong answers reveal. Question 1 answered "only null": the learner is carrying SQL's `IS NULL` over unchanged; show the first predict item. Question 2 answered "it raises an error": they have not yet separated "the conversation failed" from "the server said no"; rerun demo steps 6 and 7. Question 3 left blank: the ground rules were heard, not read; start Session 17 with them on paper.

INSTRUCTOR KIT · ANSWER KEY

Session 17: Scraping Static and Dynamic Pages

Answers, reasoning and reference output for Session 17: the 12 self-check items, predict-then-run answers, four lab solutions, both lesson quizzes, the warm-up and the exit ticket.

`Session 17` `12 self-check items` `4 lab solutions` `Confidential`

HOW TO USE THIS KEY

- The same 12 self-check items as the workbook with every option explained, then predict-then-run answers, lab reference solutions with real output, the lesson pages' quizzes, warm-up and exit ticket.
- Lab output below is from real runs of the solutions. Offline (saved pages) and online (live shop, real headless Chrome) runs gave the same numbers.
- Correct letters: Q1 C, Q3 A, Q4 D, Q5 B, Q7 B, Q9 C, Q10 D, Q11 A, Q12 D.

6.2 · PARSING HTML WITH BEAUTIFUL SOUP: SELF-CHECK

Q1

Which call gives you `None` (not an empty list, not an exception) when nothing on the page matches?

A) `soup.select("p.sale-price")`

B) `soup.find_all("p", class_="sale-price")`

C) `soup.select_one("p.sale-price")`

D) `card["data-sale-price"]` on a card without that attribute

> **Why:** `select_one()` and `find()` return one element or `None`. Calling a method on that `None` is what produces `AttributeError: 'NoneType' object has no attribute ...`.
> **A** is wrong: `select()` always returns a list; with no match it is an empty list.
> **B** is wrong: `find_all()` also returns a list, empty when nothing matches.
> **D** is wrong: Square brackets on a missing attribute raise `KeyError`. Use `.get()` to receive `None` instead.

Q2

What does this print?

```
soup = BeautifulSoup('<ul><li class="card" data-sku="A1"><b>Tent</b></li>'
                     '<li class="card"><b>Stove</b></li></ul>', "html.parser")
print([li.get("data-sku") for li in soup.select("li.card")])
```

`['A1', None]`

> **Why:** Both items match `li.card`. The first has a `data-sku`; the second has none, and `.get()` returns `None` instead of raising `KeyError`, the safe way to read an optional attribute.

Q3

Your loop follows `nav.pagination a.next` from page to page. How does it know it has reached the last page?

A) On the last page "Next" is a plain `span`, so `select_one("nav.pagination a.next")` returns `None`

B) The server answers the last page with a 404

C) The last page has no product cards

D) The loop counts to 13, the number of pages

> **Why:** The site marks the end itself: the last page has `span.next.disabled` instead of a link. The loop needs no page count, and `max_pages` stays as a seat belt.
>
> **B** is wrong: Every catalog page answers 200. Probing for a 404 sends one useless request and treats an error as normal.
>
> **C** is wrong: The last page has 2 cards. Stopping on an empty page would need one extra request past the end.
>
> **D** is wrong: A typed count is exactly what breaks when the shop adds products. Let the site say where the end is.

Q4

What does this print?

```
soup = BeautifulSoup('<p class="stock low-stock">Low stock</p>', "html.parser")
print(soup.p["class"], soup.p["class"][1].replace("-", "_"))
```

A) `['stock low-stock'] low-stock`

B) `stock low_stock`

C) `['stock', 'low-stock'] low-stock`

D) `['stock', 'low-stock'] low_stock`

> **Why:** Beautiful Soup splits `class` into a list because an element can carry several classes. Element 1 is `low-stock`, and `.replace("-", "_")` turns it into `low_stock`.
> **A** is wrong: `class` is never one string with a space in it: it is a list of class names.
> **B** is wrong: The first value printed is the whole list, not its first element.
> **C** is wrong: The replace call changed the hyphen into an underscore.

Q5

Lesson 6.2 fetches `price-list.html` with `requests` and passes `io.BytesIO(resp.content)` to `pd.read_html()`, instead of giving `read_html` the URL. Why?

A) `read_html` cannot read a URL

B) So that your User-Agent, your timeout and `raise_for_status()` apply, and pandas reads the exact bytes you checked

C) Bytes make `read_html` return numbers instead of text

D) `BytesIO` removes the second table

> **Why:** Fetching yourself keeps the polite habits of Session 16 (identify yourself, time out, check the status) and lets you save or reuse the bytes.
> **A** is wrong: It can, but then you have no say over the User-Agent, the timeout or what happens on an error.
> **C** is wrong: `read_html` reads what a person sees either way: `Price` still arrives as text such as `$299.95`.
> **D** is wrong: Both tables come back; you choose one with `attrs={"id": ...}`.

Q6

What does this print?

```
html = ('<table id="t"><tr><th>City</th><th>Price</th></tr>'
        '<tr><td>Bend</td><td>$1,299.00</td></tr></table>')
t = pd.read_html(io.StringIO(html), attrs={"id": "t"})[0]
print(t["Price"].dtype, t["Price"].str.replace("[$,]", "", regex=True).astype(float
```

`str 1299.0`

> **Why:** `read_html` reads the visible text, so `$1,299.00` arrives as a string. Removing `$` and `,` with a regular-expression replace and converting with `astype(float)` gives 1299.0.

6.3 · DYNAMIC PAGES WITH SELENIUM: SELF-CHECK

Q7

`requests` gets the Search page and finds 0 results in `#search-results`, while the browser shows 24. What should you try first?

A) Drive a headless browser with `time.sleep(5)` after `get()`

B) Find the data file the page loads (Network tab, or read its script) and, if you may, request it directly

C) Send a browser's User-Agent so the server sends the full page

D) Use `pd.read_html` on the page instead

> **Why:** The page's own script fills the list from a data file (`products.json`). One request for the file is faster, lighter for the server and sturdier than any browser automation.
> **A** is wrong: A browser is the last resort, and a fixed sleep is always too short or too long.
> **C** is wrong: The server sent the same HTML it sends everyone; the list is empty until JavaScript runs. Disguising your script also breaks the ground rules.
> **D** is wrong: There is no table in the HTML that `requests` received, and the results are not a table anyway.

Q8

What does this print?

```
print(urlencode({"q": "", "category": "Snow", "max_price": "", "in_stock": 1}))
```

`q=&category=Snow&max_price=&in_stock=1`

> **Why:** `urlencode` writes every pair, empty ones included, in dictionary order, joined by `&`. This is the exact query string the shop's search form sends when the text boxes are left empty.

Q9

Which line waits correctly for the deals status line to exist?

A) `time.sleep(3)`

B) `WebDriverWait(driver, 10).until(EC.presence_of_element_located(By.ID, "deal-status"))`

C) `WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.ID, "deal-status")))`

D) `driver.find_element(By.ID, "deal-status")`

> **Why:** An explicit wait with the locator as ONE tuple: `(By.ID, "deal-status")`. It continues the moment the element exists and raises `TimeoutException` after 10 seconds if it never does.
> **A** is wrong: A fixed sleep is too short on a slow day and wasted time on a fast one.
> **B** is wrong: The inner parentheses are missing: the condition receives two arguments instead of one locator tuple, and fails with a `TypeError`.
> **D** is wrong: That looks once, immediately, and raises `NoSuchElementException` if the script has not finished.

Q10

After clicking Submit on the GET search form, the demo waits twice: first `EC.url_contains("category=Packs")`, then for `#search-results[data-loaded="true"]`. Why not only the second wait?

A) The second condition is slower

B) `url_contains` is the only wait that raises on failure

C) The form submits twice

D) The OLD page's results are already marked loaded, so the second wait alone could pass instantly, before the new page arrives

> **Why:** Submitting loads a new page. Until it arrives, the old page still satisfies the results condition. Waiting for the address first guarantees you are looking at the new page.
> **A** is wrong: Speed is not the issue; correctness is.
> **B** is wrong: Every `wait.until` raises `TimeoutException` when its limit passes.
> **C** is wrong: One click, one new page.

Q11

You stored `status = driver.find_element(By.ID, "deal-status")`, called `driver.refresh()`, and now `status.text` raises `StaleElementReferenceException`. What happened?

A) The handle pointed at an element in the page that was replaced; find the element again after anything changes the page

B) The browser crashed and must be restarted

C) The element's text was empty

D) The wait limit was too short

> **Why:** A WebElement is a handle on one element in one loaded page. Store the data you read, not the element.
> **B** is wrong: The browser is fine; only the old handle is dead.
> **C** is wrong: An empty text is just `""`, not an exception.
> **D** is wrong: No wait is involved: the element you hold no longer exists.

Q12

What is in `driver.page_source` after the deals have loaded and Load more was clicked twice?

A) The HTML exactly as the server sent it, with an empty deal list

B) Only the 24 deal elements, without the rest of the page

C) The JSON from `deals.json`

D) The page's HTML as it is now, after its scripts ran: all 24 deals are in it

> **Why:** `page_source` is the current document, which is why Beautiful Soup finds the deals in it while the soup from `requests` found none.
> **A** is wrong: That is what `requests` receives.
> **B** is wrong: It is the whole document: head, banner, navigation and all.
> **C** is wrong: The page fetched that file, but `page_source` is HTML, not the data file.

PREDICT, THEN RUN: ANSWERS

6.2

1.

```
2 1
None
None
```

Two elements carry the class `price`; one carries `sale`. The sale paragraph has no `data-price`, so `.get()` gives `None`, and `select_one` with no match gives `None` too. Common wrong answer: `[]` for the last line (that is what `select` returns).

2.

```
https://example.com/shop/page-4.html
https://example.com/robots.txt
https://example.com/shop/products/A-1.html
```

`urljoin` resolves like a browser: a bare name replaces the last part of the path, `../` goes up one folder, and a path starting with `/` starts again at the site root. Common wrong answer: `https://example.com/shop/robots.txt` for the second line.

3.

```
'★ 4.2 (31 reviews)'
'★ 4.2 | (31 reviews)'
31
```

With no arguments `get_text()` keeps the inner spacing as it is; a separator plus `strip=True` gives tidy pieces. The regular expression `\d+` takes the first run of digits from `(31 reviews)`.

◆ 6.3

1.

```
Deals | Pinecrest Outfitters (practice site)
0 deals right after get()
```

Zero: `get()` returned before the page's script had added the deals. This is the race that explicit waits solve.

2.

```
[]
NoSuchElementException
```

`find_elements` returns an empty list; `find_element` raises `NoSuchElementException`. Common wrong answer: "both return None".

3.

```
q=rain+jacket&category=Apparel&in_stock=1
```

The space becomes `+`, the pairs keep dictionary order, and `1` becomes the text `1`.

LAB REFERENCE SOLUTIONS

◆ LAB 6.2-A · SLEEPING_BAG_SHORTLIST_SOLUTION.PY

File: `labs/module-17/l6_2-beautifulsoup/sleeping_bag_shortlist_solution.py`. Output of a real run:

```
mode: offline (saved pages in _fixtures/)
1. cards on the page: 24 | the page claims: 24 | match: True
2. listing: (24, 6) | unique SKUs: True
     sku                                    name  price  rating stock_status
SLP-1001       Pinecrest Summit 20F Sleeping Bag 307.95     4.8     in_stock
SLP-1002  Juniper Vane Foothill 30F Sleeping Bag 132.95     4.5     in_stock
SLP-1003       Tallgrass Lookout 0F Sleeping Bag 513.95     4.2    low_stock
SLP-1004 Larkspur Ridge Backcountry Sleeping Pad 187.95     4.0     in_stock
   stock: {'in_stock': 20, 'low_stock': 3, 'out_of_stock': 1}
3. sleeping bags on the page: 12 | at $260 or less and available: 6
   detail pages to fetch: 6 (not 24)
4. details as scraped:
     sku             type     weight warranty
SLP-1002 30F Sleeping Bag    0.81 kg  2 years
SLP-1007 20F Sleeping Bag    1.26 kg  2 years
SLP-1008 30F Sleeping Bag    0.95 kg  2 years
SLP-1014 30F Sleeping Bag    0.95 kg  2 years
SLP-1015  0F Sleeping Bag    1.73 kg  2 years
SLP-1019 20F Sleeping Bag Not listed Lifetime
5. the shortlist, lightest first:
     sku                                     name  price  rated_to_f  weight_kg warrant
SLP-1002   Juniper Vane Foothill 30F Sleeping Bag 132.95          30       0.81  2 year
SLP-1008       Cedar Notch Drift 30F Sleeping Bag 155.95          30       0.95  2 year
SLP-1014       Quillback Skyline 30F Sleeping Bag 121.95          30       0.95  2 year
SLP-1007 Granite Fork Switchback 20F Sleeping Bag 138.95          20       1.26  2 year
SLP-1015        Cedar Notch Ember 0F Sleeping Bag 252.95           0       1.73  2 year
SLP-1019     Pinecrest Headwater 20F Sleeping Bag 249.95          20        NaN Lifetir
6. Lightest: Juniper Vane Foothill 30F Sleeping Bag (SLP-1002), 0.81 kg, $132.95, 2 yea
   No weight listed for: SLP-1019. That is 'unknown', not 'zero' and not 'heaviest'.
   Warranties: {'2 years': 5, 'Lifetime': 1}
   saved: sleeping_bag_shortlist.csv in the output folder, 6 rows
```

- **TODO 2** `float(card.select_one("p.price")["data-price"])`; `card.select_one("p.rating").get("data-rating")` then `float(...) if rating else None`; `card.select_one("p.stock")["class"][1].replace("-", "_")`.
- **TODO 3** `listing[is_bag & (listing["price"] <= 260) & (listing["stock_status"] != "out_of_stock")]`: narrowing BEFORE fetching detail pages means 6 requests, not 24.
- **TODO 4** `{tr.th.get_text(strip=True): tr.td.get_text(strip=True) for tr in soup.select("table.specs tr")}` and `specs.get("Weight")`.
- **TODO 5** `pd.to_numeric(details["weight"].str.replace(" kg", ""), errors="coerce")` turns `Not listed` into NaN; `str.extract(r"(\d+)F")` pulls the temperature out of the type.
- **The honest sentence:** SLP-1019 has no listed weight, so it cannot be ranked; it is not the heaviest.

Common wrong answers. A `KeyError: 'data-rating'` (square brackets on an optional attribute). 24 detail requests (fetching before filtering). SLP-1019 reported as lightest or heaviest (NaN read as a number).

LAB 6.2-B · PRICE_LIST_TABLES_SOLUTION.PY

File: `labs/module-17/l6_2-beautifulsoup/price_list_tables_solution.py`. Output of a real run:

```
mode: offline (saved pages in _fixtures/)
1. tables on the page: 2 | shapes: [(242, 6), (4, 4)]
   chosen by id: (242, 6) ['SKU', 'Product', 'Category', 'Price', 'Rating', 'Availabil:
2. Price arrives as: str | first value: '$299.95'
   after cleaning: float64 | total of all list prices: 29475.9
3. availability by category:
Availability  In stock  Low stock  Out of stock
Category
Accessories         25          5             2
Apparel             26          7             2
Camp Kitchen        26          2             1
Climbing            20          3             1
Footwear            23          3             2
Packs               20          1             3
Sleeping            20          3             1
Snow                17          3             2
Tents               21          2             1
   a) most out of stock: Packs (3 products)
4. b) products with no rating: 8 | {'Footwear': 3, 'Packs': 2, 'Accessories': 2, 'Camp
   mean rating, missing left missing: 4.138 | if they had been stored as 0: 4.001
5. store hours:
     City State    Open daily  hours_open
    Boise    ID  9 am to 8 pm          11
     Bend    OR  9 am to 7 pm          10
Flagstaff    AZ 10 am to 7 pm           9
Asheville    NC 10 am to 8 pm          10
   c) open longest: Boise (11 hours a day)
6. Accessories: rows in the price list 32 | cards on the category page 32 | matched 32
   prices agree: True | 'has a rating' agrees: True
   unrated on the category page: ['ACC-1009', 'ACC-1011']
7. saved price_list_clean.csv in the output folder: (242, 6)
```

- **Tables by id**, not by position: `attrs={"id": "price-list"}` and `attrs={"id": "store-hours"}`.
- `Price` arrives as text; `.str.replace("[$,]", "", regex=True).astype(float)` gives the same total as the card scrape in the lesson (29,475.90).
- `pd.crosstab(prices["Category"], prices["Availability"])` answers (a); keeping missing ratings missing gives a mean of 4.138, while zeros would drag it to 4.001.
- Store hours: `str.extract(r"(\d+) am to (\d+) pm")`, then add 12 to the closing hour.

Common wrong answers. `tables[0]` and `tables[1]` used without the id (works today, fragile). A total of `nan` or a `TypeError` (the `$` was not removed). Store hours of -1 (forgetting that the closing time is pm).

LAB 6.3-A · BUDGET_SEARCH_SOLUTION.PY

File: `labs/module-17/l6_3-selenium/budget_search_solution.py`. Output of a real offline run (the online run with headless Chrome printed the same numbers, without the `[offline]` line):

```
mode: offline (saved pages in _fixtures/, no browser)
1. through requests: 0 results | count line: 'Loading…'
2. query string: max_price=50&in_stock=1
3. rendering the search page:
   [offline] no browser started; reading the saved finished page rendered_search_budget
4. parsed from the finished page: (60, 4) | highest price: 49.95
     sku                               name category  price
TNT-1016 Quillback Foothill Tent Footprint    Tents  41.95
TNT-1024  Cedar Notch Cairn Tent Footprint    Tents  41.95
SLP-1006 Pinecrest Backcountry Camp Pillow Sleeping  30.95
5. the page says: '73 products found' | rows parsed: 60 | MISMATCH
6. files the page's script fetches: ['deals.json', 'products.json']
   products.json: (242, 9) | $50 or less and available: 73
   the page shows the first 60 of them: True
7. budget products by category:
              on_the_page  in_the_data  not_shown
category
Accessories             2           15         13
Camp Kitchen           15           15          0
Apparel                14           14          0
Footwear               10           10          0
Climbing                9            9          0
Packs                   3            3          0
Sleeping                3            3          0
Snow                    2            2          0
Tents                   2            2          0
   13 products are in the data but not on the page, all in: ['Accessories']
   saved budget_shelf.csv in the output folder: 73 rows
```

- **TODO 1** `urlencode({"max_price": 50, "in_stock": 1})` gives `max_price=50&in_stock=1`.
- **TODO 2** `wait.until(EC.presence_of_element_located((By.CSS_SELECTOR, '#search-results[data-loaded="true"]')), message=...)`: the page's own "finished" signal.
- **TODO 3 and 4** the result cards have no `data-price`, so `float(text.lstrip("$"))`; the count is `int(re.search(r"\d+", count_text).group())`.
- **TODO 5** `shop[(shop["price"] <= 50) & (shop["stock_status"] != "out_of_stock")]`: 73 rows.
- **The finding:** the page renders only its first 60 matches. The count line (73) and the data file catch it; a scraper that trusted the rows would report 60 and miss 13 Accessories.

Common wrong answers. 0 rows (no wait, or `time.sleep` too short on a slow machine). 60 reported as the answer. 242 in step 6 (the filter in TODO 5 not applied).

LAB 6.3-B · PAGE_TRIAGE_SOLUTION.PY

File: `labs/module-17/l6_3-selenium/page_triage_solution.py`. Output of a real offline run:

```
mode: offline (saved pages in _fixtures/, no browser)
1. raw HTML of Climbing category  status 200 | items: 24
1. raw HTML of Deals              status 200 | items: 0
1. raw HTML of Search             status 200 | items: 0
2. rendering Climbing category:
   [offline] no browser started; reading the saved finished page rendered_category-clir
   items after JavaScript: 24
2. rendering Deals:
   [offline] no browser started; reading the saved finished page rendered_deals.html
   items after JavaScript: 8
2. rendering Search:
   [offline] no browser started; reading the saved finished page rendered_search_climb:
   items after JavaScript: 24
   deals status line: 'Showing 8 of 24 deals' | search count line: '24 products found'
3. files the pages' script fetches: ['deals.json', 'products.json']
4. triage:
                   in_raw_html  after_javascript                              verdict
Climbing category           24               24     static: requests + Beautiful Soup
Deals                        0                8  dynamic: request the data file first
Search                       0               24  dynamic: request the data file first
5. Climbing: category page 24 SKUs | search through the form 24 SKUs | same set: True
6. Deals: 8 on the finished page | 24 in deals.json | one request for the file beats a
   saved page_triage.csv in the output folder: 3 rows
```

- **TODO 1** the item selectors: `#deal-list li.deal` and `#search-results li`.
- **TODO 2** wait for the first deal; **TODO 3** `Select(...).select_by_visible_text("Climbing")`, submit, then two waits (`url_contains`, then the results flag).
- **TODO 4** static when raw and finished counts agree and are not zero; dynamic when the raw count is 0 and the finished one is not.
- The form search and the category page agree (24 SKUs, same set); the Deals page shows 8 of the 24 in `deals.json`.

LESSON QUIZ ANSWERS

LESSON 6.2: PARSING HTML WITH BEAUTIFUL SOUP

The five-question quiz at the end of `webscraping_beautifulsoup.html` (three options each).

ITEM	ANSWER	REASON
Question 1	**C**	A class selector matches any element whose class list includes it: 2 and 1; `select_one` returns `None` for no match.
Question 2	**A**	That card has no `span.review-count`, so `select_one()` returned `None`.
Question 3	**B**	`li.product-card p.price` names block and field by meaning, not position.
Question 4	**B**	Follow "next" until there is none, with a page limit as a seat belt.
Question 5	**A**	From bytes, the parser reads the page's own charset declaration.

LESSON 6.3: DYNAMIC PAGES WITH SELENIUM

The five-question quiz at the end of `webscraping_selenium.html` (three options each).

ITEM	ANSWER	REASON
Question 1	**B**	The deals are added by JavaScript after load; `requests` does not run JavaScript.
Question 2	**A**	An explicit wait continues as soon as the content is ready and raises clearly if it never is.
Question 3	**C**	`find_elements` returns an empty list; `find_element` raises `NoSuchElementException`.
Question 4	**B**	Request the JSON the page itself uses.
Question 5	**C**	The browser keeps running until something ends it: `quit()` belongs in `finally`.

WARM-UP AND EXIT TICKET ANSWERS

WARM-UP (RETRIEVAL FROM SESSION 16)

1. In a pymongo query, what does `{"utm": None}` match? **Answer:** Documents where `utm` is null **and** documents where it is missing. `$exists` tells the two apart.
2. Which aggregation stage does what pandas `explode` does? **Answer:** `$unwind`: one document per element of a list.

3. `requests.get()` comes back with status 404. Has an exception been raised? **Answer:** No. You hold a normal response with `.ok` False; `r.raise_for_status()` makes it loud.

4. Why does every request in this course carry `timeout=`? **Answer:** Without it `requests` waits forever for a server that has gone quiet.

5. You see `â˜…` where a star should be. What happened, and what is one fix? **Answer:** UTF-8 bytes were decoded with the wrong table (no charset sent). Set `r.encoding = "utf-8"`, or hand `r.content` to a parser that reads the page's `<meta charset>`.

EXIT TICKET

1. Which Beautiful Soup method returns `None` when nothing matches, and which returns an empty list? **Answer:** `select_one()` (and `find()`) return `None`; `select()` (and `find_all()`) return a list, empty if nothing matches.

2. Why is `WebDriverWait(...).until(...)` better than `time.sleep(5)`? **Answer:** It continues the moment the condition is true and raises a clear `TimeoutException` if it never is. A sleep is always too short or too long.

3. Before you start a browser for a page that JavaScript fills in, what do you look for first? **Answer:** The data file or API the page loads (Network tab, or the page's script). If you may request it, do that instead.

What wrong answers reveal. "`select` returns None": mixing the two families; redo predict item 1. "Sleep is fine if it is long enough": the cost has not landed; ask what a thousand pages times five seconds is. "Use Selenium" for question 3: the order of the routes needs another pass at the start of Session 18.

INSTRUCTOR KIT · ANSWER KEY

Session 18: APIs: Keys, Tokens and Polite Clients

Answers, reasoning and reference output for Session 18: the 12 self-check items, predict-then-run answers, four lab solutions, the Checkpoint 2 rubric and model answer, both lesson quizzes, the warm-up and the exit ticket.

`Session 18` `12 self-check items` `Checkpoint 2` `Confidential`

HOW TO USE THIS KEY

- The 12 self-check items with every option explained, predict-then-run answers, the four lab solutions and the Checkpoint 2 model answer and rubric with real output, both lesson quizzes, the warm-up and the exit ticket.
- Lab outputs are real offline runs (saved responses, pretend clock). Online runs against `data/mock_api.py` with the lesson flags printed the same numbers; only tokens and the mode line differ.
- Correct letters: Q1 A, Q3 C, Q4 B, Q5 D, Q6 B, Q7 C, Q10 A, Q11 D, Q12 B.

6.4 · API AUTHENTICATION: SELF-CHECK

Q1

The orders endpoint wants a key in the `X-API-Key` header. What do you get with no key at all, and with a misspelled key?

A) 401 for no key, 403 for the misspelled one

B) 403 for both

C) 401 for both

D) 404 for no key, 401 for the misspelled one

> **Why:** 401 means "I cannot tell who you are" (no credential). 403 means "I understood, and the answer is no" (a credential that is rejected). The practice API answers `missing_api_key` and `invalid_api_key`.
> **B** is wrong: With no key there is nothing to refuse: the server cannot tell who you are, which is 401.
> **C** is wrong: A wrong key was understood and rejected: 403. Sending it again will never help.
> **D** is wrong: 404 is for an address that does not exist; `/orders` exists.

Q2

What does this print? (`ApiError` is the class from the demo.)

```
err = ApiError(401, "token_expired", "Log in again.")
print(err)
print(err.code == "token_expired", isinstance(err, Exception))
```

`401 token_expired: Log in again.` then `True True`

> **Why:** `super().__init__(f"{status} {code}: {message}")` sets the text that `print(err)` shows. The code is stored as an attribute, so a program can branch on `err.code`, and `ApiError` inherits from `Exception`.

Q3

What does this print?

```
pages = {"/o?page=1": {"items": ["a", "b"], "next": "/o?page=2"},
         "/o?page=2": {"items": ["c"], "next": "/o?page=3"},
         "/o?page=3": {"items": [], "next": None}}
items, url, calls = [], "/o?page=1", 0
while url:
    page = pages[url]
    calls += 1
    items.extend(page["items"])
    url = page["next"]
print(items, calls)
```

A) `['a', 'b'] 1`

B) `['a', 'b', 'c'] 2`

C) `['a', 'b', 'c'] 3`

D) `['a', 'b', 'c', None] 3`

> **Why:** The loop reads a page, then follows `next` until it is `None`. Page 3 exists and is empty, so three pages are fetched and the items are a, b, c.
>
> **A** is wrong: The loop does not stop after the first page: page 1 has a `next`.
>
> **B** is wrong: Page 2 also points onward, so a third request is made (it returns no items).
>
> **D** is wrong: `next` is not an item; `extend` adds only the items list.

Q4

In `get_all()` the first request sends the filters in `params`, then the loop sets `params = None`. Why?

A) To save memory

B) The `next` address already carries the filters; sending them again would attach every filter twice

C) The server rejects a second page that has parameters

D) To force the server to start again at page 1

> **Why:** `next` looks like `/orders?page=2&per_page=10&since=2025-12-15&status=returned`: the filters are already in it.
> **A** is wrong: A small dictionary costs nothing.
> **C** is wrong: The server accepts parameters on every page; the problem is duplicates you did not mean to send.
> **D** is wrong: Page numbers come from `next`; dropping `params` does not reset anything.

Q5

Which header sends a bearer token correctly?

A) `X-API-Key: <token>`

B) `Authorization: <token>`

C) The token as `?token=` in the URL

D) `Authorization: Bearer <token>`

> **Why:** The value is the scheme, a space, then the token. Without the word Bearer the practice API reports `missing_token`.
> **A** is wrong: That header is for the API key, a different credential.
> **B** is wrong: This is mistake 1 in the lesson: the server does not recognize a token at all.
> **C** is wrong: URLs are written to logs and histories: never put a credential there.

Q6

You will share this notebook. Which line may stay in it?

A) `print(API_KEY)`

B) `print("key loaded:", API_KEY[:4] + "...")`

C) `API_KEY = "pinecrest-demo-key"`

D) `url = f"{API}/orders?api_key={API_KEY}"`

> **Why:** A hint confirms the secret loaded without revealing it. The key itself comes from `os.environ` at run time.
> **A** is wrong: Output cells are saved with the notebook: the whole key would be shared.
> **C** is wrong: A secret typed into code travels with every copy of the file.
> **D** is wrong: The key would end up in logs, histories and error messages.

6.5 · RATE LIMITING AND RETRIES: SELF-CHECK

Q7

A response carries `X-RateLimit-Remaining: 0` and `X-RateLimit-Reset: 3`. What should your next request do?

A) Go at once: the request itself is valid

B) Switch to a second API key

C) Wait about 3 seconds, until the window starts over

D) Stop the job: the limit is used up for the day

> **Why:** You have used this window's allowance, and the window resets in 3 seconds. Waiting is the polite move and the fastest route to a 200.
> **A** is wrong: It will be refused with 429 and count as wasted traffic.
> **B** is wrong: Getting around a limit with extra keys breaks the ground rules.
> **D** is wrong: Reset is in seconds for this API, not a daily quota.

Q8

What does this print? (`CaseInsensitiveDict` is the type of `resp.headers`.)

```
print(int(requests.utils.CaseInsensitiveDict({"retry-after": "3"}).get("Retry-After
```

`3`

Why: Header names ignore case, so `Retry-After` finds `retry-after`. The default `1` is not needed here, but it protects you when a 429 arrives without the header.

Q9

What does this print?

```
print([min(2.0, 0.2 * 2 ** n) for n in range(5)])
```

`[0.2, 0.4, 0.8, 1.6, 2.0]`

Why: Each wait doubles (exponential backoff) and `min` caps it at 2.0 seconds.

Q10

The demo's `Retry` policy has `allowed_methods=["GET", "HEAD"]`. Why not retry POST too?

A) Repeating a POST can repeat its effect, such as placing an order twice

B) urllib3 cannot retry POST

C) POST requests never fail with 503

D) A POST is always faster

Why: GET and HEAD only read, so sending them twice is harmless. A POST changes something on the server.
B is wrong: It can when told to; the policy deliberately does not.
C is wrong: Any request can meet a busy server.
D is wrong: Speed has nothing to do with whether a repeat is safe.

Q11

A cache function writes `resp.text` to disk and only then calls `check(resp)`. One request gets a 429. What happens on every later run?

A) Nothing: the cache refreshes itself

B) The next run raises `FileNotFoundError`

C) The 429 is retried automatically

D) The saved error body is served from the cache, as if it were data

> **Why:** A cache that stores a failure serves it faithfully forever. Check first, write second.
> **A** is wrong: This cache has no expiry: a file that exists is always used.
> **B** is wrong: The file exists; that is the problem.
> **C** is wrong: A cache hit never touches the network, so nothing is retried.

Q12

A job is slow because the API allows 10 requests per 5 seconds per client. A colleague suggests running three copies of the script at once. What happens?

A) It finishes three times faster

B) The limit applies to you, so the copies hit 429 three times as often and the job is no faster

C) Each copy gets its own 10 per 5 seconds

D) The server queues the extra requests politely

> **Why:** The limit counts the client (the key, or the address). Parallel copies share it; on a public service this looks like abuse.
> **A** is wrong: The allowance does not grow with the number of copies.
> **C** is wrong: The copies send the same key from the same machine: one client.
> **D** is wrong: Over the limit, the answer is 429, not a queue.

PREDICT, THEN RUN: ANSWERS

6.4

1.

```
401 403 200
```

No key 401, wrong key 403, right key 200. Common wrong answer: 401 for the wrong key.

2.

```
397 4 /orders?page=2&per_page=100&status=cancelled
```

397 cancelled orders, 4 pages of up to 100, and `next` points at page 2 with the filters already in it.

6.5

1.

```
400 False | 401 False | 404 False | 429 True | 500 True | 503 True |
```

Only temporary failures are retried: 429, 500, 503. 400, 401 and 404 describe the request. (An expired token is the one 401 worth handling, by renewing once, which is a different mechanism.)

2.

```
True False
```

`sort_keys=True` makes the two orders the same request, so the same file; a different spelling of a value is a different request.

LAB REFERENCE SOLUTIONS

LAB 6.4-A · CANCELLED_ORDERS_SOLUTION.PY

File: `labs/module-18/l6_4-api-authentication/cancelled_orders_solution.py`. Output of a real run:

```
1. key loaded: pine... (18 characters)
2. no key: 401 missing_api_key
2. a key with a typo: 403 invalid_api_key
3. 70 cancelled orders since 2025-10-01; requests needed at 25 per page: 3
4. pages read: 3 | shape: (70, 11)
   matches the promised total: True | unique ids: True | first date: 2025-10-01
5. cancelled orders by store:
           orders    value
store
Online         62  9788.71
Flagstaff       4   687.96
Boise           3    91.95
Asheville       1    35.95
   total value $10,604.57; 88.6% of the cancellations were online orders
6. 400 bad_request -> 'since' must look like 2025-01-31, got '10/01/2025'.
7. orders.csv has 70 such orders | same ids: True | same total: True
   for comparison, online orders are 65.5% of ALL orders since 2025-10-01
   Stores almost never cancel: a cancellation is something that happens to an online or
```

- **TODO 1** `API_KEY = require_env("PINECREST_API_KEY")`, a hint `API_KEY[:4] + "..."`, and `{"X-API-Key": API_KEY}`.
- **TODO 3** `probe["total"]` and `-(-total // 25)`: 70 orders, 3 requests.
- **TODO 4** the `while url:` loop with `params = None` after the first page; 70 rows, unique ids, first date 2025-10-01.
- **TODO 5** `groupby("store").agg(orders=("order_id", "count"), value=("order_total", "sum"))`; online share 88.6%.
- **TODO 6** the 400 explains itself: `'since' must look like 2025-01-31, got '10/01/2025'.`

Common wrong answers. 25 rows (one page). 140 rows or duplicates (filters sent again with `next`). "Online customers cancel more" without the base rate (65.5% of all orders are online).

LAB 6.4-B · TOKEN_LIFECYCLE_SOLUTION.PY

File: `labs/module-18/l6_4-api-authentication/token_lifecycle_solution.py`. Output of a real run (your token differs):

```
1. Bearer token wJl73K... valid for 5 seconds (your token will differ)
2. answered for: Demo Analyst | analyst
3. token without the word Bearer: 401 missing_token
3. a token nobody issued: 401 invalid_token
3. wrong password: 401 invalid_credentials
4. waiting 6 seconds for the token to expire ...
   the same token, later: 401 token_expired
5. answered for: analyst | logins so far: 2
6. the journal:
                     request  status                code                          next
                      log in     200                  ok                      use the
                 fresh token     200                  ok                      use the
token without the word Bearer     401       missing_token              stop: fix the
        a token nobody issued     401       invalid_token  stop: this token was never
               wrong password     401 invalid_credentials stop: fix the password, never
        the same token, later     401       token_expired          log in again, retry
                renewed token     200                  ok                      use the
   Only token_expired earns a second try, and exactly one.
```

TODO 2 is the header `Authorization: Bearer <token>`; TODO 5 renews only on `token_expired`, once. Note for the room: the practice API forgets an expired token the first time it reports it, so a second request with the same token gets `invalid_token`. That is why the lab lets the request in Step 5 be the one that discovers the expiry.

LAB 6.5-A · STOCK_CHECK_SOLUTION.PY

File: `labs/module-18/l6_5-rate-limiting/stock_check_solution.py`. Output of a real run:

```
this server allows: 10 requests per 5 seconds per client
1. 200 {'X-RateLimit-Limit': '10', 'X-RateLimit-Remaining': '9', 'X-RateLimit-Reset':
   a polite pace: one request every 0.55 s, so 13 requests take about 7 s
2. burst:  [200, 200, 200, 200, 200, 200, 200, 200, 200, 404, 429, 429, 429]
   answered 10 of 13, refused 3 | Retry-After: 5 s
   the server said: Too many requests. Try again in 5 second(s).
3. plan B: [200, 200, 200, 200, 200, 200, 200, 200, 200, 404, 200, 404, 200]
   refused in the end: 0 | waits: 1 | took about 5 s
4. plan A: [200, 200, 200, 200, 200, 200, 200, 200, 200, 404, 200, 404, 200]
   refused: 0 | took about 7 s
5. 9 of 9 categories answered
    category  products  out_of_stock  low_stock
       Packs        24             3          1
     Apparel        35             2          7
 Accessories        32             2          5
    Footwear        28             2          3
        Snow        22             2          3
    Sleeping        24             1          3
    Climbing        24             1          3
       Tents        24             1          2
Camp Kitchen        29             1          2
   SNW-1007: 404 No product with SKU 'SNW-1007'.
   TNT-1020: 200 in_stock
   PCK-1015: 404 No product with SKU 'PCK-1015'.
   KIT-1027: 200 in_stock
   The two 404s were each asked once in steps 3 and 4: a product that does not exist
   will not exist in five seconds either. Only the 429 earned a second try.
```

- **TODO 1** the three `X-RateLimit` headers; `PER / LIMIT * 1.1` = 0.55 s.
- **TODO 2** `float(last.headers.get("Retry-After", 1))`.
- **TODO 3** a loop over at most `max_tries`: on 429 wait as asked and try again; return anything else at once.
- **TODO 4** `throttle.wait()` before each request.
- **TODO 5** only the 429s earned a second try; the 404s were asked once.

Common wrong answers. A `TypeError` from `int(None)` (no default). A plan B that also retries 404s (the two missing SKUs are asked five times each). Plan A still refused: the interval computed as `LIMIT / PER`.

LAB 6.5-B · RETRY_AND_CACHE_SOLUTION.PY

File: `labs/module-18/l6_5-rate-limiting/retry_and_cache_solution.py`. Output of a real offline run (online, step 4 prints `200 after retrying [503, 503]` instead of the skip note):

```
this server allows: 10 requests per 5 seconds per client
1. schedule:    [0.25, 0.5, 1.0, 2.0, 3.0, 3.0]
   with jitter: [0.08, 0.08, 0.65, 0.14, 1.61, 1.1]
2. /flaky: [503, 503, 200] | Retry-After on the first: 1
3. /flaky: 200 {'status': 'ok', 'attempts_needed': 3} | tries: 3
   /products/SNW-1007: 404 not_found | tries: 1
   with max_tries=2: gave up after 2 tries (last status: 503)
4. would the policy retry?
   GET  503: True
   GET  429: True
   GET  404: False
   GET  401: False
   POST 503: False
   offline: a mounted Retry policy needs the real practice API, so this step is skipped
   Online it prints: 200 after retrying [503, 503]
5. asked 6 times: ['network', 'network', 'network', 'cache', 'cache', 'cache']
   3 requests sent, 3 answers from the cache, 3 files in the cache
   a 404 is raised, not saved: 404 not_found | files in the cache: 3
6. median list price:
          products  median_price
category
Sleeping        24        141.45
Packs           24        118.95
Climbing        24         82.45
   Six questions, three requests. Run the lab again without the rmtree line and it need
```

The schedule is `min(cap, base * 2 ** attempt)` with base 0.25 and cap 3; jitter draws `random.uniform(0, ceiling)`. The policy's `is_retry` shows the design: GET 503 and 429 yes; 404, 401 and any POST no. The cache writes only after `check()`, so the 404 is raised and not saved.

CHECKPOINT 2: RUBRIC AND MODEL ANSWER

RUBRIC (5 ROWS; SCORE EACH ROW 2, 1 OR 0)

CRITERION	FULL (2)	PARTIAL (1)	NOT YET (0)
1. Database query	Read-only connection (`?mode=ro`), ONE query with `?` parameters, correct filter (`status = 'returned'`, `order_date >= '2025-11-01'`): 118 orders, $25,191.92	Right numbers but values glued into the SQL text, or not read-only	No query, or a wrong filter (for example `>` instead of `>=`, or all statuses)

CRITERION	FULL (2)	PARTIAL (1)	NOT YET (0)
2. Credentials	Key from the environment (`os.environ` / `require_env`), sent as `X-API-Key`; only a hint printed	Works, but the key is printed in full or typed into the file	No key sent, or the key in the URL
3. Pagination and status checks	Follows `next` to the end (3 pages at 50), filters sent with the first request only, every response checked: 118 rows, unique ids	Reads all pages but sends `params` on every page, or skips the status check	Reads only the first page, or a 401 body is treated as data
4. Comparison	Compares the id sets AND the totals; reports 0 ids only in one source	Compares counts or totals only	No comparison
5. The answer	One clear sentence with both numbers that Finance could act on	A sentence without numbers, or numbers without a conclusion	No sentence

Reading the score: 9 to 10 ready for the capstone's data step; 6 to 8 one lesson to revisit (name it); 5 or less, book a catch-up on Lessons 5.1 and 6.4 before Session 20.

MODEL ANSWER

File: `labs/module-18/checkpoint-2/checkpoint2_solution.py`. The three parts:

```
conn = sqlite3.connect((DATA / "pinecrest.db").as_uri() + "?mode=ro", uri=True)
from_db = pd.read_sql(
    """
    SELECT order_id, order_date, order_total
    FROM orders
    WHERE status = ? AND order_date >= ?
    ORDER BY order_id
    """,
    conn, params=(STATUS, SINCE))
conn.close()

api_key = os.environ.get("PINECREST_API_KEY")
items, pages = [], 0
url, params = f"{API}/orders", {"status": STATUS, "since": SINCE, "per_page": 50}
while url:
    r = http.get(url, headers={"X-API-Key": api_key}, params=params, timeout=10)
    r.raise_for_status()
    data = r.json()
    items.extend(data["items"])
    pages += 1
    url = f"{API}{data['next']}" if data["next"] else None
    params = None
from_api = pd.DataFrame(items)
```

Output of a real run (offline; the online run against the practice API printed the same lines):

```
A. database: 118 returned orders since 2025-11-01, total $25,191.92
B. key loaded: pine...
   API: 3 pages, 118 rows (the API promised 118), total $25,191.92
   unique ids: True | first date: 2025-11-02
C. same order ids: True | same total value: True
   ids only in the API: 0 | ids only in the database: 0
ANSWER: Yes. Both sources hold the same 118 returned orders placed since 2025-11-01, wo
```

A typical partial answer: Part A correct; Part B reads only the first page (50 rows, $9,877.33) and concludes "No, they disagree". That is rubric row 3 at 0 and row 5 at 1: the sentence was honest about the numbers it had, but the pull was incomplete. Point at `next` in the first response.

LESSON QUIZ ANSWERS

LESSON 6.4: API AUTHENTICATION

The five-question quiz at the end of `webscraping_api_authentication.html` (three options each).

ITEM	ANSWER	REASON
Question 1	**C**	403 means understood and refused: stop and fix the credential.
Question 2	**B**	An environment variable (or a prompt), read at run time.
Question 3	**A**	Both pages are read; `i` ends at 1, the index of the last page.
Question 4	**B**	Expiry is normal: log in again and repeat the request once.
Question 5	**A**	A leaked token stops working soon and can be revoked without changing the password.

LESSON 6.5: RATE LIMITING AND RETRIES

The five-question quiz at the end of `webscraping_rate_limiting.html` (three options each).

ITEM	ANSWER	REASON
Question 1	**A**	Honor `Retry-After`: wait at least 4 seconds, then retry.
Question 2	**C**	Each wait doubles and `min` caps it at 8.0.
Question 3	**B**	Jitter spreads retries so clients that failed together do not retry together.
Question 4	**C**	A 404 describes the request: retrying cannot change it.
Question 5	**A**	Each page is saved to its own file as it arrives, and existing files are skipped.

WARM-UP AND EXIT TICKET ANSWERS

WARM-UP (RETRIEVAL FROM SESSION 17)

1. Which Beautiful Soup method returns `None` when nothing matches? **Answer:** `select_one()` (and `find()`). `select()` and `find_all()` return a list, empty when nothing matches.
2. Why does `get_soup()` pass `resp.content`, not `resp.text`? **Answer:** From bytes, Beautiful Soup reads the page's own `<meta charset>`, so the text decodes correctly whatever the server said.
3. How does a pagination loop know it has reached the last page of the catalog? **Answer:** There is no `a.next` link on the last page, so `select_one("nav.pagination a.next")` returns `None`.
4. `requests` sees an empty list where the browser shows deals. What do you try before Selenium? **Answer:** Find the data file the page loads (Network tab or the script) and request it directly, if allowed.
5. Why `WebDriverWait(...).until(...)` instead of `time.sleep()`, and where does `driver.quit()` go? **Answer:** The wait continues as soon as the condition is true and raises clearly if it never is. `quit()` goes

in a `finally` block.

EXIT TICKET

1. What is the difference between a 401 and a 403, and what does your program do with each? **Answer:** 401: no valid credential (missing, malformed, expired): fix how you send it, or renew an expired token and retry once. 403: understood and refused: stop; a person must fix the key or permission.
2. The server answers 429 with `Retry-After: 5`. What does your client do? **Answer:** Wait at least 5 seconds, then send the request again (with a maximum number of tries). Better still: pace requests so this rarely happens.
3. Name one failure that must NOT be retried, and say why. **Answer:** Any of 400, 401 (except an expired token, once), 403, 404: they describe the request, so the same request gets the same answer.

What wrong answers reveal. 401 and 403 swapped: redo predict item 1 at the start of Session 19's warm-up. "Retry immediately" for a 429: the cost to the server has not landed. "Retry everything": the difference between the request and the server's mood needs one more example.

INSTRUCTOR KIT · ANSWER KEY

Session 19: Seaborn and Plotly

Every workbook item for Session 19 with its reasoning, the four lab reference solutions explained, the lesson pages' own quiz answers, and the warm-up and exit-ticket answers.

Session 19 | 12 self-check items | Lessons 7.1–7.2 | Confidential

7.1 · STATISTICAL GRAPHICS WITH SEABORN: SELF-CHECK

Q1

Which of these is a figure-level seaborn function?

A) `sns.histplot`

B) `sns.boxplot`

C) `sns.displot`

D) `sns.heatmap`

> **Why:** `displot` creates its own figure, can facet with `col=` and `row=`, and returns a FacetGrid. The other two figure-level functions are `relplot` and `catplot`.
> **A** is wrong: `histplot` is axes-level: it takes `ax=` and returns an Axes. It is what `displot` calls for you.
> **B** is wrong: `boxplot` is axes-level; its figure-level front door is `catplot(kind="box")`.
> **D** is wrong: `heatmap` draws on one Axes and accepts `ax=`.

Q2

What does this print?

```
ax = sns.histplot(data=sales, x="revenue")
print(type(ax).__name__)
```

A) `Axes`

B) `FacetGrid`

C) `Figure`

D) `Histogram`

> **Why:** An axes-level function returns the Matplotlib Axes it drew on, so every Module 3 method (`set_title`, `set_xlabel`, ...) works on the result.
> **B** is wrong: A FacetGrid comes back from the figure-level functions (`displot`, `relplot`, `catplot`).
> **C** is wrong: The Figure is one level up: `ax.figure`.
> **D** is wrong: There is no such class; seaborn draws with ordinary Matplotlib artists.

Q3

What does this print?

```
g = sns.relplot(data=sales, x="visitors", y="revenue",
                col="store", col_wrap=2, height=2)
print(g.axes.shape)
```

`(4,)`

> **Why:** Four stores make four panels. With `col_wrap` the FacetGrid keeps its Axes in a flat array, so the shape is `(4,)`, not `(2, 2)`. Loop over it with `g.axes.flat`.

Q4

`sales` has about 60 rows for each store and month. What does `sns.lineplot(data=sales, x="month", y="revenue", hue="store", errorbar=None)` draw?

A) A zigzag through every daily row, in file order

B) One line per store through the monthly means, with no band

C) One line per store through the monthly totals

D) Nothing: it raises an error because `x` has repeated values

> **Why:** `lineplot` aggregates rows that share an x value. The default estimator is the mean, and `errorbar=None` only removes the shaded band.
> **A** is wrong: That is what `estimator=None` would do.
> **C** is wrong: The default estimator is the mean. For totals, pass `estimator="sum"`, or better, sum with `groupby` first and plot that table.
> **D** is wrong: Repeated x values are exactly the case lineplot is built for.

Q5

A chart shows the mean staff rating (1 to 5) for four stores, about 200 answers each. Every bar runs from roughly 3.2 to 5.0. What do the bars most likely show?

A) A 95% confidence interval for each mean

B) One standard error either side of each mean

C) The lowest and highest rating given

D) One standard deviation either side of each mean

> **Why:** Individual ratings vary by about one point, so mean plus or minus one standard deviation spans almost two points. That is spread of the data.
> **A** is wrong: With 200 answers the interval for a mean is only about 0.1 to 0.15 either side: far shorter.
> **B** is wrong: A standard error here is about 0.06: the bar would be barely visible.
> **C** is wrong: Ratings run from 1 to 5, so a min-to-max bar would start at 1.

Q6

What does this print? (Two lists.)

```
t = pd.DataFrame({"size": ["M", "S", "L", "S"], "n": [3, 1, 2, 5]})
fig, ax = plt.subplots()
sns.barplot(data=t, x="size", y="n", errorbar=None, ax=ax)
print([lab.get_text() for lab in ax.get_xticklabels()],
      [float(b.get_height()) for b in ax.patches])
```

`['M', 'S', 'L'] [3.0, 3.0, 2.0]`

> **Why:** Two lessons in one line. Categories are drawn in the order they first appear (M, S, L), not sorted: pass `order=` when order matters. And `barplot` aggregates: the two S rows (1 and 5) become one bar at their mean, 3.0.

7.1 · PREDICT, THEN RUN

P1

Prints:

```
g = sns.catplot(data=sales, x="store", y="visitors", col="year", kind="bar", height
print(type(g).__name__, g.axes.shape)
```

> `FacetGrid (1, 2)`

> **Why:** Two years make two panels in one row. A figure-level function returns a FacetGrid, and without `col_wrap` its Axes sit in a 2-D array: one row, two columns.

P2

Prints:

```
print(sales.groupby("store")["revenue"].agg(["std", "sem"]).round(0))
```

```
               std    sem
store
Asheville   2064.0   77.0
Bend        2546.0   95.0
Boise       3715.0  138.0
Flagstaff   2686.0  100.0
```

Why: The standard deviation (spread of single days) is about 27 times the standard error (uncertainty of the mean), because each store has about 725 days and the square root of 725 is about 27. That ratio is why `errorbar="sd"` bars are so much longer than `errorbar="se"` bars.

7.1 · THE LESSON PAGE'S OWN QUIZ

1. **B.** scatterplot is axes-level: it accepts ax= and draws on the Axes you give it. relplot would ignore ax with a warning.
2. **C.** lineplot aggregates rows that share an x: the line is the mean and the band a bootstrapped 95% confidence interval for it.
3. **A.** A figure-level function returns a FacetGrid; row="year" gives 2 rows and col="store" 4 columns: (2, 4).
4. **B.** sd bars show day-to-day spread; ci bars show uncertainty in the mean, which is tiny with 700 days.
5. **A.** Text categories are drawn in first-seen order; order= (or an ordered categorical) sets the order you mean.

7.2 · INTERACTIVE CHARTS WITH PLOTLY: SELF-CHECK

Q7

You run `fig = px.line(monthly, x="month_start", y="revenue", color="store")` in a script and nothing appears. Why?

A) The call only builds a Figure object; it is displayed when you call `fig.show()` (or write it to a file)

B) `px.line` needs `kind="line"` before it draws

C) Plotly needs an account before it can display a chart

D) `monthly` has more than one row per month, so Plotly refuses to draw

Why: Python builds a description (traces plus layout). Drawing happens in a browser, when you show the figure or open the HTML file you wrote.
B is wrong: `kind=` belongs to seaborn's figure-level functions.
C is wrong: The plotly library is open source and runs entirely on your machine.
D is wrong: Four stores with 24 months each is what `color="store"` handles: four traces.

Q8

`sales` has four stores and `promo_day` is 0 or 1. What does this print?

```
fig = px.scatter(sales, x="visitors", y="revenue",
                 color="store", facet_col="promo_day")
print(len(fig.data))
```

`8`

Why: Express makes one trace per combination of color and facet: 4 stores times 2 panels. Knowing how many traces to expect is the quickest check that a figure is what you meant.

Q9

Which `hover_data` entry shows revenue as `$27,677` in the hover label?

A) `"revenue": True`

B) `"revenue": "$%d"`

C) `"revenue": ":$,.0f"`

D) `"revenue": ":.2%"`

> **Why:** A string that starts with a colon is a d3 number format: `$` adds the dollar sign, `,` the thousands separator, `.0f` no decimals.
> **A** is wrong: `True` adds the column with its raw value: `27677.1`.
> **B** is wrong: That is Python's old percent formatting; Plotly uses d3-format after a colon.
> **D** is wrong: `.2%` multiplies by 100 and adds a percent sign: right for a rate, wrong for dollars.

Q10

What does `fig.update_layout(hovermode="x unified")` change on a line chart with four stores?

A) Zooming one panel now zooms every panel

B) One hover box appears for the x position under the mouse, listing all four stores

C) The four lines are merged into one trace

D) The x axes of all facets are forced to the same range

> **Why:** Unified hover reads a time series the way people compare it: one date, every series. It is a layout setting, so it applies to the whole figure.
> **A** is wrong: Shared zoom comes from facets sharing axes, not from the hover mode.
> **C** is wrong: The traces are untouched: the legend still switches each store on and off.
> **D** is wrong: Axis ranges are set with `update_xaxes`; the hover mode only changes the label.

Q11

What does this print?

```
totals = sales.groupby("store", as_index=False)["revenue"].sum()
fig = px.bar(totals, x="store", y="revenue")
print(fig.data[0].type, len(fig.data[0].x))
```

`bar 4`

> **Why:** Four rows in, four bars out: `px.bar` draws one bar segment per row and never aggregates. Hand it `sales` itself and the same print says `bar 2903`.

Q12

You will email one chart to a manager who is online. Your table has one row per customer, including an email column the chart does not use. What is the right way to prepare the file?

A) `fig.write_html("chart.html")` from the full table: unused columns are left out automatically

B) `fig.write_image("chart.html")`, so that no data is included

C) Put `email` in `hover_data` so the manager can check individual customers

D) Aggregate to what you mean to publish, pass only the columns the chart needs, then `fig.write_html("chart.html", include_plotlyjs="cdn")`

> **Why:** The HTML file contains the data you passed to the figure as readable text. Decide what the reader may see before you build the figure; `"cdn"` keeps the file to a few kilobytes because the reader's browser fetches plotly.js.
> **A** is wrong: Columns you did not map are indeed left out, but one point per customer still ships every customer's x, y and hover values; and the default embeds a multi-megabyte library.
> **B** is wrong: `write_image` makes a PNG, SVG or PDF chosen by the extension; `.html` is not an image format.
> **C** is wrong: That writes every email address into a file you are about to send.

7.2 · PREDICT, THEN RUN

P1

Prints:

```
fig = px.line(monthly, x="month_start", y="revenue", facet_col="store")
print(len(fig.data), len(fig.layout.annotations), fig.layout.annotations[0].text)
```

```
4 4 store=Asheville
```

Why: One trace per panel, and one annotation per panel: facet titles are layout annotations whose text is `store=Asheville`. That is why `fig.for_each_annotation(lambda a: a.update(text=a.text.split("=")[-1]))` cleans them up.

P2

Prints:

```
fig = px.histogram(sales, x="store", y="revenue", histfunc="avg")
print(fig.data[0].type, len(fig.data[0].x))
```

```
histogram 2903
```

Why: `px.histogram` lets Plotly do the averaging, but it does it in the browser: the figure still carries all 2,903 rows. The chart shows four bars; the file holds every store-day. Aggregating with pandas first sends four numbers.

7.2 · THE LESSON PAGE'S OWN QUIZ

1. **C.** The default include_plotlyjs=True embeds the whole library, so the file works offline. "cdn" needs the internet; False will not display by itself.
2. **A.** color="store" makes 4 traces, and px.bar draws one segment per row: 2,903 in total. It never aggregates.
3. **B.** Every column named in hover_data is stored in the file as readable text: the emails travel with the chart.
4. **B.** Two trace types on two y axes needs make_subplots with secondary_y and graph objects. Lines by color and facets are single Express calls.
5. **A.** Small multiples put every month on screen at once; an animation makes the reader compare from memory.

LAB REFERENCE SOLUTIONS

LAB 7.1-A · `LABS/MODULE-19/L7_1-SEABORN/TIER_ORDER_SIZES_SOLUTION.PY`

Merge check: 18,762 completed orders, 18,079 with a customer row, 683 dropped (closed accounts). 3,738 customers. Median order total: Basic $110.66, Silver $146.98, Gold $162.28. Mean completed orders per customer: Basic 2.05, Silver 5.12, Gold 10.81. Gold against Basic: order size ×1.47, orders per customer ×5.26.

How the solution works. The merge uses `validate="many_to_one"`, so pandas itself would complain if a customer id repeated in `customers`. The left panel is `sns.ecdfplot(..., hue="loyalty_tier", hue_order=tiers, log_scale=True, ax=axes[0])`: an ECDF, because three overlapping histograms of a skewed quantity are hard to read. The right panel needs **one row per customer** first (`groupby(["customer_id", "loyalty_tier"]).agg(orders=("order_id", "count"))`), then `sns.pointplot(..., errorbar=("ci", 95), seed=42)`. Finishing is Matplotlib: `set_title(..., loc="left")`, axis labels that name the unit and the bars, `sns.move_legend`.

What to accept. Any pair of titles that states the finding (order size differs modestly, order count differs a lot). A box plot or bar plot on the right is fine if the label says what is drawn. **Common wrong turn:** running `pointplot` on `table` with `y="order_total"`: that answers the first question twice. **A caution worth saying:** tiers are *defined* by completed spend, so "Gold customers order more" is partly true by construction; the chart describes how they got there, it does not explain why.

LAB 7.1-B · `LABS/MODULE-19/L7_1-SEABORN/WEBSITE_BY_AGE_SOLUTION.PY`

1,800 responses, 472 with no website rating. Means by age group: 3.69, 3.67, 3.67, 3.66, 3.72 and 65+ 3.04 (n = 291, 95% CI 2.94 to 3.15); everyone else 3.68, a gap of 0.64 points. The gap is the same for online shoppers (65+ 3.06) and store shoppers (65+ 3.02). catplot returns a FacetGrid with axes shape (1, 2). No website rating: 3% of online shoppers, 46% of store shoppers.

How the solution works. Step 1 draws the same `pointplot` twice with `errorbar=("ci", 95)` and `errorbar="sd"`. Step 2 is the hand check: `agg(n="count", mean="mean", sd="std", se="sem")`, then mean ∓ 1.96 × se. The left panel's bars match `ci_low` to `ci_high` (to within bootstrap wiggle); the right panel's match mean ± sd. Step 3 is `sns.catplot(kind="point", col="shopper", ...)`, which returns a FacetGrid: titles with `g.set_titles("{col_name} shoppers")`, saving with `g.savefig(...)`.

What it reveals. A learner who writes `g.set_title(...)` or passes `ax=` to `catplot` has not yet separated the two kinds of function. A learner who reports the 65+ gap without Step 4's caveat has the chart and not the analysis: 46% of store shoppers gave no website rating, so the store panel is the store shoppers who use the website.

LAB 7.2-A · `LABS/MODULE-19/L7_2-PLOTLY/PRODUCT_MAP_SOLUTION.PY`

242 products, 8 with no rating, 234 rows in the chart. Figure with 9 traces of type scatter. Spearman correlation of price and rating: −0.01. Lowest rated: Juniper Vane Trailhead Down Jacket, $372.95, rating 2.8, 4 reviews. product_map.html is about 31 KB with the CDN link; a product name is inside the file (True) and the cost column is not (False).

How the solution works. The table is cut to five columns **before** the figure exists. `px.scatter(..., color="category", size="review_count", log_x=True, hover_name="name", hover_data={"category": True, "price": ":$,.2f", "rating": ":.1f", "review_count": ":,"})` gives nine

traces (one per category) of type `scatter` (234 points is under the WebGL threshold of a thousand). The hover template that comes back is `<b>%{hovertext}</b><br><br>Category=%{customdata[0]}<br>List price=%{x:$,.2f}<br>Rating=%{y:.1f}<br>Reviews=%{customdata[1]:,}<extra></extra>`. On a log axis an annotation's x is `math.log10(price)`.

What to accept. Any title that states the near-zero relationship. **Common wrong answers:** the correlation printed as Pearson (−0.03 is not the number asked for, though the conclusion is the same); `"price": "$,.2f"` without the leading colon (Plotly then shows the raw value); the full `products` table passed to `px.scatter`, after which the last check prints False only by luck, because unmapped columns are not written. The habit being taught is to decide the columns on purpose.

LAB 7.2-B · `LABS/MODULE-19/L7_2-PLOTLY/PROMO_SHARE_TREND_SOLUTION.PY`

20,302 orders, 2,148 with a promo code. Weekly table (210, 4); median weekly share 6.6% online, 7.9% in store. 2 traces, hover mode x unified. Six sale weekends shaded, each four days (Friday to Monday). Share of orders using a code: 51.0% on sale-weekend days, 8.0% on all other days. January 2024 is 25.3% with no sale on: the data starts there and WELCOME10 marks a customer's first order in the data.

How the solution works. `orders["order_date"].dt.to_period("W-SUN").dt.start_time` labels each order with the Monday of its week; `groupby(["week", "channel"]).agg(orders=("order_id", "count"), share=("used_code", "mean"))` gives 105 weeks times 2 channels = 210 rows. `custom_data=["orders"]` carries the count so the hover can say `11.4% of 153 orders`. The sale weekends are found from `daily_sales.promo_day`, grouped into runs of consecutive dates with `(promo_dates.diff().dt.days != 1).cumsum()`, and shaded with `add_vrect`.

The stretch question. January 2024 shows a high share with no sale: WELCOME10 marks a customer's first order *in the data*, and the data starts on 2024-01-01, so long-standing customers look like new ones for the first weeks. It is a small preview of a trap in one capstone question; do not say which.

WORKBOOK WORKED EXAMPLES (FOR REFERENCE)

7.1 · ONE LINE, TWO BANDS

```
weather = pd.read_csv("data/store_weather.csv", parse_dates=["date"])
weather["month"] = weather["date"].dt.month
fig, axes = plt.subplots(1, 2, figsize=(10, 3.6), sharey=True, layout="constrained")
sns.lineplot(data=weather, x="month", y="high_f", hue="city",
             errorbar=("ci", 95), seed=42, ax=axes[0])
axes[0].set_title("Band: 95% CI of the mean", loc="left")
sns.lineplot(data=weather, x="month", y="high_f", hue="city",
             errorbar="sd", legend=False, ax=axes[1])
axes[1].set_title("Band: mean +/- 1 standard deviation", loc="left")
fig.savefig("highs_two_bands.png", dpi=150)
july = weather[weather["month"] == 7].groupby("city")["high_f"]
july = july.agg(n="count", mean="mean", sd="std", se="sem")
print(july.round(2))
```

```
            n   mean    sd    se
city
Asheville  62  82.81  7.48  0.95
Bend       62  82.21  7.51  0.95
Boise      62  92.00  8.73  1.11
Flagstaff  62  83.21  6.94  0.88
```

7.2 · AGGREGATE FIRST, THEN ONE CALL

```
orders = pd.read_csv("data/orders.csv")
mix = orders.groupby(["channel", "payment_method"], as_index=False).size()
mix["share"] = mix["size"] / mix.groupby("channel")["size"].transform("sum")
print(mix.round(3).to_string(index=False))
methods = ["card", "digital_wallet", "gift_card", "cash"]
fig = px.bar(mix, x="payment_method", y="share", color="channel", barmode="group",
             text_auto=".0%", category_orders={"payment_method": methods},
             labels={"share": "Share of orders", "payment_method": ""},
             title="Cash is a store-only habit; wallets are an online one")
fig.update_yaxes(tickformat=".0%")
fig.write_html("payment_mix.html", include_plotlyjs="cdn")
print(len(fig.data), "traces |", [len(t.x) for t in fig.data], "bars each")
```

```
channel payment_method  size  share
 online           card  7953  0.619
 online digital_wallet  3876  0.302
 online      gift_card  1025  0.080
  store           card  4439  0.596
  store           cash  1614  0.217
  store digital_wallet   918  0.123
  store      gift_card   477  0.064
2 traces | [3, 4] bars each
```

WARM-UP AND EXIT TICKET ANSWERS

WARM-UP (RETRIEVAL FROM SESSION 18)

1. A request comes back **401**. Another comes back **403**. What does each one tell you, and what should your client do next? **Answer:** 401: the server does not know who you are (no token, or `token_expired`): log in again and repeat the request once. 403: it knows who you are and refuses: stop and fix the credential or the permission. Neither is retried in a loop.

2. Where does an API key live while you work on a notebook you will share, and where in the request does it travel? **Answer:** In an environment variable (or typed at a prompt), read at run time; never in the code. It travels in a header (`X-API-Key`), not in the URL, because URLs end up in logs and browser history.

3. The server answers **429** with the header `Retry-After: 4`. What does a polite client do? What if the header is missing? **Answer:** Wait at least 4 seconds, then send the same request again. With no header, fall back to a default wait (exponential backoff with jitter), with a maximum number of tries.

4. What does this print?

```
print([min(4.0, 1.0 * 2 ** n) for n in range(4)])
```

   Answer: `[1.0, 2.0, 4.0, 4.0]`: the delay doubles (1, 2, 4) and then the cap of 4.0 holds it. That is exponential backoff with a ceiling; real clients add random jitter on top.

5. Name two failures a retry policy should retry and two it never should. Then: what made the Lesson 6.5 download resumable? **Answer:** Retry: 429, 5xx, timeouts (temporary). Never: 400, 401, 403, 404 (the same request will fail the same way). Resumable: each page is saved to its own file as soon as it arrives, written completely or not at all, and pages whose files exist are skipped.

EXIT TICKET

1. You call `sns.barplot(data=sales, x="store", y="revenue")` and black lines appear on the bars. What are they, by default, and which argument changes them? **Answer:** A bootstrapped 95% confidence interval for the mean of each bar (uncertainty of the estimate, not spread of the days). `errorbar=` changes it: `"sd"`, `"se"`, `("pi", 50)` or `None`. Say which one in the caption.

2. Give one job for an axes-level function and one for a figure-level function. **Answer:** Axes-level (`scatterplot`, `histplot`, `barplot`): one panel inside your own `plt.subplots()` layout, with `ax=`. Figure-level (`relplot`, `displot`, `catplot`): seaborn builds a grid of panels from a column with `col=` / `row=` and returns a FacetGrid.

3. You plot one point per customer with `email` in `hover_data` and send the HTML file to a colleague. What have you sent? **Answer:** The customer table: every x, y and hover column is stored as readable text inside the file. Aggregate first and pass Plotly only the columns the chart needs.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q3 answered `(2, 2)`:** the learner pictures the layout, not the object. With `col_wrap` the Axes array is flat. Have them print `g.axes.shape` for a call with `row=` as well.
- **Q4 answered "monthly totals":** the learner knows seaborn aggregates but not with what. Ask what `estimator=` defaults to.
- **Q5 answered "95% confidence interval":** the most important miss of the session. The learner reads every bar as uncertainty. Go back to the four-panel figure and the summary table: sd about 0.9, se about 0.06.
- **Q6 answered `[1.0, 5.0]` for S, or four bars:** the learner does not yet believe that `barplot` computes. Same root as Q4.
- **Q8 answered 4 or 2:** counts colors or facets, not their combinations. Print `[t.name for t in fig.data]`.
- **Q11 answered `bar 2903`:** read the code as if `sales` had been passed. The groupby line is the whole point: this learner will stack slivers in the project.
- **Q12 answered A:** believes "unused columns are left out" settles the privacy question. It does not: one point per customer is already the customer table.

INSTRUCTOR KIT · ANSWER KEY

Session 20: Bokeh and Streamlit

Every workbook item for Session 20 with its reasoning, the four lab reference solutions explained, the lesson pages' own quiz answers, what a good capstone plan looks like, and the warm-up and exit-ticket answers.

Session 20 | 12 self-check items | Lessons 7.3–7.4 | Confidential

7.3 · INTERACTIVE CHARTS WITH BOKEH: SELF-CHECK

Q1

A glyph is drawn with `p.scatter("visitors", "revenue", source=source, size="visitors")`. What does `size="visitors"` do?

A) Raises an error: a size must be a number

B) Reads each marker's size from the `visitors` column of the source

C) Writes the word visitors next to each marker

D) Sets every marker to the average number of visitors

Why: With a source, a string argument is always read as a column name, so each row gets its own value. A plain number means the same value for every mark.

A is wrong: A string is allowed: it means a column. `size=5` would be the fixed-size form.

C is wrong: Labels are a different glyph (text) or a hover tool.

D is wrong: Nothing is averaged: each row uses its own value, so markers would be hundreds of pixels wide here.

Q2

What does this print?

```
p = figure(width=300, height=200)
for store in ["Boise", "Bend"]:
    p.line([1, 2], [3, 4], legend_label=store)
    p.scatter([1, 2], [3, 4], legend_label=store)
print(len(p.renderers), len(p.legend.items))
```

`4 2`

> **Why:** Every glyph call adds a renderer: two stores times a line and a set of markers is 4. Glyphs that share a `legend_label` share one legend entry, so the legend has 2 items.

Q3

Two stacked time-series plots draw different columns. You want panning or zooming either one to move both along the date axis. What do you write?

A) Put both plots in one `gridplot`

B) Give both plots the same `title`

C) Build each plot from its own copy of the source

D) Create the second plot with `x_range=top.x_range`

> **Why:** A range object describes what is in view. Two plots holding the same range object can never show different views.
>
> **A** is wrong: A grid arranges plots and merges toolbars; it does not link their views.
>
> **B** is wrong: A title is text; it links nothing.
>
> **C** is wrong: Separate sources break linked selection and do nothing for panning.

Q4

On one page you wrote `ColumnDataSource(df)` twice, once for each plot. A box selection in the left plot changes nothing in the right one, and there is no error. Why?

A) The selection is stored in a source, and the two plots have two separate sources

B) Selections only link plots that also share an `x_range`

C) `box_select` works on one plot per page

D) The DataFrame was not sorted the same way in both plots

> **Why:** Two sources are two tables with the same contents. Selected row numbers are recorded on one of them and the other never hears about it. Create the source once and pass the same variable to every glyph that should be linked.
> **B** is wrong: Ranges link views (panning). Selection travels through the shared source.
> **C** is wrong: Any number of plots can carry selection tools.
> **D** is wrong: Row order is identical here; the problem is that there are two objects.

Q5

What does this print?

```
source = ColumnDataSource(pd.DataFrame({"x": [1, 2, 3], "y": [4, 5, 6]}))
print(source.column_names, len(source.data["x"]))
```

`['index', 'x', 'y'] 3`

> **Why:** Building a source from a DataFrame copies each column in and adds the DataFrame's index as a column named `index`. Every column in the source travels inside the saved HTML file, shown or not.

Q6

Which Bokeh tooltip shows a row's revenue as `$12,345`?

A) `("Revenue", "$revenue")`

B) `("Revenue", "@revenue:$,.0f")`

C) `("Revenue", "@revenue{$0,0}")`

D) `("Revenue", "%{revenue}")`

> **Why:** In a Bokeh tooltip `@name` means this row's value in column `name`, and a format in braces controls the display.
> **A** is wrong: A `$` prefix is for special fields such as `$x` and `$index`; `$revenue` is not one.
> **B** is wrong: That is the Plotly hover_data style (a colon and a d3 format). Bokeh uses braces.
> **D** is wrong: `%{...}` is Plotly's hovertemplate placeholder.

7.3 · PREDICT, THEN RUN

P1

Prints:

```
p = figure(width=300, height=200)
p.vbar(x=[1, 2, 3], top=[4, 5, 6], width=0.8)
p.line([1, 2, 3], [4, 5, 6])
print(len(p.renderers), [type(r.glyph).__name__ for r in p.renderers])
```

```
2 ['VBar', 'Line']
```

> **Why:** Each glyph call adds exactly one renderer, however many marks it draws: three bars are one `VBar` renderer, the line is one `Line` renderer.

P2

Prints:

```
a = ColumnDataSource(pd.DataFrame({"x": [1, 2, 3]}))
b = ColumnDataSource(pd.DataFrame({"x": [1, 2, 3]}))
a.selected.indices = [0, 2]
print(a.selected.indices, b.selected.indices, a is b)
```

```
[0, 2] [] False
```

Why: A selection is a list of row numbers stored on one source. A second source with the same contents is a different table and knows nothing about it: this is why two plots built from two sources do not brush each other.

7.3 · THE LESSON PAGE'S OWN QUIZ

1. **A.** Selection is recorded on the ColumnDataSource; glyphs that draw from the same source object show the same selected rows.
2. **C.** A date is stored as milliseconds; give the field a date format and name the formatter: formatters={"@date": "datetime"}.
3. **B.** A saved file has no Python process behind it; only JavaScript (CustomJS or js_link) can react in the reader's browser.
4. **C.** Two glyph calls make two renderers; the second one's glyph is a Scatter.
5. **B.** Charts that respond to each other through a shared source are Bokeh's strength. Facets with trendlines and static export are easier in Plotly.

7.4 · DASHBOARDS WITH STREAMLIT: SELF-CHECK

Q7

Line 5 of your app is `sales = pd.read_csv(...)` on a 200 MB file, with no decorator. A reader opens the page and then ticks a checkbox 10 times. How many times is the file read?

A) 1

B) 11

C) 10

D) 0

Why: The script runs once when the page opens and once more for every widget change: 1 + 10. That is why slow loading belongs in a function decorated with `@st.cache_data`.

A is wrong: That is what caching would give you. Without it, every rerun repeats line 5.

C is wrong: The first run, when the page opens, also reads the file.

D is wrong: Nothing is skipped: the whole script runs each time.

Q8

What does this print? (The check clicks the button twice.)

```
at = AppTest.from_string("""
import streamlit as st
if "n" not in st.session_state:
    st.session_state["n"] = 5
if st.button("Double"):
    st.session_state["n"] *= 2
st.write(st.session_state["n"])
""").run()
at.button[0].click().run()
at.button[0].click().run()
print(at.markdown[0].value)
```

`20`

Why: The starting value 5 is set once, because the `if "n" not in st.session_state` guard is false on every later rerun. Each click reruns the script with the button returning True, so 5 becomes 10, then 20. `st.write` shows a number as code, which is why the value prints inside backticks.

Q9

A Reset button must put a slider (which has a key) back to 0. Where does the line that sets the slider's value belong?

A) In a function passed as `on_click=`, which Streamlit runs before the rerun starts

B) In an `if st.button("Reset"):` block placed after the slider

C) Inside the function decorated with `@st.cache_data`

D) Nowhere: a widget's value cannot be changed from code

Why: A callback runs before the script, so the widget has not been created yet in that run and its value in session state may be changed.
B is wrong: By then the slider exists for this run; Streamlit raises `cannot be modified after the widget with that key is instantiated`.
C is wrong: Cached functions must not touch widgets or per-user state; the cache is shared by all users.
D is wrong: It can, through its key in session state, in a callback.

Q10

Which step of a dashboard is the right one to wrap with `@st.cache_data`?

A) Filtering the table by the stores and dates the reader chose

B) The whole app, so that nothing ever runs twice

C) A function that draws the sidebar widgets

D) Loading and preparing the CSV file, which is slow and the same whatever the widgets say

Why: Cache the slow steps that do not depend on widgets. The result is stored per set of arguments and returned without running the body again.
A is wrong: Filtering in memory is fast and must reflect the current widget values on every run.
B is wrong: A cached app could not react to its widgets.
C is wrong: Widgets do not belong inside cached functions.

Q11

What does this print? (Two numbers.)

```
at = AppTest.from_string("""
import streamlit as st
stores = st.multiselect("Stores", ["Boise", "Bend"], default=["Boise"])
if not stores:
    st.warning("Pick a store.")
    st.stop()
st.metric("Stores chosen", len(stores))
""").run()
at.multiselect[0].set_value([]).run()
print(len(at.warning), len(at.metric))
```

`1 0`

> **Why:** With no store chosen the script shows the warning and `st.stop()` ends that run, so the metric line is never reached: one warning, zero metrics. That is the polite way to handle an empty or half-finished input.

Q12

What does `st.button("Go")` return?

A) True from the first click onward

B) The number of times it has been clicked

C) True only during the one rerun caused by a click, otherwise False

D) The label of the button

> **Why:** A button is not a switch. It reports "I was just clicked" for a single run of the script. Anything that must persist after that run goes into session state.
> **A** is wrong: That would be a toggle or checkbox (`st.toggle`, `st.checkbox`).
> **B** is wrong: Streamlit keeps no count; keep one yourself in session state.
> **D** is wrong: Widgets return the reader's input, not their own label.

7.4 · PREDICT, THEN RUN

P1

Prints:

```
at = AppTest.from_string("""
import streamlit as st
total = st.session_state.get("total", 0)
if st.button("Add 10"):
    total += 10
st.write("Total:", total)
""").run()
for _ in range(3):
    at.button[0].click().run()
print(at.markdown[-1].value)
```

> `Total: `10``

> **Why:** Reading from session state is not enough: the new total is never written back, so every rerun starts from 0 again and one click's worth is all that ever shows. The fix is `st.session_state["total"] = total` inside the `if`, or a callback.

P2

Prints:

```
at = AppTest.from_string("""
import streamlit as st
@st.cache_data
def square(n):
    st.session_state["runs"] = st.session_state.get("runs", 0) + 1
    return n * n
answers = [square(3), square(3), square(4)]
st.write(st.session_state["runs"])
""").run()
print(at.markdown[0].value)
```

> `2`

> **Why:** The cache stores one result per set of arguments. `square(3)` runs the body once and the second `square(3)` is answered from the cache; `square(4)` is a new argument, so the body runs again: 2 runs for 3 calls.

7.4 · THE LESSON PAGE'S OWN QUIZ

1. **B.** Any widget change reruns the whole script from the top; the widget returns its new value.
2. **A.** total is reset to 0 on every rerun and the button is True for one run only, so it shows 10 after each click, never more.
3. **C.** With the same argument and code, st.cache_data returns a copy of the stored result and skips the function body.
4. **A.** An app is a running Python process: it must run on a machine she can reach, with the packages and the data there.
5. **C.** Recalculating totals for any stores and dates and downloading the rows needs Python behind the controls. Hover and legend clicks (A) and linked highlighting (B) work in standalone files.

LAB REFERENCE SOLUTIONS

LAB 7.3-A · `LABS/MODULE-20/L7_3-BOKEH/MARGIN_BRUSH_SOLUTION.PY`

234 products with a rating; margin runs from 34% to 58%. Source columns: index, name, category, price, margin, rating, review_count (no cost). `renderers: 1 + 1 | same source: True`. The left plot has seven tools (BoxSelect, LassoSelect, Tap, WheelZoom, Pan, Reset, Hover). margin_brush.html is about 29 KB. Margin of 50% or more: 67 products, mean rating 4.17; under 50%: 167 products, mean rating 4.12; Spearman correlation of margin and rating +0.01.

How the solution works. `margin` is computed in pandas and `cost` is left out of `table` before `ColumnDataSource(table)` is made, so cost never reaches the file. One `style` dictionary (`source=source, nonselection_alpha=0.1, selection_color="#d97706"`) is passed to both `scatter` calls, which guarantees the two glyphs share the source. Both figures get `tools="box_select,lasso_select,tap,wheel_zoom,pan,reset"` with `active_drag="box_select"`, and a `HoverTool` whose tooltips use `@price{$0,0.00}` and `@margin{0%}`. The page is `row(left, right)`, written with `output_file(...)` and `save(...)`.

What to accept. Any two plots on one source with a working selection. **The finding is a null:** mean rating 4.17 against 4.12 and a rank correlation of +0.01. Learners who report "high-margin products are rated higher" from the 0.05 gap have not looked at the spread; it is a useful rehearsal for the capstone's honest-null standard. **Common wrong turn:** `ColumnDataSource(table)` typed inside each `scatter` call: the page works, the brush does not.

LAB 7.3-B · `LABS/MODULE-20/L7_3-BOKEH/CATEGORY_PICKER_SOLUTION.PY`

234 rated products in 9 categories. `view shows: category = Tents | 24 products to start with`. category_picker.html (cdn) is about 21 KB; category_picker_offline.html (inline) about 1,750 KB. Counts the page should show: Accessories 30, Apparel 35, Camp Kitchen 28, Climbing 24, Footwear 25, Packs 22, Sleeping 24, Snow 22, Tents 24.

How the solution works. The scatter draws through `view=CDSView(filter=GroupFilter(column_name="category", group="Tents"))`. The `Select` widget's `CustomJS` sets `flt.group = cb_obj.value`, rewrites the plot title, then loops over `source.data["category"]` to count matches and writes the sentence into a `Div`. The slider needs no JavaScript from the learner: `slider.js_link("value", line, "location")` moves a `Span`. Everything is built inside `build_page()` so that the second save (with `mode="inline"`) gets fresh objects; reusing the first page's objects raises `RuntimeError: Models must be owned by only a single document`.

Checking a learner's page: pick each category and compare the counter with the printed table (Apparel 35 is the largest, Packs and Snow 22 the smallest). A counter that never changes usually means `count` was not passed in `args`; the browser console (F12) says `count is not defined`.

LAB 7.4-B · `LABS/MODULE-20/L7_4-STREAMLIT/GEAR_SHORTLIST_SOLUTION.PY`

24 tents. The first metric reads 0 at the start, 1 after adding the first tent, still 1 after adding it again and after choosing a second tent, 2 after adding the second ($259.90 in total: $41.95 + $217.95), and 0 after Clear. The third metric, the ordinary counter, reads 1 on every line. The unfinished starter prints 0, 1, 0, 1, 0: the list forgets.

How the solution works. `st.session_state["shortlist"]` is created once (`if "shortlist" not in st.session_state`). The selectbox has `key="choice"`, so the callback `add_to_shortlist` can read the current choice from session state and append it if it is new; `clear_shortlist` assigns an empty list. Both buttons use `on_click=`; neither is wrapped in `if`. The page is drawn from `tents[tents["name"].isin(st.session_state["shortlist"])]`. The ordinary variable `reruns_plain` is left in on purpose: it reads 1 on every line of the check.

What the unfinished starter teaches. Its check prints 0, 1, 0, 1, 0: the click's rerun shows one tent, and the very next rerun (choosing another tent) empties the list. **Common wrong answers:** keeping `if left.button(...)` *and* adding `on_click=` (the tent is appended twice per click, hidden only by the duplicate guard); creating the list without the `if ... not in` guard (it is emptied on every rerun, the same bug in new clothes).

LAB 7.4-A · `LABS/MODULE-20/L7_4-STREAMLIT/CATALOG_BROWSER_SOLUTION.PY` AND `TEST_CATALOG_BROWSER.PY`

Start (Tents): Products 24, Median price $298.45, Mean rating 4.25. Tents and Sleeping under $100: 12 products, median $46.95, mean rating 4.07; with In stock only: 11. No category: the warning shows, 0 metrics, 0 errors. After Reset: ['Tents'], (0, 620), False, 24 products. `test_catalog_browser.py`: 3 tests pass.

How the solution works. `load_products` is decorated with `@st.cache_data` and returns only the eight columns the page uses. Three sidebar widgets have keys (`categories`, `price`, `in_stock`); their starting values are set through session state by `reset_filters()`, which is also the Reset button's `on_click` callback. After filtering, `if view.empty: st.warning(...); st.stop()` ends the run politely. Three `st.metric` calls sit in `st.columns(3)`; the table is `st.dataframe(table, hide_index=True, width="stretch")`; the download is `table.to_csv(index=False)`.

The file's last lines decide how it was started: `get_script_run_ctx(suppress_warning=True)` is not None under `streamlit run` (and under AppTest), so `main()` runs; under plain `python` the file calls

`self_check()`, which drives the app with `AppTest.from_file(__file__)`. The pytest file is the same idea in three named tests: opens with Tents (24 products), an empty selection warns without an error, Reset restores all three filters.

Common wrong answers: resetting with `st.session_state["price"] = (0, 620)` inside `if st.button("Reset"):` after the slider (the red box says the key cannot be modified after the widget is instantiated); giving the multiselect both `default=` and a session-state value (Streamlit warns); dollar signs in `st.caption` text, which Markdown reads as math.

WORKBOOK WORKED EXAMPLES (FOR REFERENCE)

7.3 · A SOURCE, BARS AND A FORMATTED HOVER

```
import pandas as pd
from bokeh.models import ColumnDataSource, NumeralTickFormatter
from bokeh.plotting import figure, output_file, save

weather = pd.read_csv("data/store_weather.csv", parse_dates=["date"])
ash = weather[weather["city"] == "Asheville"].copy()
ash["month"] = ash["date"].dt.strftime("%b")
ash["rainy"] = ash["precip_in"] >= 0.10
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
          "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]
table = ash.groupby("month").agg(high=("high_f", "mean"), rainy=("rainy", "mean"))
table = table.loc[months].reset_index()
source = ColumnDataSource(table)
tips = [("Month", "@month"), ("Rainy days", "@rainy{0%}"),
        ("Mean high", "@high{0} F")]
p = figure(width=700, height=300, x_range=months, tools="save", tooltips=tips,
           title="Asheville: July is the rainiest month")
p.vbar(x="month", top="rainy", width=0.8, source=source, color="#667eea")
p.yaxis.formatter = NumeralTickFormatter(format="0%")
p.y_range.start = 0
output_file("asheville_rain.html", title="Asheville rain")
save(p)
print(source.column_names, "|", len(p.renderers), "renderer,", len(table), "bars")
print(table.round(2).iloc[[0, 5, 6, 7, 11]].to_string(index=False))
```

```
['index', 'month', 'high', 'rainy'] | 1 renderer, 12 bars
month  high  rainy
  Jan 47.87   0.21
  Jun 84.33   0.28
  Jul 82.81   0.39
  Aug 83.39   0.34
  Dec 51.32   0.19
```

```
import pandas as pd
import streamlit as st

@st.cache_data
def load_weather(path):
    return pd.read_csv(path, parse_dates=["date"])

weather = load_weather("data/store_weather.csv")
city = st.selectbox("City", ["Boise", "Bend", "Flagstaff", "Asheville"])
inches = st.slider("A rainy day has at least this many inches",
                   0.05, 1.00, 0.10, step=0.05)
one = weather[weather["city"] == city]
rainy = one["precip_in"] >= inches
st.metric("Rainy days", f"{rainy.mean():.0%}",
          help="Share of the 731 days in 2024 and 2025")
st.caption(f"{int(rainy.sum())} of {len(one)} days in {city}")
```

```
from streamlit.testing.v1 import AppTest

def report():
    print(at.selectbox[0].value, at.slider[0].value, "->", at.metric[0].value,
          "|", at.caption[0].value)

at = AppTest.from_file("rain_app.py", default_timeout=30).run()
report()
at.selectbox[0].select("Asheville").run()
report()
at.slider[0].set_value(0.5).run()
report()
```

```
Boise 0.1 -> 9% | 68 of 731 days in Boise
Asheville 0.1 -> 24% | 172 of 731 days in Asheville
Asheville 0.5 -> 6% | 44 of 731 days in Asheville
```

CAPSTONE KICKOFF: WHAT A GOOD PLAN LOOKS LIKE

There is no answer to mark tonight. Read each plan for five things, in the order of the frame in the workbook: (1) a **question with a decision** behind it, not a topic; (2) the **tables** named and the **unit of analysis** stated (one row per store-day, per customer, per product); (3) **one main comparison** and a measure with a method that fits it (proportions, medians of a skewed amount, a ratio); (4) a named **risk to fairness** with a check; (5) an **expected chart** and what would change the learner's mind. A plan with all five is at Proficient on the first rubric row; one that

also says in advance what result would change the reader's mind is heading for Exemplary. The instructor answer notes for the four questions are in `final-assessment.html`. They are not for learners, tonight or later.

WARM-UP AND EXIT TICKET ANSWERS

WARM-UP (RETRIEVAL FROM SESSION 19)

1. Which kind of seaborn function takes `ax=`, and what does the other kind return? **Answer:** Axes-level functions (`scatterplot`, `barplot`, `histplot`, ...) take `ax=` and return that Axes. Figure-level functions (`relplot`, `displot`, `catplot`) make their own figure and return a FacetGrid.

2. `sns.barplot` draws a black line on each bar. With default settings, what is it? Name one setting that shows spread instead. **Answer:** A bootstrapped 95% confidence interval for the mean (uncertainty of the estimate). `errorbar="sd"` or `errorbar=("pi", 50)` shows spread of the data.

3. You pass 2,903 daily rows to `px.bar(sales, x="store", y="revenue")`. What do you get, and what should you have done? **Answer:** 2,903 bar segments stacked into four piles, with no error: Plotly Express does not aggregate. Group with pandas first (`groupby("store", as_index=False)["revenue"].mean()`), then plot four rows.

4. `monthly` has one row per store and month (4 stores, 24 months). What does this print?

```
fig = px.line(monthly, x="month_start", y="revenue", color="store")
print(len(fig.data), fig.data[0].type, len(fig.data[0].x))
```

   Answer: `4 scatter 24`: one trace per store, each a `scatter` trace in lines mode holding 24 points. A figure is data: you can count it before you look at it.

5. You wrote a chart with `fig.write_html("chart.html", include_plotlyjs="cdn")`. Name two things that are true of that file. **Answer:** It is small because the reader's browser fetches plotly.js from a public server, so the reader must be online; and it contains the data you passed to the figure (every x, y and hover column) as readable text.

EXIT TICKET

1. Two Bokeh plots on one page. What must they share for a box selection in one to highlight the same rows in the other? What must they share to pan together? **Answer:** The same `ColumnDataSource` object for linked selection (brushing). The same range object (`x_range=first.x_range`) for linked panning and zooming.

2. A reader changes one widget in your Streamlit app. What runs? Name the place for a value that must survive, and the decorator for a slow step that must not repeat. **Answer:** The whole script reruns from the top with the widget's new value. `st.session_state` keeps values across reruns (per browser tab); `@st.cache_data` stores a function's result so the slow body runs once.

3. Write your capstone question letter (A, B, C or D) and the one comparison your plan is built on. **Answer:** No single answer. Check that it is a comparison (two or more groups, or before and after against something), not a topic, and that the learner can say what would make it unfair. Use the cards to balance your coaching time in Session 21.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q2 answered `2 2` or `4 4`:** renderers and legend entries are being confused. One glyph call, one renderer; one label, one legend entry.
- **Q3 answered A (gridplot):** thinks layout links plots. Layout arranges; a shared range links.
- **Q4 answered B:** has merged the two kinds of link. Ask: "What did we share for panning? What for brushing?"
- **Q6 answered B:** carried Plotly's `:$,.0f` into Bokeh. The braces are the tell.
- **Q7 answered 1:** has not yet accepted the rerun model; assumes line 5 is skipped once the data is loaded, as it would be in a notebook kernel.
- **Q8 answered `10` or `5`:** believes the `if "n" not in` block runs every time (5) or that the second click starts over (10). Trace the guard.
- **Q9 answered B:** will meet the red box in the lab. Let them, then ask what runs before the script.
- **Q12 answered A:** treats a button as a switch; the shortlist lab will expose it.

INSTRUCTOR KIT · ANSWER KEY

Session 21: Capstone Studio and Showcase

The twelve self-check items with reasoning, the predict items, the lesson page's quiz, the scaffold lab's reference numbers, and the warm-up and exit-ticket answers. Capstone answer notes and the rubric are in final-assessment.html.

Session 21 | 12 self-check items | Lesson 8.1 | Confidential

8.1 · THE PINECREST REPORT: SELF-CHECK

Q1

Which of these is a capstone question rather than a topic?

A) Asheville store performance

B) An exploration of customer acquisition channels

C) Did products repriced on 2025-03-01 sell fewer units, March to December, than comparable products whose price did not change?

D) Everything about the WELCOME10 code

Why: It names what is compared with what, over which period, with a measure: someone could predict the notebook from it. That is the done-when test of Milestone 1.

A is wrong: A topic: no comparison, no measure, no decision.

B is wrong: Still a topic; "exploration" is a step, not an answer someone can act on.

D is wrong: Unbounded: it invites six comparisons and finishes none.

Q2

You compare a strongly right-skewed amount (customer spend) between two groups of about 500 people each. Which summary and interval suit it best?

A) The difference in medians with a bootstrap 95% confidence interval, reported next to the means

B) The difference in means with no interval, since the groups are large

C) Whichever of mean or median gives the smaller p-value

D) The maximum of each group, since large customers matter most

> **Why:** Skew makes the mean sensitive to a few large values; the median describes the typical customer, and the bootstrap gives an interval without assuming a shape (Lesson 8.1, Exercise 2). Showing both is honest.
> **B** is wrong: Large groups shrink the interval; they do not remove the need to report it.
> **C** is wrong: Choosing the statistic by its p-value is searching until something is significant.
> **D** is wrong: A maximum is one customer; it says nothing about the group.

Q3

What does this print?

```
orders = pd.DataFrame({"order_id": [1, 2], "order_total": [100.0, 50.0]})
lines = pd.DataFrame({"order_id": [1, 1, 1, 2], "sku": ["A", "B", "C", "A"]})
merged = orders.merge(lines, on="order_id")
print(len(merged), merged["order_total"].sum())
```

```
4 350.0
```

> **Why:** Order 1 matches three lines, so its total of 100 is repeated three times: 300 + 50 = 350, for 150 of real revenue. This is the merge that multiplies rows (Common Mistake 2). Sum each quantity at the level where it lives, and print row counts around every merge.

Q4

Your main result: a difference of +1.2 percentage points (95% CI −1.5 to +4.0), p = 0.39. Which sentence belongs in the report?

A) The new approach works: the rate went up by 1.2 points

B) We found no clear difference: the data are consistent with anything from 1.5 points lower to 4.0 points higher, so a gain larger than about 4 points is unlikely

C) There is no difference between the groups

D) The result is not significant, so the analysis failed

Why: An honest null gives the estimate, the interval and what the interval rules out. It scores as well on the rubric as a dramatic discovery.
A is wrong: Reports the point estimate as if it were certain; the interval includes zero and negative values.
C is wrong: "No evidence of a difference" is not "no difference": the data cannot rule out a gain of up to about 4 points.
D is wrong: A well-measured null is a complete answer.

Q5

You compare how many customers "came back" after their first order. Some first orders were in November 2025 and the data end on 2025-12-31. What makes the comparison fair?

A) Count anyone who ever ordered again, whenever their first order was

B) Drop every customer whose first order was in 2025

C) Use months since signup as the outcome instead

D) Give everyone the same window (for example 180 days) and keep only customers whose first order was at least 180 days before the data end

Why: A fixed follow-up window gives every customer the same chance to return. Without it, a group with more late first orders looks worse for no reason except time.
A is wrong: Customers with late first orders have had little time: the measure mixes behavior with calendar position.
B is wrong: Throws away data without fixing the problem: 2024 first orders still differ in follow-up time.
C is wrong: Changes the question; time since signup is not coming back.

Q6

What does this print?

```
from itertools import combinations
channels = ["organic", "paid_search", "social", "referral", "in_store"]
print(len(list(combinations(channels, 2))))
```

`10`

Why: Five groups give 10 pairs. At a 5% threshold, comparing ten pairs of identical groups would produce a "significant" result about 40% of the time by chance, which is why the plan names the main comparison first and pairwise p-values are adjusted (for example with Holm's method) and their number reported.

Q7

Sales are seasonal and the company is growing. A change happened on a known date to some products only. Which comparison is fairest?

A) The change in the affected products, against the change in unaffected products over the same months a year earlier (a difference in differences)

B) The two months before the date against the ten months after it, for the affected products

C) Affected against unaffected products, after the date only

D) The year before against the year after, for all products together

Why: Comparing the same calendar months removes seasonality; comparing with products that did not change removes company growth. What is left is closer to the effect of the change, though the products were not chosen at random.

B is wrong: Before-and-after across seasons: January and February are slow months, so almost anything looks better afterward.

C is wrong: The two groups may have differed before the change; this cannot tell.

D is wrong: Mixes the affected products with the rest and keeps the growth.

Q8

Two conversion rates, 10% and 9%. What does this print? (It is Cohen's h, the effect size for two proportions.)

```
print(round(2 * np.arcsin(np.sqrt(0.10)) - 2 * np.arcsin(np.sqrt(0.09)), 3))
```

`0.034`

Why: h compares proportions on a scale where 0.2 is small, 0.5 medium and 0.8 large. One point on a 9% base is h of about 0.03: a very small effect, even if a large sample makes it significant. Report it next to the p-value.

Q9

Which caveat is specific enough to earn credit in the Communication row of the rubric?

A) More data is needed

B) Correlation is not causation

C) Customers chose whether to use the code, so those who used it may differ from those who did not in ways the data do not record

D) The results may not be accurate

Why: It names who was compared, why they may differ, and what the data cannot see. A reader learns exactly how far the claim goes.

A is wrong: The lesson names this as the generic caveat to avoid.

B is wrong: True everywhere and therefore says nothing about this analysis.

D is wrong: Vague: which results, inaccurate how, and why?

Q10

What is the first sentence after the question in report.md?

A) A description of the files you loaded

B) The answer, with the main number and its uncertainty

C) A list of the methods you used

D) A thank-you to the reader

> **Why:** The reader asked a question. Leading with the answer is the template's first rule; everything the reader does not need in order to trust it stays in the notebook.
> **A** is wrong: That belongs in the three-line Method section at the end.
> **C** is wrong: A lab diary, the fifth common mistake.
> **D** is wrong: Spends the reader's first seconds on nothing.

Q11

What does this print? (Run it twice: does the answer change?)

```
rng = np.random.default_rng(42)
print(rng.choice([10, 20, 30, 40], size=3).tolist())
```

`[10, 40, 30]`

> **Why:** A seeded generator gives the same draws on every run, so a bootstrap interval or a sample in the notebook reproduces exactly after Restart and Run All. Unseeded randomness is one of the Developing-level faults in the Reproducibility row.

Q12

You have three minutes in the showcase. What do you say first?

A) How long the analysis took

B) Every chart you made while exploring

C) The cleaning steps, in order

D) The question, then the answer with its uncertainty, pointing at the one figure

> **Why:** The showcase is the one-page report spoken aloud: question, answer, evidence, one caveat. The audience remembers the figure and the sentence.
> **A** is wrong: Effort is not evidence.
> **B** is wrong: Exploration charts are for you; one finished figure carries the argument.
> **C** is wrong: The journey belongs in the notebook.

PREDICT, THEN RUN

P1

Prints:

```
first = pd.to_datetime(pd.Series(["2024-03-01", "2025-11-20"]))
print(((pd.Timestamp("2025-12-31") - first).dt.days >= 180).tolist())
```

```
[True, False]
```

> **Why:** Only the first customer has 180 days of follow-up before the data end. The second cannot be judged on a 180-day window yet: keep them out of that comparison rather than counting them as "did not come back".

P2

Prints:

```
sales = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
by_month = sales.groupby(sales["date"].dt.month)["revenue"].mean()
print(by_month.loc[[2, 8, 12]].round(0).to_dict())
```

```
{2: 5551.0, 8: 8684.0, 12: 9281.0}
```

Why: Mean daily store revenue in August is far above February, and December is higher still. Any before-and-after comparison that crosses seasons measures the calendar first; compare the same months a year apart, or against a group that did not change.

THE LESSON PAGE'S OWN QUIZ

1. **C.** Each order's total was repeated once per order line, so revenue was counted several times.
2. **B.** With 20,000 per group a tiny difference reaches significance; d = 0.02 says it is very small.
3. **A.** Online shoppers mostly skipped the question, so the mean describes mostly store shoppers.
4. **C.** Staff are scheduled for expected demand, which also drives revenue; the correlation cannot show what adding staff would do.
5. **A.** One static figure with a finding-first title in a one-page report suits a reader with a PDF and five minutes.

LAB REFERENCE SOLUTION

LAB 8.1 · `LABS/MODULE-21/L8_1-CAPSTONE/CAPSTONE_SCAFFOLD_SOLUTION.PY`

Online orders paid by digital wallet were returned at 7.3% (n = 3,765) against 7.1% for card (n = 7,731): +0.2 points (95% CI −0.8 to +1.3), p = 0.66, h = +0.009. Without footwear 5.1% vs 4.8% (+0.3, CI −0.7 to +1.3); with footwear 14.7% vs 15.1% (−0.4, CI −3.2 to +2.6). Cancelled orders are excluded (they never shipped) and 998 gift-card orders are dropped by design; the table has 11,496 rows. The figure `figure_1.png` shows both subgroups with 95% error bars.

What it models: a plan written before code; `overview()` checks; one `build_table()` with row counts; one helper that produces all five parts of the sentence; a robustness check inside subgroups that are known to matter (footwear); a finding-first title; three plain sentences. **What to accept from a learner:** any correct comparison with all five parts and an honest reading. **Common wrong turn:** keeping cancelled orders in the denominator, which lowers both rates slightly and changes nothing important, but should be a stated decision.

WORKBOOK WORKED EXAMPLE (FOR REFERENCE)

A STRONG SENTENCE FOR A NULL RESULT

```
from statsmodels.stats.proportion import confint_proportions_2indep, proportions_ztest

survey = pd.read_csv("data/survey.csv")
customers = pd.read_csv("data/customers.csv")
t = survey.merge(customers[["customer_id", "marketing_opt_in"]], on="customer_id",
                 validate="one_to_one").dropna(subset=["recommend"])
t["promoter"] = t["recommend"] >= 9
g = t.groupby("marketing_opt_in")["promoter"].agg(["sum", "count"])
yes, no = g.loc[True], g.loc[False]
p1, p0 = yes["sum"] / yes["count"], no["sum"] / no["count"]
z, p = proportions_ztest([yes["sum"], no["sum"]], [yes["count"], no["count"]])
low, high = confint_proportions_2indep(yes["sum"], yes["count"], no["sum"], no["count"]
h = 2 * np.arcsin(np.sqrt(p1)) - 2 * np.arcsin(np.sqrt(p0))
print(f"Promoters (recommend 9 or 10): {p1:.1%} of opted-in customers (n={yes['count']}
      f"against {p0:.1%} of the others (n={no['count']}), a difference of {(p1 - p0) *
      f"(95% CI {low * 100:+.1f} to {high * 100:+.1f}; z-test p = {p:.2f}; h = {h:.3f})
```

```
Promoters (recommend 9 or 10): 32.3% of opted-in customers (n=1020) against 31.3% of th
```

WARM-UP AND EXIT TICKET ANSWERS

WARM-UP

1. Two Bokeh plots on one page: what must they share for a box selection in one to highlight the same rows in the other? **Answer:** The same `ColumnDataSource` object. (Sharing a range object links panning and zooming instead.)

2. A Bokeh file saved with `output_file` has a dropdown. Which kind of callback works for a reader who opens the file? **Answer:** JavaScript: `CustomJS` (or `js_link`). A Python `on_change` callback needs a running Bokeh server.

3. A reader moves a Streamlit slider. What runs? **Answer:** The whole script, from the top, with the slider returning its new value.

4. Where does a value live that must survive reruns? And what stops a slow CSV load repeating on every click? **Answer:** `st.session_state` (per browser tab); `@st.cache_data` on the loading function (shared by all users).

5. Your capstone: say the question letter and the headline comparison in one sentence: "I am comparing ___ with ___ using ___." **Answer:** No single answer. Listen for a comparison rather than a topic, and note who has none yet: they get the first coaching round.

EXIT TICKET

1. Write your main result as one sentence with all five parts. **Answer:** Estimate for each group, the difference, its 95% interval, the test and p-value, and the effect size, in words a manager can repeat. A null result also says how large a difference the data can rule out.
2. Name one caveat that is specific to your question and data (not "more data is needed"). **Answer:** Check it is specific: who or what is missing, what could not be separated, or why "because" is too strong for this design.
3. What will you do in the 75 minutes of capstone work left after today, in order? **Answer:** Finish the figure, write report.md (Answer first, then Caveats), then Restart and Run All and check every number in the report against the fresh output, score yourself on the rubric, submit.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q2 answered B:** believes large samples excuse a missing interval. Expect a report with bare means.
- **Q3 answered `2 150.0`:** has not yet seen a merge repeat a value; check this learner's revenue totals for Questions B and D.
- **Q4 answered C:** turns "no evidence" into "no difference". The single most common Communication fault in capstone reports.
- **Q5 answered A:** the time-to-return trap. Ask the learner to print the latest first-order date in their comparison.
- **Q6 answered 5 or 20:** counts groups, or ordered pairs. Ten unordered pairs.
- **Q7 answered B:** the before-and-after trap; this learner needs the same-months comparison before submitting a Question D report.
- **Q9 answered B:** a generic caveat. Ask: "true of this analysis in what way?"
- **Q11 answered "it changes":** has not seen a seeded generator; check their bootstrap cells for a seed.

GRADING

The rubric, its weighting, scoring guidance, submission rules and the instructor answer notes for the four questions are in `final-assessment.html`.